

TEC LE COURS



THE KEEP

Programmation d'un moteur de jeu en 2.5D complet, étape par étape.

Cours en 4 parties, avec le code de chaque étape.

TANGUY CHENIER



SOMMAIRE

Partie 1. Le moteur de rendu	5
1. Le premier fichier	6
2. La fenêtre	8
3. Le tampon de pixels	10
4. La boucle principale	14
5. La représentation du niveau	17
6. Le joueur et ses déplacements	21
7. Le lancer de rayons	27
8. La correction de la distance	35
9. L'éclairage	38
10. La carte en médaillon	42
11. Le glissement le long des murs	44
 Partie 2. Les textures	 47
1. Le plein écran	48
2. L'image suit la fenêtre	51
3. Une texture dessinée par le programme	56
4. Les murs habillés	60
5. La lecture au bon endroit	64
6. Le sol	66
7. La brume au sol	71
8. Le plafond	73
9. Le rangement avant la livraison	77
10. Un fichier par métier	82
 Partie 3. Le niveau, la lumière et les portes	 89
1. Le niveau lu dans un fichier	90
2. Le point de départ et la sortie	94
3. Des plaques de métal	97
4. Plusieurs sortes de murs	102
5. Les portes	107
6. Les néons	114
7. La lumière vient du niveau	119
8. Trois couleurs de néon et un tube qui faiblit	128
9. La fenêtre décide de l'image	134
10. La carte se remplit	139
11. Une police en cent quatre-vingts octets	143
12. Le badge, la trappe et un mot au joueur	150
 Partie 4. Quelque chose dans le noir	 156
1. Le fichier d'une planche d'images	157

SOMMAIRE SUITE

2. Une créature dans le couloir	161
3. Le tampon de profondeur	167
4. Cinq angles, trois en miroir	171
5. Elle vous voit, et elle vient	175
6. Une barre d'acier	181
7. Le coup est un cône	187
8. Elle rend les coups	191
9. Une barre et un nombre	195
10. Elle tient sur le sol	198
11. Un cri porte	200
12. Ce qui traîne au sol	203
13. Une arme de poing, et elle coûte	209
14. Où le projectile est allé	215
15. La lourde	219
16. Mesurer avant de dire que c'est lent	224
17. La sortie	230
18. Personne aux touches	233

Partie 4. Le son, l'histoire et la fin

239

1. Le son est calculé lui aussi	240
2. Le rectangle autour d'une planche	253
3. La tôle arrachée	255
4. Trois sortes de portes	260
5. La carte est un radar	264
6. L'histoire, les bandes noires et le journal	266
7. La souris tourne la tête	279
8. L'équipage, les rôdeurs et la chose du fond	285
9. Le combat s'entend et se sent	300
10. L'histoire est déclenchée par des lieux	304
11. La borne ne joue jamais deux fois la même partie	309
12. Le niveau, du sas au fond de la soute	312

Comment suivre ce cours

Ce cours vous apprend à programmer un moteur de jeu en 2.5D, en C, sans moteur ni bibliothèque graphique, du premier fichier vide jusqu'à un niveau complet avec ses créatures, ses portes, sa lumière, son son et son histoire. Le jeu obtenu s'appelle The Keep. Il utilise la méthode de Wolfenstein 3D, le lancer de rayons, un monde plat vu en perspective dans lequel les murs ont tous la même hauteur. Cette méthode tient en quelques centaines de lignes, et elle se comprend entièrement.

Le cours ne suppose aucune connaissance en mathématiques au-delà des quatre opérations, et il explique chaque notion au moment où le programme en a besoin, les coordonnées, les vecteurs, la rotation, la projection, les couleurs, les filtres sonores. Il suppose en revanche que vous savez lire un programme en C, déclarer une variable, écrire une fonction et une boucle. Tout le reste est expliqué.

Le cours est organisé en quatre parties et soixante-trois chapitres. Chaque chapitre ajoute une chose au jeu, et se termine par un programme qui compile et qui se lance. Vous pouvez suivre le cours en tapant le code chapitre après chapitre, ou le lire avec le code final ouvert à côté. Le code complet, avec un commit par chapitre, se trouve sur GitHub, et le jeu se joue aussi dans le navigateur. Les adresses sont à la fin du livre.

Chaque chapitre suit le même ordre. Il commence par son objectif, puis explique ce qu'il faut comprendre, avec une figure quand un dessin vaut mieux qu'un texte. Quand il y a des calculs, une page les refait avec des nombres que vous pouvez vérifier à la main. Viennent ensuite les pages de code, qui montrent les lignes à écrire, en clair, dans le fichier tel qu'il est à la fin du chapitre, avec les lignes déjà présentes en gris et des explications ligne par ligne en marge. Une image montre enfin ce que vous devez voir à l'écran.

Les figures ne sont pas des croquis. Elles sont calculées par les mêmes formules que le jeu, et les textures et les planches montrées sont celles que le programme dessine. Les mots en vert sont des noms du code. Un vocabulaire ferme les chapitres qui introduisent des notions, et un exercice propose une modification à essayer.

Sur Linux, le jeu se compile avec gcc, les en-têtes de Xlib et celles d'ALSA, par la ligne de commande ci-dessous. Le jeu a été écrit en public, en quatre séances filmées, et les chapitres suivent l'ordre dans lequel il a été construit, parce que cet ordre est celui dans lequel on le comprend.

```
gcc -O2 -o keep src/*.c -Iinclude -lX11 -lm -lasound
```

VOCABULAIRE

Un chapitre est une unité du cours, quelques dizaines de lignes qui ajoutent une chose au jeu et se terminent par un lancement.

1

PREMIERE PARTIE

Le moteur de rendu

Dans cette première partie, vous allez écrire le cœur du moteur dans un seul fichier de quatre cents lignes. À la fin, votre programme affichera un couloir vu à la première personne, dans lequel le joueur se déplace au clavier.

11 CHAPITRES 396 LIGNES 1 FICHIER LANGAGE C

Le premier fichier

```
main.c: In function 'main':
main.c:10:23: error: storage size of 'screen' isn't known
    10 |         struct screen screen;
        |                     ^~~~~~
main.c:11:14: warning: implicit declaration of function 'screen_open' [-Wimplicit-function-declaration]
    11 |         if (!screen_open(&screen))
        |                     ^~~~~~
```

Figure 1.1. Le message du compilateur à la fin du chapitre. La structure et la fonction ne sont pas encore écrites.

Dans ce chapitre, vous allez créer le premier fichier du moteur et écrire la fonction principale avant tout le reste, en laissant le compilateur vous dire ce qui manque.

Le moteur tient dans un seul fichier, `main.c`. Il commence par trois lignes de commentaire qui disent ce que nous allons faire, puis par deux inclusions. La première donne accès à la bibliothèque Xlib, avec laquelle le programme parlera au serveur graphique. La seconde donne accès aux fonctions d'entrée et de sortie du langage C, comme `getchar` et `fprintf`.

La fonction `main` est écrite avant tout le reste. Elle déclare une variable `screen` de type `struct screen`, appelle la fonction `screen_open` pour ouvrir la fenêtre, attend une touche avec `getchar`, puis referme la connexion au serveur graphique avec `XCLOSEDisplay`.

Ni la structure ni la fonction n'existent encore. Le programme est écrit dans l'ordre où l'on réfléchit. On décrit d'abord ce que le programme doit faire, puis on écrit ce qui manque. Le compilateur le signale aussitôt, et ses deux messages nomment exactement ce qu'il reste à écrire, la structure `screen` et la fonction `screen_open`. C'est l'objet du chapitre suivant.

VOCABULAIRE

Xlib est la bibliothèque qui permet à un programme d'ouvrir une fenêtre et d'y dessiner sous le système X, le serveur graphique de Linux. **Une structure** est un type de donnée composé de plusieurs champs. La structure `screen` regroupera tout ce qui concerne la fenêtre.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

MAIN C LIGNES 1 A 17

```

1 // wolfenstein 3d drew its corridors with one ray per column, in 1992
2 // let us write that engine in c, from this empty file
3 // x11 gives us a window and a block of memory, the rest is ours
4
5 #include <X11/Xlib.h>
6 #include <stdio.h>
7
8 int main(void)
9 {
10     struct screen screen;
11     if (!screen_open(&screen))
12         return 1;
13
14     getchar();
15     XCloseDisplay(screen.display);
16     return 0;
17 }
```

Ligne par ligne

- 5** L'en-tête de Xlib. Toutes les fonctions dont le nom commence par un X viennent de cette bibliothèque.
- 10** La variable qui contiendra la fenêtre. Le type `struct screen` n'existe pas encore, et le compilateur le dit.
- 11** L'appel de la fonction qui ouvrira la fenêtre. Elle renverra 1 en cas de succès et 0 en cas d'échec, d'où le point d'exclamation qui teste l'échec.
- 14** Le programme attend une touche dans le terminal. Sans cette attente, la fenêtre se fermerait aussitôt ouverte.

La fenêtre



Figure 2.1. La fenêtre de 640 sur 400 pixels, vide, avec la couleur de fond choisie.

Dans ce chapitre, vous allez ouvrir une fenêtre avec la bibliothèque Xlib et comprendre ce que le serveur graphique attend d'un programme.

Nous écrivons maintenant la structure et la fonction annoncées au chapitre précédent. La structure `screen` conserve les trois objets que Xlib demande à chaque appel, la connexion au serveur graphique, la fenêtre elle-même et un contexte graphique, c'est-à-dire un jeu de réglages de dessin.

La fonction `screen_open` ouvre la connexion avec `XOpenDisplay`, puis crée une fenêtre de 640 pixels de large sur 400 de haut avec `XCreateSimpleWindow`. Une fenêtre créée n'est pas encore visible. C'est l'appel à `XMapWindow` qui la fait apparaître à l'écran, et l'appel à `XFlush` qui envoie réellement les ordres au serveur, car Xlib les accumule dans une file d'attente.

Entre les deux, le programme donne un nom et une classe à la fenêtre. Le gestionnaire de fenêtres s'en sert pour écrire la barre de titre et pour reconnaître le programme. Le fond de la fenêtre reçoit une couleur bleu nuit, définie par la constante `WINDOW_BG`.

VOCABULAIRE

Le serveur graphique est le programme qui gère l'écran, le clavier et la souris. Les applications lui envoient des ordres, comme ouvrir une fenêtre ou dessiner un rectangle. **Un contexte graphique** est un ensemble de réglages de dessin conservé par le serveur, que l'on passe à chaque fonction de dessin.

EXERCICE

Modifiez les constantes `WIN_WIDTH` et `WIN_HEIGHT`, recompilez et observez la nouvelle taille de la fenêtre. Puis changez `WINDOW_BG` pour obtenir un fond rouge, dont la valeur est `0xff0000`.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

MAIN C LIGNES 4 A 44

```

4
5 #include <X11/Xlib.h>
6 #include <X11/Xutil.h>
7 #include <stdio.h>
8
9 #define WIN_WIDTH 640
10 #define WIN_HEIGHT 400
11
12 #define WINDOW_BG 0x182636
13
14 struct screen {
15     Display *display;
16     Window window;
17     GC gc;
18 };
19
20 static int screen_open(struct screen *screen)
21 {
22     screen->display = XOpenDisplay(NULL);
23     if (!screen->display) {
24         fprintf(stderr, "no X display\n");
25         return 0;
26     }
27
28     int number = DefaultScreen(screen->display);
29     screen->window = XCreateSimpleWindow(screen->display,
30         RootWindow(screen->display, number),
31         0, 0, WIN_WIDTH, WIN_HEIGHT, 0, 0, WINDOW_BG);
32
33     // the window manager wants to know who it is drawing a title bar for
34     XClassHint class_hint = { "raycaster", "Raycaster" };
35     XSetClassHint(screen->display, screen->window, &class_hint);
36     XStoreName(screen->display, screen->window, "raycaster");
37
38     XMapWindow(screen->display, screen->window);
39     screen->gc = XCreateGC(screen->display, screen->window, 0, NULL);
40     XFlush(screen->display);
41     return 1;
42 }
43
44 int main(void)

```

Ligne par ligne

- 22** L'ouverture de la connexion. L'argument **NULL** désigne l'écran par défaut, celui indiqué par la variable d'environnement **DISPLAY**.
- 24** Si la connexion échoue, le programme l'écrit sur la sortie d'erreur et renvoie 0. L'appelant saura qu'il ne doit pas continuer.
- 29** La création de la fenêtre. Les arguments sont la fenêtre parente, la position, la taille, l'épaisseur de la bordure, la couleur de la bordure et la couleur du fond.
- 34** Le nom et la classe de la fenêtre, pour le gestionnaire de fenêtres.
- 38** La fenêtre devient visible.
- 40** Les ordres accumulés partent vers le serveur.

Le tampon de pixels

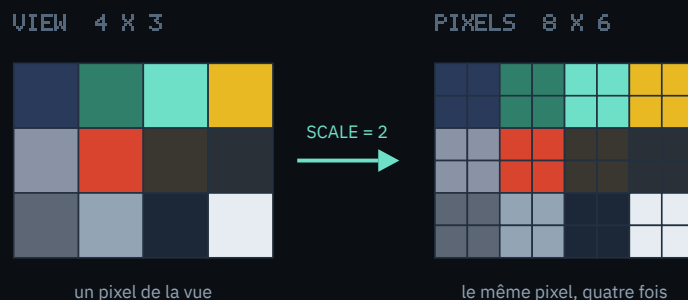


Figure 3.1. L'agrandissement d'un facteur 2. Chaque pixel du tableau `view` est recopié quatre fois dans le tableau `pixels`, en un carré de 2 sur 2. La fenêtre affiche le second tableau.

Dans ce chapitre, vous allez apprendre ce qu'est un tampon de pixels, dessiner dedans nombre par nombre, et l'envoyer à la fenêtre.

Un moteur de rendu n'appelle pas une fonction de dessin pour chaque point. Il écrit directement les couleurs dans un tableau de nombres, un nombre par pixel, puis envoie le tableau entier à la fenêtre. Ce tableau s'appelle le tampon de pixels. Toute l'image du jeu y est construite avant d'être affichée.

Le jeu dessine en 320 pixels de large sur 200 de haut, dans le tableau `view`. Cette taille est celle des jeux de 1992, et elle divise par quatre le travail de calcul par rapport à la fenêtre. La fonction `screen_present` agrandit ensuite l'image pour remplir la fenêtre de 640 sur 400. Chaque pixel de la vue devient un carré de 2 pixels sur 2, comme le montre la figure 3.1. Le facteur d'agrandissement est la constante `SCALE`.

L'agrandissement écrit dans un second tableau, `pixels`, de la taille de la fenêtre. Ce tableau est enveloppé dans une `XImage`, la structure par laquelle Xlib accepte une image en mémoire. L'appel `XPutImage` copie enfin l'image dans la fenêtre. Pour vérifier que tout fonctionne, la fonction `main` remplit la vue d'un dégradé, du vert dans le sens horizontal et du bleu dans le sens vertical.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

MAIN C LIGNES 6 A 18

```
6  #include <X11/Xutil.h>
7  #include <stdio.h>
8  #include <stdlib.h>
9
10 // the world is drawn at 320x200 and blown up to fill the window: chunky
11 // pixels are not nostalgia, they are four times fewer rays to trace
12 #define VIEW_WIDTH  320
13 #define VIEW_HEIGHT 200
14 #define SCALE        2
15 #define WIN_WIDTH    (VIEW_WIDTH * SCALE)
16 #define WIN_HEIGHT   (VIEW_HEIGHT * SCALE)
17
18 #define WINDOW_BG    0x182636
```

MAIN C LIGNES 22 A 31

```
22  Window  window;
23  GC      gc;
24  XImage  *image;
25  unsigned int *pixels;
26 };
27
28 // every pixel of the game lands here first
29 static unsigned int view[VIEW_WIDTH * VIEW_HEIGHT];
30
31 static int screen_open(struct screen *screen)
```

MAIN C LIGNES 38 A 43

```
38
39  int number = DefaultScreen(screen->display);
40  Visual *visual = DefaultVisual(screen->display, number);
41  int depth = DefaultDepth(screen->display, number);
42  screen->window = XCreateSimpleWindow(screen->display,
43  RootWindow(screen->display, number),
```

MAIN C LIGNES 51 A 86

```
51  XMapWindow(screen->display, screen->window);
52  screen->gc = XCreateGC(screen->display, screen->window, 0, NULL);
53
54  // XCreateImage copies nothing: we keep writing into our own buffer
and
55  // the server reads it from there
56  screen->pixels = malloc(WIN_WIDTH * WIN_HEIGHT * sizeof(unsigned
int));
57  if (!screen->pixels) {
58      fprintf(stderr, "out of memory\n");
59      return 0;
60  }
61
62  screen->image = XCreateImage(screen->display, visual, depth, ZPixmap,
0,
63      (char *)screen->pixels, WIN_WIDTH, WIN_HEIGHT, 32, 0);
64  XFlush(screen->display);
65  return 1;
66 }
67
68 // blow the 320x200 view up into the window, one source pixel per block
69 static void screen_present(struct screen *screen)
```

Ligne par ligne

- 12** La largeur de la vue, en pixels. La hauteur est de 200, et le rapport est celui d'un écran de l'époque.
- 29** Le tampon de pixels. Un tableau d'entiers non signés, un par pixel, rangé ligne par ligne. Le pixel de coordonnées (x;y) est à l'indice `y * VIEW_WIDTH + x`.
- 56** Le second tableau, à la taille de la fenêtre, alloué en mémoire. Le programme vérifie que l'allocation a réussi avant de continuer.
- 62** La structure `XImage` qui enveloppe le tableau. Elle ne copie rien, elle se contente de mémoriser l'adresse du tableau et sa taille.

PARTIE 1 CHAPITRE 3 LE CODE

SUITE

```
70 {
71     for (int y = 0; y < VIEW_HEIGHT; y++) {
72         unsigned int *row = view + y * VIEW_WIDTH;
73         for (int copy = 0; copy < SCALE; copy++) {
74             unsigned int *out = screen->pixels + (y * SCALE + copy) *
WIN_WIDTH;
75             for (int x = 0; x < VIEW_WIDTH; x++) {
76                 unsigned int color = row[x];
77                 for (int again = 0; again < SCALE; again++)
78                     *out++ = color;
79             }
80         }
81     }
82     XPutImage(screen->display, screen->window, screen->gc, screen->image,
83         0, 0, 0, 0, WIN_WIDTH, WIN_HEIGHT);
84     XFlush(screen->display);
85 }
86
```

MAIN C LIGNES 91 A 100

```
91     return 1;
92
93     // something to look at: green across the screen, blue down it
94     for (int y = 0; y < VIEW_HEIGHT; y++)
95         for (int x = 0; x < VIEW_WIDTH; x++)
96             view[y * VIEW_WIDTH + x] = (x << 8) | y;
97
98     screen_present(&screen);
99     getchar();
100     XCloseDisplay(screen.display);
```

73 Chaque ligne de la vue est recopiée **SCALE** fois, et dans chaque ligne chaque pixel est recopié **SCALE** fois. C'est l'agrandissement de la figure 3.1.

82 L'image entière est envoyée à la fenêtre en un seul appel.

96 Le dégradé de test. La valeur de x est placée dans l'octet du vert et celle de y dans l'octet du bleu.



Figure 3.2. Le dégradé de test, du vert vers la droite et du bleu vers le bas. Chaque pixel de la vue occupe un carré de 2 pixels sur 2 dans la fenêtre.

VOCABULAIRE

Le tampon de pixels est le tableau en mémoire dans lequel l'image est construite avant d'être affichée. **Un pixel** est un point de l'image. Sa couleur est un nombre entier.

La boucle principale



Figure 4.1. Le dégradé défile vers la gauche, une colonne par image. La fenêtre se ferme avec la touche Échap ou avec le bouton de fermeture.

Dans ce chapitre, vous allez écrire la boucle principale, celle qui produit les images l'une après l'autre, et lire le clavier.

Un jeu ne s'arrête pas après avoir affiché une image. Il recommence sans cesse la même suite d'opérations, lire les touches, mettre à jour le monde, dessiner, afficher. Cette répétition s'appelle la boucle principale, et chaque tour de boucle produit une image.

Avant la boucle, la fonction `screen_open` demande deux choses de plus au serveur. Elle s'abonne au message envoyé lorsque l'utilisateur clique sur le bouton de fermeture de la fenêtre, faute de quoi le serveur couperait la connexion sans prévenir. Elle demande aussi à recevoir les événements du clavier, en désactivant la répétition automatique des touches. Sans cette précaution, une touche maintenue enfoncée produirait une suite de relâchements et d'appuis, et le joueur avancerait par à-coups.

La fonction `screen_read_events` lit tous les événements en attente. Elle reconnaît le message de fermeture et la touche Échap, et note dans la structure `keys` que le programme doit quitter. La boucle principale s'arrête dès que ce drapeau est levé. Pour montrer que la boucle tourne, le dégradé se déplace d'un pixel à chaque image.

VOCABULAIRE

La boucle principale est la boucle qui produit les images l'une après l'autre, tant que le jeu n'est pas terminé. **Un événement** est un message du serveur graphique, comme l'appui d'une touche ou une demande de fermeture.

EXERCICE

Ajoutez une touche qui arrête le défilement du dégradé. Il faut un nouveau champ dans `struct keys`, un nouveau `case` dans la lecture des événements, et un test dans la boucle avant d'augmenter `frame`.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

MAIN C LIGNES 4 A 9

```
4
5 #include <X11/Xlib.h>
6 #include <X11/XKBlib.h>
7 #include <X11/keysym.h>
8 #include <X11/Xutil.h>
9 #include <stdio.h>
```

MAIN C LIGNES 25 A 29

```
25 GC gc;
26 XImage *image;
27 Atom close_message;
28 unsigned int *pixels;
29 };
```

MAIN C LIGNES 52 A 67

```
52 XStoreName(screen->display, screen->window, "raycaster");
53
54 // without this the close button cuts the connection under our feet;
55 // with it we get a message and we decide what to do
56 screen->close_message = XInternAtom(screen->display,
"WM_DELETE_WINDOW", False);
57 XSetWMProtocols(screen->display, screen->window, &screen-
>close_message, 1);
58
59 // and without this one, holding a key gives us release/press pairs
60 // thirty times a second, so the player would stutter
61 Bool supported;
62 XkbSetDetectableAutoRepeat(screen->display, True, &supported);
63
64 XSelectInput(screen->display, screen->window, KeyPressMask |
KeyReleaseMask);
65
66 XMapWindow(screen->display, screen->window);
67 screen->gc = XCreateGC(screen->display, screen->window, 0, NULL);
```

MAIN C LIGNES 79 A 107

```
79 XFlush(screen->display);
80 return 1;
81 }
82
83 struct keys {
84     int quit;
85 };
86
87 static void screen_read_events(struct screen *screen, struct keys *keys)
88 {
89     while (XPending(screen->display)) {
90         XEvent event;
91         XNextEvent(screen->display, &event);
92
93         if (event.type == ClientMessage &&
94             (Atom)event.xclient.data.l[0] == screen->close_message)
95             keys->quit = 1;
96
97         if (event.type == KeyPress || event.type == KeyRelease) {
98             int down = event.type == KeyPress;
```

Ligne par ligne

- 56** Le message de fermeture porte un nom, converti en un identifiant numérique appelé atome. Le programme annonce au serveur qu'il veut le recevoir.
- 62** La répétition automatique devient détectable. Une touche maintenue ne produit plus qu'un seul appui.
- 64** Le programme s'abonne aux appuis et aux relâchements de touches.
- 89** Tant que des événements attendent, le programme les lit un par un. La boucle ne bloque jamais.

PARTIE 1 CHAPITRE 4 LE CODE

SUITE

```
99         KeySym key = XkbKeycodeToKeysym(screen->display,
100             event.xkey.keycode, 0, 0);
101         switch (key) {
102             case XK_Escape: keys->quit = down; break;
103         }
104     }
105 }
106 }
107
```

MAIN C LIGNES 131 A 147

```
131     return 1;
132
133     struct keys keys = { 0 };
134     int frame = 0;
135
136     while (!keys.quit) {
137         screen_read_events(&screen, &keys);
138         frame++;
139         // something to look at: green across the screen, blue down it
140         for (int y = 0; y < VIEW_HEIGHT; y++)
141             for (int x = 0; x < VIEW_WIDTH; x++)
142                 view[y * VIEW_WIDTH + x] = (((x + frame) & 0xff) << 8) |
143 y;
144         screen_present(&screen);
145     }
146     XCloseDisplay(screen.display);
147     return 0;

```

99 Le code de la touche dépend du clavier. Il est converti en un symbole indépendant du matériel, comme `XK_Escape`.

102 La touche Échap lève le drapeau de sortie.

136 La boucle principale. Elle tourne tant que personne n'a demandé de quitter.

La représentation du niveau

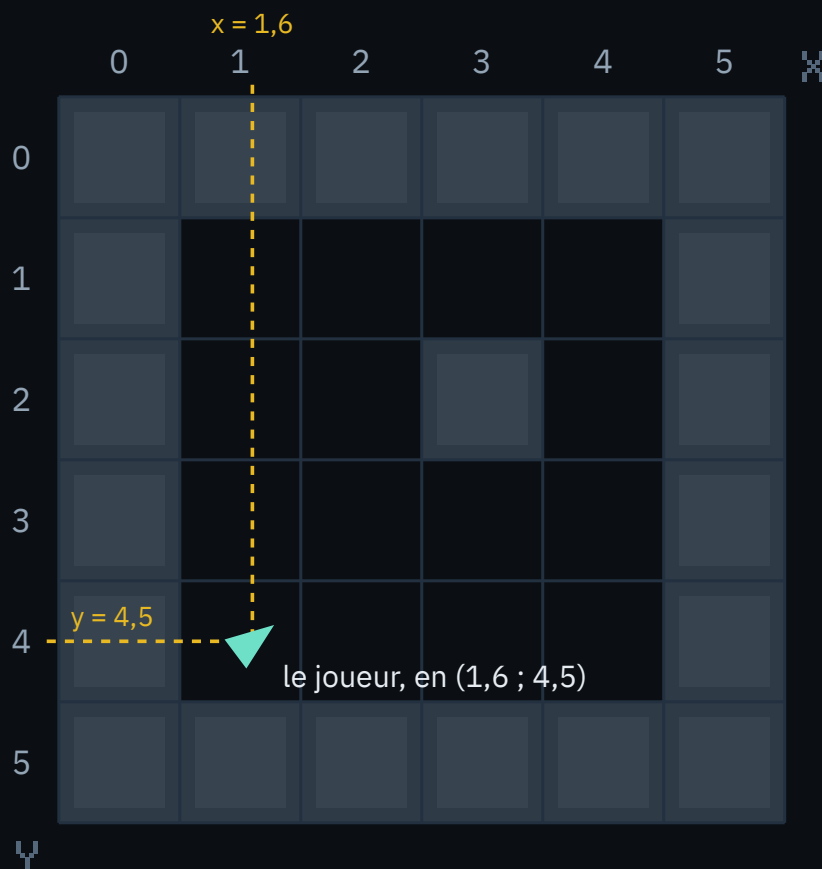


Figure 5.1. Une petite grille de 6 cases sur 6, et son écriture en caractères. Les cases grises sont des murs. Le triangle représente le joueur, placé en (1,6 ; 4,5). Le joueur n'est pas forcément au centre d'une case, c'est pourquoi sa position s'écrit avec des nombres à virgule.

Dans ce chapitre, vous allez représenter le monde du jeu par une grille de cases, apprendre à repérer une case par ses coordonnées, et dessiner cette grille.

Avant d'afficher une image du monde, le programme a besoin de connaître le niveau. Dans *The Keep*, le niveau est une grille de cases, comme une feuille de papier quadrillé. Chaque case contient soit un mur, soit du vide.

Dans cette première partie, le niveau est écrit directement dans le programme, sous la forme d'un tableau de 24 chaînes de caractères, une par ligne de la grille. Le caractère dièse représente un mur et le point représente une case vide. La fonction `is_wall` répond à la seule question que le moteur posera au niveau, cette case contient-elle un mur. Une case située hors de la grille est considérée comme un mur, ce qui empêche le joueur de sortir du monde.

Chaque case est repérée par deux coordonnées. La coordonnée x est le numéro de la colonne, comptée à partir de la gauche, et la coordonnée y est le numéro de la ligne, comptée à partir du haut. La numérotation commence à zéro. La case en haut à gauche a donc pour coordonnées (0 ; 0). La fonction `render_map` dessine la grille en grand, un carré de 8 pixels par case, pour vérifier le niveau à l'écran.

VOCABULAIRE

Une case est un élément de la grille. Elle contient un mur ou du vide. **Les coordonnées** x et y indiquent la position d'un point dans la grille.

La représentation du niveau, suite

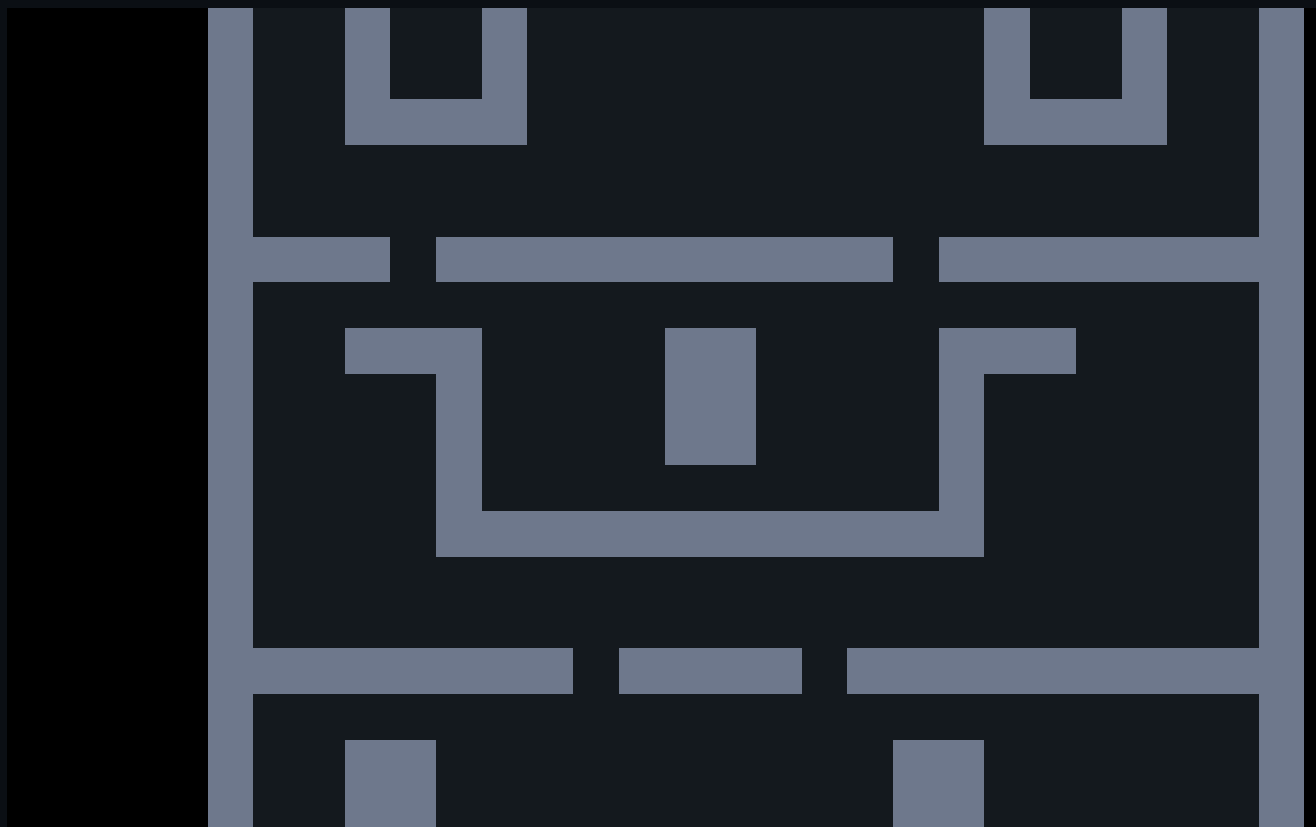


Figure 5.2. Le niveau de 24 cases sur 24, dessiné en grand dans la fenêtre. Les murs sont gris et les cases vides sont sombres.

EXERCICE

Modifiez le tableau `map` pour ajouter une pièce, puis relancez le programme. Le dessin doit suivre. Vérifiez que chaque ligne garde exactement 24 caractères.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

MAIN C LIGNES 18 A 69

```

18 #define WIN_HEIGHT (VIEW_HEIGHT * SCALE)
19
20 #define MAP_WIDTH 24
21 #define MAP_HEIGHT 24
22
23 #define MAP_CELL 3
24 #define MAP_LEFT 4
25 #define MAP_TOP 4
26
27 #define MAP_WALL 0x6f7b8f
28 #define MAP_FLOOR 0x161a22
29 #define MAP_HEADING 0xe8b04b
30 #define MAP_PLAYER 0xd8534f
31
32 #define WINDOW_BG 0x182636
33
34 // the whole world: a wall is anything that is not a dot
35 static const char *map[MAP_HEIGHT] = {
36     "#####",
37     "#.....#",
38     "#.####.####.",
39     "#...#.....#.#.",
40     "#...#.....#.#.",
41     "#.####.####.",
42     "#.....#",
43     "#.....#",
44     "####.#####.#####",
45     "#.....#",
46     "#.###...##...###...#",
47     "#...#...##...#...#.",
48     "#...#...##...#...#.",
49     "#...#...##...#...#.",
50     "#...#####...#.",
51     "#.....#",
52     "#.....#",
53     "#####.####.#####",
54     "#.....#",
55     "#..##.....##...#.",
56     "#..##.....##...#.",
57     "#.....#",
58     "#.....#",
59     "#####",
60 };
61
62 static int is_wall(int x, int y)
63 {
64     if (x < 0 || x >= MAP_WIDTH || y < 0 || y >= MAP_HEIGHT)
65         return 1;
66     return map[y][x] != '.';
67 }
68
69 struct screen {

```

MAIN C LIGNES 172 A 188

```

172 }
173
174 // the map, drawn big while it is the only thing we have to look at
175 static void render_map(int cell, int left, int top)

```

Ligne par ligne

- 35** Le niveau. Un tableau de chaînes de caractères, une par ligne. La première chaîne est la ligne du haut.
- 62** La question posée au niveau. Elle reçoit des coordonnées entières, celles d'une case.
- 64** Hors de la grille, tout est mur. Le moteur n'aura jamais à vérifier les bornes ailleurs.
- 66** Tout caractère autre que le point est un mur. Le tableau se lit d'abord par ligne, donc par y, puis par colonne.

PARTIE 1 CHAPITRE 5 LE CODE

SUITE

```
176 {
177     for (int y = 0; y < MAP_HEIGHT; y++)
178         for (int x = 0; x < MAP_WIDTH; x++) {
179             unsigned int color = is_wall(x, y) ? MAP_WALL : MAP_FLOOR;
180             for (int line = 0; line < cell; line++)
181                 for (int column = 0; column < cell; column++)
182                     view[(top + y * cell + line) * VIEW_WIDTH
183                         + left + x * cell + column] = color;
184         }
185 }
186
187 int main(void)
188 {
```

179 Deux couleurs, une pour les murs et une pour le vide.

199 La grille est dessinée avec des cases de 8 pixels, à partir du point (64 ; 4) de la vue.

MAIN C LIGNES 197 A 201

```
197     screen_read_events(&screen, &keys);
198     frame++;
199     render_map(8, 64, 4);
200
201     screen_present(&screen);
```


Le joueur et ses déplacements

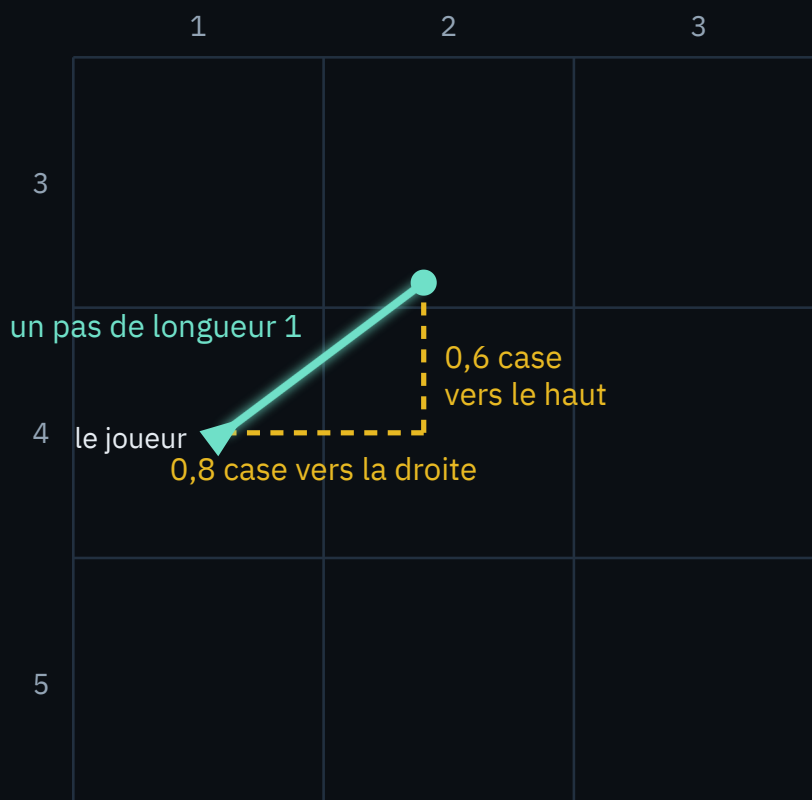


Figure 6.1. Un pas de longueur 1 dans la direction $(0,8 ; -0,6)$, en gros plan. Le pas déplace le joueur de 0,8 case vers la droite et de 0,6 case vers le haut. Le point vert marque le point d'arrivée.

Dans ce chapitre, vous allez placer le joueur dans la grille, représenter la direction de son regard par deux nombres, et le faire marcher et tourner au clavier.

Le joueur est décrit par une structure de six nombres. Les deux premiers, `x` et `y`, sont sa position dans la grille. Les deux suivants, `dir_x` et `dir_y`, sont la direction dans laquelle il regarde. Les deux derniers, `plane_x` et `plane_y`, décrivent la largeur de son champ de vision et seront expliqués au chapitre 7.

La direction n'est pas représentée par un angle, mais par deux nombres, comme la position. Ces deux nombres indiquent le déplacement du joueur lorsqu'il avance d'une longueur de 1 dans la direction de son regard. Par exemple, la direction $(0,8 ; -0,6)$ signifie qu'un pas de longueur 1 déplace le joueur de 0,8 case vers la droite et de 0,6 case vers le haut. Le signe moins correspond au haut de la grille, puisque la coordonnée `y` augmente vers le bas.

Au départ, le joueur est placé en $(2,5 ; 6,5)$, au centre d'une case, et regarde vers la droite, dans la direction $(1 ; 0)$. Les flèches du clavier le font avancer, reculer et tourner. Les touches `a` et `d` le font glisser sur le côté, sans tourner.

Pour tourner, le programme fait pivoter le vecteur de direction d'un angle `a`. La figure 6.2 montre le calcul. Un vecteur $(x ; y)$ tourné de l'angle `a` devient $(x \cos a - y \sin a ; x \sin a + y \cos a)$. La direction $(1 ; 0)$ tournée de 90 degrés devient ainsi $(0 ; 1)$, c'est-à-dire vers le bas de la grille. Le vecteur `plane` subit la même rotation, pour rester perpendiculaire à la direction.

Le joueur et ses déplacements, suite

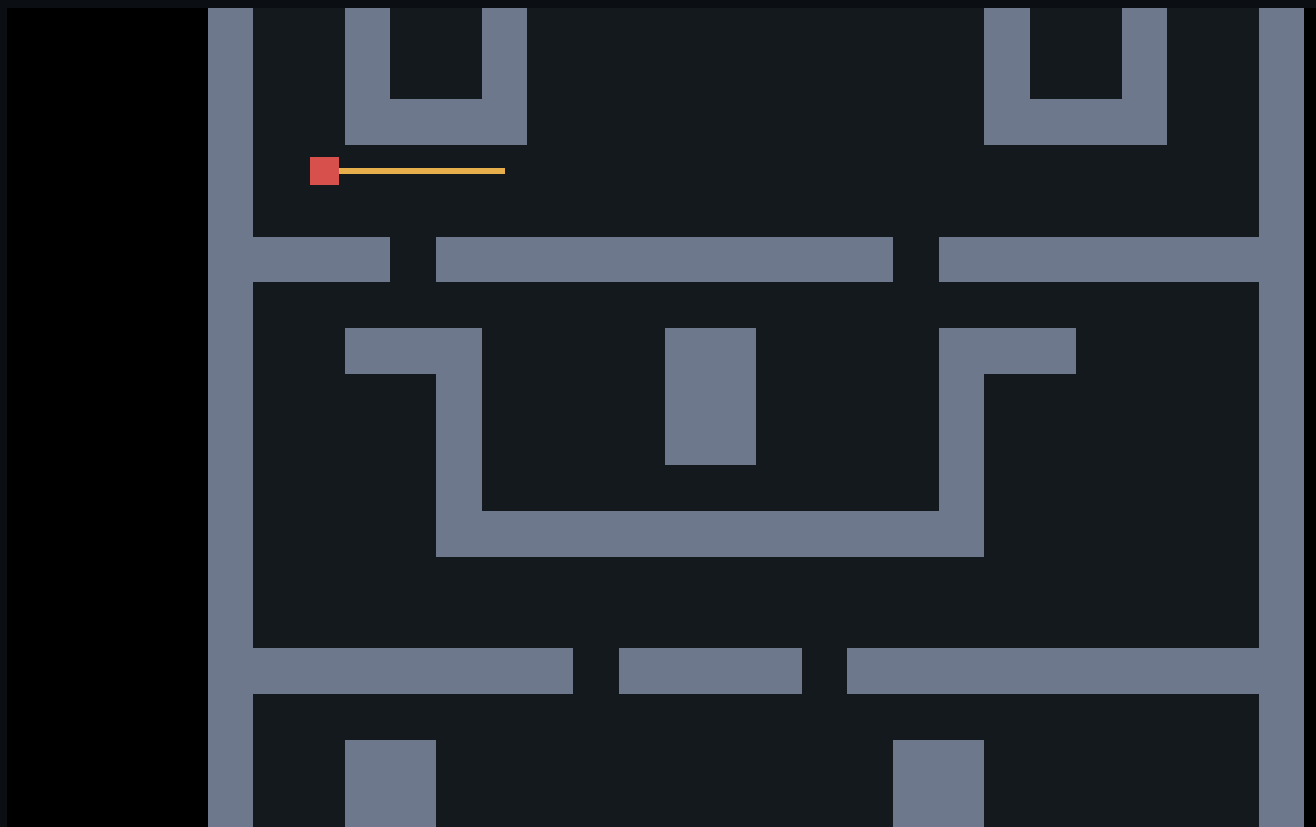


Figure 6.2. Le joueur, en rouge, est dessiné sur la grille avec un trait jaune qui indique la direction de son regard.

Les déplacements dépendent du temps et non du nombre d'images. À chaque tour de boucle, le programme mesure le temps écoulé depuis l'image précédente et multiplie la vitesse par ce temps. Un joueur avance ainsi de 3 cases par seconde, que la machine produise 30 ou 200 images par seconde. La fonction `now_in_seconds` lit une horloge qui ne recule jamais, même si l'heure du système est modifiée.

Enfin, le programme s'endort à la fin de chaque tour de boucle pour ne pas dépasser 60 images par seconde. Sans cette pause, la boucle occuperait un processeur entier à dessiner la même image des milliers de fois.

VOCABULAIRE

La direction est le couple de nombres qui indique où mène un pas de longueur 1. **Un radian** est l'unité d'angle des fonctions cos et sin du langage C. Un tour complet vaut 2π radians.

La rotation de la direction

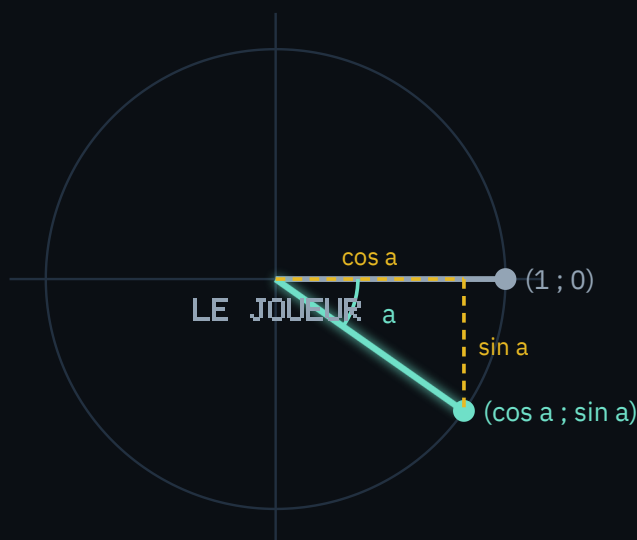


Figure 6.2. La direction (1 ; 0), en gris, tournée d'un angle a . Le vecteur obtenu, en vert, a pour composantes $\cos a$ et $\sin a$, en jaune. Toutes les directions de longueur 1 se trouvent sur le cercle.

Tourner la tête revient à faire pivoter le vecteur de direction autour du joueur. La longueur du vecteur reste 1, seul son angle change. La figure 6.2 montre la direction (1 ; 0), qui pointe vers la droite, tournée d'un angle a . Sa nouvelle valeur est $(\cos a ; \sin a)$, car le cosinus est la projection du vecteur sur l'axe horizontal et le sinus sa projection sur l'axe vertical.

Pour un vecteur quelconque $(x ; y)$, la formule générale est la suivante. La nouvelle composante horizontale vaut $x \cos a - y \sin a$, et la nouvelle composante verticale vaut $x \sin a + y \cos a$. Cette formule est celle de la rotation, et c'est exactement ce que la fonction `turn_player` calcule.

Prenons un exemple. Le joueur regarde vers la droite, en (1 ; 0), et tourne de 90 degrés. Le cosinus de 90 degrés vaut 0 et son sinus vaut 1. La nouvelle direction est donc $(1 \times 0 - 0 \times 1 ; 1 \times 1 + 0 \times 0) = (0 ; 1)$. Le joueur regarde maintenant vers le bas de la grille. Dans le programme, l'angle est exprimé en radians. Un tour complet vaut 2π radians, soit environ 6,28.

EXERCICE

Calculez la direction (1 ; 0) tournée de 180 degrés, dont le cosinus vaut -1 et le sinus 0. Vous devez trouver (-1 ; 0), le joueur regarde vers la gauche.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

MAIN C LIGNES 6 A 15

```
6  #include <X11/XKLib.h>
7  #include <X11/keysym.h>
8  #include <math.h>
9  #include <X11/Xutil.h>
10 #include <stdio.h>
11 #include <stdlib.h>
12 #include <time.h>
13 #include <unistd.h>
14
15 // the world is drawn at 320x200 and blown up to fill the window: chunky
```

MAIN C LIGNES 23 A 32

```
23 #define MAP_WIDTH  24
24 #define MAP_HEIGHT 24
25
26 #define WALK_SPEED 3.0
27 #define TURN_SPEED 2.2
28
29 // how wide the view is: 0.66 against a unit direction is about 66
   degrees
30 #define FIELD_OF_VIEW 0.66
31
32 #define MAP_CELL    3
```

MAIN C LIGNES 138 A 145

```
138
139 struct keys {
140     int forward, back, left, right, strafe_left, strafe_right, quit;
141 };
142
143 static void screen_read_keys(struct screen *screen, struct keys *keys)
144 {
145     while (XPending(screen->display)) {
```

MAIN C LIGNES 157 A 166

```
157         switch (key) {
158             case XK_Escape: keys->quit = down; break;
159             case XK_Up:     keys->forward = down; break;
160             case XK_Down:   keys->back = down; break;
161             case XK_Left:   keys->left = down; break;
162             case XK_Right:  keys->right = down; break;
163             case XK_a:      keys->strafe_left = down; break;
164             case XK_d:      keys->strafe_right = down; break;
165         }
166     }
```

MAIN C LIGNES 187 A 220

```
187 }
188
189 struct player {
190     double x, y;                // where we stand, in map squares
191     double dir_x, dir_y;        // a unit vector: where the eyes point
192     double plane_x, plane_y;    // the camera plane, as wide as the view
   is
193 };
194
195 static void move_player(struct player *player, double step_x, double
   step_y)
```

Ligne par ligne

190 La position du joueur, en cases. Les nombres à virgule permettent de se trouver n'importe où dans une case.

195 Le déplacement. La case d'arrivée est vérifiée avant de bouger, ce qui empêche de traverser un mur.

PARTIE 1 CHAPITRE 6 LE CODE

SUITE

```
196 {
197     if (!is_wall((int)(player->x + step_x), (int)(player->y + step_y))) {
198         player->x += step_x;
199         player->y += step_y;
200     }
201 }
202
203 // turning is one rotation matrix, applied to both vectors at once
204 static void turn_player(struct player *player, double angle)
205 {
206     double cosine = cos(angle);
207     double sine = sin(angle);
208
209     double dir_x = player->dir_x;
210     player->dir_x = dir_x * cosine - player->dir_y * sine;
211     player->dir_y = dir_x * sine + player->dir_y * cosine;
212     double plane_x = player->plane_x;
213     player->plane_x = plane_x * cosine - player->plane_y * sine;
214     player->plane_y = plane_x * sine + player->plane_y * cosine;
215 }
216
217 // the map, drawn big while it is the only thing we have to look at
218 static void render_map(const struct player *player, int cell, int left,
219 int top)
220 {
221     for (int y = 0; y < MAP_HEIGHT; y++)
```

MAIN C LIGNES 226 A 246

```
226         + left + x * cell + column] = color;
227     }
228
229     // where we stand, and which way we look
230     int dot_x = left + (int)(player->x * cell);
231     int dot_y = top + (int)(player->y * cell);
232     for (int step = 3; step < 4 * cell; step++)
233         view[(dot_y + (int)(player->dir_y * step)) * VIEW_WIDTH
234             + dot_x + (int)(player->dir_x * step)] = MAP_HEADING;
235     for (int line = -2; line <= 2; line++)
236         for (int column = -2; column <= 2; column++)
237             view[(dot_y + line) * VIEW_WIDTH + dot_x + column] =
238 MAP_PLAYER;
239
240 static double now_in_seconds(void)
241 {
242     struct timespec moment;
243     clock_gettime(CLOCK_MONOTONIC, &moment);
244     return moment.tv_sec + moment.tv_nsec / 1e9;
245 }
246
```

MAIN C LIGNES 251 A 287

```
251     return 1;
252
253     struct player player = {
254         .x = 2.5, .y = 6.5,
255         .dir_x = 1.0, .dir_y = 0.0,
256         .plane_x = 0.0, .plane_y = FIELD_OF_VIEW,
257     };
258     struct keys keys = { 0 };
259     double last = now_in_seconds();
260
261     while (!keys.quit) {
262         double moment = now_in_seconds();
263         double elapsed = moment - last;
264         last = moment;
265     }
```

210 La formule de la rotation, appliquée à la direction. L'ancienne valeur de `dir_x` a été mise de côté juste avant, car elle est encore utilisée à la ligne suivante.

230 La position du joueur, convertie en pixels de la grille dessinée.

243 L'horloge monotone. Elle compte les secondes depuis le démarrage de la machine et ne recule jamais.

256 Le vecteur du champ de vision, perpendiculaire à la direction. Sa longueur, 0,66, donne un angle de vue d'environ 66 degrés.

263 Le temps écoulé depuis l'image précédente, en secondes. Toutes les vitesses sont multipliées par cette valeur.

PARTIE 1 CHAPITRE 6 LE CODE

SUITE

```
266         screen_read_keys(&screen, &keys);
267
268         // every move is scaled by the time the last frame took, so the
game
269         // runs at the same speed whatever the machine is doing
270         double forward = (keys.forward - keys.back) * WALK_SPEED *
elapsed;
271         double sideways = (keys.strafe_right - keys.strafe_left) *
WALK_SPEED * elapsed;
272         double turn = (keys.right - keys.left) * TURN_SPEED * elapsed;
273
274         move_player(&player, player.dir_x * forward + player.plane_x *
sideways,
275                   player.dir_y * forward + player.plane_y * sideways);
276         turn_player(&player, turn);
277         render_map(&player, 8, 64, 4);
278         screen_present(&screen);
279
280         // nothing here needs more than sixty frames a second, and
without
281         // this the loop eats a whole core to draw the same thing twice
282         double spare = 1.0 / 60.0 - (now_in_seconds() - moment);
283         if (spare > 0)
284             usleep((useconds_t)(spare * 1e6));
285
286     }
287     XCloseDisplay(screen.display);
```

270 La distance à parcourir pendant cette image. Les deux touches se soustraient, si les deux sont enfoncées le joueur ne bouge pas.

284 La pause qui limite la boucle à 60 images par seconde.

Le lancer de rayons

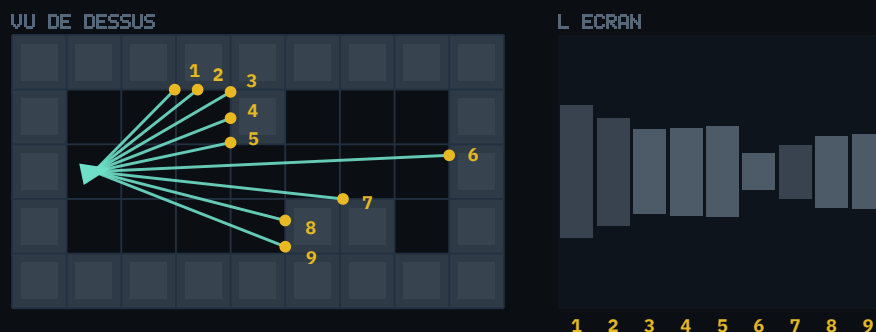


Figure 7.1. À gauche, le niveau vu de dessus, avec le joueur et neuf de ses rayons. À droite, l'image produite pour ces neuf colonnes. Le rayon 1 rencontre un mur proche et sa bande est haute. Le rayon 6 parcourt une longue distance et sa bande est basse. Les bandes claires correspondent aux murs atteints sur une ligne verticale de la grille et les bandes sombres aux murs atteints sur une ligne horizontale.

Dans ce chapitre, vous allez apprendre le lancer de rayons, la méthode qui transforme la grille en couloir vu à la première personne, avec tous ses calculs.

L'image du jeu est composée de 320 colonnes de pixels. Pour chaque colonne, le programme calcule la distance entre le joueur et le premier mur situé dans la direction correspondante. C'est le principe du lancer de rayons, celui de Wolfenstein 3D en 1992.

Une fois cette distance connue, le programme dessine dans la colonne une bande verticale dont la hauteur dépend de la distance. La bande est haute lorsque le mur est proche et basse lorsqu'il est loin. Les 320 bandes placées côte à côte forment l'image du couloir. Tout le moteur repose sur ce principe. Les textures, les portes et les monstres des parties suivantes viennent s'ajouter sur ces bandes.

Pour mesurer la distance, le programme trace un rayon, c'est-à-dire une demi-droite qui part de la position du joueur dans la direction associée à la colonne. Il cherche ensuite le point où ce rayon rencontre le premier mur.

La direction de chaque rayon se construit à partir des deux vecteurs du joueur. Le vecteur **dir** pointe vers le milieu de l'écran. Le vecteur **plane** est perpendiculaire à **dir** et sa longueur, 0,66, fixe la largeur du champ de vision. La figure 7.2 montre les deux vecteurs et les rayons.

Pour la colonne **x**, le programme calcule d'abord un nombre **camera** qui vaut -1 pour la colonne de gauche, 0 pour la colonne du milieu et +1 pour la colonne de droite. La direction du rayon est alors **dir** plus **plane** multiplié par **camera**. Le rayon du milieu est donc la direction du regard, et les rayons des bords s'écartent d'autant plus que **plane** est long.

Un champ de vision de 0,66 correspond à un angle d'environ 66 degrés, proche de celui de la vision humaine à l'écran. Une valeur plus grande donne un effet de grand angle, une valeur plus petite un effet de zoom.

Le vecteur du champ de vision

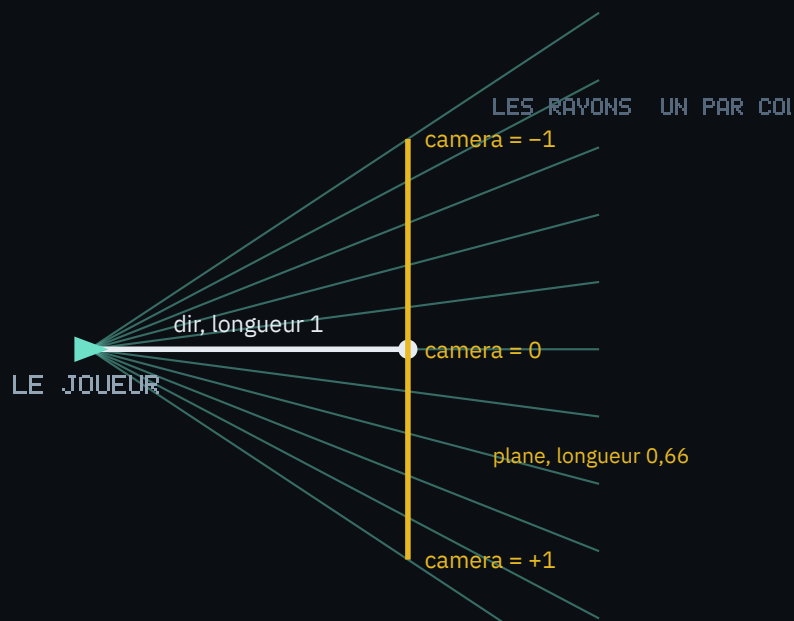


Figure 7.2. Le joueur regarde dans la direction `dir`, de longueur 1. Le segment jaune est le vecteur `plane`, parcouru de `-1` à `+1` par le nombre `camera`. Les rayons partent du joueur et passent par ce segment, un par colonne de l'image.

Le vecteur `plane` est obtenu en faisant tourner la direction d'un quart de tour, puis en la multipliant par 0,66. Pour une direction $(dir_x ; dir_y)$, le vecteur perpendiculaire est $(-dir_y ; dir_x)$. Au départ, le joueur regarde dans la direction $(1 ; 0)$, et son vecteur `plane` vaut donc $(0 ; 0,66)$.

La figure 7.2 montre le segment décrit par `plane` lorsque `camera` varie de `-1` à `+1`. Ce segment est l'écran lui-même, vu de dessus et placé à une case du joueur. Chaque colonne de l'image correspond à un point de ce segment, et le rayon de la colonne passe par ce point.

Pour la colonne 80 sur 320, le calcul donne `camera` = $2 \times 80 \div 320 - 1 = -0,5$. Avec la direction de départ, le rayon vaut $(1 ; 0) + (0 ; 0,66) \times (-0,5) = (1 ; -0,33)$. Il pointe vers la droite et légèrement vers le haut. Ce vecteur n'a pas une longueur de 1, et le chapitre 8 montrera que c'est une chance.

EXERCICE

Calculez le rayon de la colonne 320, la dernière. Vous devez trouver `camera` = 1 et un rayon de $(1 ; 0,66)$.

Le parcours du rayon dans la grille

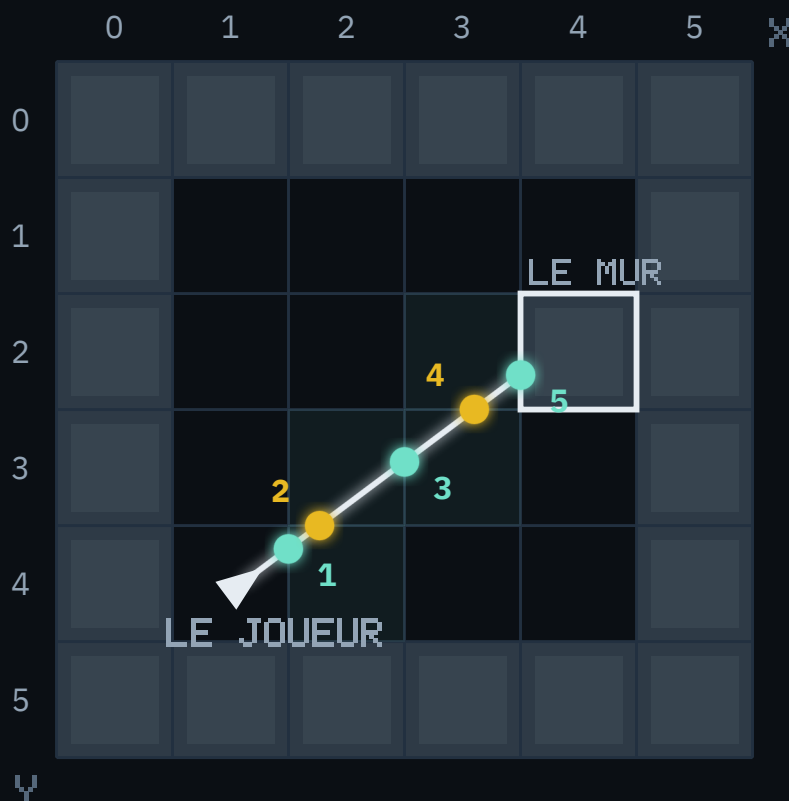


Figure 7.3. Le rayon part du joueur, placé en (1,6 ; 4,5), dans la direction (0,8 ; -0,6). Les points verts marquent les lignes verticales franchies et les points jaunes les lignes horizontales. Au cinquième franchissement, le rayon entre dans la case (4 ; 2), qui contient un mur. Le parcours s'arrête.

Pour trouver le point où le rayon rencontre un mur, le programme parcourt la grille case par case en suivant le rayon. À chaque ligne de la grille franchie, il vérifie si la case dans laquelle le rayon entre contient un mur.

Le rayon franchit deux sortes de lignes, les lignes verticales, en vert sur la figure 7.3, et les lignes horizontales, en jaune. Le programme n'avance pas d'une longueur fixe. À chaque étape, il détermine laquelle des deux prochaines lignes est la plus proche, et il avance directement jusqu'à elle.

Pour cela, il conserve deux longueurs, la longueur du rayon jusqu'à la prochaine ligne verticale et la longueur du rayon jusqu'à la prochaine ligne horizontale. À chaque étape, il choisit la plus petite des deux, il fait entrer le rayon dans la case correspondante, puis il augmente cette longueur de la valeur nécessaire pour traverser une case.

POURQUOI AUCUNE CASE N'EST OUBLIÉE

Deux lignes voisines sont séparées d'exactly une case. Le programme avance toujours jusqu'à la plus proche, le rayon ne peut donc pas en sauter une.

Un exemple de calcul

PAS	VERTICALE	HORIZONTALE	CHOIX	CASE
1	0,5	0,833	verticale, 0,5	(2 ; 4)
2	1,75	0,833	horizontale, 0,833	(2 ; 3)
3	1,75	2,5	verticale, 1,75	(3 ; 3)
4	3	2,5	horizontale, 2,5	(3 ; 2)
5	3	4,167	verticale, 3	(4 ; 2) mur

Tableau 7.1. Le parcours du rayon, une ligne du tableau par franchissement. À chaque étape, le programme compare les deux longueurs, choisit la plus petite, fait entrer le rayon dans la case suivante, puis ajoute à la longueur choisie la valeur nécessaire pour traverser une case, 1,25 pour une ligne verticale et 1,667 pour une ligne horizontale. À la cinquième étape, la case (4 ; 2) contient un mur. La longueur du rayon vaut alors 3. C'est la distance recherchée pour cette colonne.

EXERCICE

Refaites le calcul avec le joueur placé au centre de sa case, en (1,5 ; 4,5). La première ligne verticale se trouve alors à une demi-case, et la longueur de départ vaut $0,5 \times 1,25 = 0,625$. La suite du calcul est identique.

Reprenons la figure 7.3 et effectuons les calculs à la main. Le joueur est en (1,6 ; 4,5) et regarde dans la direction (0,8 ; -0,6). Les calculs ne font intervenir que des divisions, des multiplications et des additions.

La longueur nécessaire pour traverser une case

Un pas de longueur 1 déplace le rayon de 0,8 case vers la droite. Pour traverser une case entière dans le sens horizontal, le rayon doit donc avancer d'une longueur de $1 \div 0,8 = 1,25$. Dans le programme, cette valeur s'appelle `delta_x`. De même, un pas de longueur 1 déplace le rayon de 0,6 case vers le haut, et la traversée d'une case dans le sens vertical demande une longueur de $1 \div 0,6 = 1,667$. Cette valeur s'appelle `delta_y`.

La longueur jusqu'à la première ligne

Le joueur est en $x = 1,6$. La prochaine ligne verticale est en $x = 2$, à 0,4 case de distance, ce qui correspond à une longueur de rayon de $0,4 \times 1,25 = 0,5$. Cette valeur s'appelle `side_x`. Dans le sens vertical, le joueur est en $y = 4,5$ et la prochaine ligne horizontale est en $y = 4$, à une demi-case, soit une longueur de $0,5 \times 1,667 = 0,833$. Cette valeur s'appelle `side_y`. Le parcours peut alors commencer.

La hauteur des murs à l'écran

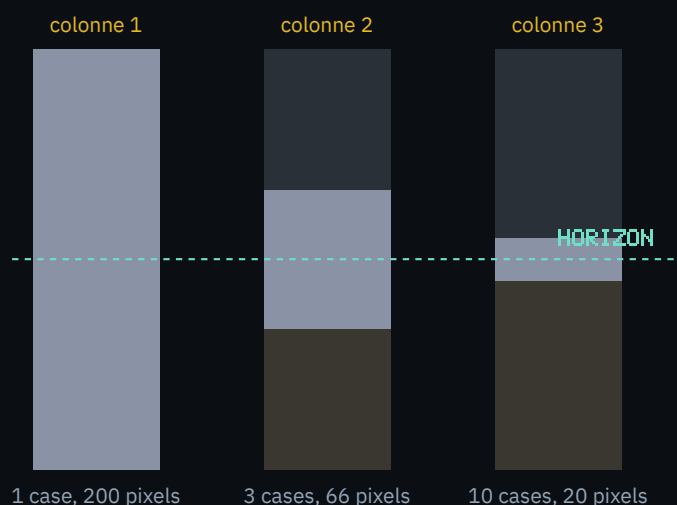


Figure 7.4. Trois colonnes de l'image, pour des murs situés à 1, 3 et 10 cases. La bande du mur est centrée sur l'horizon et sa hauteur est 200 divisée par la distance. Le plafond occupe le haut de la colonne et le sol le bas.

La distance connue, il reste à dessiner la colonne. Un objet paraît d'autant plus petit qu'il est éloigné. Un mur situé à deux cases paraît deux fois plus petit qu'un mur situé à une case. La hauteur de la bande est donc la hauteur de l'écran divisée par la distance.

L'écran fait 200 pixels de haut. Un mur à une case de distance occupe toute la hauteur, 200 pixels. Un mur à 3 cases occupe $200 \div 3 = 66$ pixels, et un mur à 10 cases en occupe 20. La bande est centrée sur l'horizon, c'est-à-dire sur la ligne du milieu de l'écran, à 100 pixels du haut. Pour une bande de 66 pixels, le haut de la bande est donc à $100 - 33 = 67$ pixels du haut de l'écran.

Au-dessus de la bande, la colonne reçoit la couleur du plafond. En dessous, elle reçoit la couleur du sol. Une colonne est ainsi toujours composée de trois morceaux, le plafond, le mur et le sol, et l'image entière n'est qu'une suite de 320 colonnes de ce genre.

VOCABULAIRE

L'**horizon** est la ligne du milieu de l'écran, à hauteur des yeux du joueur. Les murs sont centrés sur cette ligne.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

MAIN C LIGNES 20 A 24

```
20 #define WIN_WIDTH (VIEW_WIDTH * SCALE)
21 #define WIN_HEIGHT (VIEW_HEIGHT * SCALE)
22 #define HORIZON (VIEW_HEIGHT / 2)
23
24 #define MAP_WIDTH 24
```

MAIN C LIGNES 27 A 34

```
27 #define WALK_SPEED 3.0
28 #define TURN_SPEED 2.2
29
30 #define CEILING_COLOR 0x2a3038
31 #define FLOOR_COLOR 0x3a3630
32 #define WALL_COLOR 0x8a93a5
33
34 // how wide the view is: 0.66 against a unit direction is about 66
    degrees
```

MAIN C LIGNES 220 A 292

```
220 }
221
222 static void draw_column(int x, int top, int bottom, unsigned int color)
223 {
224     if (top < 0)
225         top = 0;
226     if (bottom > VIEW_HEIGHT)
227         bottom = VIEW_HEIGHT;
228
229     for (int y = top; y < bottom; y++)
230         view[y * VIEW_WIDTH + x] = color;
231 }
232
233 // how far the ray is from the first grid line it will cross
234 static double first_line(double position, double direction)
235 {
236     double cell = position - floor(position);
237     return direction < 0 ? cell : 1.0 - cell;
238 }
239
240 // one ray per column of the screen. the ray walks the grid square by
    square
241 // until it meets a wall, and how far it went decides how tall to draw it
242 static void render_walls(const struct player *player)
243 {
244     for (int x = 0; x < VIEW_WIDTH; x++) {
245         // -1 on the left edge of the screen, +1 on the right edge
246         double camera = 2.0 * x / VIEW_WIDTH - 1.0;
247         double ray_x = player->dir_x + player->plane_x * camera;
248         double ray_y = player->dir_y + player->plane_y * camera;
249
250         int map_x = (int)player->x;
251         int map_y = (int)player->y;
252
253         // the distance the ray covers to cross one whole square
254         double delta_x = ray_x == 0.0 ? 1e30 : fabs(1.0 / ray_x);
255         double delta_y = ray_y == 0.0 ? 1e30 : fabs(1.0 / ray_y);
256         double side_x = first_line(player->x, ray_x) * delta_x;
257         double side_y = first_line(player->y, ray_y) * delta_y;
```

Ligne par ligne

- 22** L'horizon, la ligne du milieu de l'écran. Les bandes de mur sont centrées dessus.
- 222** Le dessin d'un morceau de colonne. Les bornes sont ramenées dans l'écran, car un mur très proche déborde en haut et en bas.
- 236** La fraction de case déjà parcourue. La fonction `floor` arrondit vers le bas.
- 246** Le nombre `camera`, de -1 à $+1$ sur la largeur de l'écran.
- 247** La direction du rayon, la direction du regard plus une part du vecteur `plane`.
- 254** La longueur nécessaire pour traverser une case dans le sens horizontal, $1 \div 0,8 = 1,25$ dans l'exemple du tableau 7.1. La valeur `1e30` est un nombre très grand, utilisé lorsque le rayon est exactement vertical.
- 256** La longueur jusqu'à la première ligne verticale, $0,4 \times 1,25 = 0,5$ dans l'exemple.

PARTIE 1 CHAPITRE 7 LE CODE

SUITE

```
258     int step_x = ray_x < 0 ? -1 : 1;
259     int step_y = ray_y < 0 ? -1 : 1;
260
261     // always step along the axis whose grid line is nearest. that is
262     // the whole trick: no square is missed, and none is visited
263     twice
264     int side = 0;
265     while (!is_wall(map_x, map_y)) {
266         if (side_x < side_y) {
267             side_x += delta_x;
268             map_x += step_x;
269             side = 0;
270         } else {
271             side_y += delta_y;
272             map_y += step_y;
273             side = 1;
274         }
275     }
276     // how far the ray actually travelled before it hit something
277     double travelled = side == 0 ? side_x - delta_x : side_y -
delta_y;
278     double distance = travelled * hypot(ray_x, ray_y);
279     if (distance < 0.02)
280         distance = 0.02;
281
282     int height = (int)(VIEW_HEIGHT / distance);
283     int top = HORIZON - height / 2;
284
285     draw_column(x, 0, top, CEILING_COLOR);
286     draw_column(x, top, top + height, WALL_COLOR);
287     draw_column(x, top + height, VIEW_HEIGHT, FLOOR_COLOR);
288 }
289 }
290
291 // the map, drawn big while it is the only thing we have to look at
292 static void render_map(const struct player *player, int cell, int left,
int top)
```

MAIN C LIGNES 349 A 353

```
349     player.dir_y * forward + player.plane_y * sideways);
350     turn_player(&player, turn);
351     render_walls(&player);
352     screen_present(&screen);
353
```

- 258** Le sens de parcours sur chaque axe.
- 264** La boucle du parcours. Chaque tour correspond à une ligne du tableau 7.1.
- 265** La plus petite des deux longueurs gagne. Le programme la prolonge d'une case et passe à la case suivante sur cet axe.
- 277** La longueur du rayon au moment où il est entré dans le mur. La dernière prolongation, faite après l'entrée, est retirée.
- 278** La distance réelle, longueur du rayon multipliée par la longueur de son vecteur. Cette ligne est la cause du défaut visible sur la figure 7.5, et le chapitre 8 la remplace.
- 282** La hauteur de la bande, la hauteur de l'écran divisée par la distance.
- 283** Le haut de la bande, centrée sur l'horizon.
- 285** Les trois morceaux de la colonne, le plafond, le mur et le sol.

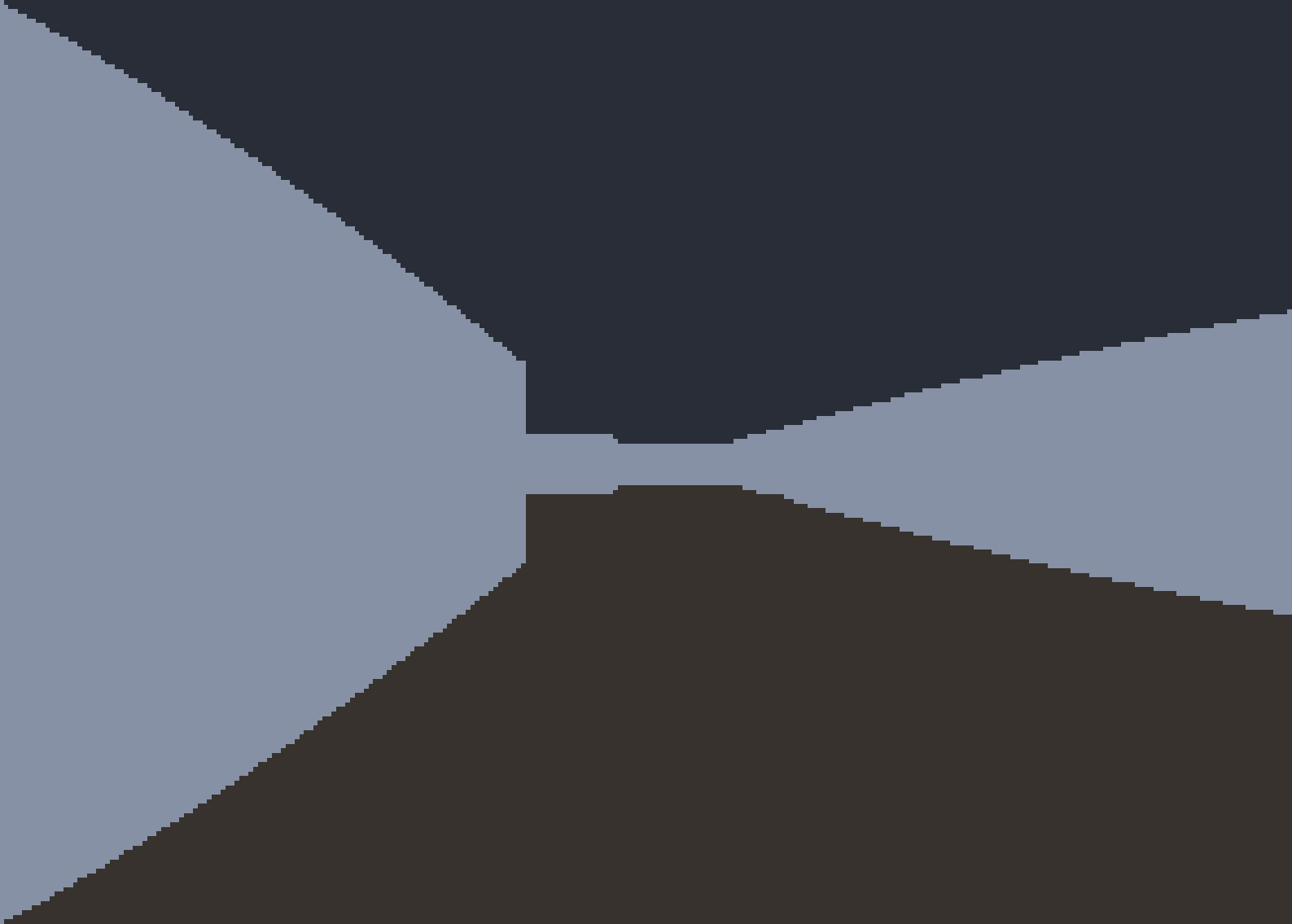


Figure 7.2. La première image en relief. Les murs sont d'une seule teinte et leur hauteur ne dépend que de la distance. Les murs droits paraissent courbés, un défaut corrigé au chapitre suivant.

VOCABULAIRE

Le lancer de rayons est la méthode qui calcule une image en suivant, pour chaque colonne de l'écran, un rayon parti de l'œil jusqu'au premier obstacle. **Le champ de vision** est l'angle que couvre l'image, environ 66 degrés dans The Keep.

La correction de la distance



Figure 8.1. Un mur droit devant le joueur. La distance mesurée le long du rayon, en vert, est plus grande pour les rayons des bords. La distance au plan de la caméra, en pointillés, est la même pour tous les rayons. C'est elle qui donne des murs droits.

Dans ce chapitre, vous allez comprendre pourquoi les murs paraissent courbés et corriger la mesure de la distance.

Sur l'image du chapitre précédent, les murs droits paraissent courbés. Le défaut vient de la distance utilisée pour calculer la hauteur des bandes. Le programme mesurait la longueur du rayon depuis le joueur jusqu'au mur. Or, devant un mur droit, les rayons des bords de l'écran sont plus longs que le rayon du milieu, comme le montre la figure 8.1. Les colonnes des bords recevaient donc des bandes plus basses, et le mur semblait s'éloigner sur les côtés.

La bonne mesure est la distance entre le mur et le plan de la caméra, c'est-à-dire la longueur du rayon projetée sur la direction du regard. Devant un mur droit, cette distance est la même pour toutes les colonnes, et le mur apparaît droit.

Il se trouve que le programme possédait déjà cette valeur. Le vecteur du rayon n'a pas une longueur de 1, il vaut `dir` plus une part de `plane`. La longueur `travelled` du parcours est comptée en multiples de ce vecteur, et cette mesure est précisément la distance au plan de la caméra. La multiplication par la longueur du vecteur, ajoutée au chapitre 7, la transformait en distance réelle. Il suffit de la retirer.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

MAIN C LIGNES 274 A 283

```

274     }
275
276     // how far the ray actually distance before it hit something
277     double distance = side == 0 ? side_x - delta_x : side_y -
    delta_y;
278     // measured from the eye, the corners of a wall are farther than
    its
279     // middle, so a straight wall bends into a fishbowl. what we want
    is
280     // the distance on the camera plane, and the ray was already
    giving
281     // it to us before we "corrected" it
282     if (distance < 0.02)
283         distance = 0.02;

```

Ligne par ligne

277 La distance au plan de la caméra. C'est la valeur qui s'appelait **travelled** au chapitre 7, sans la multiplication qui la déformait.

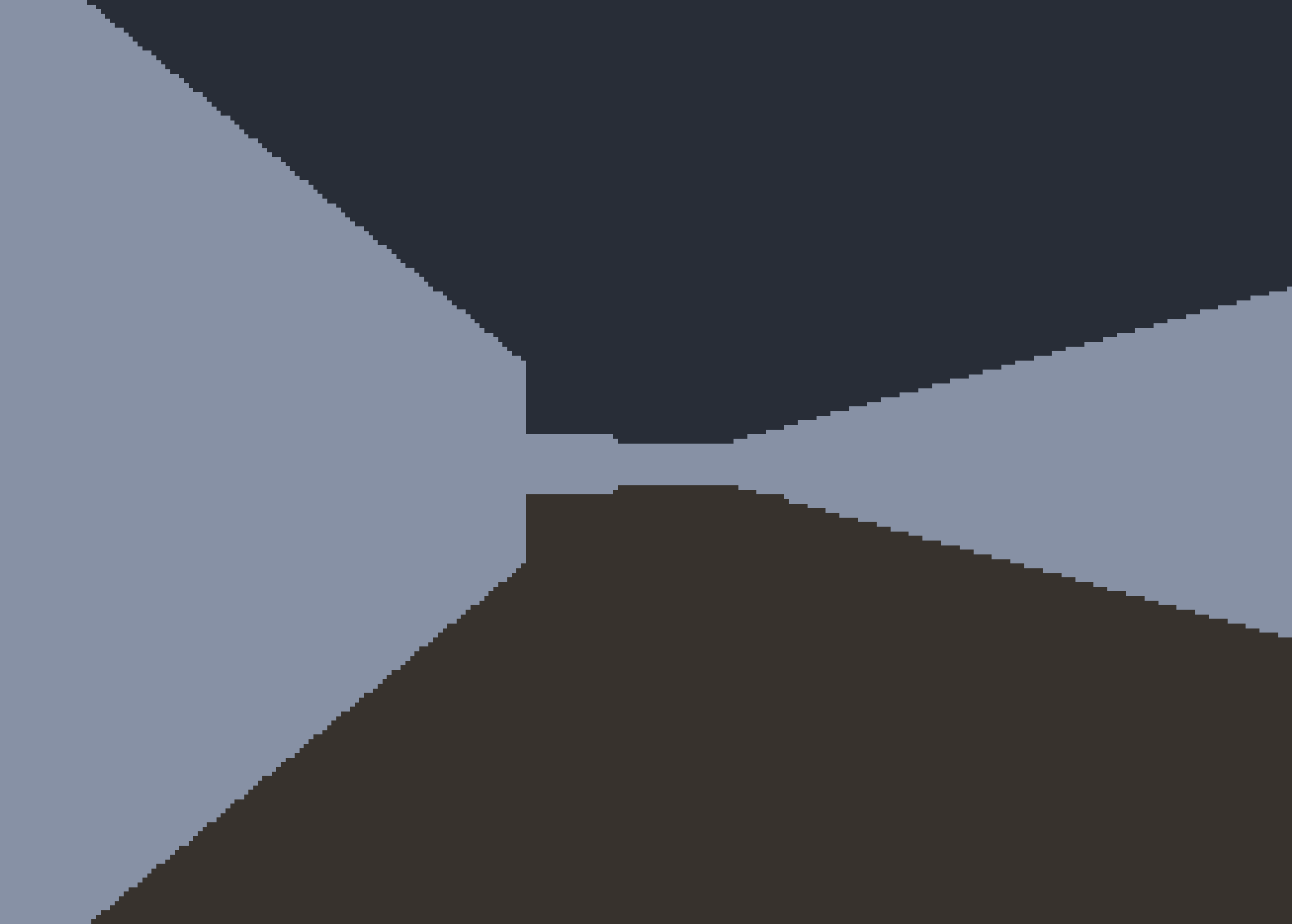


Figure 8.2. Le même couloir, avec la distance au plan de la caméra. Les murs sont droits.

VOCABULAIRE

Le plan de la caméra est la droite qui passe par le joueur et qui est perpendiculaire à la direction de son regard. C'est l'écran vu de dessus.

EXERCICE

Remettez la multiplication par `hypot(ray_x, ray_y)` et tournez sur place devant un mur. Observez la courbure, puis retirez-la de nouveau.

L'éclairage

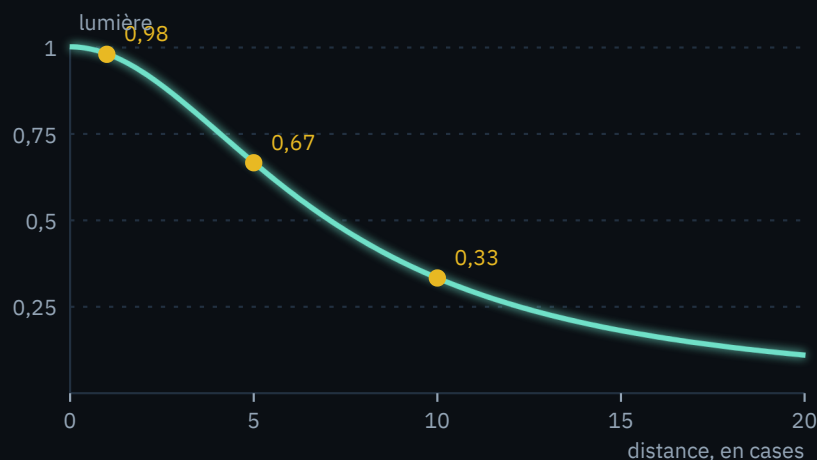


Figure 9.1. Le facteur de lumière en fonction de la distance, calculé par la fonction `fog`. La formule est $1 \div (1 + 0,02 \times \text{distance}^2)$. Les points jaunes marquent les distances de 1, 5 et 10 cases.

Dans ce chapitre, vous allez éclairer les murs, apprendre comment une couleur est rangée dans un nombre, et faire disparaître le fond des couloirs dans la brume.

Avec une seule teinte de mur, deux murs qui se rejoignent à angle droit ont exactement la même couleur, et l'angle disparaît. Nous apportons deux améliorations. Les murs atteints sur une ligne verticale de la grille reçoivent une teinte claire, et ceux atteints sur une ligne horizontale une teinte sombre. La variable `side` du parcours, qui mémorise le type de la dernière ligne franchie, sert précisément à ce choix.

La seconde amélioration est la brume. Plus un mur est loin, plus il est sombre. La fonction `fog` calcule un facteur de lumière compris entre 0 et 1 à partir de la distance, selon la courbe de la figure 9.1. À une case, la lumière vaut 0,98. À cinq cases, elle vaut 0,67. À dix cases, elle ne vaut plus que 0,33. Les couloirs prennent ainsi de la profondeur.

La fonction `shade` applique ce facteur à une couleur. Une couleur est composée de trois nombres, le rouge, le vert et le bleu, rangés dans un seul entier. La fonction les sépare, multiplie chacun par le facteur de lumière, puis les rassemble. La page suivante détaille cette opération.

Une couleur dans un nombre



Figure 9.2. La couleur `0x8a93a5` dans ses 24 bits utiles. Chaque octet est une composante. Le décalage de 16 bits fait descendre le rouge en bas du nombre, celui de 8 bits fait descendre le vert, et le bleu est déjà en bas.

Une couleur est enregistrée dans un seul nombre entier de 32 bits. Les trois composantes rouge, verte et bleue occupent chacune un octet, c'est-à-dire une valeur de 0 à 255. Le rouge occupe l'octet du milieu haut, le vert l'octet suivant et le bleu l'octet du bas. La couleur `0x8a93a5` des murs clairs se décompose ainsi en rouge 138, vert 147 et bleu 165.

Pour assombrir une couleur, le programme extrait chaque composante, la multiplie par un facteur compris entre 0 et 1, puis remet les trois résultats à leur place. Avec un facteur de 0,667, les murs clairs deviennent rouge 92, vert 98 et bleu 110, une teinte plus sombre mais de la même couleur.

L'extraction se fait avec deux opérations sur les bits. Le décalage vers la droite, écrit `>>`, fait descendre l'octet voulu tout en bas du nombre. Le masque `& 0xff` ne garde que cet octet. Le décalage vers la gauche, écrit `<<`, fait l'opération inverse au moment de recomposer la couleur.

EXERCICE

Décomposez la couleur du sol, `0x3a3630`, puis appliquez un facteur de 0,5. Vous devez trouver rouge 29, vert 27 et bleu 24.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

MAIN C LIGNES 30 A 35

```
30 #define CEILING_COLOR 0x2a3038
31 #define FLOOR_COLOR 0x3a3630
32 #define WALL_LIGHT 0x8a93a5
33 #define WALL_DARK 0x5d6675
34
35 // how wide the view is: 0.66 against a unit direction is about 66
    degrees
```

MAIN C LIGNES 221 A 241

```
221 }
222
223 static unsigned int shade(unsigned int color, double light)
224 {
225     if (light > 1.0)
226         light = 1.0;
227
228     unsigned int red = (unsigned int)(((color >> 16) & 0xff) * light);
229     unsigned int green = (unsigned int)(((color >> 8) & 0xff) * light);
230     unsigned int blue = (unsigned int)((color & 0xff) * light);
231     return (red << 16) | (green << 8) | blue;
232 }
233
234 // far away is dark. one line, and the corridors have depth
235 static double fog(double distance)
236 {
237     return 1.0 / (1.0 + distance * distance * 0.02);
238 }
239
240 static void draw_column(int x, int top, int bottom, unsigned int color)
241 {
```

MAIN C LIGNES 305 A 313

```
305
306     draw_column(x, 0, top, CEILING_COLOR);
307     // the two orientations must not share a shade, or every corner
308     // disappears
309     double light = fog(distance);
310     draw_column(x, top, top + height,
311         shade(side == 0 ? WALL_LIGHT : WALL_DARK, light));
312     draw_column(x, top + height, VIEW_HEIGHT, FLOOR_COLOR);
313 }
```

Ligne par ligne

223 L'assombrissement d'une couleur par un facteur de lumière.

228 La composante rouge, extraite par un décalage de 16 bits et un masque, puis multipliée par le facteur.

231 Les trois composantes remises en place, par des décalages et un ou bit à bit.

237 La courbe de la brume. Le carré de la distance fait chuter la lumière de plus en plus vite.

311 La teinte dépend de la ligne franchie, puis la brume s'applique.

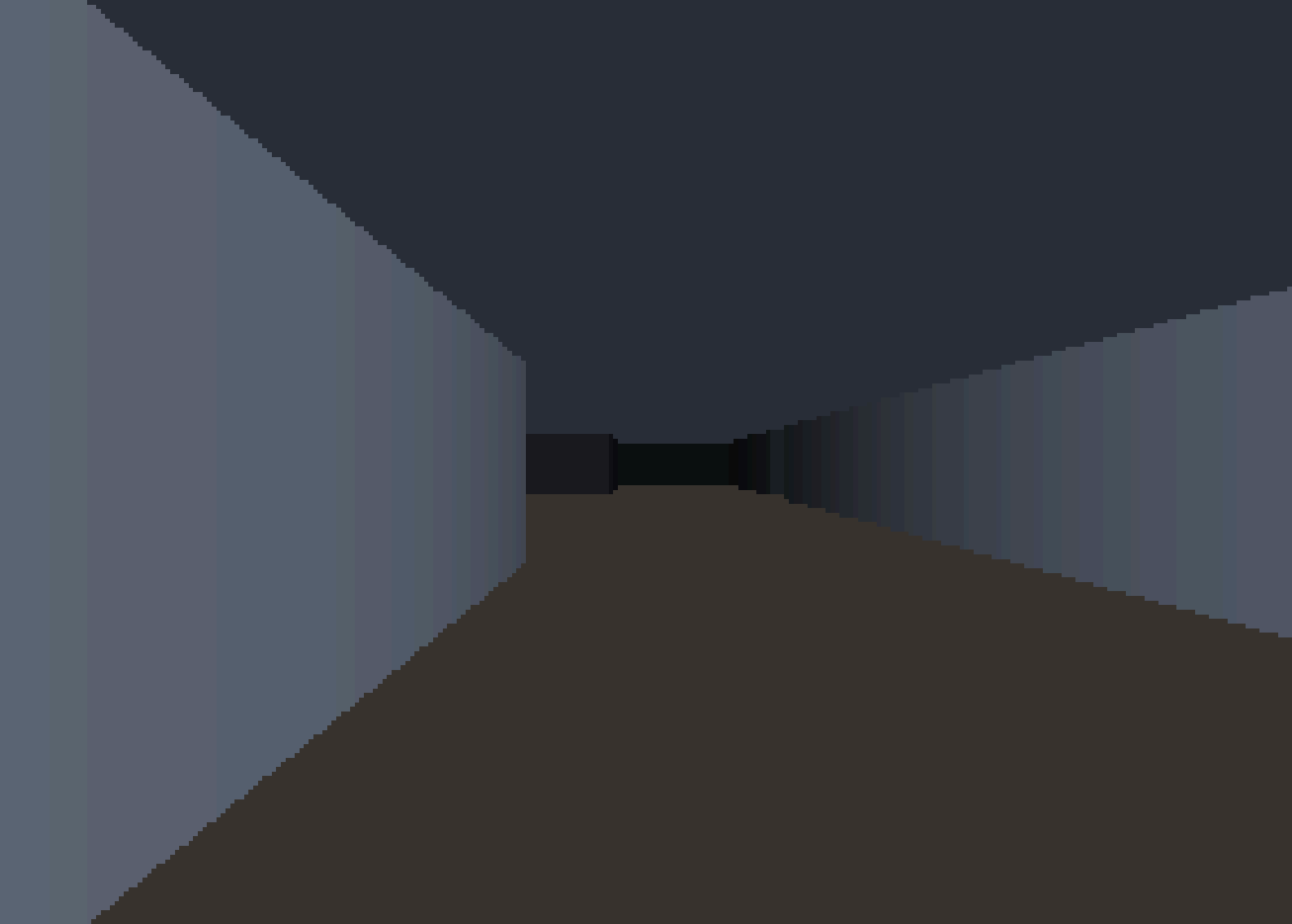


Figure 9.2. Le couloir avec deux teintes de mur et la brume. Les angles sont visibles et le fond du couloir s'assombrit.

VOCABULAIRE

La **brume** est l'assombrissement des murs avec la distance. Un **octet** est un groupe de 8 bits, qui représente un nombre de 0 à 255. Une couleur occupe trois octets.

La carte en médaillon

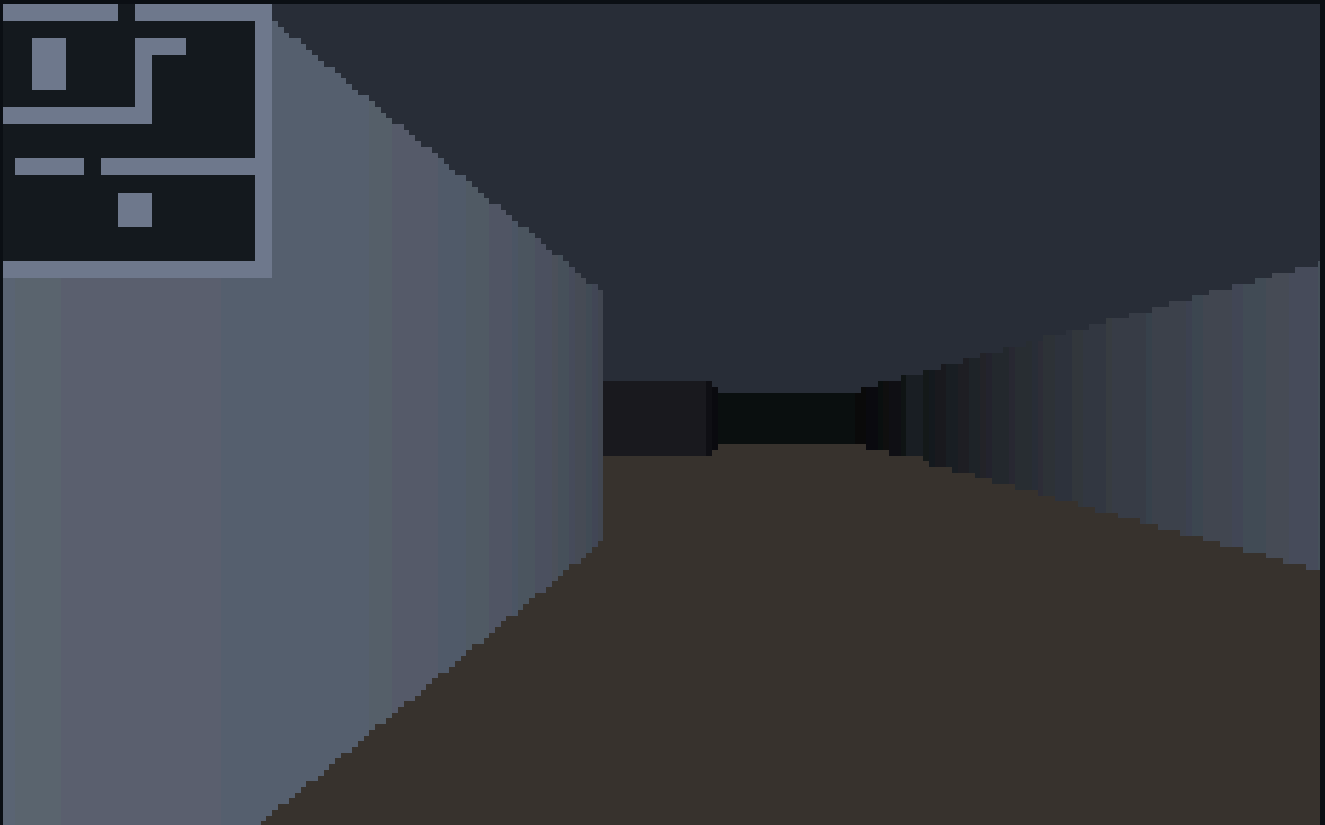


Figure 10.1. Le couloir, avec la carte en médaillon dans le coin. Le point rouge est le joueur.

Dans ce chapitre, vous allez réduire la carte à un médaillon dans le coin de l'écran.

La grille dessinée en grand au chapitre 5 a servi à vérifier le niveau et les déplacements. Maintenant que le couloir s'affiche, la carte devient un médaillon dans le coin supérieur gauche de l'écran, avec des cases de 3 pixels. La fonction `render_map` ne change pas, seuls ses arguments changent.

La carte est dessinée après les murs, pour se trouver par-dessus l'image. L'ordre des appels de dessin est celui des couches de l'image finale. Ce qui est dessiné en dernier recouvre ce qui a été dessiné avant.

EXERCICE

Déplacez la carte dans le coin inférieur droit en modifiant les constantes `MAP_LEFT` et `MAP_TOP`. La carte fait 24 cases de 3 pixels, soit 72 pixels de côté, et la vue fait 320 sur 200.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

MAIN C LIGNES 314 A 318

```
314 }  
315  
316 // the same map, now small enough to live in a corner  
317 static void render_map(const struct player *player, int cell, int left,  
    int top)  
318 {
```

MAIN C LIGNES 375 A 379

```
375     turn_player(&player, turn);  
376     render_walls(&player);  
377     render_map(&player, MAP_CELL, MAP_LEFT, MAP_TOP);  
378     screen_present(&screen);  
379
```

Ligne par ligne

377 Le même dessin qu'au chapitre 5, avec des cases de 3 pixels, à 4 pixels des bords.

Le glissement le long des murs



Figure 11.1. Le joueur avance en biais vers un mur. À gauche, le déplacement est testé en bloc et refusé, le joueur ne bouge pas. À droite, chaque axe est testé séparément. Le déplacement vertical est refusé, le déplacement horizontal a lieu, et le joueur glisse le long du mur.

Dans ce chapitre, vous allez corriger les collisions pour que le joueur glisse le long des murs au lieu de s'y arrêter.

Jusqu'ici, un déplacement qui menait dans un mur était refusé en entier. Un joueur qui avançait en biais vers un mur s'arrêtait net, alors qu'il devrait glisser le long du mur, comme dans tous les jeux du genre. La figure 11.1 compare les deux comportements.

La solution consiste à tester les deux axes séparément. Le déplacement horizontal est vérifié et appliqué seul, puis le déplacement vertical. Si l'un des deux mène dans un mur, seul celui-là est refusé, et l'autre a lieu. Le joueur glisse.

Le programme ajoute aussi une marge de 0,2 case devant le joueur. Sans elle, la position pourrait s'approcher d'un mur jusqu'à le toucher, et l'œil se retrouverait dans l'épaisseur du mur. La marge est ajoutée dans le sens du déplacement, comme un nez qui touche le mur avant les yeux.

EXERCICE

Mettez la marge à 0 et avancez contre un mur. L'image se met à clignoter, car l'œil entre dans le mur et le rayon en sort aussitôt. Remettez ensuite la valeur.

Le glissement le long des murs, suite

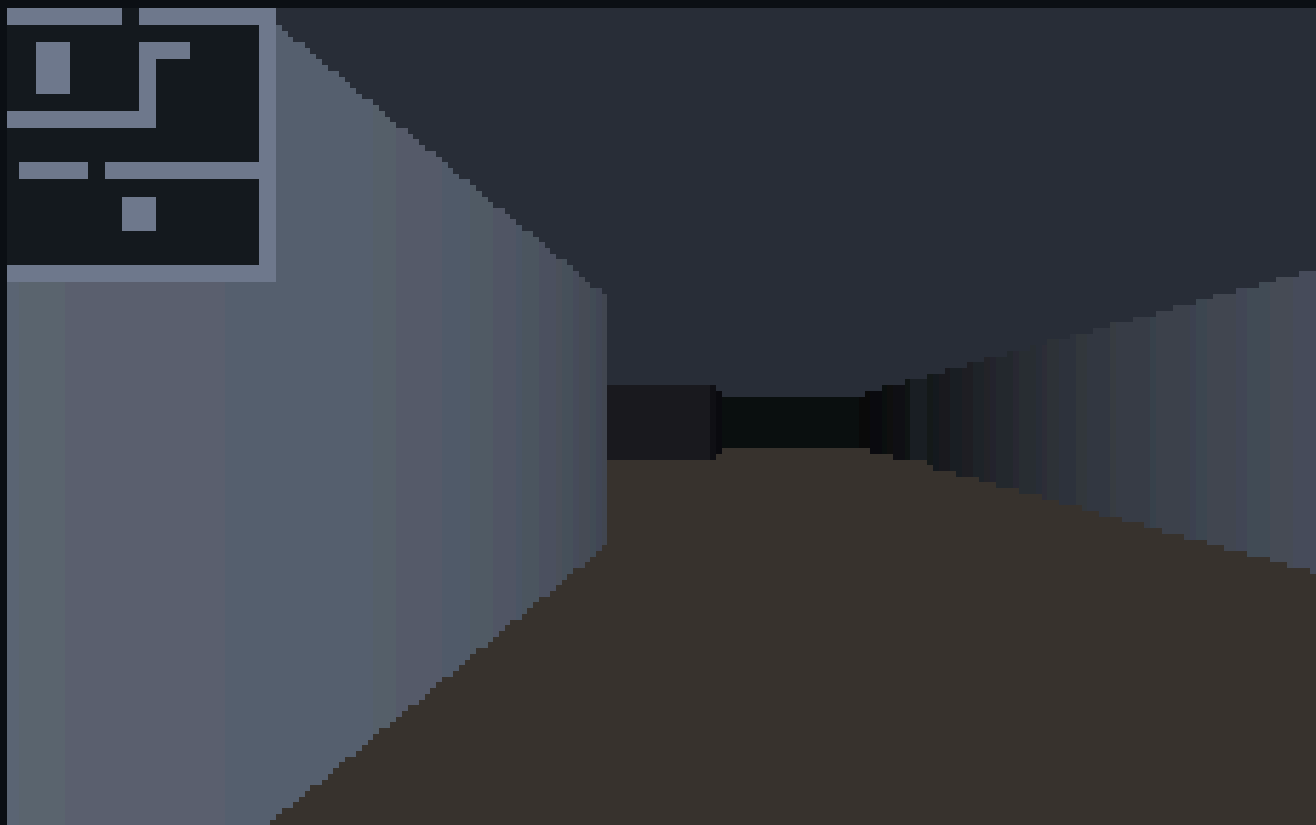


Figure 11.2. Le couloir à la fin de la première partie. Le joueur glisse le long des murs et ne peut plus coller son œil contre eux.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

MAIN C LIGNES 199 A 214

```
199 };
200
201 // each axis is tested on its own, so a shoulder against a wall keeps
    sliding
202 // instead of stopping the player dead. the margin is the nose: without
    it we
203 // walk until the eyes are inside the texture
204 static void move_player(struct player *player, double step_x, double
    step_y)
205 {
206     const double margin = 0.2;
207     double nose_x = step_x > 0 ? margin : -margin;
208     double nose_y = step_y > 0 ? margin : -margin;
209
210     if (!is_wall((int)(player->x + step_x + nose_x), (int)player->y))
211         player->x += step_x;
212     if (!is_wall((int)player->x, (int)(player->y + step_y + nose_y)))
213         player->y += step_y;
214 }
```

Ligne par ligne

- 206** La marge, en cases. Le joueur s'arrête à cette distance des murs.
- 207** La marge prend le signe du déplacement, elle est toujours devant le joueur.
- 210** Le test du seul déplacement horizontal. La coordonnée y n'a pas encore bougé.
- 212** Le test du seul déplacement vertical, avec la coordonnée x déjà mise à jour.

2

DEUXIEME PARTIE

Les textures

Dans cette deuxième partie, vous allez habiller le moteur. Les murs, le sol et le plafond recevront des textures, c'est-à-dire des images dessinées par le programme lui-même au démarrage. La fenêtre apprendra à changer de taille, et vous rangerez le code dans cinq fichiers.

10 CHAPITRES 5 FICHIERS 798 LIGNES LANGAGE C

Le plein écran

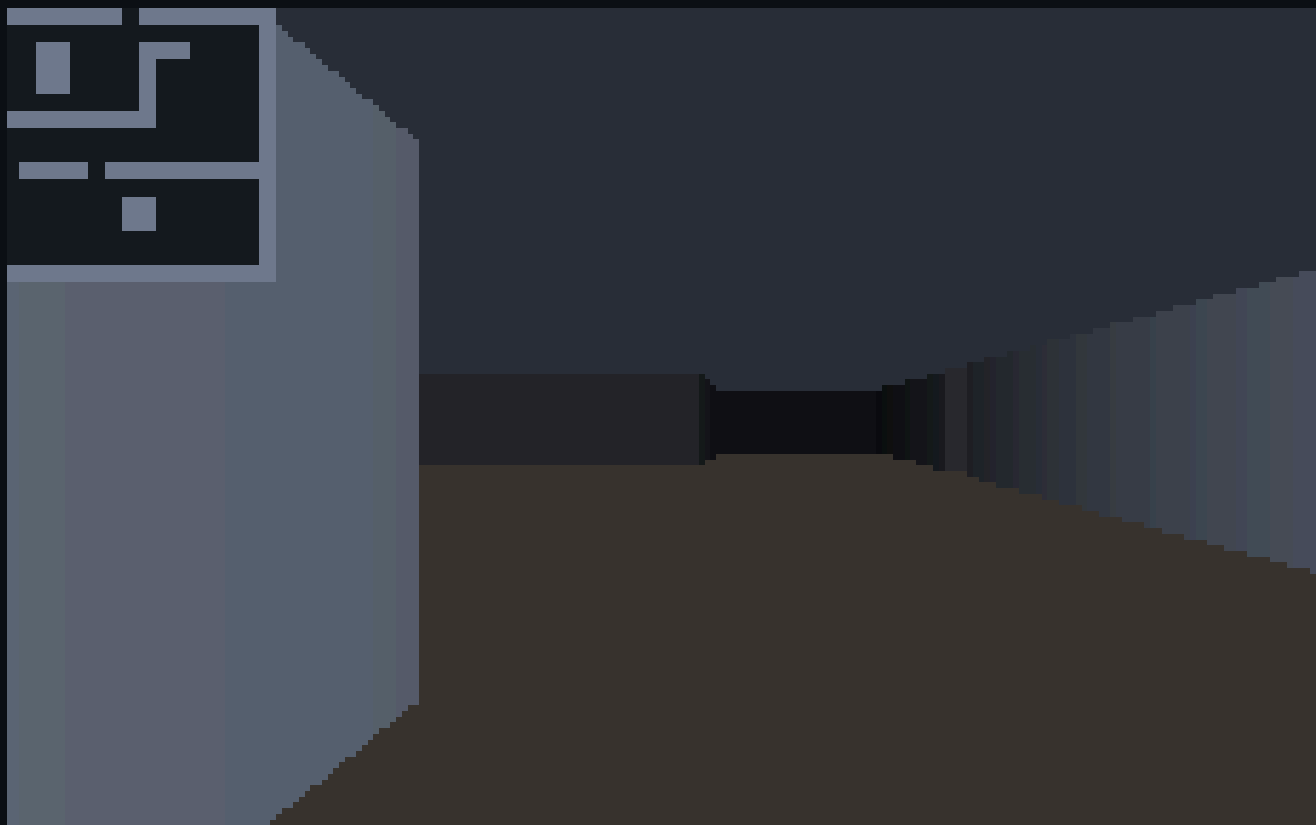


Figure 1.1. La fenêtre au démarrage, centrée sur l'écran et nommée The Keep.

Dans ce chapitre, vous allez donner un nom à la fenêtre et lui apprendre le plein écran.

La fenêtre de la première partie faisait 640 pixels sur 400, une taille fixe. Nous lui donnons un nom, The Keep, et la touche F11 pour passer en plein écran et en revenir.

Xlib ne connaît pas le plein écran. Une fenêtre en plein écran est simplement une fenêtre déplacée en haut à gauche et agrandie à la taille de l'écran. C'est ce que fait la fonction `screen_fullscreen`, avec les dimensions de l'écran obtenues par `DisplayWidth` et `DisplayHeight`. La fonction `screen_windowed` fait l'opération inverse, et place la fenêtre au centre de l'écran.

La fonction `screen_toggle_fullscreen` retient l'état courant dans le champ `full` de la structure et passe de l'un à l'autre. La touche F11 l'appelle, à l'appui seulement, sinon le relâchement de la touche ferait revenir la fenêtre à sa taille initiale.

EXERCICE

Ajoutez une seconde touche pour le plein écran, par exemple F, en ajoutant un `case XK_f` à côté de celui de F11.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

MAIN C LIGNES 45 A 49

```
45 #define MAP_PLAYER 0xd8534f
46
47 #define GAME_NAME "The Keep"
48 #define WINDOW_BG 0x182636
49
```

MAIN C LIGNES 90 A 132

```
90 Atom close_message;
91 unsigned int *pixels;
92 int full; // are we filling the screen?
93 };
94
95 // every pixel of the game lands here first
96 static unsigned int view[VIEW_WIDTH * VIEW_HEIGHT];
97
98 // the window at the size of the picture, in the middle of the screen
99 static void screen_windowed(struct screen *screen)
100 {
101     int number = DefaultScreen(screen->display);
102     int full_width = DisplayWidth(screen->display, number);
103     int full_height = DisplayHeight(screen->display, number);
104
105     XMoveResizeWindow(screen->display, screen->window,
106         (full_width - WIN_WIDTH) / 2, (full_height - WIN_HEIGHT) / 2,
107         WIN_WIDTH, WIN_HEIGHT);
108 }
109
110 // and full screen, without asking anyone: there is no xlib call for it,
111 // only
112 // a window as big as the screen
113 static void screen_fullscreen(struct screen *screen)
114 {
115     int number = DefaultScreen(screen->display);
116
117     XMoveResizeWindow(screen->display, screen->window, 0, 0,
118         DisplayWidth(screen->display, number),
119         DisplayHeight(screen->display, number));
120 }
121 static void screen_toggle_fullscreen(struct screen *screen)
122 {
123     screen->full = !screen->full;
124     if (screen->full)
125         screen_fullscreen(screen);
126     else
127         screen_windowed(screen);
128
129     XFlush(screen->display);
130 }
131
132 static int screen_open(struct screen *screen)
```

MAIN C LIGNES 148 A 152

```
148 XClassHint class_hint = { "raycaster", "Raycaster" };
149 XSetClassHint(screen->display, screen->window, &class_hint);
150 XStoreName(screen->display, screen->window, GAME_NAME);
151
152 // without this the close button cuts the connection under our feet;
```

Ligne par ligne

- 92** Le drapeau du plein écran, 1 quand la fenêtre remplit l'écran.
- 102** La largeur de l'écran, demandée au serveur graphique.
- 105** Une seule fonction déplace et redimensionne la fenêtre. Elle est appelée avec le centre de l'écran en mode fenêtre et avec le coin en haut à gauche en plein écran.
- 123** L'état bascule. Le point d'exclamation inverse une valeur logique.

PARTIE 2 CHAPITRE 1 LE CODE

SUITE

MAIN C LIGNES 162 A 169

```
162     XSelectInput(screen->display, screen->window, KeyPressMask |
      KeyReleaseMask);
163
164     screen_windowed(screen);
165     XMapWindow(screen->display, screen->window);
166     screen->gc = XCreateGC(screen->display, screen->window, 0, NULL);
167     screen->full = 0;
168
169     // XCreateImage copies nothing: we keep writing into our own buffer
      and
```

209 La touche F11. Le test if (down) ne garde que l'appui.

MAIN C LIGNES 207 A 214

```
207         case XK_a:      keys->strafe_left = down; break;
208         case XK_d:      keys->strafe_right = down; break;
209         case XK_F11:
210             if (down)
211                 screen_toggle_fullscreen(screen);
212             break;
213     }
214 }
```

L'image suit la fenêtre



Figure 2.1. La même image de 320 sur 200 dans deux fenêtres. Le programme choisit le plus grand agrandissement entier qui tient, 2 puis 2, et centre l'image. Le reste de la fenêtre garde la couleur de fond.

Dans ce chapitre, vous allez faire suivre la taille de la fenêtre à l'image, avec un agrandissement entier et un centrage.

En plein écran, l'image de 640 pixels sur 400 restait dans un coin de la fenêtre. Nous faisons suivre la fenêtre à l'image, quelle que soit sa taille. L'image du jeu garde ses 320 pixels sur 200, et le programme choisit le plus grand agrandissement entier qui tient dans la fenêtre, puis centre l'image. Un agrandissement entier est indispensable. Avec un facteur comme 2,5, certains pixels de l'image seraient dessinés sur 2 pixels et d'autres sur 3, et les murs scintilleraient à chaque mouvement.

La fonction `screen_fit` calcule ce facteur, ainsi que la position du coin de l'image dans la fenêtre. La fonction `screen_resize` alloue un nouveau tableau de pixels à la taille de la fenêtre et une nouvelle `XImage`, après avoir libéré les anciens. Le serveur prévient le programme de chaque changement de taille par un événement `ConfigureNotify`, que le programme reçoit parce qu'il s'y est abonné avec `StructureNotifyMask`.

La fonction `screen_present` tient compte du facteur et du coin. Elle commence par remplir la fenêtre entière de la couleur de fond, car l'image ne couvre plus forcément toute la fenêtre. Elle vérifie aussi que chaque ligne et chaque colonne restent dans la fenêtre, pour le cas où la fenêtre est plus petite que l'image. Enfin, le programme indique au gestionnaire de fenêtres une taille minimale, celle de l'image, en dessous de laquelle il n'y aurait plus rien à montrer.

VOCABULAIRE

Un événement `ConfigureNotify` est le message envoyé par le serveur graphique quand la fenêtre change de taille ou de position.

EXERCICE

Lancez le jeu, puis agrandissez la fenêtre à la souris. L'image saute d'un agrandissement au suivant, 2, 3, puis 4, et reste toujours centrée.

L'image suit la fenêtre, suite

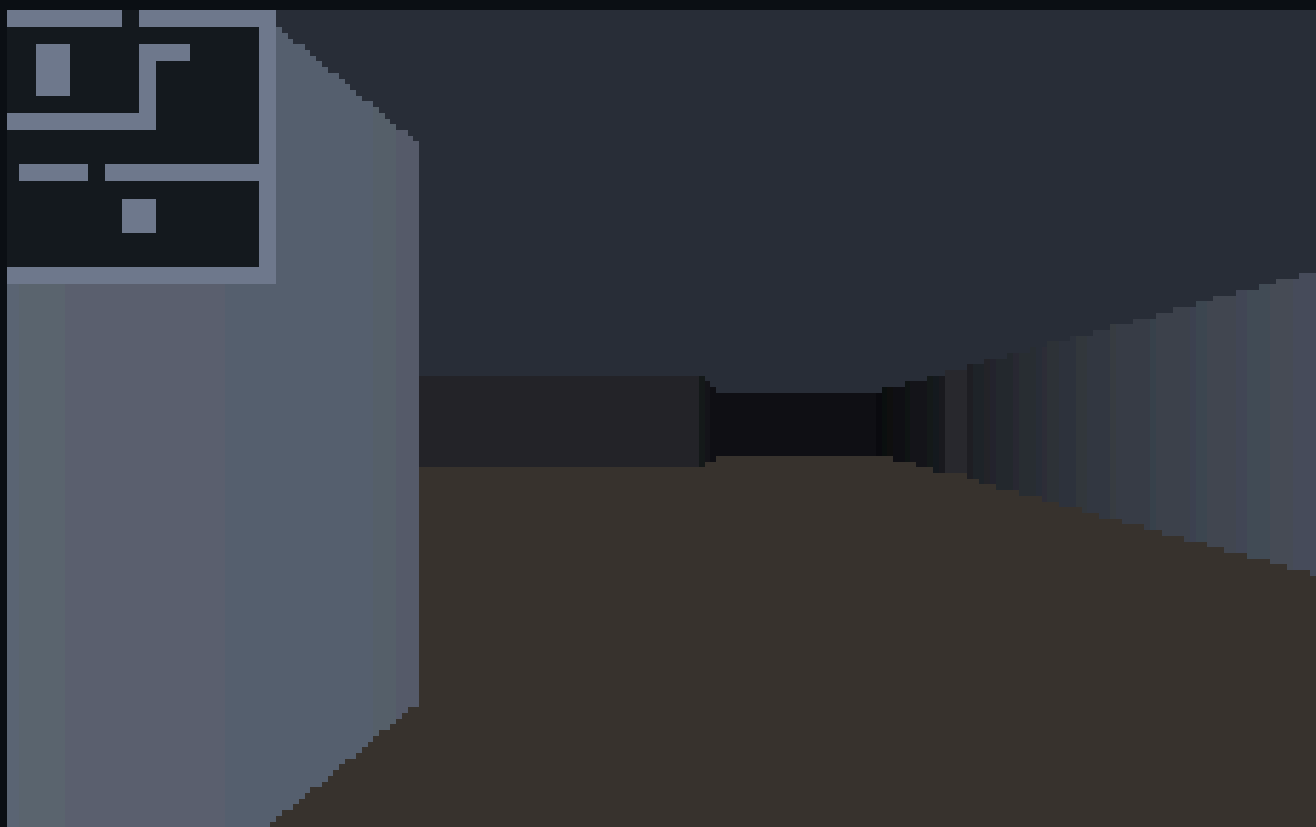


Figure 2.2. L'image centrée dans la fenêtre, avec la couleur de fond autour.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

MAIN C LIGNES 90 A 98

```
90     Atom      close_message;
91     unsigned int *pixels;
92     // the window is the player's to resize; the picture keeps its own
size
93     Visual *visual;
94     int depth;
95     int width, height;
96     int scale, left, top;
97     int full;                                // are we filling the screen?
98 };
```

MAIN C LIGNES 135 A 182

```
135 }
136
137 // how big a block each view pixel becomes, and where the picture sits in
the
138 // window. a whole number of pixels per pixel, or the walls shimmer
139 static void screen_fit(struct screen *screen)
140 {
141     int by_width = screen->width / VIEW_WIDTH;
142     int by_height = screen->height / VIEW_HEIGHT;
143
144     screen->scale = by_width < by_height ? by_width : by_height;
145     if (screen->scale < 1)
146         screen->scale = 1;
147     // a window smaller than the picture would give a negative corner,
and we
148     // would copy pixels in front of the buffer
149     screen->left = (screen->width - VIEW_WIDTH * screen->scale) / 2;
150     if (screen->left < 0)
151         screen->left = 0;
152     screen->top = (screen->height - VIEW_HEIGHT * screen->scale) / 2;
153     if (screen->top < 0)
154         screen->top = 0;
155 }
156
157 // a new window size means a new buffer and a new image. XDestroyImage
frees
158 // the pixels it was handed, so the old buffer goes with the old image
159 static int screen_resize(struct screen *screen, int width, int height)
160 {
161     unsigned int *pixels = malloc((size_t)width * height *
sizeof(unsigned int));
162     if (!pixels) {
163         fprintf(stderr, "out of memory\n");
164         return 0;
165     }
166
167     if (screen->image)
168         XDestroyImage(screen->image);
169
170     screen->pixels = pixels;
171     screen->width = width;
172     screen->height = height;
173     // XCreateImage copies nothing: we keep writing into our own buffer
and
174     // the server reads it from there
```

Ligne par ligne

- 96** Le facteur d'agrandissement et la position du coin de l'image dans la fenêtre, recalculés à chaque changement de taille.
- 144** Le plus petit des deux facteurs possibles, en largeur et en hauteur, pour que l'image tienne dans les deux sens.
- 149** La marge de gauche, la moitié de la place restante.
- 168** L'ancienne image est libérée. Cette fonction libère aussi le tableau de pixels qui lui avait été confié.

PARTIE 2 CHAPITRE 2 LE CODE

SUITE

```
175     screen->image = XCreateImage(screen->display, screen->visual, screen->depth,
176     ZPixmap, 0, (char *)pixels, width, height, 32, 0);
177     screen_fit(screen);
178     return screen->image != NULL;
179 }
180
181 static int screen_open(struct screen *screen)
182 {
```

203 La taille minimale de la fenêtre, indiquée au gestionnaire de fenêtres.

220 L'abonnement aux changements de taille, en plus des touches.

253 Le message de changement de taille. Le programme ne réalloue que si la taille a réellement changé.

285 La fenêtre entière reçoit la couleur de fond avant que l'image soit copiée au centre.

MAIN C LIGNES 199 A 208

```
199     XStoreName(screen->display, screen->window, GAME_NAME);
200
201     // the window may grow as much as it likes, but never below the
picture:
202     // under that there is nothing left to show
203     XSizeHints hints = { .flags = PMinSize,
204     .min_width = VIEW_WIDTH, .min_height = VIEW_HEIGHT };
205     XSetWMNormalHints(screen->display, screen->window, &hints);
206
207     // without this the close button cuts the connection under our feet;
208     // with it we get a message and we decide what to do
```

MAIN C LIGNES 215 A 233

```
215     XkbSetDetectableAutoRepeat(screen->display, True, &supported);
216
217     // StructureNotifyMask is what tells us the window changed size.
without
218     // it the picture stays in the corner when the window grows
219     XSelectInput(screen->display, screen->window,
220     KeyPressMask | KeyReleaseMask | StructureNotifyMask);
221
222     screen_windowed(screen);
223     XMapWindow(screen->display, screen->window);
224     screen->gc = XCreateGC(screen->display, screen->window, 0, NULL);
225
226     screen->visual = visual;
227     screen->depth = depth;
228     screen->image = NULL;
229     screen->full = 0;
230
231     if (!screen_resize(screen, WIN_WIDTH, WIN_HEIGHT))
232         return 0;
233
```

MAIN C LIGNES 249 A 259

```
249     (Atom)event.xclient.data.l[0] == screen->close_message)
250     keys->quit = 1;
251
252     // the server tells us the new size; the picture follows it
253     if (event.type == ConfigureNotify &&
254     (event.xconfigure.width != screen->width ||
255     event.xconfigure.height != screen->height))
256         screen_resize(screen, event.xconfigure.width,
257         event.xconfigure.height);
258
259     if (event.type == KeyPress || event.type == KeyRelease) {
```

MAIN C LIGNES 281 A 311

```
281 static void screen_present(struct screen *screen)
282 {
283     // whatever the picture does not cover stays the window colour
284     for (int i = 0; i < screen->width * screen->height; i++)
285         screen->pixels[i] = WINDOW_BG;
286
287     int scale = screen->scale;
288     // a window narrower than the picture: we draw the columns that fit,
and
```

PARTIE 2 CHAPITRE 2 LE CODE

SUITE

```
289 // this does not change from one row to the next
290 int visible = (screen->width - screen->left) / scale;
291 if (visible > VIEW_WIDTH)
292     visible = VIEW_WIDTH;
293 for (int y = 0; y < VIEW_HEIGHT; y++) {
294     unsigned int *row = view + y * VIEW_WIDTH;
295     for (int copy = 0; copy < scale; copy++) {
296         int line = screen->top + y * scale + copy;
297         if (line >= screen->height)
298             continue;
299         unsigned int *out = screen->pixels + line * screen->width
300             + screen->left;
301         for (int x = 0; x < visible; x++) {
302             unsigned int color = row[x];
303             for (int again = 0; again < scale; again++)
304                 *out++ = color;
305         }
306     }
307 }
308 XPutImage(screen->display, screen->window, screen->gc, screen->image,
309     0, 0, 0, 0, screen->width, screen->height);
310 XFlush(screen->display);
311 }
```

290 Le nombre de colonnes de l'image qui tiennent dans la fenêtre.

MAIN C LIGNES 510 A 517

```
510
511 }
512
513 // XDestroyImage frees the buffer it was given, so this is the whole
514 // clean-up: the picture, then the connection
515 XDestroyImage(screen.image);
516 XCloseDisplay(screen.display);
517 return 0;
```

Une texture dessinée par le programme

LA TEXTURE DES MURS 64 X 64

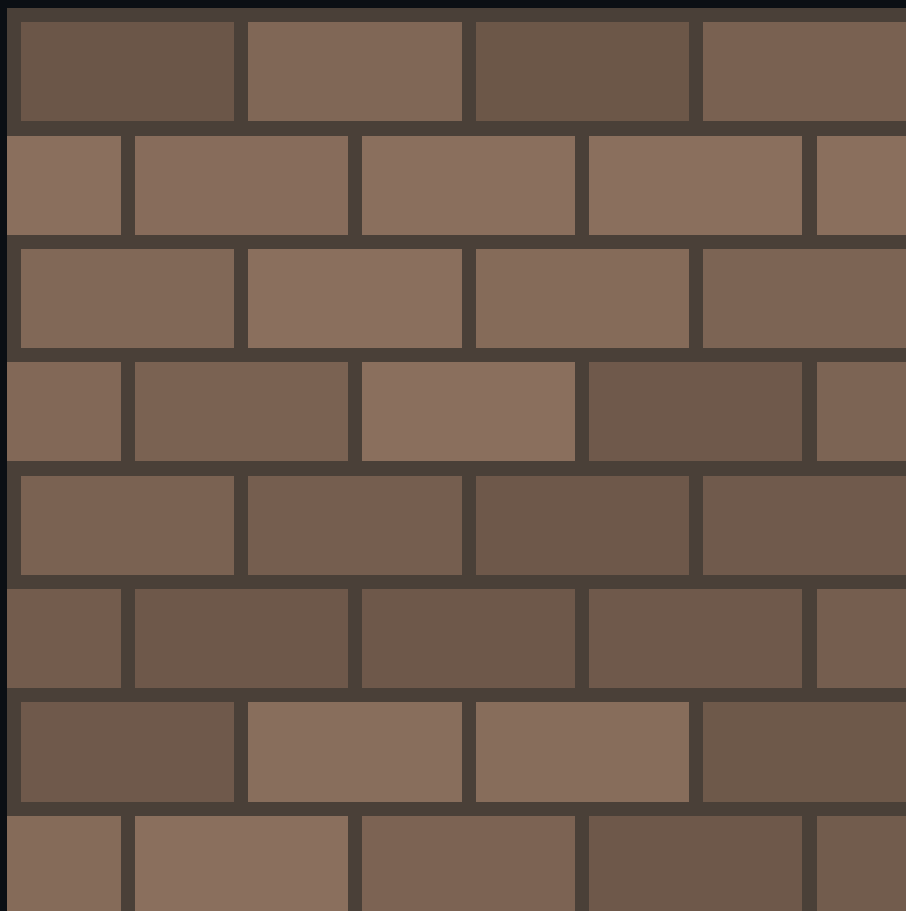


Figure 3.1. La texture des briques, calculée par la fonction `make_wall_texture` et agrandie quatre fois. Les rangées sont décalées d'une demi-brique et chaque brique a sa teinte.

Dans ce chapitre, vous allez dessiner votre première texture par le code, un mur de briques, et apprendre à produire des nombres qui semblent tirés au hasard.

Une texture est une image carrée que le moteur plaque sur les murs. Dans *The Keep*, aucune image n'est chargée depuis un fichier. Les textures sont dessinées par le programme au démarrage, pixel par pixel, et rangées dans un tableau de 64 pixels sur 64. Une texture de 64 pixels suffit à l'image de 320 sur 200, et sa taille est une puissance de deux, ce qui permet de revenir au début de la texture par une simple opération sur les bits.

La texture des murs représente des briques. Chaque rangée fait 8 pixels de haut et chaque brique 16 pixels de large. Une rangée sur deux est décalée d'une demi-brique, comme dans un vrai mur. Les joints, c'est-à-dire la première ligne de chaque rangée et la première colonne de chaque brique, reçoivent une couleur sombre. Chaque brique reçoit une teinte légèrement différente de ses voisines, ce qui évite un mur plat et régulier.

Cette variation vient de la fonction `noise`. Elle transforme deux nombres entiers, ici le numéro de la brique et celui de la rangée, en un nombre compris entre 0 et 1 qui semble tiré au hasard. Le calcul est toujours le même pour les mêmes entrées, et une brique garde donc sa teinte d'une image à l'autre. Pour vérifier le dessin, la fonction `show_texture` affiche la texture dans le coin de l'écran, en attendant que les murs la portent.

Une texture dessinée par le programme, suite

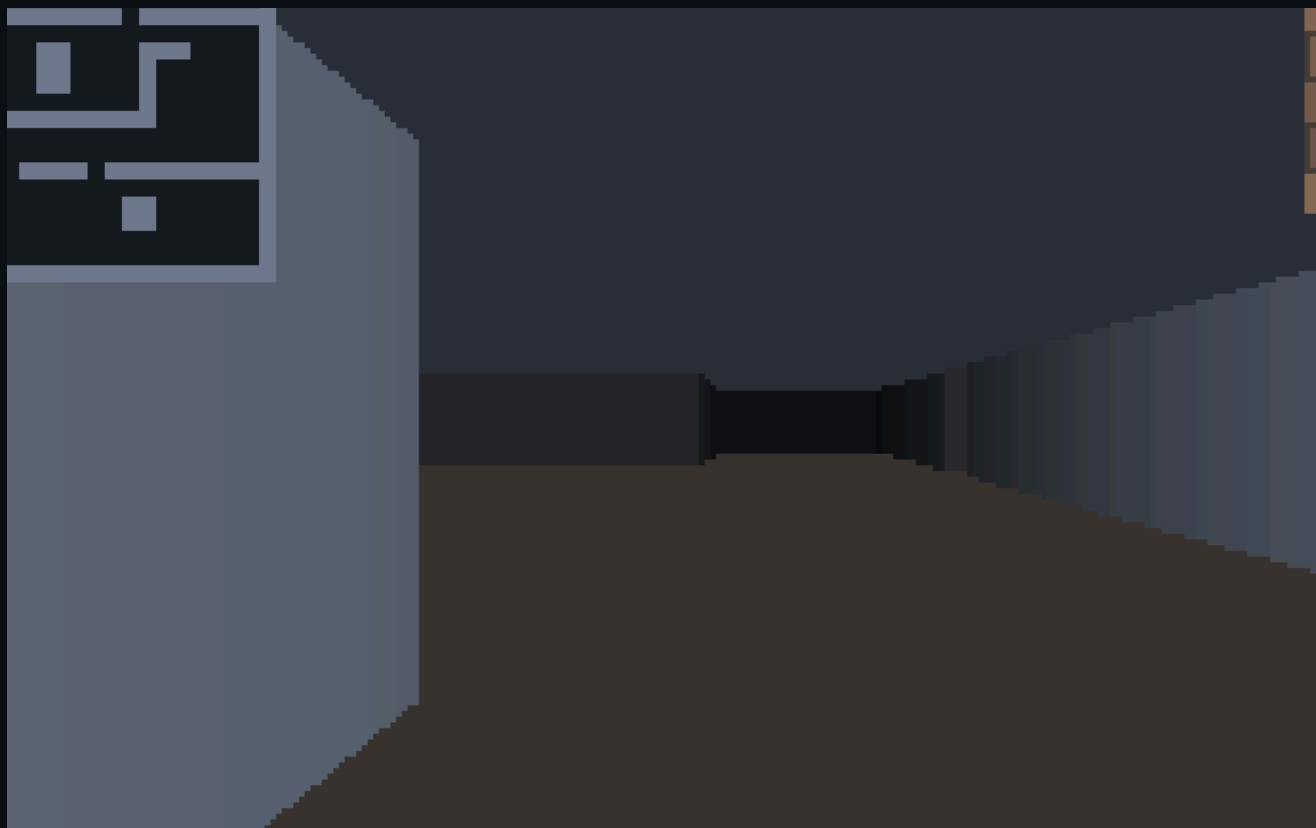


Figure 3.2. La texture des briques affichée dans le coin en haut à droite de l'écran, au-dessus du couloir encore sans texture.

VOCABULAIRE

Une **texture** est une image plaquée sur une surface du monde, ici un carré de 64 pixels de côté. Une **puissance de deux** est un nombre comme 2, 4, 8, 16, 32 ou 64. Un tableau de cette taille se parcourt en boucle avec un masque de bits au lieu d'une division.

EXERCICE

Changez **BRICK_WIDTH** en 32 et **BRICK_HEIGHT** en 16. Les briques deviennent deux fois plus grandes. Puis essayez une valeur de **BRICK_VARY** de 0 pour voir un mur sans variation de teinte.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

MAIN C LIGNES 17 A 21

```
17 #define VIEW_WIDTH  320
18 #define VIEW_HEIGHT 200
19 #define SCALE        4
20 #define WIN_WIDTH    (VIEW_WIDTH * SCALE)
21 #define WIN_HEIGHT   (VIEW_HEIGHT * SCALE)
```

MAIN C LIGNES 47 A 64

```
47 #define GAME_NAME    "The Keep"
48 #define WINDOW_BG    0x182636
49
50 // a texture is a square of pixels drawn once at startup. a power of two,
   so
51 // wrapping around it is a bitwise and instead of a division
52 #define TEX_SIZE      64
53 #define TEX_MASK      (TEX_SIZE - 1)
54
55 // a brick wall: the size of one brick, its two colours, and how far one
   brick
56 // is allowed to differ from the next so that the wall is not flat
57 #define BRICK_WIDTH   16
58 #define BRICK_HEIGHT  8
59 #define BRICK_COLOR    0x8a6f5d
60 #define BRICK_JOINT   0x4a4038
61 #define BRICK_LIGHT    0.78
62 #define BRICK_VARY     0.30
63
64 // the whole world: a wall is anything that is not a dot
```

MAIN C LIGNES 114 A 119

```
114 // every pixel of the game lands here first
115 static unsigned int view[VIEW_WIDTH * VIEW_HEIGHT];
116
117 static unsigned int wall_texture[TEX_SIZE * TEX_SIZE];
118
119 // the window at the size of the picture, in the middle of the screen
```

MAIN C LIGNES 362 A 375

```
362 }
363
364 // the same square always gets the same number between 0 and 1: a brick
   keeps
365 // its shade from one frame to the next, and the wall does not crawl. the
   three
366 // constants are arbitrary large odd numbers, any others would do as well
367 static double noise(int x, int y)
368 {
369     unsigned int n = (unsigned int)(x * 374761393 + y * 668265263);
370     n = (n ^ (n >> 13)) * 1274126177u;
371     return (double)((n >> 16) & 0xffff) / 65535.0;
372 }
373
374 static unsigned int shade(unsigned int color, double light)
375 {
```

MAIN C LIGNES 398 A 420

```
398     for (int y = top; y < bottom; y++)
399         view[y * VIEW_WIDTH + x] = color;
400 }
```

Ligne par ligne

- 53** Le masque, 63. Un et bit à bit avec 63 ramène n'importe quel nombre entre 0 et 63, comme un reste de division par 64, mais beaucoup plus vite.
- 367** Le générateur de nombres qui semblent tirés au hasard. Les trois grands nombres impairs mélangent les bits des deux entrées.
- 371** Seize bits du résultat, ramenés entre 0 et 1.

PARTIE 2 CHAPITRE 3 LE CODE

SUITE

```
401
402 // bricks: rows half a brick apart, a mortar line between them, and each
403 // brick a shade of its own. drawn once, read a million times
404 static void make_wall_texture(void)
405 {
406     for (int y = 0; y < TEX_SIZE; y++) {
407         for (int x = 0; x < TEX_SIZE; x++) {
408             int row = y / BRICK_HEIGHT;
409             int shift = (row & 1) ? BRICK_WIDTH / 2 : 0;
410             int column = (x + shift) / BRICK_WIDTH;
411             int joint = y % BRICK_HEIGHT == 0 ||
412                 (x + shift) % BRICK_WIDTH == 0;
413             unsigned int color = joint ? BRICK_JOINT
414                 : shade(BRICK_COLOR,
415                     BRICK_LIGHT + BRICK_VARY * noise(column, row));
416             wall_texture[y * TEX_SIZE + x] = color;
417         }
418     }
419 }
420
```

409 Les rangées impaires sont décalées d'une demi-brique.

411 Un joint est la première ligne d'une rangée ou la première colonne d'une brique.

415 La teinte de la brique, une base de 0,78 plus jusqu'à 0,30 selon la brique.

532 La texture est dessinée une seule fois, avant la boucle principale.

MAIN C LIGNES 484 A 496

```
484 }
485
486 // a look at what we just made, until the walls can wear it
487 static void show_texture(const unsigned int *texture)
488 {
489     for (int y = 0; y < TEX_SIZE; y++)
490         for (int x = 0; x < TEX_SIZE; x++)
491             view[y * VIEW_WIDTH + VIEW_WIDTH - TEX_SIZE + x] =
492                 texture[y * TEX_SIZE + x];
493 }
494
495 // the same map, now small enough to live in a corner
496 static void render_map(const struct player *player, int cell, int left,
497     int top)
```

MAIN C LIGNES 529 A 535

```
529     return 1;
530
531     // the textures cost nothing to keep and everything to draw: once,
532     here
533     make_wall_texture();
534
535     struct player player = {
536         .x = 2.5, .y = 6.5,
```

MAIN C LIGNES 557 A 561

```
557     turn_player(&player, turn);
558     render_walls(&player);
559     show_texture(wall_texture);
560     render_map(&player, MAP_CELL, MAP_LEFT, MAP_TOP);
561     screen_present(&screen);
```

Les murs habillés

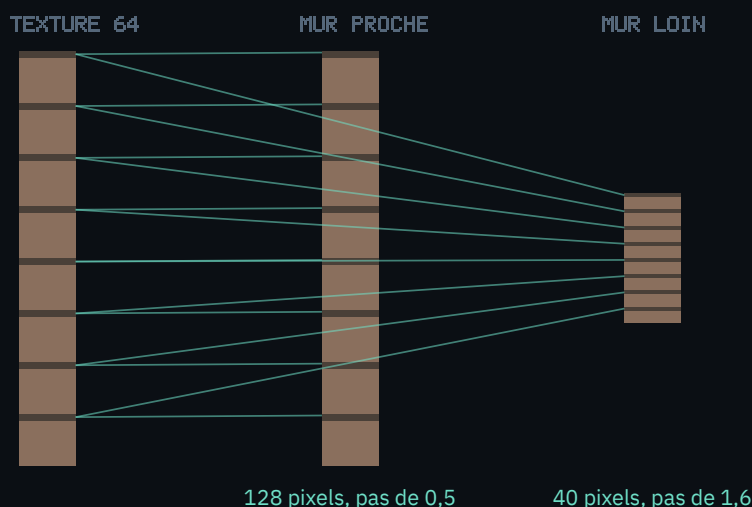


Figure 4.1. Une colonne de la texture, à gauche, dessinée dans deux bandes de l'écran. Pour un mur proche, la bande de 128 pixels avance d'un demi-pixel de texture par pixel d'écran. Pour un mur lointain, la bande de 40 pixels avance de 1,6 pixel de texture par pixel d'écran.

Dans ce chapitre, vous allez plaquer la texture sur les murs, colonne par colonne, et calculer où le rayon a touché le mur.

Nous plaquons maintenant la texture sur les murs. Pour chaque colonne de l'écran, le programme doit répondre à deux questions. Quelle colonne de la texture faut-il dessiner, et comment répartir ses 64 pixels sur la hauteur de la bande.

La première question dépend de l'endroit où le rayon a touché le mur. Le programme calcule la position du point d'impact le long de la face touchée, un nombre entre 0 et 1 appelé `wall_x`, puis le multiplie par 64. La page suivante détaille ce calcul. La seconde question est réglée par la fonction `draw_wall_column`. Elle parcourt la bande de haut en bas et avance dans la texture d'un pas de 64 divisé par la hauteur de la bande à chaque pixel. Un mur proche, dont la bande est haute, avance lentement dans la texture et chaque pixel de la texture s'étale sur plusieurs pixels de l'écran. Un mur lointain avance vite et saute des pixels de la texture.

Le programme assombrit aussi les faces touchées sur une ligne horizontale, en multipliant leur lumière par 0,68. Les deux teintes de mur de la première partie sont ainsi conservées avec la texture.

La position du rayon sur le mur

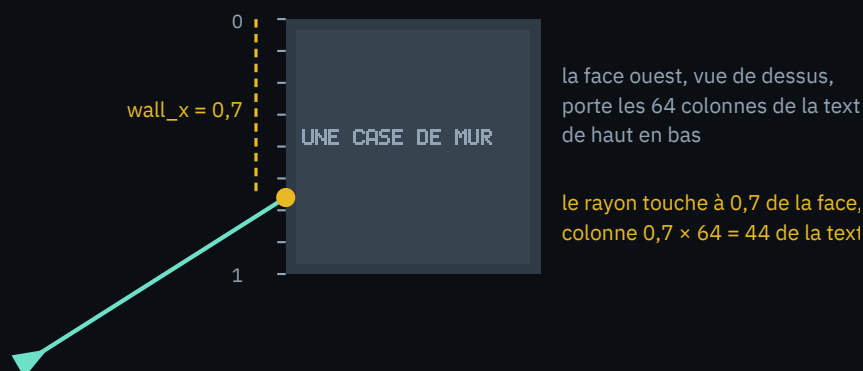


Figure 4.2. Une case de mur vue de dessus. Le rayon touche la face ouest à 0,7 de sa longueur, comptée depuis le coin du haut. La colonne de texture à dessiner est la colonne 44.

Pour savoir quelle colonne de la texture dessiner, le programme doit savoir où le rayon a touché le mur, le long de la face touchée. Cette position s'appelle `wall_x`. Elle vaut 0 à une extrémité de la face et 1 à l'autre, et la colonne de texture est `wall_x` multiplié par 64.

Le calcul dépend de la ligne franchie en dernier. Si le rayon a franchi une ligne verticale de la grille, il a touché une face orientée nord-sud, et sa position le long de cette face est donnée par la coordonnée y du point d'impact. Cette coordonnée vaut celle du joueur plus la distance multipliée par la composante y du rayon. Si le rayon a franchi une ligne horizontale, c'est la coordonnée x qui compte.

Le résultat est un nombre comme 6,7. Seule la partie après la virgule compte, car elle indique la position dans la case. La fonction `floor` donne la partie entière, 6, et la soustraction laisse 0,7, le cas de la figure 4.2.

EXERCICE

Le joueur est en (2,5 ; 6,5) et son rayon (1 ; -0,33) touche un mur à la distance 4 sur une ligne verticale. Vous devez trouver $6,5 - 1,32 = 5,18$, donc `wall_x = 0,18`, et la colonne 11.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

MAIN C LIGNES 61 A 67

```
61 #define BRICK_LIGHT 0.78
62 #define BRICK_VARY 0.30
63
64 // a wall met on a north-south line keeps this much of its light
65 #define SIDE_LIGHT 0.68
66
67 // the whole world: a wall is anything that is not a dot
```

MAIN C LIGNES 403 A 428

```
403 }
404
405 // one column of the wall, read down one column of the texture
406 static void draw_wall_column(int x, int top, int height, int tex_x,
407     double light, int dark)
408 {
409     double step = (double)TEX_SIZE / height;
410     double tex_y = 0.0;
411     int y = top;
412
413     if (y < 0)
414         y = 0;
415
416     int bottom = top + height;
417     if (bottom > VIEW_HEIGHT)
418         bottom = VIEW_HEIGHT;
419
420     for (; y < bottom; y++) {
421         unsigned int color = wall_texture[((int)tex_y & TEX_MASK) *
TEX_SIZE + tex_x];
422         view[y * VIEW_WIDTH + x] = shade(color, dark ? light * SIDE_LIGHT
: light);
423         tex_y += step;
424     }
425 }
426
427 // bricks: rows half a brick apart, a mortar line between them, and each
428 // brick a shade of its own. drawn once, read a million times
```

MAIN C LIGNES 500 A 513

```
500
501     draw_column(x, 0, top, CEILING_COLOR);
502     // where along the wall the ray landed, between 0 and 1. that is
503     // the column of the texture to read
504     double wall_x = side == 0 ? player->y + distance * ray_y
505         : player->x + distance * ray_x;
506     wall_x -= floor(wall_x);
507     int tex_x = (int)(wall_x * TEX_SIZE);
508
509     // the two orientations must not share a shade, or every corner
510     // disappears
511     draw_wall_column(x, top, height, tex_x, fog(distance), side ==
1);
512     draw_column(x, top + height, VIEW_HEIGHT, FLOOR_COLOR);
513 }
```

Ligne par ligne

- 65** La part de lumière conservée par les faces touchées sur une ligne horizontale.
- 409** Le pas dans la texture pour un pixel de l'écran, 64 divisé par la hauteur de la bande.
- 421** Le pixel de la texture, à la ligne `tex_y` et à la colonne `tex_x`. Le masque ramène la ligne entre 0 et 63.
- 504** La position du point d'impact le long de la face, selon le type de ligne franchie.
- 506** Seule la partie après la virgule est conservée.
- 507** La colonne de texture à dessiner, entre 0 et 63.

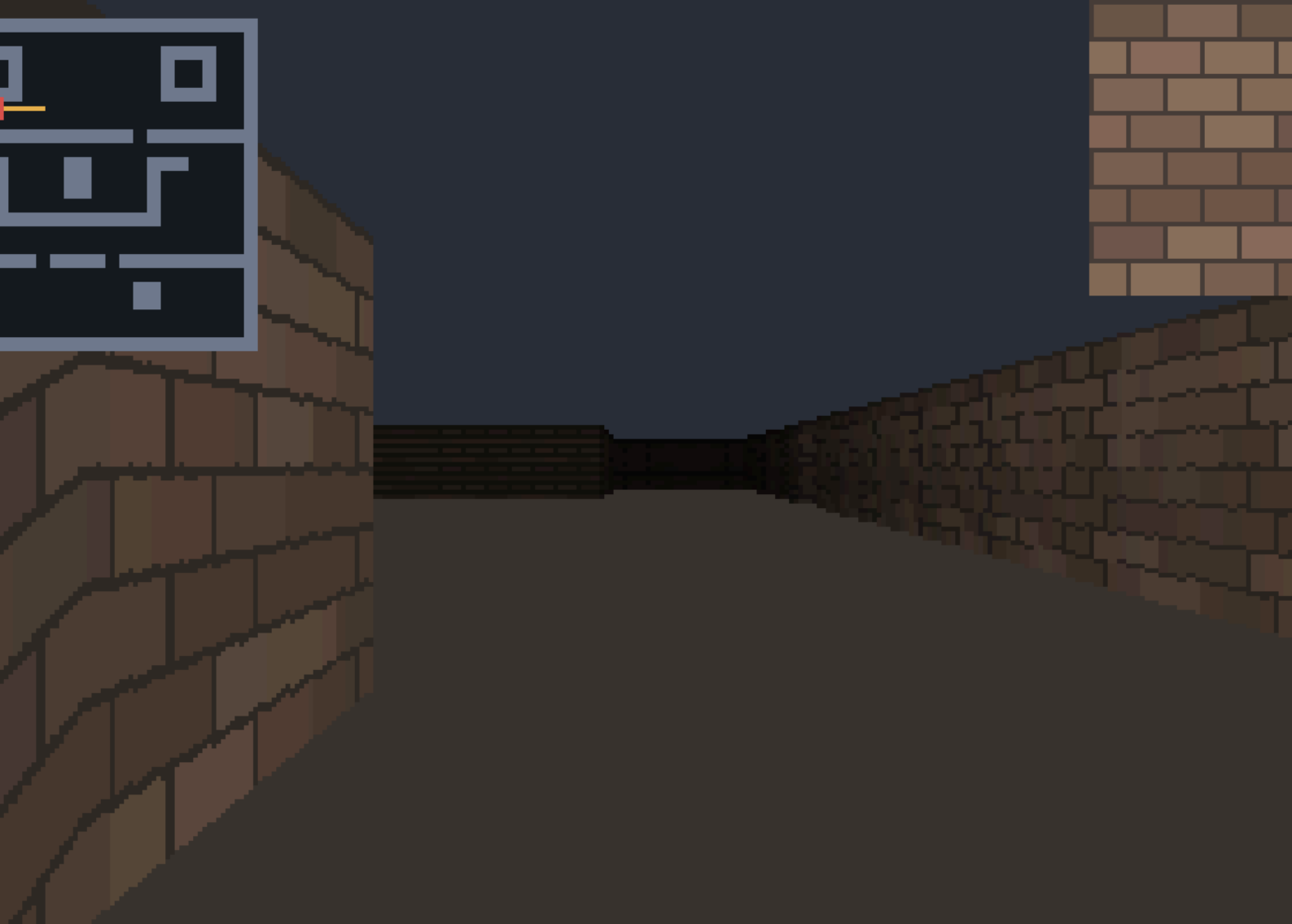


Figure 4.2. Le couloir avec les murs de briques. La texture suit les murs et s'assombrit avec la distance.

VOCABULAIRE

Le plaquage de texture est l'opération qui fait correspondre chaque pixel d'une surface à l'écran à un pixel de la texture.

La lecture au bon endroit



Figure 5.1. Le couloir avec les deux corrections. Les briques ne glissent plus quand le joueur approche d'un mur.

Dans ce chapitre, vous allez corriger deux défauts du plaquage, les briques qui glissent devant un mur proche et les faces dessinées en miroir.

Deux défauts restent visibles après le chapitre précédent. Le premier apparaît devant un mur très proche, dont la bande dépasse l'écran en haut et en bas. Le programme commençait à lire la texture en haut de l'écran, comme si c'était le haut de la bande, et les briques glissaient sous les pieds du joueur à mesure qu'il avançait. La correction consiste à calculer la ligne de texture qui correspond réellement au haut de l'écran. Si le haut de la bande est à 30 pixels au-dessus de l'écran, la lecture commence 30 pas plus loin dans la texture.

Le second défaut concerne les deux faces d'un même mur. Vues de deux côtés opposés, elles étaient dessinées en miroir l'une de l'autre, car la position `wall_x` est comptée dans le même sens pour les deux. Le programme retourne donc la colonne de texture pour les faces regardées depuis l'est ou depuis le sud. Le sens de lecture est alors le même partout, et un joueur qui contourne un pilier voit ses briques se suivre.

EXERCICE

Retirez la première correction, en remettant `y = 0` sans le calcul de `tex_y`, et avancez vers un mur. Observez les briques qui glissent, puis remettez la correction.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

MAIN C LIGNES 411 A 421

```
411     int y = top;
412
413     // the wall can be taller than the screen: we start reading in the
414     // middle of the texture rather than clamping, or the bricks slide
415     // under our feet as we walk towards them
416     if (y < 0) {
417         tex_y = -top * step;
418         y = 0;
419     }
420
421     int bottom = top + height;
```

MAIN C LIGNES 511 A 518

```
511     wall_x -= floor(wall_x);
512     int tex_x = (int)(wall_x * TEX_SIZE);
513     // the two faces we can see of the same wall must not be mirror
514     // images of each other
515     if ((side == 0 && ray_x > 0) || (side == 1 && ray_y < 0))
516         tex_x = TEX_SIZE - 1 - tex_x;
517
518     // the two orientations must not share a shade, or every corner
```

Ligne par ligne

417 Le haut de la bande est à **top** pixels, un nombre négatif quand la bande dépasse. La lecture commence à la ligne de texture correspondante.

515 Les faces vues depuis l'ouest et depuis le nord sont retournées.

516 La colonne 0 devient la colonne 63, et inversement.

Le sol



Figure 6.1. Le joueur vu de côté. L'œil est à une demi-case au-dessus du sol, et l'écran est devant lui. La ligne 150 de l'écran regarde le sol à 2 cases, et la ligne 190 le regarde à 1,11 case. Les deux triangles formés par l'œil, l'écran et le sol ont la même forme, c'est ce qui donne la formule.

Dans ce chapitre, vous allez dessiner le sol, ligne par ligne, avec la géométrie qui donne la distance du sol sous chaque ligne de l'écran.

Le sol reçoit à son tour une texture, des dalles carrées de 32 pixels avec un joint sombre et un grain calculé par la fonction `noise`. Le dessin du sol ne se fait pas colonne par colonne comme les murs, mais ligne par ligne. La raison est géométrique. Tous les pixels d'une même ligne de l'écran, sous l'horizon, regardent des points du sol situés à la même distance du joueur.

Cette distance se calcule avec la figure 6.1. L'œil du joueur est à une demi-case au-dessus du sol, à mi-hauteur des murs. La ligne de l'écran située juste sous l'horizon regarde très loin. La dernière ligne regarde juste sous les pieds du joueur. Entre les deux, la distance vaut la hauteur de l'écran divisée par le double de l'écart entre la ligne et l'horizon, soit $200 \div (2 \times y - 200)$.

Le point du sol vu par le premier pixel d'une ligne se trouve sur le rayon de gauche, celui de la colonne 0, à cette distance. Le programme avance ensuite d'un pas fixe pour chaque pixel vers la droite, et lit la texture à chaque pas. Une ligne entière du sol ne coûte ainsi que deux additions par pixel.

La distance du sol, ligne par ligne

LIGNE	DENOMINATEUR	DISTANCE	RESULTAT
101	$2 \times 101 - 200 = 2$	$200 \div 2$	100 cases
110	$2 \times 110 - 200 = 20$	$200 \div 20$	10 cases
150	$2 \times 150 - 200 = 100$	$200 \div 100$	2 cases
190	$2 \times 190 - 200 = 180$	$200 \div 180$	1,11 case
199	$2 \times 199 - 200 = 198$	$200 \div 198$	1,01 case

Tableau 6.1. La distance du sol pour cinq lignes de l'écran. Le dénominateur vaut le double de la ligne moins la hauteur de l'écran. Plus la ligne est basse, plus le sol est proche.

La distance du sol sous une ligne de l'écran se calcule avec la formule $200 \div (2 \times y - 200)$, où y est le numéro de la ligne, compté depuis le haut de l'écran. Le tableau 6.1 donne quelques valeurs.

La ligne 101, juste sous l'horizon, correspond à un sol situé à 100 cases, si loin qu'il se confond avec l'horizon. La ligne 150, au milieu de la moitié basse de l'écran, correspond à un sol situé à 2 cases. La dernière ligne, la 199, correspond au sol situé juste sous les pieds du joueur, à 1,01 case.

Une fois la distance connue, le programme calcule le point du sol vu par le pixel de gauche de la ligne, en avançant de cette distance le long du rayon de gauche, dont la direction est `dir - plane`. Puis il avance d'un pas constant pour chaque pixel vers la droite. Le pas vaut la largeur du plan de la caméra, $2 \times \text{plane}$, multipliée par la distance et divisée par les 320 pixels de la ligne. Ce sont les variables `step_x` et `step_y`.

EXERCICE

Calculez la distance du sol pour la ligne 125. Vous devez trouver $200 \div 50 = 4$ cases.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

MAIN C LIGNES 62 A 72

```
62 #define BRICK_VARY 0.30
63
64 // a tiled floor: a tile is square, so one size is enough
65 #define TILE_SIZE 32
66 #define TILE_COLOR 0x4a4a52
67 #define TILE_JOINT 0x2e2e34
68 #define TILE_LIGHT 0.85
69 #define TILE_VARY 0.25
70
71 // a wall met on a north-south line keeps this much of its light
72 #define SIDE_LIGHT 0.68
```

MAIN C LIGNES 126 A 130

```
126
127 static unsigned int wall_texture[TEX_SIZE * TEX_SIZE];
128 static unsigned int floor_texture[TEX_SIZE * TEX_SIZE];
129
130 // the window at the size of the picture, in the middle of the screen
```

MAIN C LIGNES 457 A 516

```
457 }
458
459 // the ground: square tiles with a joint, and a grain that keeps the eye
    busy
460 static void make_floor_texture(void)
461 {
462     for (int y = 0; y < TEX_SIZE; y++) {
463         for (int x = 0; x < TEX_SIZE; x++) {
464             int joint = x % TILE_SIZE == 0 || y % TILE_SIZE == 0;
465             unsigned int color = joint ? TILE_JOINT
466                                     : shade(TILE_COLOR,
467                                             TILE_LIGHT + TILE_VARY * noise(x, y));
468             floor_texture[y * TEX_SIZE + x] = color;
469         }
470     }
471 }
472
473 // how far the ray is from the first grid line it will cross
474 static double first_line(double position, double direction)
475 {
476     double cell = position - floor(position);
477     return direction < 0 ? cell : 1.0 - cell;
478 }
479
480 // the ground, one screen row at a time. every pixel of a row is the same
481 // distance away, so the walk across the floor is a straight line, and
    the row
482 // costs two additions per pixel
483 static void render_floor(const struct player *player)
484 {
485     // the ray through the left edge of the screen. the same for every
    row,
486     // so it is worked out once
487     double left_x = player->dir_x - player->plane_x;
488     double left_y = player->dir_y - player->plane_y;
489
490     for (int y = HORIZON + 1; y < VIEW_HEIGHT; y++) {
491         // how far the ground under this row is: a row one pixel below
```

Ligne par ligne

460 La texture des dalles, 32 pixels de côté, avec un joint sur la première ligne et la première colonne de chaque dalle.

487 Le rayon de la colonne 0, le plus à gauche, calculé une fois pour toutes les lignes.

PARTIE 2 CHAPITRE 6 LE CODE

SUITE

```
492 // the horizon is very far, the bottom row is right at our feet,
493 // and the eye is half a wall above the floor
494 double distance = (double)VIEW_HEIGHT / (2 * y - VIEW_HEIGHT);
495
496 // where that row starts on the floor, and what one pixel to the
497 // right is worth. the camera plane spans from -plane to +plane,
498 // so the whole row is two planes wide
499 double step_x = distance * 2.0 * player->plane_x / VIEW_WIDTH;
500 double step_y = distance * 2.0 * player->plane_y / VIEW_WIDTH;
501 double ground_x = player->x + distance * left_x;
502 double ground_y = player->y + distance * left_y;
503
504 for (int x = 0; x < VIEW_WIDTH; x++) {
505     int tex_x = (int)(ground_x * TEX_SIZE) & TEX_MASK;
506     int tex_y = (int)(ground_y * TEX_SIZE) & TEX_MASK;
507     int at = tex_y * TEX_SIZE + tex_x;
508
509     view[y * VIEW_WIDTH + x] = floor_texture[at];
510
511     ground_x += step_x;
512     ground_y += step_y;
513 }
514 }
515 }
516
```

494 La distance du sol sous la ligne y, la formule du tableau 6.1.

499 Le pas sur le sol pour un pixel vers la droite. La ligne entière couvre deux fois le vecteur **plane**.

501 Le point du sol vu par le premier pixel de la ligne.

505 La colonne de texture, la position sur le sol multipliée par 64 et ramenée entre 0 et 63 par le masque. La texture se répète ainsi sur chaque case.

511 Le pas vers la droite, deux additions par pixel.

MAIN C LIGNES 628 A 632

```
628 // the textures cost nothing to keep and everything to draw: once,
    here
629     make_wall_texture();
630     make_floor_texture();
631
632     struct player player = {
```

MAIN C LIGNES 654 A 658

```
654         player.dir_y * forward + player.plane_y * sideways);
655     turn_player(&player, turn);
656     render_floor(&player);
657     render_walls(&player);
658     show_texture(wall_texture);
```

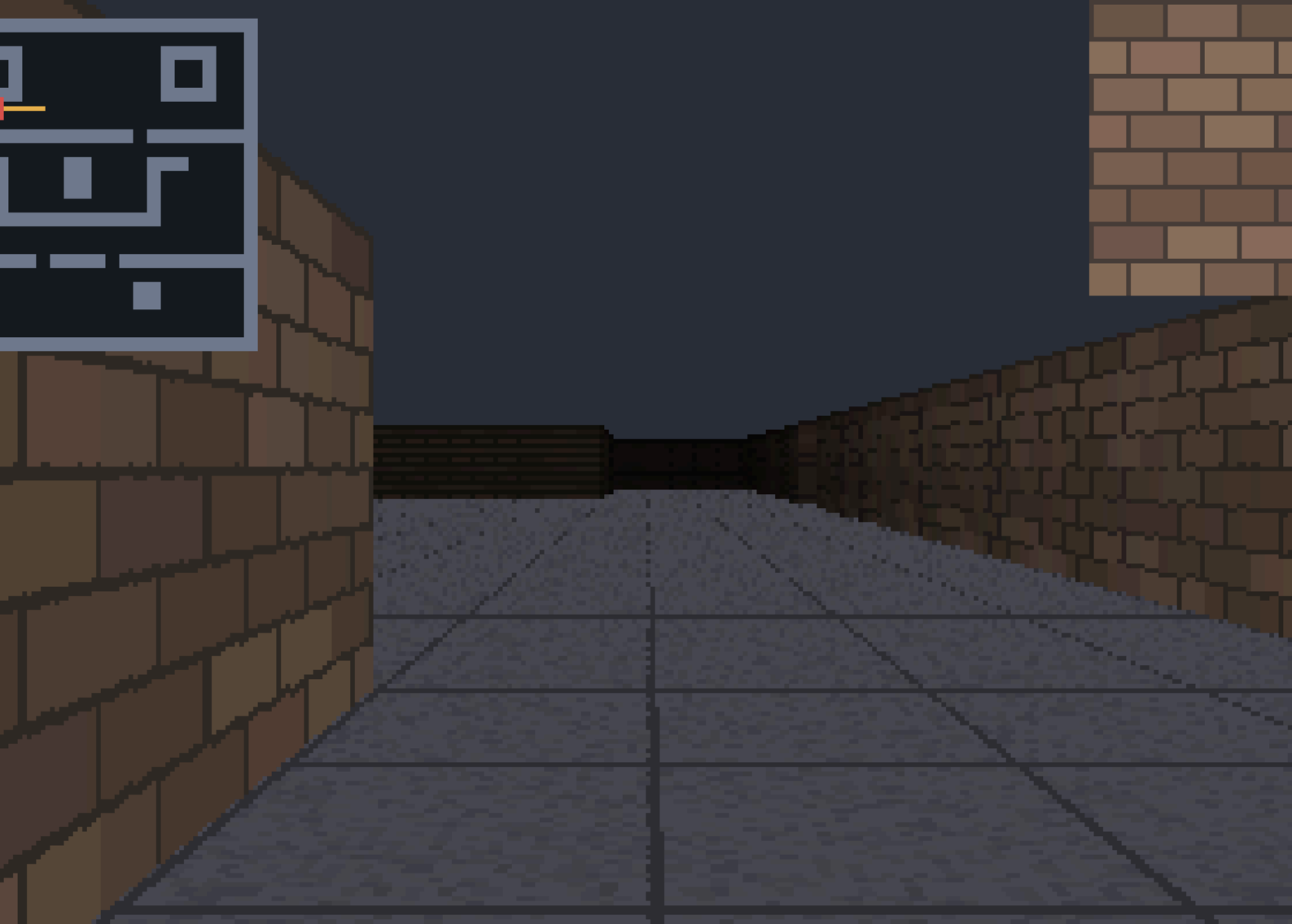


Figure 6.2. Le sol dallé sous les murs de briques. Le plafond est encore d'une seule couleur.

VOCABULAIRE

L'horizon est la ligne du milieu de l'écran. Les lignes du dessous montrent le sol, celles du dessus le plafond.

La brume au sol



Figure 7.1. Le sol assombri avec la distance, comme les murs.

Dans ce chapitre, vous allez appliquer la brume au sol.

Le sol du chapitre précédent est éclairé de la même façon près du joueur et au loin, alors que les murs s'assombrissent avec la distance. Nous appliquons au sol la même fonction `fog` qu'aux murs.

Comme tous les pixels d'une ligne sont à la même distance, la lumière est calculée une seule fois par ligne, avant la boucle sur les pixels, et la fonction `shade` s'applique à chaque pixel lu dans la texture. Le coût est minime et le sol rejoint les murs dans la brume.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

MAIN C LIGNES 501 A 505

```
501     double ground_x = player->x + distance * left_x;
502     double ground_y = player->y + distance * left_y;
503     double light = fog(distance);
504
505     for (int x = 0; x < VIEW_WIDTH; x++) {
```

MAIN C LIGNES 508 A 512

```
508         int at = tex_y * TEX_SIZE + tex_x;
509
510         view[y * VIEW_WIDTH + x] = shade(floor_texture[at], light);
511
512         ground_x += step_x;
```

Ligne par ligne

503 La lumière de la ligne, calculée une fois.

510 Chaque pixel du sol est assombri avant d'être écrit.

Le plafond



Figure 8.1. Les trois textures du jeu, telles que le programme les dessine au démarrage. Les briques, les dalles et les panneaux du plafond.

Dans ce chapitre, vous allez dessiner le plafond, qui est le reflet du sol, avec sa propre texture.

Le plafond est le reflet du sol. La ligne située à 50 pixels sous l'horizon et la ligne située à 50 pixels au-dessus regardent des points situés à la même distance, l'un sur le sol, l'autre au plafond. Le programme écrit donc deux pixels à chaque pas de la boucle du sol, celui de la ligne `y` sur le sol et celui de la ligne `199 - y` au plafond, avec les mêmes coordonnées de texture et la même lumière. La fonction est renommée `render_floor_and_ceiling`.

Le plafond reçoit sa propre texture, des panneaux de 16 pixels de haut, plus sombres que le sol, avec un grain plus grossier obtenu en divisant la coordonnée `x` par 8 avant l'appel à `noise`. Le sol et le plafond ne se ressemblent ainsi jamais, ce qui aide le joueur à savoir où est le haut.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

MAIN C LIGNES 69 A 81

```
69 #define TILE_VARY 0.25
70
71 // a panelled ceiling: panels are wide and flat, so a height and not a
    size,
72 // and a coarser grain than stone
73 #define PANEL_HEIGHT 16
74 #define PANEL_GRAIN 8
75 #define PANEL_COLOR 0x343b46
76 #define PANEL_JOINT 0x232932
77 #define PANEL_LIGHT 0.90
78 #define PANEL_VARY 0.15
79
80 // a wall met on a north-south line keeps this much of its light
81 #define SIDE_LIGHT 0.68
```

MAIN C LIGNES 136 A 140

```
136 static unsigned int wall_texture[TEX_SIZE * TEX_SIZE];
137 static unsigned int floor_texture[TEX_SIZE * TEX_SIZE];
138 static unsigned int ceiling_texture[TEX_SIZE * TEX_SIZE];
139
140 // the window at the size of the picture, in the middle of the screen
```

MAIN C LIGNES 481 A 498

```
481 }
482
483 // the ceiling: wide panels, darker, so up and down never look alike
484 static void make_ceiling_texture(void)
485 {
486     for (int y = 0; y < TEX_SIZE; y++) {
487         for (int x = 0; x < TEX_SIZE; x++) {
488             int joint = y % PANEL_HEIGHT == 0;
489             unsigned int color = joint ? PANEL_JOINT
490                 : shade(PANEL_COLOR,
491                     PANEL_LIGHT + PANEL_VARY * noise(x / PANEL_GRAIN,
492                         y));
493             ceiling_texture[y * TEX_SIZE + x] = color;
494         }
495     }
496
497 // how far the ray is from the first grid line it will cross
498 static double first_line(double position, double direction)
```

MAIN C LIGNES 502 A 509

```
502 }
503
504 // the ground and the sky, one screen row at a time. every pixel of a row
    is
505 // the same distance away, so the walk across the floor is a straight
    line, and
506 // the row costs two additions per pixel
507 static void render_floor_and_ceiling(const struct player *player)
508 {
509     // the ray through the left edge of the screen. the same for every
    row,
```

MAIN C LIGNES 533 A 537

```
533
```

Ligne par ligne

488 Un panneau n'a de joint qu'entre les rangées.

491 Le grain grossier, huit pixels de large partagent la même valeur.

PARTIE 2 CHAPITRE 8 LE CODE

SUITE

```
534         view[y * VIEW_WIDTH + x] = shade(floor_texture[at], light);
535         view[(VIEW_HEIGHT - 1 - y) * VIEW_WIDTH + x] =
            shade(ceiling_texture[at], light);
536
537         ground_x += step_x;
```

MAIN C LIGNES 654 A 658

```
654     make_wall_texture();
655     make_floor_texture();
656     make_ceiling_texture();
657
658     struct player player = {
```

MAIN C LIGNES 680 A 684

```
680         player.dir_y * forward + player.plane_y * sideways);
681     turn_player(&player, turn);
682     render_floor_and_ceiling(&player);
683     render_walls(&player);
684     show_texture(wall_texture);
```

535 Le pixel du plafond, sur la ligne symétrique de celle du sol par rapport à l'horizon.

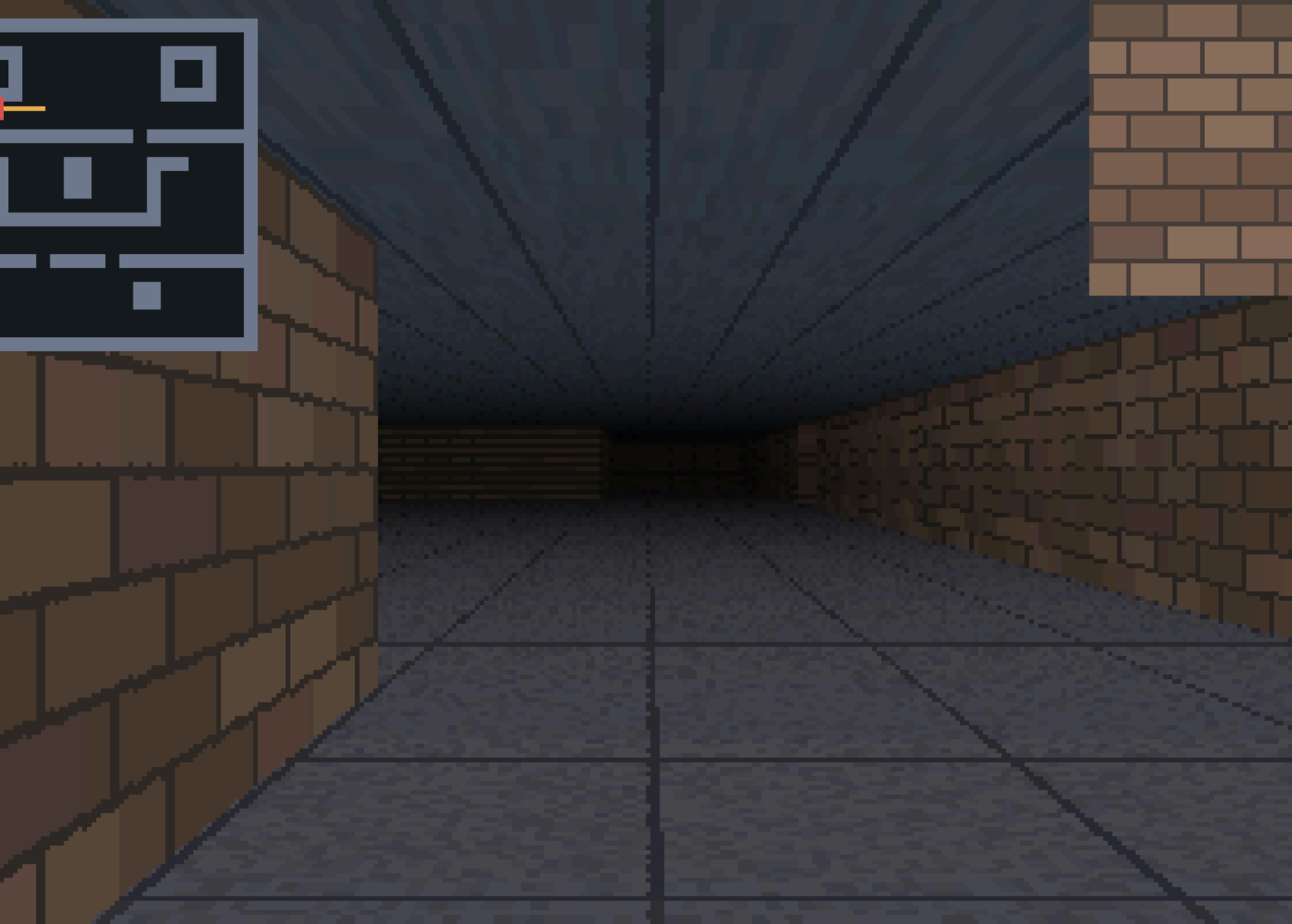


Figure 8.2. Le couloir complet, avec les briques, les dalles et les panneaux du plafond.

Le rangement avant la livraison

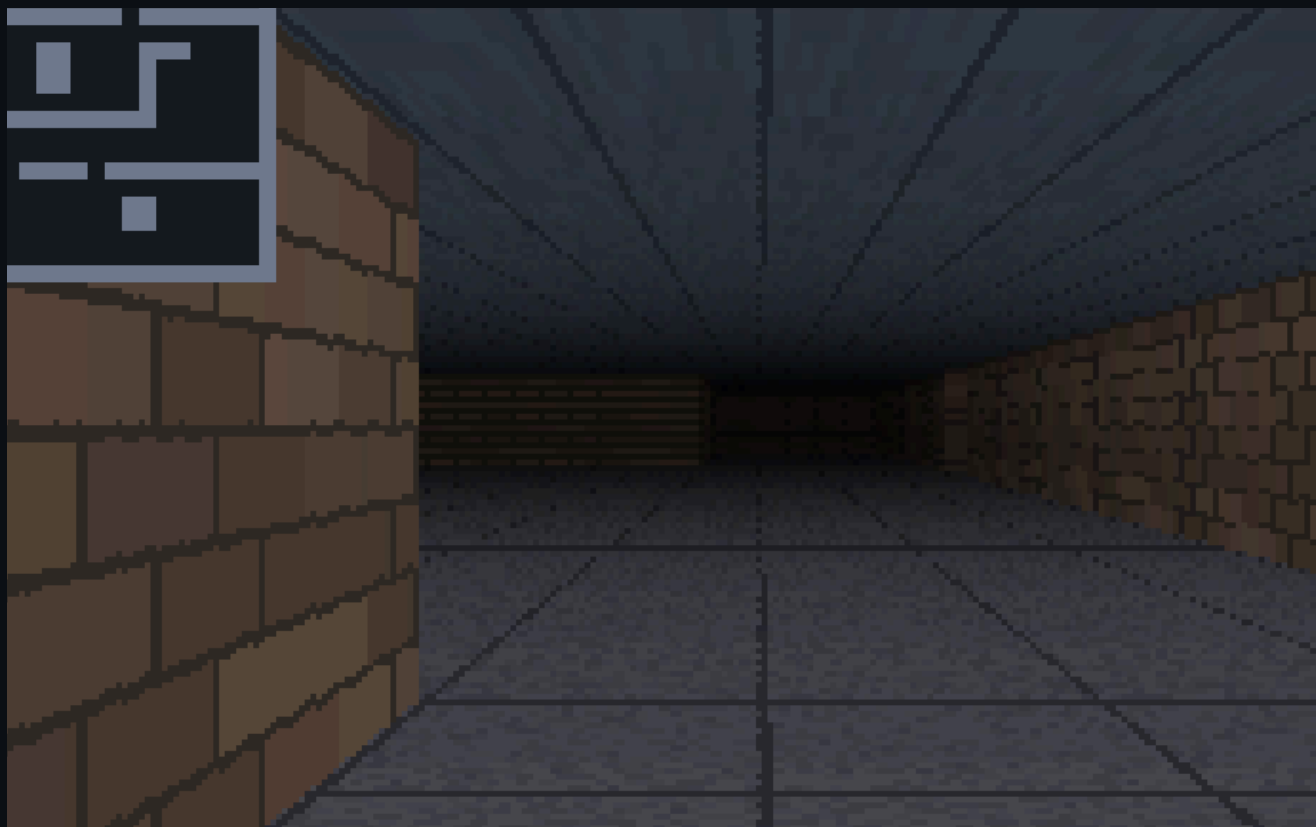


Figure 9.1. L'image ne change pas. Le rangement est invisible à l'écran, et c'est le but.

Dans ce chapitre, vous allez ranger le programme, nommer chaque nombre, séparer le lancer de rayons du dessin, et corriger un défaut découvert en relisant.

Le programme fonctionne, et nous le rangeons avant de le publier. Ranger ne change pas ce que le programme fait. Cela consiste à donner un nom à chaque nombre, à confier une seule tâche à chaque fonction, et à corriger les défauts découverts en relisant.

Les nombres reçoivent des noms. Le 32 de `XCreateImage` devient `SCANLINE_PAD`, le 60 de la pause devient `FRAMES_PER_SECOND`, la position de départ devient `START_X` et `START_Y`, et la densité de la brume, la distance minimale et la valeur très grande du parcours deviennent `FOG_DENSITY`, `NEAR_CLIP` et `VERY_FAR`. Un nom dit pourquoi le nombre est là, et le commentaire au-dessus dit d'où il vient.

Le parcours du rayon sort de `render_walls` pour devenir la fonction `cast_ray`, qui renvoie une structure `hit` avec la distance, la position sur le mur et le type de ligne franchie. La fonction `render_walls` ne fait plus que dessiner. La relecture révèle aussi un vrai défaut. Le trait qui indique la direction du joueur sur la carte était écrit sans vérifier qu'il restait dans l'écran, et il écrivait en mémoire hors du tableau de pixels. La nouvelle fonction `put_pixel` vérifie les bornes avant d'écrire.

VOCABULAIRE

Un **nombre magique** est un nombre écrit tel quel dans le code, sans nom qui dise à quoi il sert. Le rangement les remplace par des constantes nommées.

EXERCICE

Cherchez dans le code un nombre qui n'a pas encore de nom, par exemple le 4 de `render_map`, et donnez-lui une constante.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

MAIN C LIGNES 6 A 10

```
6 #include <X11/XKLib.h>
7 #include <X11/keysym.h>
8 #include <X11/Xutil.h>
9 #include <math.h>
10 #include <stdio.h>
```

MAIN C LIGNES 22 A 58

```
22 #define HORIZON      (VIEW_HEIGHT / 2)
23
24 // XCreateImage wants to know how each row of pixels is padded, in bits.
   this
25 // is NOT the colour depth: 32 is what a modern display expects
26 #define SCANLINE_PAD 32
27
28 #define MAP_WIDTH  24
29 #define MAP_HEIGHT 24
30
31 #define WALK_SPEED 3.0
32 #define TURN_SPEED 2.2
33 #define FRAMES_PER_SECOND 60
34
35 // where the player stands when the game opens: a free square in the
   corridor
36 #define START_X 2.5
37 #define START_Y 6.5
38
39 // how wide the view is: 0.66 against a unit direction is about 66
   degrees
40 #define FIELD_OF_VIEW 0.66
41
42 // how fast the light falls off with distance
43 #define FOG_DENSITY  0.02
44 // never divide by less than this: a wall right against the eye
45 #define NEAR_CLIP    0.02
46 // a ray parallel to an axis never crosses that axis's grid lines
47 #define VERY_FAR     1e30
48
49 #define MAP_CELL  3
50 #define MAP_LEFT  4
51 #define MAP_TOP   4
52
53 // the player on the map: half the side of his square, and how many cells
   long
54 // the line showing where he looks
55 #define MAP_DOT    2
56 #define MAP_ARROW  4
57
58 #define MAP_WALL   0x6f7b8f
```

MAIN C LIGNES 227 A 231

```
227 // the server reads it from there
228 screen->image = XCreateImage(screen->display, screen->visual, screen->
   >depth,
229     ZPixmap, 0, (char *)pixels, width, height, SCANLINE_PAD, 0);
230 screen_fit(screen);
231 return screen->image != NULL;
```

MAIN C LIGNES 394 A 398

```
394 player->dir_x = dir_x * cosine - player->dir_y * sine;
```

Ligne par ligne

- 26** Le 32 de `XCreateImage`, qui n'est pas la profondeur des couleurs mais l'alignement des lignes de pixels.
- 47** La valeur donnée à un rayon parallèle à un axe, qui ne franchira jamais de ligne de cet axe.

PARTIE 2 CHAPITRE 9 LE CODE

SUITE

```
395     player->dir_y = dir_x * sine + player->dir_y * cosine;
396
397     double plane_x = player->plane_x;
398     player->plane_x = plane_x * cosine - player->plane_y * sine;
```

MAIN C LIGNES 414 A 419

```
414     if (light > 1.0)
415         light = 1.0;
416     if (light < 0.0)
417         light = 0.0;
418
419     unsigned int red = (unsigned int)(((color >> 16) & 0xff) * light);
```

MAIN C LIGNES 423 A 431

```
423 }
424
425 // far away is dark: light is 1 at the eye and falls towards 0 with the
426 // square of the distance. one line, and the corridors have depth
427 static double fog(double distance)
428 {
429     return 1.0 / (1.0 + distance * distance * FOG_DENSITY);
430 }
431
```

MAIN C LIGNES 550 A 611

```
550 }
551
552 // what a ray found: how far it went, which way the wall it met faces,
553 // and
554 // where along that wall it landed
554 struct hit {
555     double distance;
556     double wall_x;
557     int side;
558 };
559
560 // walk the grid square by square until we meet a wall, always stepping
561 // along
562 // the axis whose grid line is nearest: no square is missed, and none is
563 // visited twice
563 static struct hit cast_ray(const struct player *player, double ray_x,
564 double ray_y)
565 {
566     int map_x = (int)player->x;
567     int map_y = (int)player->y;
568
569     // the distance the ray covers to cross one whole square
570     double delta_x = ray_x == 0.0 ? VERY_FAR : fabs(1.0 / ray_x);
571     double delta_y = ray_y == 0.0 ? VERY_FAR : fabs(1.0 / ray_y);
572     double side_x = first_line(player->x, ray_x) * delta_x;
573     double side_y = first_line(player->y, ray_y) * delta_y;
574     int step_x = ray_x < 0 ? -1 : 1;
575     int step_y = ray_y < 0 ? -1 : 1;
576
577     // always step along the axis whose grid line is nearest. that is
578     // the whole trick: no square is missed, and none is visited twice
579     int side = 0;
580     while (!is_wall(map_x, map_y)) {
581         if (side_x < side_y) {
582             side_x += delta_x;
583             map_x += step_x;
584             side = 0;
585         } else {
586             side_y += delta_y;
587             map_y += step_y;
588             side = 1;
589         }
590     }
591     struct hit hit;
592     hit.distance = side_x < side_y ? side_x : side_y;
593     hit.wall_x = map_x;
594     hit.side = side;
595     return hit;
596 }
```

416 Une lumière négative rendrait des couleurs absurdes. Elle est ramenée à zéro.

554 Ce qu'un rayon rapporte, la distance, la position sur le mur et le type de ligne.

563 Le parcours du rayon, désormais une fonction à part, qui reçoit la direction du rayon et renvoie ce qu'il a trouvé.

PARTIE 2 CHAPITRE 9 LE CODE

SUITE

```
588     }
589 }
590
591 // how far it went, measured on the camera plane and not from the
eye:
592 // from the eye the corners of a wall are farther than its middle,
and
593 // a straight wall would bend into a fishbowl
594 double distance = side == 0 ? side_x - delta_x : side_y - delta_y;
595 if (distance < NEAR_CLIP)
596     distance = NEAR_CLIP;
597
598 // where along the wall it landed, between 0 and 1: that is the
column
599 // of the texture to read
600 double wall_x = side == 0 ? player->y + distance * ray_y
601     : player->x + distance * ray_x;
602 wall_x -= floor(wall_x);
603
604 struct hit hit = { .distance = distance, .wall_x = wall_x, .side =
side };
605 return hit;
606 }
607
608 // one ray per column of the screen, and how far that ray went decides
how
609 // tall the wall is drawn: near is tall, far is short
610 static void render_walls(const struct player *player)
611 {
```

637 L'écriture d'un pixel, refusée hors de l'écran. Le trait de direction de la carte écrivait hors du tableau.

MAIN C LIGNES 615 A 642

```
615     double ray_x = player->dir_x + player->plane_x * camera;
616     double ray_y = player->dir_y + player->plane_y * camera;
617     struct hit hit = cast_ray(player, ray_x, ray_y);
618
619     int height = (int)(VIEW_HEIGHT / hit.distance);
620     int top = HORIZON - height / 2;
621
622     int tex_x = (int)(hit.wall_x * TEX_SIZE);
623     // the two faces we can see of the same wall must not be mirror
624     // images of each other
625     if ((hit.side == 0 && ray_x > 0) || (hit.side == 1 && ray_y < 0))
626         tex_x = TEX_SIZE - 1 - tex_x;
627
628     // the two orientations must not share a shade, or every corner
629     // disappears
630     draw_wall_column(x, top, height, tex_x, fog(hit.distance),
631         hit.side == 1);
632 }
633 }
634
635 // one pixel, if it is on the screen at all. the heading line runs off
the
636 // map, and until now it wrote wherever that landed in memory
637 static void put_pixel(int x, int y, unsigned int color)
638 {
639     if (x >= 0 && x < VIEW_WIDTH && y >= 0 && y < VIEW_HEIGHT)
640         view[y * VIEW_WIDTH + x] = color;
641 }
642
```

MAIN C LIGNES 649 A 665

```
649     for (int line = 0; line < cell; line++)
650         for (int column = 0; column < cell; column++)
651             put_pixel(left + x * cell + column,
```

PARTIE 2 CHAPITRE 9 LE CODE

SUITE

```
652             top + y * cell + line, color);
653         }
654
655         // where we stand, and which way we look
656         int dot_x = left + (int)(player->x * cell);
657         int dot_y = top + (int)(player->y * cell);
658         for (int step = MAP_DOT + 1; step < MAP_ARROW * cell; step++)
659             put_pixel(dot_x + (int)(player->dir_x * step),
660                     dot_y + (int)(player->dir_y * step), MAP_HEADING);
661         for (int line = -MAP_DOT; line <= MAP_DOT; line++)
662             for (int column = -MAP_DOT; column <= MAP_DOT; column++)
663                 put_pixel(dot_x + column, dot_y + line, MAP_PLAYER);
664     }
665 }
```

726 Le contexte graphique est libéré avant la fermeture, comme l'image.

MAIN C LIGNES 682 A 688

```
682     make_ceiling_texture();
683
684     // facing east, with the camera plane across the line of sight
685     struct player player = {
686         .x = START_X, .y = START_Y,
687         .dir_x = 1.0, .dir_y = 0.0,
688         .plane_x = 0.0, .plane_y = FIELD_OF_VIEW,
```

MAIN C LIGNES 704 A 711

```
704         double turn = (keys.right - keys.left) * TURN_SPEED * elapsed;
705
706         double step_x = player.dir_x * forward + player.plane_x *
sideways;
707         double step_y = player.dir_y * forward + player.plane_y *
sideways;
708
709         move_player(&player, step_x, step_y);
710         turn_player(&player, turn);
711         render_floor_and_ceiling(&player);
```

MAIN C LIGNES 714 A 720

```
714         screen_present(&screen);
715
716         // no need to draw faster than that, and without this the loop
717         // eats a whole core to draw the same thing twice
718         double spare = 1.0 / FRAMES_PER_SECOND - (now_in_seconds() -
moment);
719         if (spare > 0)
720             usleep((useconds_t)(spare * 1e6));
```

MAIN C LIGNES 724 A 728

```
724         // clean-up: the picture, then the connection
725         XDestroyImage(screen.image);
726         XFreeGC(screen.display, screen.gc);
727         XCloseDisplay(screen.display);
728         return 0;
```

Un fichier par métier



Figure 10.1. L'image est la même. Le programme est maintenant réparti en cinq fichiers de code et quatre en-têtes.

Dans ce chapitre, vous allez répartir le programme en plusieurs fichiers, avec des en-têtes, et apprendre comment le C relie ces fichiers entre eux.

Sept cents lignes dans un seul fichier deviennent difficiles à parcourir. Nous répartissons le programme en cinq fichiers, un par métier. Le fichier `world.c` contient le niveau et le joueur. Le fichier `screen.c` contient tout ce qui parle à Xlib, la fenêtre, les événements et l'affichage. Le fichier `texture.c` contient les textures, la fonction `noise` et la fonction `shade`. Le fichier `render.c` contient le lancer de rayons, le sol, le plafond et la carte. Le fichier `main.c` ne garde que la boucle principale.

Chaque fichier de code est accompagné d'un fichier d'en-tête, avec l'extension `.h`, qui déclare ce que les autres fichiers ont le droit d'utiliser. Un en-tête contient les constantes, les structures et les prototypes des fonctions, c'est-à-dire leur signature sans leur corps. Les tableaux partagés, comme le tampon de pixels `view` et les trois textures, sont déclarés avec le mot `extern`, qui signifie que le tableau existe dans un autre fichier.

Chaque en-tête commence par une garde d'inclusion, trois lignes qui empêchent de l'inclure deux fois dans le même fichier. Les fonctions internes à un fichier gardent le mot `static`, les autres le perdent pour devenir visibles. La compilation prend maintenant tous les fichiers de `src` et l'option `-Iinclude`, qui indique où trouver les en-têtes. Le code des fonctions ne change pas, il change seulement de fichier. Les pages qui suivent montrent les quatre en-têtes et le nouveau `main.c`.

Un fichier par métier, suite

VOCABULAIRE

Un **en-tête** est un fichier `.h` qui déclare les constantes, les structures et les fonctions qu'un fichier de code offre aux autres. Un **prototype** est la déclaration d'une fonction sans son corps, son nom, son type de retour et ses arguments. Une **garde d'inclusion** est le couple `#ifndef` et `#define` qui protège un en-tête contre une double inclusion.

EXERCICE

Compilez avec la ligne `gcc src/*.c -Iinclude -o keep -lX11 -lm`. Puis retirez `-lm` et lisez le message de l'éditeur de liens, qui ne trouve plus `cos` ni `floor`.

Les en-têtes et le nouveau main.c

Les quatre en-têtes en entier, puis le fichier main.c réduit à la boucle principale. Les fonctions déplacées dans les autres fichiers ne sont pas reproduites, elles n'ont pas changé.

INCLUDE WORLD H LIGNES 1 A 25

```
1  #ifndef WORLD_H
2  #define WORLD_H
3
4  #define MAP_WIDTH  24
5  #define MAP_HEIGHT 24
6
7  // where the player stands when the game opens: a free square in the
   corridor
8  #define START_X 2.5
9  #define START_Y 6.5
10
11 #define WALK_SPEED 3.0
12 #define TURN_SPEED 2.2
13 struct player {
14     double x, y;           // where we stand, in map squares
15     double dir_x, dir_y;   // a unit vector: where the eyes point
16     double plane_x, plane_y; // the camera plane, as wide as the view
   is
17 };
18
19 #define FRAMES_PER_SECOND 60
20
21 int is_wall(int x, int y);
22 void move_player(struct player *player, double step_x, double step_y);
23 void turn_player(struct player *player, double angle);
24
25 #endif
```

INCLUDE SCREEN H LIGNES 1 A 50

```
1  #ifndef SCREEN_H
2  #define SCREEN_H
3
4  #include <X11/Xlib.h>
5
6  // the world is drawn at 320x200 and blown up to fill the window: chunky
7  // pixels are not nostalgia, they are four times fewer rays to trace
8  #define VIEW_WIDTH  320
9  #define VIEW_HEIGHT 200
10 #define SCALE        4
11 #define WIN_WIDTH    (VIEW_WIDTH * SCALE)
12 #define WIN_HEIGHT   (VIEW_HEIGHT * SCALE)
13 #define HORIZON      (VIEW_HEIGHT / 2)
14
15 // XCreateImage wants to know how each row of pixels is padded, in bits.
   this
16 // is NOT the colour depth: 32 is what a modern display expects
17 #define SCANLINE_PAD 32
18
19 #define GAME_NAME    "The Keep"
20 #define WINDOW_BG    0x182636
21
22 struct screen {
23     Display *display;
24     Window  window;
25     GC      gc;
26     XImage  *image;
27     Atom     close_message;
```

Ligne par ligne

- 1** La garde d'inclusion. Si `WORLD_H` est déjà défini, tout le fichier est ignoré.
- 21** Le prototype de la fonction, sans son corps. Le corps est dans `world.c`.

PARTIE 2 CHAPITRE 10 LE CODE

SUITE

```
28     unsigned int *pixels;
29     // the window is the player's to resize; the picture keeps its own
    size
30     Visual *visual;
31     int depth;
32     int width, height;
33     int scale, left, top;
34     int full;                // are we filling the screen?
35 };
36
37 struct keys {
38     int forward, back, left, right, strafe_left, strafe_right, quit;
39 };
40
41 // the frame buffer belongs to the screen; everyone else just writes in
    it
42 extern unsigned int view[VIEW_WIDTH * VIEW_HEIGHT];
43
44 int screen_open(struct screen *screen);
45 int screen_resize(struct screen *screen, int width, int height);
46 void screen_toggle_fullscreen(struct screen *screen);
47 void screen_read_keys(struct screen *screen, struct keys *keys);
48 void screen_present(struct screen *screen);
49
50 #endif
```

42 Le tampon de pixels est déclaré ici et défini dans `screen.c`. Les autres fichiers y écrivent.

37 Les textures appartiennent à `texture.c`, mais `render.c` les lit.

INCLUDE TEXTURE H LIGNES 1 A 47

```
1  #ifndef TEXTURE_H
2  #define TEXTURE_H
3
4  // a texture is a square of pixels drawn once at startup. a power of two,
    so
5  // wrapping around it is a bitwise and instead of a division
6  #define TEX_SIZE    64
7  #define TEX_MASK    (TEX_SIZE - 1)
8
9  // a brick wall: the size of one brick, its two colours, and how far one
    brick
10 // is allowed to differ from the next so that the wall is not flat
11 #define BRICK_WIDTH  16
12 #define BRICK_HEIGHT 8
13 #define BRICK_COLOR  0x8a6f5d
14 #define BRICK_JOINT  0x4a4038
15 #define BRICK_LIGHT  0.78
16 #define BRICK_VARY   0.30
17
18 // a tiled floor: a tile is square, so one size is enough
19 #define TILE_SIZE    32
20 #define TILE_COLOR   0x4a4a52
21 #define TILE_JOINT   0x2e2e34
22 #define TILE_LIGHT   0.85
23 #define TILE_VARY    0.25
24
25 // a panelled ceiling: panels are wide and flat, so a height and not a
    size,
26 // and a coarser grain than stone
27 #define PANEL_HEIGHT 16
28 #define PANEL_GRAIN  8
29 #define PANEL_COLOR  0x343b46
30 #define PANEL_JOINT  0x232932
31 #define PANEL_LIGHT  0.90
32 #define PANEL_VARY   0.15
33
34 // a wall met on a north-south line keeps this much of its light
35 #define SIDE_LIGHT   0.68
36
37 extern unsigned int wall_texture[TEX_SIZE * TEX_SIZE];
38 extern unsigned int floor_texture[TEX_SIZE * TEX_SIZE];
```

PARTIE 2 CHAPITRE 10 LE CODE

SUITE

```
39 extern unsigned int ceiling_texture[TEX_SIZE * TEX_SIZE];
40
41 double noise(int x, int y);
42 unsigned int shade(unsigned int color, double light);
43 void make_wall_texture(void);
44 void make_floor_texture(void);
45 void make_ceiling_texture(void);
46
47 #endif
```

INCLUDE RENDER H LIGNES 1 A 35

```
1 #ifndef RENDER_H
2 #define RENDER_H
3
4 #include "world.h"
5
6 // how wide the view is: 0.66 against a unit direction is about 66
  degrees
7 #define FIELD_OF_VIEW 0.66
8
9 // how fast the light falls off with distance
10 #define FOG_DENSITY 0.02
11 // never divide by less than this: a wall right against the eye
12 #define NEAR_CLIP 0.02
13 // a ray parallel to an axis never crosses that axis's grid lines
14 #define VERY_FAR 1e30
15
16 #define MAP_CELL 3
17 #define MAP_LEFT 4
18 #define MAP_TOP 4
19
20 // the player on the map: half the side of his square, and how many cells
  long
21 // the line showing where he looks
22 #define MAP_DOT 2
23 #define MAP_ARROW 4
24
25 #define MAP_WALL 0x6f7b8f
26 #define MAP_FLOOR 0x161a22
27 #define MAP_HEADING 0xe8b04b
28 #define MAP_PLAYER 0xd8534f
29
30 double fog(double distance);
31 void render_floor_and_ceiling(const struct player *player);
32 void render_walls(const struct player *player);
33 void render_map(const struct player *player, int cell, int left, int
  top);
34
35 #endif
```

SRC MAIN C LIGNES 1 A 78

```
1 // wolfenstein 3d drew its corridors with one ray per column, in 1992
2 // let us write that engine in c, from this empty file
3 // x11 gives us a window and a block of memory, the rest is ours
4
5 #include <X11/Xlib.h>
6 #include <X11/Xutil.h>
7 #include <time.h>
8 #include <unistd.h>
9
10 #include "render.h"
11 #include "screen.h"
12 #include "texture.h"
13 #include "world.h"
14
15 static double now_in_seconds(void)
16 {
17     struct timespec moment;
```

4

L'en-tête du rendu a besoin de la structure du joueur, il inclut donc l'en-tête du monde.

10

Le fichier principal inclut les quatre en-têtes, entre guillemets parce qu'ils sont dans le projet et non dans le système.

PARTIE 2 CHAPITRE 10 LE CODE

SUITE

```
18     clock_gettime(CLOCK_MONOTONIC, &moment);
19     return moment.tv_sec + moment.tv_nsec / 1e9;
20 }
21
22 int main(void)
23 {
24     struct screen screen;
25     if (!screen_open(&screen))
26         return 1;
27
28     // the textures cost nothing to keep and everything to draw: once,
    here
29     make_wall_texture();
30     make_floor_texture();
31     make_ceiling_texture();
32
33     // facing east, with the camera plane across the line of sight
34     struct player player = {
35         .x = START_X, .y = START_Y,
36         .dir_x = 1.0, .dir_y = 0.0,
37         .plane_x = 0.0, .plane_y = FIELD_OF_VIEW,
38     };
39     struct keys keys = { 0 };
40     double last = now_in_seconds();
41
42     while (!keys.quit) {
43         double moment = now_in_seconds();
44         double elapsed = moment - last;
45         last = moment;
46
47         screen_read_keys(&screen, &keys);
48
49         // every move is scaled by the time the last frame took, so the
    game
50         // runs at the same speed whatever the machine is doing
51         double forward = (keys.forward - keys.back) * WALK_SPEED *
    elapsed;
52         double sideways = (keys.strafe_right - keys.strafe_left) *
    WALK_SPEED * elapsed;
53         double turn = (keys.right - keys.left) * TURN_SPEED * elapsed;
54
55         double step_x = player.dir_x * forward + player.plane_x *
    sideways;
56         double step_y = player.dir_y * forward + player.plane_y *
    sideways;
57
58         move_player(&player, step_x, step_y);
59         turn_player(&player, turn);
60         render_floor_and_ceiling(&player);
61         render_walls(&player);
62         render_map(&player, MAP_CELL, MAP_LEFT, MAP_TOP);
63         screen_present(&screen);
64
65         // no need to draw faster than that, and without this the loop
66         // eats a whole core to draw the same thing twice
67         double spare = 1.0 / FRAMES_PER_SECOND - (now_in_seconds() -
    moment);
68         if (spare > 0)
69             usleep((useconds_t)(spare * 1e6));
70     }
71
72     // XDestroyImage frees the buffer it was given, so this is the whole
73     // clean-up: the picture, then the connection
74     XDestroyImage(screen.image);
75     XFreeGC(screen.display, screen.gc);
76     XCloseDisplay(screen.display);
77     return 0;
```

22 La boucle principale, seule fonction restée dans ce fichier avec l'horloge.

PARTIE 2 CHAPITRE 10 LE CODE

SUITE

78 }

3

TROISIEME PARTIE

Le niveau, la lumière et les portes

Dans cette troisième partie, vous allez faire sortir le monde du programme. Le niveau sera lu dans un fichier texte, les murs deviendront des plaques de métal, les portes s'ouvriront, les néons éclaireront, et le jeu parlera au joueur avec une police dessinée en cent quatre-vingts octets.

12 CHAPITRES 7 FICHIERS 1229 LIGNES LANGAGE C

Le niveau lu dans un fichier



Figure 1.1. Le niveau de la troisième partie, dessiné à partir du fichier `levels/keep.txt`. Le point 1 est le départ et le point 2 la sortie. Les traits de couleur sont les portes et les néons, décrits dans les chapitres suivants.

Dans ce chapitre, vous allez lire le niveau dans un fichier texte au lieu de l'écrire dans le programme, et manipuler de la mémoire dont la taille n'est pas connue à l'avance.

Jusqu'ici, le niveau était écrit dans le programme. Pour déplacer un couloir, il fallait recompiler. Nous faisons lire le niveau dans un fichier texte au démarrage. Le fichier contient une ligne de caractères par ligne de la grille, avec les mêmes conventions que le tableau de la première partie, un dièse ou un chiffre pour un mur et un point pour le vide.

La fonction `load_level` ouvre le fichier, lit les lignes une par une avec `fgets`, retire le retour à la ligne, et copie chaque ligne en mémoire avec `strdup`. Comme le nombre de lignes n'est pas connu à l'avance, le tableau des lignes est agrandi au fur et à mesure avec `realloc`, en doublant sa taille à chaque fois qu'il est plein. La largeur du niveau est celle de la ligne la plus longue.

Le tableau `map` n'est plus de taille fixe. Il devient un pointeur vers des lignes, et sa largeur et sa hauteur deviennent deux variables, `map_width` et `map_height`. La fonction `is_wall` vérifie en plus que la colonne demandée existe dans la ligne, car les lignes du fichier peuvent avoir des longueurs différentes.

VOCABULAIRE

Un **pointeur** est une variable qui contient l'adresse d'une zone de mémoire. Le tableau `map` est un pointeur vers un tableau de pointeurs, un par ligne. **realloc** agrandit une zone de mémoire déjà allouée, en conservant son contenu.

EXERCICE

Ajoutez une ligne au fichier du niveau, puis lancez le jeu. Aucune recompilation n'est nécessaire.

Le niveau lu dans un fichier, suite



Figure 1.2. Le premier niveau lu dans un fichier, avec les textures de la deuxième partie.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE WORLD H LIGNES 2 A 10

```
2  #define WORLD_H
3
4  // the level is read at startup, so its size is not known at compile time
5  extern char **map;
6  extern int map_width, map_height;
7
8  #define LEVEL_FILE "levels/keep.txt"
9
10 #define WALK_SPEED 3.0
```

INCLUDE WORLD H LIGNES 18 A 22

```
18 #define FRAMES_PER_SECOND 60
19
20 int load_level(const char *path);
21 int is_wall(int x, int y);
22 void move_player(struct player *player, double step_x, double step_y);
```

SRC MAIN C LIGNES 22 A 28

```
22 int main(void)
23 {
24     if (!load_level(LEVEL_FILE))
25         return 1;
26
27     struct screen screen;
28     if (!screen_open(&screen))
```

SRC MAIN C LIGNES 36 A 40

```
36     // facing east, with the camera plane across the line of sight
37     struct player player = {
38         .x = 3.5, .y = 3.5,
39         .dir_x = 1.0, .dir_y = 0.0,
40         .plane_x = 0.0, .plane_y = FIELD_OF_VIEW,
```

SRC RENDER C LIGNES 180 A 185

```
180 void render_map(const struct player *player, int cell, int left, int top)
181 {
182     for (int y = 0; y < map_height; y++)
183         for (int x = 0; x < map_width; x++) {
184             unsigned int color = is_wall(x, y) ? MAP_WALL : MAP_FLOOR;
185             for (int line = 0; line < cell; line++)
```

SRC WORLD C LIGNES 2 A 59

```
2
3  #include <math.h>
4  #include <stdio.h>
5  #include <stdlib.h>
6  #include <string.h>
7
8  // the world comes out of a file: a table compiled in means rebuilding
   the
9  // game to move one corridor
10 char **map;
11 int map_width, map_height;
12
13 // what the level file can hold. a wall is anything that is not floor.
14 int load_level(const char *path)
15 {
```

Ligne par ligne

5

Le niveau, un pointeur vers des lignes. Sa taille n'est plus connue à la compilation.

PARTIE 3 CHAPITRE 1 LE CODE

SUITE

```
16 FILE *f = fopen(path, "r");
17 if (!f) {
18     fprintf(stderr, "no level at %s\n", path);
19     return 0;
20 }
21
22 char line[512];
23 int room = 0;
24 map_height = 0;
25 map_width = 0;
26 while (fgets(line, sizeof line, f)) {
27     int length = (int)strlen(line);
28     while (length > 0 && (line[length - 1] == '\n' || line[length -
1] == '\r'))
29         line[--length] = '\0';
30     if (length == 0)
31         continue;
32     if (map_height == room) {
33         room = room ? room * 2 : 32;
34         char **bigger = realloc(map, (size_t)room * sizeof(*map));
35         if (!bigger) {
36             fclose(f);
37             return 0;
38         }
39         map = bigger;
40     }
41     map[map_height] = strdup(line);
42     if (!map[map_height]) {
43         fclose(f);
44         return 0;
45     }
46     if (length > map_width)
47         map_width = length;
48     map_height++;
49 }
50 fclose(f);
51 return map_height > 0;
52 }
53
54 int is_wall(int x, int y)
55 {
56     if (x < 0 || x >= map_width || y < 0 || y >= map_height
57         || x >= (int)strlen(map[y]))
58         return 1;
59     return map[y][x] != '.';
```

- 16** L'ouverture du fichier en lecture. Un échec renvoie **NULL**, et le programme le signale.
- 26** Une ligne à la fois, dans un tampon de 512 caractères.
- 28** Le retour à la ligne est retiré. Un fichier venu de Windows en a deux.
- 33** La place est doublée quand elle manque, 32 lignes d'abord, puis 64, puis 128.
- 41** La ligne est copiée dans une zone de mémoire à sa taille.
- 57** Une colonne au-delà de la fin de la ligne est un mur.

Le point de départ et la sortie



Figure 2.1. Le premier écran, avec le joueur placé sur la case S et tourné vers le couloir le plus long.

Dans ce chapitre, vous allez placer le départ et la sortie dans le fichier du niveau, et choisir la direction du regard au démarrage.

Le fichier du niveau dit maintenant où le joueur commence et où se trouve la sortie. La lettre S marque la case de départ et la lettre E la case de sortie. La fonction `level_marks` parcourt la grille pour les trouver, et place le joueur au centre de la case de départ.

La direction du regard au départ est choisie par la fonction `face_the_open`. Elle mesure, dans les quatre directions, la longueur du couloir libre devant le joueur, et retient la plus longue. Le premier écran du jeu montre ainsi un couloir et non un mur à deux pas.

La boucle principale vérifie à chaque image si le joueur se trouve sur la case de sortie. Si c'est le cas, le jeu se termine. Le niveau a désormais un but. La fonction `is_wall` doit aussi apprendre que les lettres S et E sont des cases libres.

EXERCICE

Déplacez la lettre S dans le fichier du niveau, dans une autre pièce. Le joueur y commence, tourné vers le passage le plus long.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE WORLD H LIGNES 19 A 24

```
19
20 int load_level(const char *path);
21 void level_marks(double *start_x, double *start_y, int *exit_x, int
  *exit_y,
22         double *dir_x, double *dir_y);
23 int is_wall(int x, int y);
24 void move_player(struct player *player, double step_x, double step_y);
```

SRC MAIN C LIGNES 36 A 47

```
36 // facing east, with the camera plane across the line of sight
37 struct player player = {
38     .x = 0.0, .y = 0.0,
39     .dir_x = 1.0, .dir_y = 0.0,
40     .plane_x = 0.0, .plane_y = FIELD_OF_VIEW,
41 };
42 // the file says where we start and where the way out is
43 int exit_x, exit_y;
44 level_marks(&player.x, &player.y, &exit_x, &exit_y,
45             &player.dir_x, &player.dir_y);
46 struct keys keys = { 0 };
47 double last = now_in_seconds();
```

SRC MAIN C LIGNES 53 A 61

```
53
54     screen_read_keys(&screen, &keys);
55
56     // the way out. the level had no end: one walked until one
57     // stopped. standing on it closes the keep behind us.
58     if ((int)player.x == exit_x && (int)player.y == exit_y)
59         keys.quit = 1;
60
61     // every move is scaled by the time the last frame took, so the
game
```

SRC WORLD C LIGNES 52 A 106

```
52 }
53
54 // which way to look when the game opens: down the longest clear line, so
the
55 // first thing anyone sees is a corridor and not a wall two steps away
56 static void face_the_open(double x, double y, double *dir_x, double
*dir_y)
57 {
58     const double way[4][2] = { {1, 0}, {0, 1}, {-1, 0}, {0, -1} };
59     double best = -1.0;
60     *dir_x = 1.0;
61     *dir_y = 0.0;
62     for (int i = 0; i < 4; i++) {
63         double d = 0.0;
64         while (d < 20.0 && !is_wall((int)(x + way[i][0] * (d + 0.5)),
(int)(y + way[i][1] * (d + 0.5))))
65             d += 0.5;
66         if (d > best) {
67             best = d;
68             *dir_x = way[i][0];
69             *dir_y = way[i][1];
70         }
71     }
72 }
73 }
```

Ligne par ligne

- 58** La case du joueur est comparée à celle de la sortie. La partie entière des coordonnées donne la case.
- 58** Les quatre directions, est, sud, ouest et nord.
- 64** La distance libre dans une direction, mesurée par demi-cases jusqu'à un mur ou jusqu'à 20 cases.

PARTIE 3 CHAPITRE 2 LE CODE

SUITE

```
74
75 // 'S' is where we stand at the first frame, 'E' is the way out
76 void level_marks(double *start_x, double *start_y, int *exit_x, int
    *exit_y,
77                 double *dir_x, double *dir_y)
78 {
79     *start_x = 1.5;
80     *start_y = 1.5;
81     *exit_x = -1;
82     *exit_y = -1;
83     for (int y = 0; y < map_height; y++)
84         for (int x = 0; map[y][x]; x++) {
85         if (map[y][x] == 'S') {
86             *start_x = x + 0.5;
87             *start_y = y + 0.5;
88         } else if (map[y][x] == 'E') {
89             *exit_x = x;
90             *exit_y = y;
91         }
92     }
93     face_the_open(*start_x, *start_y, dir_x, dir_y);
94 }
95
96 int is_wall(int x, int y)
97 {
98     if (x < 0 || x >= map_width || y < 0 || y >= map_height
99         || x >= (int)strlen(map[y]))
100         return 1;
101     // the file carries more than walls and floor: where the player
102     // starts, and where the way out is. only a wall stops anyone.
103     char c = map[y][x];
104     return !(c == '.' || c == 'S' || c == 'E');
105 }
106
```

85 La case de départ. Le joueur est placé en son centre, d'où le 0,5.

104 Trois caractères sont des cases libres.

Des plaques de métal



Figure 3.1. Les trois textures de plaques, telles que le programme les dessine à cette étape. Le mur porte quatre panneaux et des rivets, le sol seize dalles serrées, et le plafond est presque noir.

Dans ce chapitre, vous allez dessiner des plaques de métal avec un joint, un dégradé, un biseau et des rivets, et apprendre à mélanger deux couleurs.

The Keep se passe dans une station, et non dans un château. Les briques laissent la place à des plaques de métal. Une plaque est décrite par une structure `plate`, avec quatre couleurs, le métal, sa teinte en pleine lumière, sa teinte dans l'ombre et la couleur du joint, ainsi que le nombre de panneaux en largeur et en hauteur et la présence de rivets. La texture passe à 128 pixels de côté.

La fonction `make_plating` dessine une texture entière à partir d'une structure `plate`. Un joint creux sépare les panneaux, avec l'ombre du panneau du dessus. Chaque panneau reçoit un dégradé du haut vers le bas, car la lumière vient du plafond, un biseau clair sur ses bords supérieur et gauche et sombre sur les deux autres, et quatre rivets aux coins, chacun avec son côté éclairé. Un grain léger, calculé par la fonction `smooth_noise`, évite les surfaces parfaitement plates.

La fonction `mix` mélange deux couleurs dans une proportion donnée, composante par composante. La fonction `relief` s'en sert pour tirer la couleur du métal vers sa teinte claire ou vers sa teinte sombre, selon le signe d'une valeur de relief. Le sol et le plafond sont dessinés par la même fonction, avec d'autres structures `plate`. La fonction `smooth_noise` lisse le bruit de la deuxième partie. Elle tire une valeur par carré de 8 pixels et glisse d'une valeur à la suivante avec une courbe qui part et arrive à plat, sans quoi la grille des carrés se verrait.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE RENDER H LIGNES 3 A 12

```
3
4 #include "world.h"
5
6 // a colour is three bytes in one number: where each one starts, and what
  it
7 // takes to keep a single one. every blend of colours starts here
8 #define RED_SHIFT 16
9 #define GREEN_SHIFT 8
10 #define CHANNEL 0xff
11
12 // how wide the view is: 0.66 against a unit direction is about 66
  degrees
```

INCLUDE TEXTURE H LIGNES 4 A 27

```
4 // a texture is a square of pixels drawn once at startup. a power of two,
  so
5 // wrapping around it is a bitwise and instead of a division
6 #define TEX_SIZE 128
7 #define TEX_MASK (TEX_SIZE - 1)
8
9 // a station wall is panels, not bricks: a hollow seam, a bevel lit at
  the
10 // top left, a gradient inside the panel, rivets at the corners
11 struct plate {
12     unsigned int color;           // the sheet metal
13     unsigned int light;          // what it becomes in full light
14     unsigned int dark;           // and in shadow
15     unsigned int seam;           // the hollow joint between two panels
16     int across, down;            // how many panels across, and down
17     int rivets;                  // are there rivets at the corners?
18 };
19
20 #define SEAM_W 3                  // how wide a seam is
21 #define RIVET_IN 6                // how far from the corner a rivet sits
22 #define PLATE_SHEEN 0.34          // the gradient down a panel
23 #define PLATE_EDGE 0.30           // how strong the bevel is
24 #define PLATE_GRAIN 0.10          // and how strong the grain is
25 #define GRAIN_CELL 8              // the size of a patch of grain
26
27 // a wall met on a north-south line keeps this much of its light
```

INCLUDE TEXTURE H LIGNES 33 A 38

```
33
34 double noise(int x, int y);
35 // two colours blended: part = 0 gives the first, 1 gives the second
36 unsigned int mix(unsigned int a, unsigned int b, double part);
37 unsigned int shade(unsigned int color, double light);
38 void make_wall_texture(void);
```

SRC TEXTURE C LIGNES 1 A 5

```
1 #include "texture.h"
2
3 #include "render.h"
4
5 unsigned int wall_texture[TEX_SIZE * TEX_SIZE];
```

SRC TEXTURE C LIGNES 17 A 51

```
17 }
18
```

Ligne par ligne

- 11** Tout ce qui décrit une plaque. Quatre couleurs, deux nombres de panneaux et la présence de rivets.

PARTIE 3 CHAPITRE 3 LE CODE

SUITE

```
19 // noise in patches, not per pixel: one draw per pixel gives salt and
    pepper.
20 // we draw one value every `cell` pixels and slide between them
21 static double smooth_noise(int x, int y, int cell)
22 {
23     int cells = TEX_SIZE / cell;
24     int gx = x / cell, gy = y / cell;
25     double tx = (double)(x % cell) / cell;
26     double ty = (double)(y % cell) / cell;
27     // a curve that leaves and lands flat: without it the grid shows
28     tx = tx * tx * (3.0 - 2.0 * tx);
29     ty = ty * ty * (3.0 - 2.0 * ty);
30     int dx = (gx + 1) % cells, dy = (gy + 1) % cells;
31     double top = noise(gx, gy) * (1.0 - tx) + noise(dx, gy) * tx;
32     double bottom = noise(gx, dy) * (1.0 - tx) + noise(dx, dy) * tx;
33     return top * (1.0 - ty) + bottom * ty;
34 }
35
36 unsigned int mix(unsigned int a, unsigned int b, double part)
37 {
38     if (part < 0.0)
39         part = 0.0;
40     if (part > 1.0)
41         part = 1.0;
42     unsigned int out = 0;
43     for (int d = 0; d <= RED_SHIFT; d += GREEN_SHIFT) {
44         double ca = (a >> d) & CHANNEL, cb = (b >> d) & CHANNEL;
45         out |= (unsigned int)(ca + (cb - ca) * part) << d;
46     }
47     return out;
48 }
49
50 unsigned int shade(unsigned int color, double light)
51 {
```

28 La courbe qui part et arrive à plat. Sans elle, la grille des carrés de bruit se voit.

36 Le mélange de deux couleurs, composante par composante. La boucle parcourt les trois décalages, 0, 8 et 16.

82 Le joint, sur les trois premiers pixels de chaque panneau.

SRC TEXTURE C LIGNES 61 A 145

```
61 }
62
63 // above zero toward the light, below toward the shadow: one grey
    lightened
64 // and darkened gives cardboard, two colours give relief
65 static unsigned int relief(const struct plate *p, double v)
66 {
67     return v > 0.0 ? mix(p->color, p->light, v) : mix(p->color, p->dark,
    -v);
68 }
69
70 // sheet metal: a panel, its seam, its bevel, its rivets. every surface
    in
71 // the level comes out of here, with a panel size and four colours
72 static void make_plating(unsigned int *out, const struct plate *p)
73 {
74     int wide = TEX_SIZE / p->across, high = TEX_SIZE / p->down;
75
76     for (int y = 0; y < TEX_SIZE; y++) {
77         for (int x = 0; x < TEX_SIZE; x++) {
78             int dx = x % wide, dy = y % high;
79
80             // the seam is hollow, and the panel above casts its
            // shadow into it. without it the panels float
81             if (dx < SEAM_W || dy < SEAM_W) {
82                 double hollow = (dx == SEAM_W - 1 || dy == SEAM_W - 1)
                    ? 0.15 : 0.70;
83                 out[y * TEX_SIZE + x] = mix(p->seam, p->dark, hollow);
84                 continue;
85             }
86
87             // the light comes from the ceiling: the top of the
```

PARTIE 3 CHAPITRE 3 LE CODE

SUITE

```
90         // panel takes it, the bottom much less
91         double v = PLATE_SHEEN * (0.5 - (double)dy / high);
92         v += PLATE_GRAIN * (smooth_noise(x, y, GRAIN_CELL) - 0.5);
93
94         // the bevel: the edge in the light at the top and the
95         // left, in shadow at the bottom and the right
96         if (dx == SEAM_W || dy == SEAM_W)
97             v += PLATE_EDGE;
98         else if (dx == wide - 1 || dy == high - 1)
99             v -= PLATE_EDGE;
100
101         // four rivets to a panel, each with its own lit side:
102         // that is what makes one believe the metal
103         if (p->rivets) {
104             int rx = (dx < wide / 2 ? dx : wide - 1 - dx) - RIVET_IN;
105             int ry = (dy < high / 2 ? dy : high - 1 - dy) - RIVET_IN;
106             if (rx * rx + ry * ry <= 2)
107                 v += rx + ry < 0 ? 0.60 : -0.40;
108         }
109         out[y * TEX_SIZE + x] = relief(p, v);
110     }
111 }
112 }
113
114 // the plates of the station: four colours and two numbers each, and that
115 // is
116 // all that separates the gangway from the deck underfoot
117 static const struct plate HULL = {
118     .color = 0x424b59, .light = 0x9aa9bb, .dark = 0x141821,
119     .seam = 0x0d1016, .across = 2, .down = 2, .rivets = 1,
120 };
121 static const struct plate DECK = {
122     .color = 0x2f3540, .light = 0x6d7b8c, .dark = 0x0f1218,
123     .seam = 0x090b10, .across = 4, .down = 4, .rivets = 0,
124 };
125 static const struct plate ROOF = {
126     .color = 0x232833, .light = 0x525d6d, .dark = 0x080a0e,
127     .seam = 0x06080b, .across = 2, .down = 2, .rivets = 0,
128 };
129 // the ordinary wall of the gangway
130 void make_wall_texture(void)
131 {
132     make_plating(wall_texture, &HULL);
133 }
134
135 // the floor: dark tiles, tight, no rivets
136 void make_floor_texture(void)
137 {
138     make_plating(floor_texture, &DECK);
139 }
140
141 // the ceiling: nearly black, and that is on purpose
142 void make_ceiling_texture(void)
143 {
144     make_plating(ceiling_texture, &ROOF);
145 }
```

91 Le dégradé, positif en haut du panneau et négatif en bas.

96 Le biseau clair sur les bords du haut et de gauche.

106 Un rivet, un petit cercle de rayon environ 1,4 pixel près de chaque coin.

116 La plaque des murs, quatre panneaux et des rivets.



Figure 3.2. Le couloir en plaques de métal. Les briques ont disparu.

VOCABULAIRE

Un **biseau** est le bord incliné d'un panneau. Il est clair du côté de la lumière et sombre du côté opposé. Une **interpolation** est le calcul d'une valeur intermédiaire entre deux valeurs connues. La fonction **mix** interpole entre deux couleurs.

Plusieurs sortes de murs



Figure 4.1. Les trois textures de mur. Le fichier du niveau choisit l'une d'elles pour chaque case de mur, par un chiffre.

Dans ce chapitre, vous allez donner plusieurs sortes de murs au niveau, choisies par un chiffre dans le fichier.

Un niveau dont tous les murs se ressemblent est difficile à lire. Nous donnons au jeu trois sortes de murs. Le tableau `wall_texture` devient un tableau de trois textures, et le fichier du niveau choisit la texture de chaque mur par un chiffre, 1 pour la première, 2 pour la deuxième et 3 pour la troisième. Tout autre caractère de mur, comme le dièse, donne la première, et un ancien niveau reste lisible.

La fonction `wall_kind` lit ce chiffre dans la grille. Le résultat du lancer de rayons, la structure `hit`, reçoit un champ `kind` qui transporte cette information jusqu'à la fonction de dessin. Les trois textures sont trois structures `plate`, avec des panneaux plus ou moins grands et des couleurs différentes.

EXERCICE

Ajoutez une quatrième plaque avec ses propres couleurs, portez `WALL_KINDS` à 4, et utilisez le chiffre 4 dans le fichier du niveau.

Plusieurs sortes de murs, suite

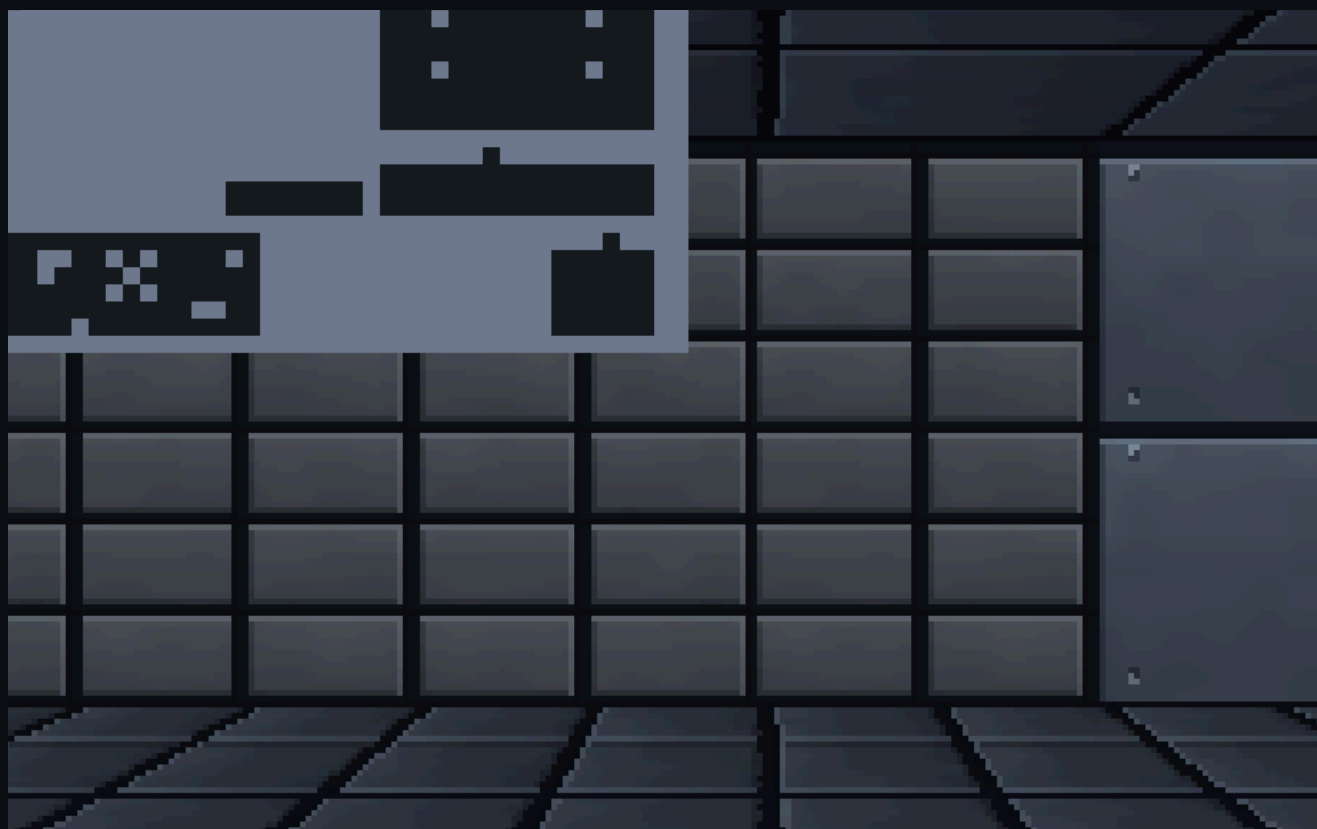


Figure 4.2. Deux sortes de murs dans la même image, la cloison à petits panneaux et la coque à rivets.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE TEXTURE H LIGNES 28 A 36

```
28 #define SIDE_LIGHT 0.68
29
30 // several kinds of wall, so one room is not the next: a panel size and
  four
31 // colours, still drawn in code
32 #define WALL_KINDS 3
33
34 extern unsigned int wall_texture[WALL_KINDS][TEX_SIZE * TEX_SIZE];
35 extern unsigned int floor_texture[TEX_SIZE * TEX_SIZE];
36 extern unsigned int ceiling_texture[TEX_SIZE * TEX_SIZE];
```

INCLUDE TEXTURE H LIGNES 40 A 44

```
40 unsigned int mix(unsigned int a, unsigned int b, double part);
41 unsigned int shade(unsigned int color, double light);
42 void make_wall_textures(void);
43 void make_floor_texture(void);
44 void make_ceiling_texture(void);
```

INCLUDE WORLD H LIGNES 22 A 26

```
22 double *dir_x, double *dir_y;
23 int is_wall(int x, int y);
24 int wall_kind(int x, int y);
25 void move_player(struct player *player, double step_x, double step_y);
26 void turn_player(struct player *player, double angle);
```

SRC MAIN C LIGNES 30 A 34

```
30
31 // the textures cost nothing to keep and everything to draw: once,
  here
32 make_wall_textures();
33 make_floor_texture();
34 make_ceiling_texture();
```

SRC RENDER C LIGNES 15 A 19

```
15 // one column of the wall, read down one column of the texture
16 static void draw_wall_column(int x, int top, int height, int tex_x,
17 double light, int dark, int kind)
18 {
19 double step = (double)TEX_SIZE / height;
```

SRC RENDER C LIGNES 34 A 38

```
34
35 for (; y < bottom; y++) {
36 unsigned int color = wall_texture[kind][((int)tex_y & TEX_MASK) *
  TEX_SIZE + tex_x];
37 view[y * VIEW_WIDTH + x] = shade(color, dark ? light * SIDE_LIGHT
  : light);
38 tex_y += step;
```

SRC RENDER C LIGNES 92 A 97

```
92 double wall_x;
93 int side;
94 // which of the walls it is: the level file says so, by a digit
95 int kind;
96 };
97
```

SRC RENDER C LIGNES 140 A 145

```
140 wall_x -= floor(wall_x);
141
```

Ligne par ligne

- 32** Le nombre de sortes de murs.
- 34** Un tableau de textures, une par sorte de mur.
- 36** La lecture se fait dans la texture de la bonne sorte.

PARTIE 3 CHAPITRE 4 LE CODE

SUITE

```
142     struct hit hit = { .distance = distance, .wall_x = wall_x, .side =
      side,
143         .kind = wall_kind(map_x, map_y) };
144     return hit;
145 }
```

SRC RENDER C LIGNES 168 A 172

```
168     // disappears
169     draw_wall_column(x, top, height, tex_x, fog(hit.distance),
170         hit.side == 1, hit.kind);
171 }
172 }
```

SRC TEXTURE C LIGNES 3 A 7

```
3 #include "render.h"
4
5 unsigned int wall_texture[WALL_KINDS][TEX_SIZE * TEX_SIZE];
6 unsigned int floor_texture[TEX_SIZE * TEX_SIZE];
7 unsigned int ceiling_texture[TEX_SIZE * TEX_SIZE];
```

SRC TEXTURE C LIGNES 118 A 129

```
118     .seam = 0xd1016, .across = 2, .down = 2, .rivets = 1,
119 };
120 static const struct plate BULK = {
121     .color = 0x474c54, .light = 0x8b939d, .dark = 0x12151a,
122     .seam = 0xc0e12, .across = 4, .down = 6, .rivets = 0,
123 };
124 static const struct plate PARTITION = {
125     .color = 0x39404c, .light = 0x828d9c, .dark = 0x101319,
126     .seam = 0xa0d12, .across = 2, .down = 4, .rivets = 1,
127 };
128 static const struct plate DECK = {
129     .color = 0x2f3540, .light = 0x6d7b8c, .dark = 0xf1218,
```

SRC TEXTURE C LIGNES 136 A 144

```
136
137 // the ordinary wall of the gangway
138 void make_wall_textures(void)
139 {
140     make_plating(wall_texture[0], &BULK);
141     make_plating(wall_texture[1], &HULL);
142     make_plating(wall_texture[2], &PARTITION);
143 }
144
```

SRC WORLD C LIGNES 1 A 4

```
1 #include "world.h"
2 #include "texture.h"
3
4 #include <math.h>
```

SRC WORLD C LIGNES 95 A 111

```
95 }
96
97 // the level file says which wall it is: '1' is the first, '2' the second
   and
98 // so on. anything else is the first, so an old level still reads.
99 int wall_kind(int x, int y)
100 {
101     if (x < 0 || y < 0 || x >= map_width || y >= map_height
102         || x >= (int)strlen(map[y]))
103         return 0;
104     char c = map[y][x];
105     if (c >= '1' && c < '1' + WALL_KINDS)
106         return c - '1';
107     return 0;
108 }
109
110 int is_wall(int x, int y)
```

143 Le rayon rapporte la sorte du mur qu'il a touché.

105 Le chiffre du fichier, converti en numéro de texture. Le caractère 1 donne 0.

PARTIE 3 CHAPITRE 4 LE CODE

SUITE

111 {

Les portes

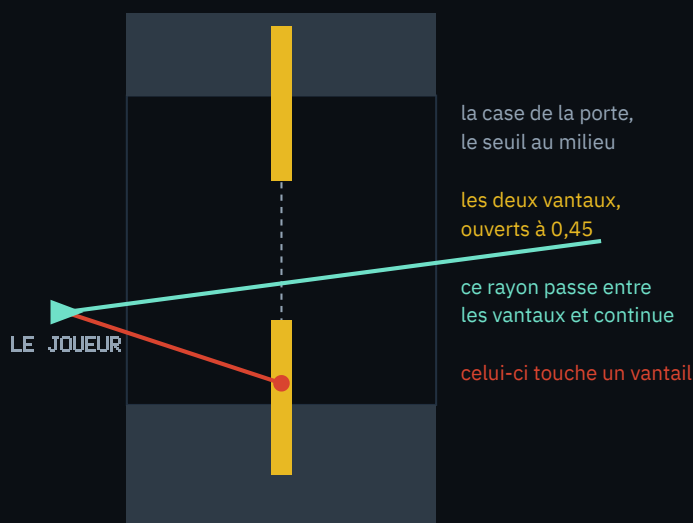


Figure 5.1. Une porte vue de dessus, ouverte à 0,45. Les deux vantaux, en jaune, ont glissé dans les murs voisins. Le rayon vert passe entre les vantaux et continue. Le rayon rouge touche un vantail et s'arrête au seuil.

Dans ce chapitre, vous allez écrire des portes qui s'ouvrent en deux, avec la géométrie qui fait passer le rayon entre les vantaux.

Une porte est une case du niveau, marquée par le signe plus, dont l'ouverture varie de 0, fermée, à 1, entièrement ouverte. Le tableau `doors` conserve cette ouverture pour chaque case du niveau. Le joueur pousse une porte avec la touche Espace. La fonction `push_door` cherche une porte devant lui, jusqu'à 2,6 cases de distance, et met son ouverture en marche. La fonction `move_doors` fait avancer toutes les portes en cours d'ouverture, en 1,2 seconde. Une porte ouverte à plus de 0,8 laisse passer le joueur, et ne se referme jamais.

La porte est dessinée comme un mur, avec une texture de deux vantaux séparés par une bande de danger et une rainure centrale. Mais elle ne remplit pas sa case. Elle se trouve au milieu, sur une ligne appelée le seuil, et ses deux vantaux glissent dans les murs de chaque côté. La figure 5.1 montre cette géométrie vue de dessus.

Le lancer de rayons doit en tenir compte. Quand le parcours du rayon entre dans une case de porte, la fonction `door_leaf` calcule le point où le rayon coupe le seuil. Si ce point tombe dans l'espace ouvert entre les deux vantaux, le rayon continue son chemin. Sinon, il s'arrête sur le vantail, et la texture est décalée de la distance dont le vantail a glissé, pour que le dessin bouge avec lui.

L'orientation du seuil dépend des murs voisins. Si les cases au nord et au sud de la porte sont des murs, la porte est prise dans un mur qui va du nord au sud, et le seuil est une ligne verticale au milieu de la case. Sinon, le seuil est horizontal.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

Ligne par ligne

27 Le temps d'ouverture d'une porte, en secondes.

INCLUDE SCREEN H LIGNES 37 A 41

```
37 struct keys {
38     int forward, back, left, right, strafe_left, strafe_right, quit;
39     int push;          // a press on space, waiting to be read
40 };
41
```

INCLUDE TEXTURE H LIGNES 25 A 46

```
25 #define GRAIN_CELL 8          // the size of a patch of grain
26
27 // the door: two leaves that part, a hazard band, a seam down the middle.
  it
28 // has to read as a door before anyone knows what to do with it
29 #define DOOR_STEEL 0x4d5866
30 #define DOOR_LIGHT 0x9daab8
31 #define DOOR_DARK 0x121620
32 #define DOOR_SEAM 0x0b0e14
33 #define HAZARD 0xd8a13a      // the diagonal stripes
34 #define HAZARD_DARK 0x1b1f27
35 #define DOOR_BAND 22         // how tall the hazard band is
36 #define DOOR_STRIPE 10       // how wide one stripe is
37
38 // a wall met on a north-south line keeps this much of its light
39 #define SIDE_LIGHT 0.68
40
41 // several kinds of wall, so one room is not the next: a panel size and
  four
42 // colours, still drawn in code
43 #define WALL_KINDS 4
44 #define DOOR_TEXTURE 3       // which of the four is the door
45
46 extern unsigned int wall_texture[WALL_KINDS][TEX_SIZE * TEX_SIZE];
```

INCLUDE WORLD H LIGNES 23 A 39

```
23 int is_wall(int x, int y);
24 int wall_kind(int x, int y);
25 // a door takes a second to open, and once it is open this far one walks
26 // between the two leaves
27 #define DOOR_SECONDS 1.2
28 #define DOOR_WALKABLE 0.8
29 // how far a push reaches: one opens a door from where one stands, not
  with
30 // one's nose against the leaf
31 #define DOOR_REACH 2.6
32
33 int is_door(int x, int y);
34 // from 0 (shut) to 1 (both leaves tucked into the wall)
35 double door_at(int x, int y);
36 void push_door(const struct player *player);
37 void move_doors(double elapsed);
38 void move_player(struct player *player, double step_x, double step_y);
39 void turn_player(struct player *player, double angle);
```

SRC MAIN C LIGNES 54 A 63

```
54     screen_read_keys(&screen, &keys);
55
56     if (keys.push) {
57         push_door(&player);
58         keys.push = 0;
```


PARTIE 3 CHAPITRE 5 LE CODE

SUITE

```
59     }
60     move_doors(elapsed);
61
62     // the way out. the level had no end: one walked until one
63     // stopped. standing on it closes the keep behind us.
```

SRC RENDER C LIGNES 86 A 121

```
86 }
87
88 // the door opens in two, and in the middle of its square: the ray passes
89 // between the leaves, and that is how one sees it open instead of vanish
90 static int door_leaf(const struct player *player, double ray_x, double
ray_y,
91     int mx, int my, double *distance, double *along)
92 {
93     // which way does the wall that holds the door run? that is what says
94     // which plane the threshold lies in
95     int north_south = is_wall(mx, my - 1) && is_wall(mx, my + 1);
96     double toward = north_south ? ray_x : ray_y;
97     if (toward == 0.0)
98         return 0; // the ray runs along the threshold
99
100     double d = north_south ? (mx + 0.5 - player->x) / ray_x
101         : (my + 0.5 - player->y) / ray_y;
102     if (d <= 0.0)
103         return 0;
104     double u = north_south ? player->y + d * ray_y - my
105         : player->x + d * ray_x - mx;
106     if (u < 0.0 || u > 1.0)
107         return 0; // it passes beside the threshold
108
109     double gap = u - 0.5;
110     double half_open = door_at(mx, my) / 2.0;
111     if (gap < half_open && -gap < half_open)
112         return 0; // it passes between the two leaves
113
114     *distance = d;
115     // the leaf has slid: the texture slides with it
116     *along = gap < 0.0 ? u + half_open : u - half_open;
117     return 1;
118 }
119
120 // what a ray found: how far it went, which way the wall it met faces,
and
121 // where along that wall it landed
```

SRC RENDER C LIGNES 147 A 151

```
147 // the whole trick: no square is missed, and none is visited twice
148 int side = 0;
149 while (1) {
150     if (side_x < side_y) {
151         side_x += delta_x;
```

SRC RENDER C LIGNES 157 A 178

```
157     side = 1;
158     }
159
160     // a door does not stop a whole square: we ask the
161     // leaf itself, and the ray carries on if it is open
162     if (is_door(map_x, map_y)) {
163         double d, along;
164         if (!door_leaf(player, ray_x, ray_y, map_x, map_y,
```

95 L'orientation de la porte, déduite des murs voisins.

100 La distance à laquelle le rayon coupe le seuil, au milieu de la case.

111 Le point d'impact est dans l'espace entre les deux vantaux. Le rayon continue.

116 La position sur le vantail, décalée de ce qu'il a glissé, pour que la texture le suive.

PARTIE 3 CHAPITRE 5 LE CODE

SUITE

```
165             &d, &along))
166             continue;
167             int right = is_wall(map_x, map_y - 1)
168             && is_wall(map_x, map_y + 1);
169             struct hit door = {
170                 .distance = d < NEAR_CLIP ? NEAR_CLIP : d,
171                 .wall_x = along, .side = right ? 0 : 1,
172                 .kind = DOOR_TEXTURE };
173             return door;
174         }
175         if (is_wall(map_x, map_y))
176             break;
177     }
178 }
```

153 Les rayures de la bande de danger, inclinées dans un sens sur le vantail gauche et dans l'autre sur le droit.

SRC SCREEN C LIGNES 174 A 183

```
174         case XK_a:      keys->strafe_left = down; break;
175         case XK_d:      keys->strafe_right = down; break;
176         case XK_space:
177             // a short press must not be lost: press and
178             // release can fall in the same frame
179             if (down)
180                 keys->push = 1;
181             break;
182         case XK_F11:
183             if (down)
```

SRC TEXTURE C LIGNES 136 A 177

```
136
137 // the ordinary wall of the gangway
138 // the door opens in two, and the drawing says so before it ever moves
139 static void make_door(unsigned int *out)
140 {
141     static const struct plate LEAF = {
142         .color = DOOR_STEEL, .light = DOOR_LIGHT, .dark = DOOR_DARK,
143         .seam = DOOR_SEAM, .across = 2, .down = 3, .rivets = 1,
144     };
145     make_plating(out, &LEAF);
146
147     int middle = TEX_SIZE / 2, band = TEX_SIZE / 2 - DOOR_BAND / 2;
148     for (int y = 0; y < TEX_SIZE; y++) {
149         for (int x = 0; x < TEX_SIZE; x++) {
150             // the hazard band: diagonals, and they turn the
151             // other way at the middle, so the leaves read apart
152             if (y >= band && y < band + DOOR_BAND) {
153                 int slant = x < middle ? x + y : x - y + TEX_SIZE;
154                 unsigned int c = (slant / DOOR_STRIPE) % 2
155                     ? HAZARD : HAZARD_DARK;
156                 double wear = 0.25 * smooth_noise(x, y, 4);
157                 out[y * TEX_SIZE + x] = mix(c, DOOR_DARK, wear);
158             }
159             // the centre seam: this is where the two leaves meet
160             int from_middle = x - middle < 0 ? middle - x : x - middle;
161             if (from_middle < 2)
162                 out[y * TEX_SIZE + x] = DOOR_SEAM;
163             else if (from_middle < 4)
164                 out[y * TEX_SIZE + x] = mix(out[y * TEX_SIZE + x],
165                     DOOR_DARK, 0.55);
166         }
167     }
168 }
169
170 void make_wall_textures(void)
171 {
172     make_plating(wall_texture[0], &BULK);
173     make_plating(wall_texture[1], &HULL);
174     make_plating(wall_texture[2], &PARTITION);
175 }
```

PARTIE 3 CHAPITRE 5 LE CODE

SUITE

```
175     make_door(wall_texture[3]);
176 }
177
```

SRC WORLD C LIGNES 11 A 18

```
11 char **map;
12 int map_width, map_height;
13
14 // how far each door has opened, one number per square: a door is a
  place,
15 // not an object
16 static double *doors;
17
18 // what the level file can hold. a wall is anything that is not floor.
```

SRC WORLD C LIGNES 54 A 63

```
54 }
55 fclose(f);
56 if (map_height > 0) {
57     free(doors);
58     doors = calloc((size_t)map_width * map_height, sizeof(*doors));
59     if (!doors)
60         return 0;
61 }
62 return map_height > 0;
63 }
```

SRC WORLD C LIGNES 105 A 155

```
105 }
106
107 // we look a few steps ahead, not at arm's length: nobody walks into a
  door
108 // to open it, one pushes it from where one stands
109 void push_door(const struct player *player)
110 {
111     for (double d = 0.6; d <= DOOR_REACH; d += 0.4) {
112         int x = (int)(player->x + player->dir_x * d);
113         int y = (int)(player->y + player->dir_y * d);
114         if (is_door(x, y)) {
115             if (doors[y * map_width + x] == 0.0)
116                 doors[y * map_width + x] = 0.001;
117             return;
118         }
119         if (is_wall(x, y))
120             return; // a wall between us: nothing to push
121     }
122 }
123
124 // the two leaves take a second to part, and they never close again:
  coming
125 // back this way should be a short cut, not a chore
126 void move_doors(double elapsed)
127 {
128     if (!doors)
129         return;
130     for (int i = 0; i < map_width * map_height; i++)
131         if (doors[i] > 0.0 && doors[i] < 1.0) {
132             doors[i] += elapsed / DOOR_SECONDS;
133             if (doors[i] > 1.0)
134                 doors[i] = 1.0;
135         }
136 }
137
138 int is_door(int x, int y)
139 {
140     if (x < 0 || y < 0 || x >= map_width || y >= map_height
141         || x >= (int)strlen(map[y]))
```

16 L'ouverture de chaque case du niveau, entre 0 et 1. Une case qui n'est pas une porte reste à 0.

111 La recherche d'une porte devant le joueur, par pas de 0,4 case, jusqu'à 2,6 cases.

132 Chaque porte en cours d'ouverture avance en proportion du temps écoulé.

PARTIE 3 CHAPITRE 5 LE CODE

SUITE

```
142         return 0;
143     return map[y][x] == '+';
144 }
145
146 double door_at(int x, int y)
147 {
148     if (x < 0 || y < 0 || x >= map_width || y >= map_height
149         || !doors)
150         return 0.0;
151     return doors[y * map_width + x];
152 }
153
154 // the level file says which wall it is: '1' is the first, '2' the second
155 // and
156 // so on. anything else is the first, so an old level still reads.
```

176 Une porte est un mur tant qu'elle n'est pas ouverte à 0,8.

SRC WORLD C LIGNES 173 A 178

```
173     // starts, and where the way out is. only a wall stops anyone.
174     char c = map[y][x];
175     if (c == '+')
176         return door_at(x, y) < DOOR_WALKABLE;
177     return !(c == '.' || c == 'S' || c == 'E');
178 }
```



Figure 5.2. Une porte fermée au bout du couloir, avec sa bande de danger. La touche Espace l'ouvre en deux.

VOCABULAIRE

Le seuil est la ligne au milieu de la case de la porte sur laquelle les vantaux se trouvent. **Un vantail** est l'une des deux moitiés de la porte. Chacune glisse dans le mur de son côté.

Les néons



Figure 6.1. La texture du mur qui porte un néon, et celle du plafond. Le tube est blanc au centre, vert au bord, et son halo s'étend sur le métal autour.

Dans ce chapitre, vous allez dessiner un néon avec son halo et le poser sur les murs et au plafond.

Une station a besoin de lumières. Nous dessinons un néon, un tube dans son boîtier de métal, entouré d'un halo. La fonction `neon_strip` le peint sur une texture existante, à une position et avec des dimensions données. Le tube est blanc au centre et vert au bord, et le halo s'atténue avec le carré de la distance au tube. Sans le halo, le tube ne serait qu'un rectangle blanc collé sur le mur.

Le fichier du niveau place les néons par la lettre T sur une case de mur. Cette case reçoit la texture de la coque avec un tube vertical au milieu. Le plafond reçoit lui aussi un tube, horizontal, au centre de sa texture. Comme la texture du plafond se répète à chaque case, une ligne de néons court au plafond de tous les couloirs, et cette ligne qui s'éloigne donne aux couloirs leur perspective.

Les néons, suite



Figure 6.2. Les néons sur les murs et au plafond. Ils sont dessinés, mais n'éclairent pas encore.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE TEXTURE H LIGNES 36 A 54

```
36 #define DOOR_STRIPE 10          // how wide one stripe is
37
38 // the neon is the only thing in the level that shines, and it is what
   gives
39 // its colour to everything else
40 #define NEON_CORE 0xeaffffb
41 #define NEON_TUBE 0x9ff0e4
42 #define NEON_EDGE 0x3aa896
43 #define HOUSING 0x1a1f28        // the metal housing that holds it
44
45 // a wall met on a north-south line keeps this much of its light
46 #define SIDE_LIGHT 0.68
47
48 // several kinds of wall, so one room is not the next: a panel size and
   four
49 // colours, still drawn in code
50 #define WALL_KINDS 5
51 #define DOOR_TEXTURE 3          // which of the five is the door
52 #define LAMP_TEXTURE 4          // and the wall that carries a strip
53
54 extern unsigned int wall_texture[WALL_KINDS][TEX_SIZE * TEX_SIZE];
```

INCLUDE WORLD H LIGNES 23 A 27

```
23 int is_wall(int x, int y);
24 int wall_kind(int x, int y);
25 int is_lamp(int x, int y);
26 // a door takes a second to open, and once it is open this far one walks
27 // between the two leaves
```

SRC TEXTURE C LIGNES 136 A 176

```
136
137 // the ordinary wall of the gangway
138 // a neon strip in its housing: the housing, the tube, and the glow
   around
139 // it. without the glow the tube is a white rectangle stuck on the wall
140 static void neon_strip(unsigned int *out, int cx, int cy, int half_w,
141     int half_h)
142 {
143     for (int y = 0; y < TEX_SIZE; y++)
144         for (int x = 0; x < TEX_SIZE; x++) {
145             int dx = x - cx < 0 ? cx - x : x - cx;
146             int dy = y - cy < 0 ? cy - y : y - cy;
147             if (dx <= half_w + 4 && dy <= half_h + 4) {
148                 // the housing, with its lit edge
149                 out[y * TEX_SIZE + x] = mix(HOUSING, 0xffffffff,
150                     x - cx <= -half_w - 3 || y - cy <= -half_h - 3
151                     ? 0.25 : 0.0);
152             }
153             if (dx <= half_w && dy <= half_h) {
154                 // the tube: white at the core, teal at the edge
155                 double edge = (double)dx / (half_w + 1);
156                 out[y * TEX_SIZE + x] = edge < 0.45 ? NEON_CORE
157                     : edge < 0.8 ? NEON_TUBE : NEON_EDGE;
158             }
159         }
160     // and the glow, dying with the square of the distance
161     for (int y = 0; y < TEX_SIZE; y++)
```

Ligne par ligne

- 140** Le dessin d'un néon sur une texture, centré en (cx ; cy), avec sa demi-largeur et sa demi-hauteur.
- 155** La distance au centre du tube, entre 0 et 1, qui choisit entre le cœur, le tube et le bord.

PARTIE 3 CHAPITRE 6 LE CODE

SUITE

```
162         for (int x = 0; x < TEX_SIZE; x++) {
163             double px = (x - cx) / (half_w + 34.0);
164             double py = (y - cy) / (half_h + 34.0);
165             double d = px * px + py * py;
166             int dx = x - cx < 0 ? cx - x : x - cx;
167             int dy = y - cy < 0 ? cy - y : y - cy;
168             if (d >= 1.0 || (dx <= half_w + 4 && dy <= half_h + 4))
169                 continue;
170             out[y * TEX_SIZE + x] = mix(out[y * TEX_SIZE + x],
171                 NEON_TUBE, 0.40 * (1.0 - d) * (1.0 - d));
172         }
173     }
174
175     // the door opens in two, and the drawing says so before it ever moves
176     static void make_door(unsigned int *out)
```

165 Le carré de la distance au tube, pour le halo. Au-delà de 1, il n'y a plus de halo.

211 Le tube du mur, vertical, 8 pixels de large et 68 de haut.

234 Le tube du plafond, horizontal.

153 La lettre T du fichier marque un mur qui porte un néon.

SRC TEXTURE C LIGNES 205 A 215

```
205 }
206
207 // the wall that carries a strip: sheet metal, with the neon in it
208 static void make_lamp_wall(unsigned int *out, const struct plate *p)
209 {
210     make_plating(out, p);
211     neon_strip(out, TEX_SIZE / 2, TEX_SIZE / 2, 4, 34);
212 }
213
214 void make_wall_textures(void)
215 {
```

SRC TEXTURE C LIGNES 218 A 222

```
218     make_plating(wall_texture[2], &PARTITION);
219     make_door(wall_texture[3]);
220     make_lamp_wall(wall_texture[4], &HULL);
221 }
222
```

SRC TEXTURE C LIGNES 227 A 235

```
227 }
228
229 // the ceiling carries the strips: a line of neon running away from us
230 // is what gives a corridor its perspective
231 void make_ceiling_texture(void)
232 {
233     make_plating(ceiling_texture, &ROOF);
234     neon_strip(ceiling_texture, TEX_SIZE / 2, TEX_SIZE / 2, 34, 5);
235 }
```

SRC WORLD C LIGNES 144 A 157

```
144 }
145
146 // a strip sits on a wall: the file puts a letter where the wall would
147 // be,
148 // and that letter carries the neon
149 int is_lamp(int x, int y)
150 {
151     if (x < 0 || y < 0 || x >= map_width || y >= map_height
152         || x >= (int)strlen(map[y]))
153         return 0;
154     return map[y][x] == 'T';
155 }
156
157 double door_at(int x, int y)
158 {
```

SRC WORLD C LIGNES 170 A 175

```
170     return 0;
171     char c = map[y][x];
172     if (c == 'T')
173         return LAMP_TEXTURE;
```

PARTIE 3 CHAPITRE 6 LE CODE

SUITE

```
174     if (c >= '1' && c < '1' + WALL_KINDS)
175         return c - '1';
```

La lumière vient du niveau

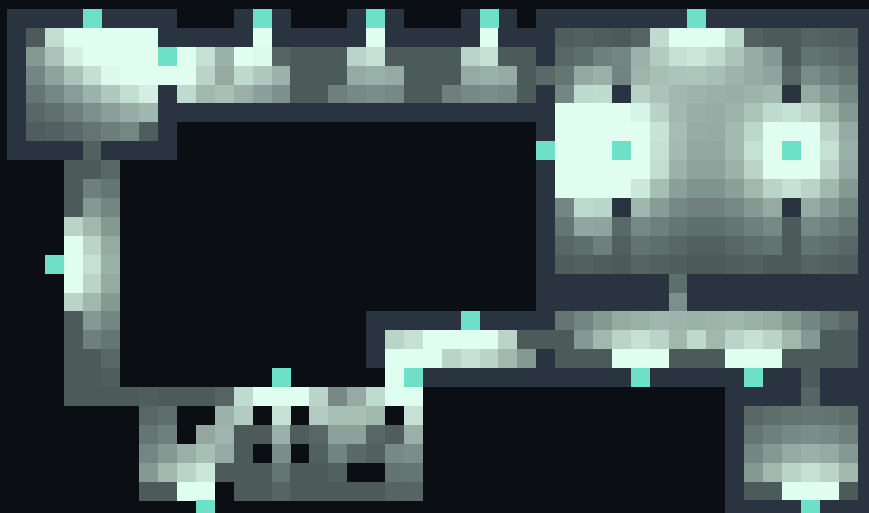


Figure 7.1. La carte de lumière du niveau, calculée comme le fait la fonction `light_map`. Les cases claires sont les plus éclairées. Les carrés verts sont les néons posés sur les murs. La lumière ne traverse pas les murs.

Dans ce chapitre, vous allez calculer une carte de lumière à partir des néons du niveau, et donner une couleur à la lumière.

Jusqu'ici, la brume assombrissait tout de la même façon, en fonction de la seule distance au joueur. Nous remplaçons la brume par une lumière qui vient des néons du niveau. Le nouveau fichier `light.c` calcule, au chargement, la quantité de lumière reçue par chaque case du niveau. C'est la carte de lumière, un tableau `bright` avec un nombre par case.

Chaque case reçoit d'abord une lumière de base de 0,30, celle des néons du plafond, présents partout. Puis chaque néon posé sur un mur ajoute de la lumière aux cases situées à moins de 7 cases de lui, d'autant plus qu'elles sont proches, et seulement si aucun mur ne se trouve entre le néon et la case. La fonction `sees` vérifie cette ligne de vue en parcourant les cases intermédiaires. Le résultat est la figure 7.1, des flaques de lumière autour des néons et de la pénombre entre elles.

Au moment du dessin, la fonction `lit_at` lit cette carte entre les cases, en mélangeant les valeurs des quatre cases voisines du point demandé. Sans ce mélange, le sol montrerait des rectangles de lumière aux bords nets.

Le joueur porte aussi sa propre lampe. La fonction `lamp` donne sa lumière en fonction de la distance, selon la courbe de la figure 7.2. Elle tombe avec le carré de la distance, s'éteint sur le dernier quart de sa portée, et ne dépasse jamais 7 cases. Au-delà, c'est le noir. Le sol reçoit la lampe portée plus la lumière de sa case. Un mur reçoit la lumière de la case d'où on le regarde, réduite s'il est vu de biais, car une surface vue de face prend la lumière et une surface vue de profil beaucoup moins.

Un mur n'est pas non plus éclairé de la même façon du haut en bas. La fonction `at_height` donne le plus de lumière à la hauteur du néon, à 45 pour cent de la hauteur du mur, et en retire vers le plafond et vers le sol. Sans cette variation, une colonne de mur serait une bande plate. Enfin, les néons vivent. La fonction `lamp_flicker` fait varier leur intensité de quelques centièmes, avec deux ondes de fréquences sans rapport, pour que l'œil ne trouve pas de rythme.

La lampe portée par le joueur

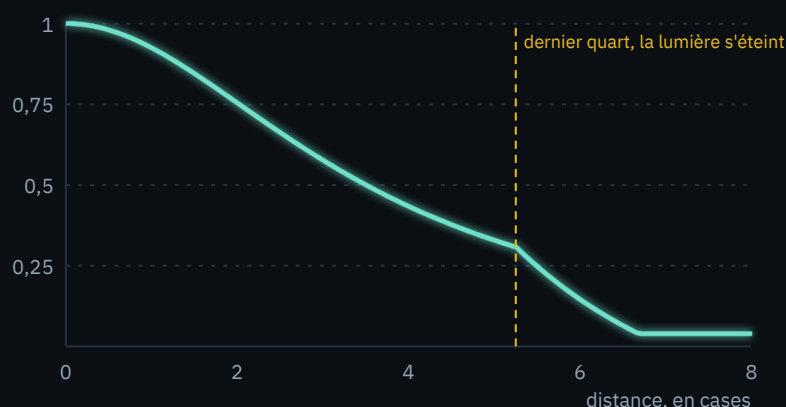


Figure 7.2. La lumière de la lampe portée en fonction de la distance, en cases. Elle décroît avec le carré de la distance et s'éteint sur le dernier quart de sa portée de 7 cases.

La lumière d'une lampe décroît avec le carré de la distance, comme dans la réalité. À une distance d , avec une portée de 7 cases, la fonction `Lamp` calcule d'abord la fraction de portée $p = d \div 7$, puis la lumière $1 \div (1 + 4 p^2)$. À 1 case, la lumière vaut 0,92. À 3,5 cases, à mi-portée, elle vaut 0,5. À 7 cases, elle vaudrait 0,2, mais la fonction l'éteint avant.

Sur le dernier quart de la portée, entre 5,25 et 7 cases, la lumière est multipliée par un facteur qui va de 1 à 0. Sans cette extinction progressive, le bord de la lumière serait un anneau net à 7 cases du joueur, et l'œil le verrait aussitôt. Au-delà de la portée, la lumière garde une valeur plancher de 0,04, qui empêche le noir absolu.

La lampe portée ne fait pas tout. Elle est multipliée par 0,30 avant d'être ajoutée à la lumière de la case. Elle sert à garder les pieds du joueur hors de l'obscurité, et ce sont les néons du niveau qui font l'image.

EXERCICE

Calculez la lumière de la lampe à 2 cases. La fraction p vaut $2 \div 7 = 0,286$, son carré 0,082, et la lumière $1 \div (1 + 0,327) = 0,75$.

La couleur de la lumière

CANAL	PLAQUE	PART DE LUMIERE	RESULTAT
rouge	66	$0,5 \times 242 + 0,11 \times 44 = 125,8$	$66 \times 125,8 \div 255 = 32$
vert	75	$0,5 \times 255 + 0,11 \times 74 = 135,6$	$75 \times 135,6 \div 255 = 39$
bleu	89	$0,5 \times 252 + 0,11 \times 128 = 140,1$	$89 \times 140,1 \div 255 = 48$

Tableau 7.1. La plaque des murs sous une demi-part de néon et la pénombre. La couleur du néon est `0xf2fffc`, soit rouge 242, vert 255 et bleu 252, et celle de la nuit `0x2c4a80`, soit rouge 44, vert 74 et bleu 128.

Jusqu'ici, la lumière ne faisait qu'assombrir. La fonction `shade` multipliait les trois composantes d'une couleur par le même facteur. Une plaque éclairée et la même plaque dans l'ombre avaient la même couleur, plus ou moins sombre. Une vraie lumière a une couleur. Le néon est d'un blanc à peine teinté de vert, et l'ombre entre deux néons est franchement bleue.

La fonction `tint` reçoit deux quantités, la part de néon et la part de pénombre. Pour chaque composante, elle calcule la lumière reçue, la part de néon multipliée par la composante de la couleur du néon plus la part de pénombre multipliée par la composante de la couleur de la nuit. Puis elle multiplie la composante de la plaque par cette lumière et divise par 255.

Le tableau 7.1 fait le calcul pour la plaque des murs, de couleur `0x424b59`, avec une part de néon de 0,5 et une part de pénombre de 0,11. La plaque prend la teinte du néon, et le bleu de la nuit, plus fort dans la troisième colonne, reste présent dans l'ombre.

EXERCICE

Refaites le calcul avec une part de néon de 0. Il ne reste que la nuit, et la plaque devient bleue et très sombre, rouge 1, vert 3 et bleu 6.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE LIGHT H LIGNES 1 A 50

```

1  #ifndef LIGHT_H
2  #define LIGHT_H
3
4  // the light comes from the strips set in the level: pools of it,
   darkness
5  // between them, and enough to find one's way.
6
7  // how far the lamp we carry reaches, in squares. beyond that it is black
8  #define LAMP_REACH  7.0
9  // what is left of it right at the edge
10 #define LAMP_FLOOR   0.04
11 // a neon pulses, very little and very fast
12 #define LAMP_HUM     0.035
13 #define LAMP_HZ      9.0
14
15 // a surface facing us takes the light, one seen edge on does not:
   without
16 // that, everything close by shines like a glowing camera
17 #define LAMP_AMBIENT 0.30    // what a surface takes even facing away
18
19 // how many squares a strip on a wall reaches
20 #define LAMP_ON_WALL 7.0
21 // the ceiling strips run everywhere: they give every square its base
   light,
22 // and the ones on the walls make the pools
23 #define STRIP_LIGHT  0.30
24 // the blue gloom behind it all, faint: the neon is what makes the
   picture
25 #define GLOOM        0.11
26 // the glow we carry ourselves, faint: it keeps us out of pure black
27 #define CARRIED      0.30
28 // the ceiling takes less than the floor: a strip lights downward
29 #define CEILING_PART 0.45
30
31 // a strip hangs at head height: with no height a column of wall is lit
   the
32 // same from floor to ceiling, and comes out flat
33 #define LAMP_HEIGHT  0.45    // where the strip hangs, 0 at the wall
   top
34 #define UNDER_ROOF  1.25    // how much the top of the wall loses
35 #define AT_THE_FOOT  0.52    // and how much its foot loses
36
37 // the light map of the level, worked out once at load time
38 void light_map(void);
39 double lit_here(int x, int y);
40 // the same, continuous: we read between the squares
41 double lit_at(double x, double y);
42
43 // what a point on a wall takes by its height, 0 at the top, 1 at the
   foot
44 double at_height(double v);
45 // what the carried lamp gives at that distance, at this instant
46 double lamp(double distance);
47 // the neon is alive: called once a frame, with the clock
48 void lamp_flicker(double seconds);
49
50 #endif

```

PARTIE 3 CHAPITRE 7 LE CODE

SUITE

INCLUDE TEXTURE H LIGNES 43 A 56

```
43 #define HOUSING    0x1a1f28    // the metal housing that holds it
44
45 // the colour of light: a plate under a neon and the same plate in shadow
46 // differ by colour, not only by how much light they take
47 #define LIGHT_LAMP  0xf2fffc    // a white barely turned teal
48 // and the shadow is frankly blue: the contrast between the two is what
49 // makes the picture, not the difference in brightness
50 #define LIGHT_NIGHT 0x2c4a80
51
52 // the same colour, taken under this much neon and this much gloom
53 unsigned int tint(unsigned int color, double lamp, double night);
54
55 // a wall met on a north-south line keeps this much of its light
56 #define SIDE_LIGHT  0.68
```

SRC LIGHT C LIGNES 1 A 115

```
1  #include "light.h"
2  #include "world.h"
3
4  #include <math.h>
5  #include <stdlib.h>
6
7  // what the neons are worth at this instant, around 1
8  static double neon = 1.0;
9
10 // two waves at unrelated speeds never repeat: the eye stops finding a
11 // pattern in them.
12 void lamp_flicker(double seconds)
13 {
14     neon = 1.0 + LAMP_HUM * (sin(seconds * LAMP_HZ)
15                             + 0.6 * sin(seconds * LAMP_HZ * 2.37));
16 }
17
18 // the light falls with the square of the distance, like the real thing,
19 // and
20 // stops at LAMP_REACH: beyond it lies the dark, not a grey wall
21 double lamp(double distance)
22 {
23     double part = distance / LAMP_REACH;
24     double light = neon / (1.0 + 4.0 * part * part);
25
26     if (part >= 1.0)
27         return LAMP_FLOOR;
28     // the last quarter fades out, or the edge of the light is a hard
29     // ring
30     if (part > 0.75)
31         light *= (1.0 - part) * 4.0;
32     if (light < LAMP_FLOOR)
33         light = LAMP_FLOOR;
34     if (light > 1.0)
35         light = 1.0;
36     return light;
37 }
38
39 // `v` runs from 0 at the top of the wall to 1 at its foot. the most
40 // light
41 // is at strip height, and it falls away on both sides.
42 double at_height(double v)
43 {
44     double gap = v - LAMP_HEIGHT;
45     double loss = gap < 0.0 ? UNDER_ROOF * -gap : AT_THE_FOOT * gap;
46     return 1.0 - loss;
47 }
```

14

Le scintillement des néons, deux ondes de fréquences sans rapport, d'amplitude 0,035.

23

La lampe portée, qui décroît avec le carré de la fraction de portée.

30

L'extinction sur le dernier quart de la portée.

43

La perte de lumière en s'éloignant de la hauteur du néon, plus forte vers le plafond que vers le sol.

PARTIE 3 CHAPITRE 7 LE CODE

SUITE

```
47 // what each square takes, worked out once at load time: rendering only
48 // has
49 // to read an array.
50 static double *bright;
51 static int sees(int x1, int y1, int x2, int y2)
52 {
53     // a wall between the two, and the strip lights nothing
54     double dx = x2 - x1, dy = y2 - y1;
55     double n = fabs(dx) > fabs(dy) ? fabs(dx) : fabs(dy);
56     if (n < 1.0)
57         return 1;
58     for (double i = 1.0; i < n; i += 1.0) {
59         int x = (int)(x1 + dx * i / n + 0.5);
60         int y = (int)(y1 + dy * i / n + 0.5);
61         if (is_wall(x, y) && !is_door(x, y))
62             return 0;
63     }
64     return 1;
65 }
66
67 void light_map(void)
68 {
69     free(bright);
70     bright = calloc((size_t)map_width * map_height, sizeof(*bright));
71     if (!bright)
72         return;
73     // the base light of the ceiling strips, everywhere at once
74     for (int i = 0; i < map_width * map_height; i++)
75         bright[i] = STRIP_LIGHT;
76     for (int y = 0; y < map_height; y++)
77         for (int x = 0; x < map_width; x++) {
78             if (!is_lamp(x, y))
79                 continue;
80             for (int j = y - LAMP_ON_WALL; j <= y + LAMP_ON_WALL; j++)
81                 for (int i = x - LAMP_ON_WALL; i <= x + LAMP_ON_WALL;
82                     i++) {
83                     if (i < 0 || j < 0 || i >= map_width || j >=
84                         map_height)
85                         continue;
86                     double d = hypot(i - x, j - y);
87                     if (d > LAMP_ON_WALL || !sees(x, y, i, j))
88                         continue;
89                     double part = d / LAMP_ON_WALL;
90                     bright[j * map_width + i] += 1.0 / (1.0 + 3.0 * part
91                         * part)
92                         * (1.0 - part);
93                 }
94             }
95         }
96     // what one square takes from the strips of the level
97     double lit_here(int x, int y)
98     {
99         if (!bright || x < 0 || y < 0 || x >= map_width || y >= map_height)
100             return 0.0;
101         double v = bright[y * map_width + x] * neon;
102         return v > 1.0 ? 1.0 : v;
103     }
104     // light does not stop at the edge of a square: we read between the four
105     // neighbours, or the floor and the walls get hard rectangles
106     double lit_at(double x, double y)
107     {
108         double fx = x - 0.5, fy = y - 0.5;
109         int x0 = (int)floor(fx), y0 = (int)floor(fy);
```

51 La ligne de vue entre deux cases. Les cases intermédiaires sont parcourues une par une.

88 La lumière qu'un néon ajoute à une case, selon sa distance.

105 La lecture de la carte entre les cases, un mélange des quatre voisines.

PARTIE 3 CHAPITRE 7 LE CODE

SUITE

```
109     double ax = fx - x0, ay = fy - y0;
110     double a = lit_here(x0, y0), b = lit_here(x0 + 1, y0);
111     double c = lit_here(x0, y0 + 1), d = lit_here(x0 + 1, y0 + 1);
112     double top = a + (b - a) * ax;
113     double bottom = c + (d - c) * ax;
114     return top + (bottom - top) * ay;
115 }
```

34 La lumière selon la hauteur sur le mur, la plus forte à la hauteur du néon.

74 La part de la lampe portée pour une ligne du sol.

SRC MAIN C LIGNES 9 A 13

```
9
10 #include "render.h"
11 #include "light.h"
12 #include "screen.h"
13 #include "texture.h"
```

SRC MAIN C LIGNES 32 A 36

```
32     // the textures cost nothing to keep and everything to draw: once,
    here
33     make_wall_textures();
34     light_map();
35     make_floor_texture();
36     make_ceiling_texture();
```

SRC MAIN C LIGNES 61 A 65

```
61     }
62     move_doors(elapsed);
63     lamp_flicker(moment);
64
65     // the way out. the level had no end: one walked until one
```

SRC RENDER C LIGNES 1 A 4

```
1 #include "render.h"
2 #include "light.h"
3
4 #include "screen.h"
```

SRC RENDER C LIGNES 9 A 13

```
9 // one column of the wall, read down one column of the texture
10 static void draw_wall_column(int x, int top, int height, int tex_x,
11     double warm, int dark, int kind)
12 {
13     double step = (double)TEX_SIZE / height;
```

SRC RENDER C LIGNES 29 A 39

```
29     for (; y < bottom; y++) {
30         unsigned int color = wall_texture[kind][((int)tex_y & TEX_MASK) *
    TEX_SIZE + tex_x];
31         double lit = dark ? warm * SIDE_LIGHT : warm;
32         // and the height: under the vault it is dark, at strip
33         // height it is day. without this the wall is a flat wash.
34         lit *= at_height(tex_y / TEX_SIZE);
35         if (lit < 0.0)
36             lit = 0.0;
37         view[y * VIEW_WIDTH + x] = tint(color, lit, GLOOM);
38         tex_y += step;
39     }
```

SRC RENDER C LIGNES 70 A 76

```
70     double ground_x = player->x + distance * left_x;
71     double ground_y = player->y + distance * left_y;
72     // the floor takes what its square takes: that is what
73     // lays the pools of light on the ground
74     double base = CARRIED * lamp(distance);
75
76     for (int x = 0; x < VIEW_WIDTH; x++) {
```

SRC RENDER C LIGNES 79 A 88

```
79         int at = tex_y * TEX_SIZE + tex_x;
80
```

PARTIE 3 CHAPITRE 7 LE CODE

SUITE

```
81         double light = base + lit_at(ground_x, ground_y);
82         if (light > 1.0)
83             light = 1.0;
84         view[y * VIEW_WIDTH + x] = tint(floor_texture[at], light,
GLOOM);
85         view[(VIEW_HEIGHT - 1 - y) * VIEW_WIDTH + x] =
86             tint(ceiling_texture[at], light * CEILING_PART, GLOOM);
87
88         ground_x += step_x;
```

SRC RENDER C LIGNES 221 A 242

```
221         tex_x = TEX_SIZE - 1 - tex_x;
222
223         // how square this wall is to us: a north-south wall met by a ray
224         // going mostly east is seen head on, and catches the light
225         double facing = hit.side == 0 ? ray_x : ray_y;
226         double length = sqrt(ray_x * ray_x + ray_y * ray_y);
227         if (length > 0.0)
228             facing /= length;
229         // this wall takes what the square we look at it from
230         // takes, at the point we hit, plus the glow we carry
231         double on_wall = lit_at(player->x + ray_x * hit.distance * 0.94,
232             player->y + ray_y * hit.distance * 0.94);
233         double lit = on_wall + CARRIED * lamp(hit.distance);
234         if (lit > 1.0)
235             lit = 1.0;
236         // a wall facing us takes the neon, a wall seen at an angle
237         // far less
238         double face = LAMP_AMBIENT + (1.0 - LAMP_AMBIENT)
239             * (facing < 0 ? -facing : facing);
240         draw_wall_column(x, top, height, tex_x, lit * face,
241             hit.side == 1, hit.kind);
242     }
```

SRC TEXTURE C LIGNES 44 A 65

```
44         double ca = (a >> d) & CHANNEL, cb = (b >> d) & CHANNEL;
45         out |= (unsigned int)(ca + (cb - ca) * part) << d;
46     }
47     return out;
48 }
49
50 // each channel of the surface, times what the light gives it there: the
51 // neon gives green and blue, the gloom almost only blue
52 unsigned int tint(unsigned int color, double lamp, double night)
53 {
54     unsigned int out = 0;
55     for (int d = 0; d <= RED_SHIFT; d += GREEN_SHIFT) {
56         // two constants: dividing them by 255 at every pixel
57         // of every frame costs three divisions for nothing
58         double part = lamp * ((LIGHT_LAMP >> d) & CHANNEL)
59             + night * ((LIGHT_NIGHT >> d) & CHANNEL);
60         double v = ((color >> d) & CHANNEL) * part * (1.0 / CHANNEL);
61         if (v > CHANNEL)
62             v = CHANNEL;
63         out |= (unsigned int)v << d;
64     }
65     return out;
```

225 De combien le mur fait face au joueur. Un mur vu de face reçoit toute la lumière, un mur vu de profil en reçoit 30 pour cent.

52 La couleur d'une plaque sous une part de néon et une part de pénombre, détaillée dans le tableau 7.1.



Figure 7.2. Les flaques de lumière sous les néons, et la pénombre bleue entre elles. Les murs vus de face sont plus clairs que les murs vus de biais.

VOCABULAIRE

La carte de lumière est le tableau qui donne, pour chaque case du niveau, la lumière qu'elle reçoit des néons. Elle est calculée une fois au chargement. **La ligne de vue** entre deux cases est libre quand aucun mur ne se trouve entre elles.

Trois couleurs de néon et un tube qui faiblit

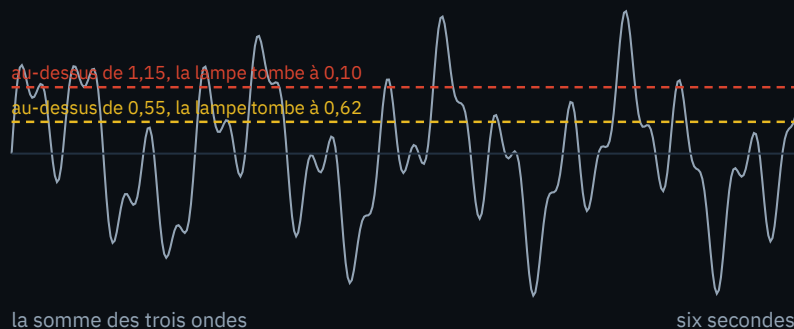


Figure 8.1. La somme des trois ondes du tube en panne, sur six secondes. Chaque fois que la courbe dépasse un seuil, le tube tombe. L'œil ne trouve pas de rythme, parce que les trois fréquences n'ont pas de rapport simple entre elles.

Dans ce chapitre, vous allez colorer les néons par zone et faire clignoter un tube en fin de vie avec trois ondes.

Le niveau se divise en zones, et chaque zone a sa lumière. Le néon ordinaire est vert, le néon de secours est ambré et celui de la salle des machines est d'un blanc froid. Les lettres T, R et B du fichier choisissent la couleur, et la fonction `neon_strip` reçoit désormais les trois couleurs du tube en argument.

La lettre F marque un tube en fin de vie. Il ne scintille pas, il coupe. La fonction `lamp_flicker` additionne trois ondes de fréquences sans rapport, et compare la somme à deux seuils, comme le montre la figure 8.1. Au-dessus du premier seuil, le tube tombe à 10 pour cent. Entre les deux, il tombe à 62 pour cent. En dessous, il éclaire normalement. La lumière des tubes en panne est conservée dans une carte séparée, `faulty`, pour que leurs coupures n'affectent que leurs propres flaques.

EXERCICE

Remplacez un T par un F dans le fichier du niveau, et regardez la flaque de lumière sous ce néon tomber et revenir.

Trois couleurs de néon et un tube qui faiblit, suite

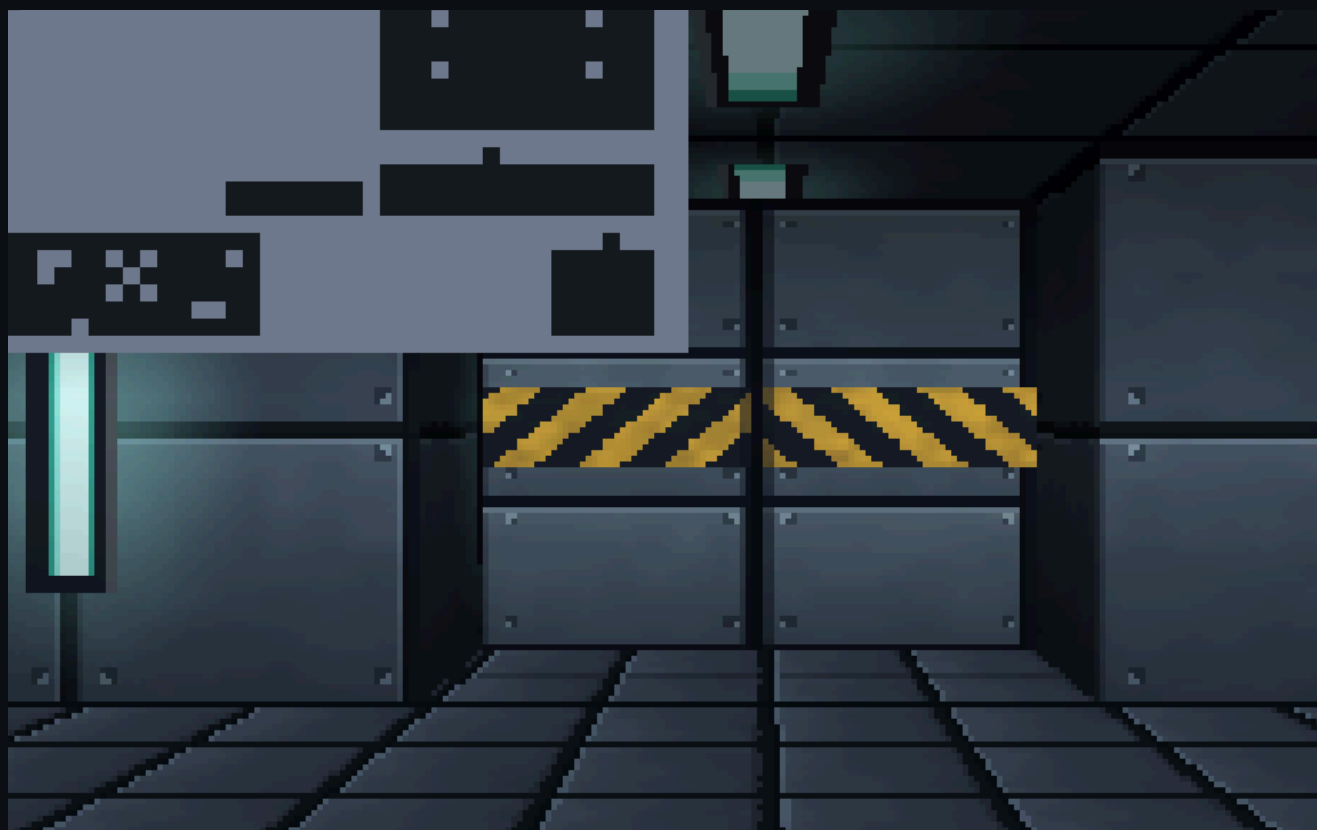


Figure 8.2. Un néon ambré de secours à côté du néon vert. Le tube marqué F s'éteint par à-coups.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE LIGHT H LIGNES 12 A 18

```
12 #define LAMP_HUM      0.035
13 #define LAMP_HZ       9.0
14 // and a failing tube goes out for good for the length of a beat
15 #define FAULT_LOW     0.10
16 #define FAULT_HALF    0.62
17
18 // a surface facing us takes the light, one seen edge on does not:
    without
```

INCLUDE TEXTURE H LIGNES 41 A 50

```
41 #define NEON_TUBE     0x9ff0e4
42 #define NEON_EDGE     0x3aa896
43 #define ALARM_CORE    0xffe9c2
44 #define ALARM_TUBE    0xf0a53c
45 #define ALARM_EDGE    0x8c3c10
46 #define COLD_CORE     0xf4f8ff
47 #define COLD_TUBE     0xbcd2f0
48 #define COLD_EDGE     0x5070a0
49 #define HOUSING       0x1a1f28    // the metal housing that holds it
50
```

INCLUDE TEXTURE H LIGNES 64 A 74

```
64 // several kinds of wall, so one room is not the next: a panel size and
    four
65 // colours, still drawn in code
66 #define WALL_KINDS 7
67 #define DOOR_TEXTURE 3            // which of the seven is the door
68 // three colours of strip, one per zone: the gangway in service, the
    machine
69 // room, and the hold on its emergency lighting
70 #define LAMP_TEXTURE 4           // the ordinary strip, teal
71 #define ALARM_TEXTURE 5          // the emergency one, amber
72 #define COLD_TEXTURE 6           // and the cold white of the machines
73
74 extern unsigned int wall_texture[WALL_KINDS][TEX_SIZE * TEX_SIZE];
```

INCLUDE WORLD H LIGNES 24 A 29

```
24 int wall_kind(int x, int y);
25 int is_lamp(int x, int y);
26 // and this one no longer holds: it flickers
27 int lamp_faulty(int x, int y);
28 // a door takes a second to open, and once it is open this far one walks
29 // between the two leaves
```

SRC LIGHT C LIGNES 5 A 12

```
5 #include <stdlib.h>
6
7 // what the neons are worth at this instant, around 1, and what a failing
8 // tube is worth, which is another matter
9 static double neon = 1.0;
10 static double fault = 1.0;
11
12 // two waves at unrelated speeds never repeat: the eye stops finding a
```

SRC LIGHT C LIGNES 16 A 26

```
16     neon = 1.0 + LAMP_HUM * (sin(seconds * LAMP_HZ)
17         + 0.6 * sin(seconds * LAMP_HZ * 2.37));
```

PARTIE 3 CHAPITRE 8 LE CODE

SUITE

```
18
19 // the failing tube does not modulate, it cuts: three unrelated
20 // waves, two thresholds, and the wobble seems to give up.
21 double wobble = sin(seconds * 13.7) + sin(seconds * 4.3)
22               + sin(seconds * 31.1) * 0.5;
23 fault = wobble > 1.15 ? FAULT_LOW
24         : wobble > 0.55 ? FAULT_HALF : neon;
25 }
26
```

SRC LIGHT C LIGNES 57 A 62

```
57 // to read an array.
58 static double *bright;
59 // and what the failing tubes give, kept apart: they have a life of their
   own
60 static double *faulty;
61
62 static int sees(int x1, int y1, int x2, int y2)
```

SRC LIGHT C LIGNES 79 A 86

```
79 {
80     free(bright);
81     free(faulty);
82     bright = calloc((size_t)map_width * map_height, sizeof(*bright));
83     faulty = calloc((size_t)map_width * map_height, sizeof(*faulty));
84     if (!bright || !faulty)
85         return;
86     // the base light of the ceiling strips, everywhere at once
```

SRC LIGHT C LIGNES 99 A 108

```
99         continue;
100         double part = d / LAMP_ON_WALL;
101         double received = 1.0 / (1.0 + 3.0 * part * part)
102             * (1.0 - part);
103         if (lamp_faulty(x, y))
104             faulty[j * map_width + i] += received;
105         else
106             bright[j * map_width + i] += received;
107     }
108 }
```

SRC LIGHT C LIGNES 114 A 119

```
114 if (!bright || x < 0 || y < 0 || x >= map_width || y >= map_height)
115     return 0.0;
116 double v = bright[y * map_width + x] * neon
117           + faulty[y * map_width + x] * fault;
118 return v > 1.0 ? 1.0 : v;
119 }
```

SRC TEXTURE C LIGNES 154 A 166

```
154
155 // the ordinary wall of the gangway
156 // the three colours of a tube, from the core to the edge
157 static const unsigned int NEON[3] = { NEON_CORE, NEON_TUBE, NEON_EDGE };
158 static const unsigned int ALARM[3] = { ALARM_CORE, ALARM_TUBE, ALARM_EDGE
   };
159 static const unsigned int COLD[3] = { COLD_CORE, COLD_TUBE, COLD_EDGE };
160
161 // a neon strip in its housing: the housing, the tube, and the glow
   around
162 // it. without the glow the tube is a white rectangle stuck on the wall
163 static void neon_strip(unsigned int *out, int cx, int cy, int half_w,
164                       int half_h, const unsigned int *glow)
165 {
166     for (int y = 0; y < TEX_SIZE; y++)
```

21 Les trois ondes. Les fréquences 13,7, 4,3 et 31,1 n'ont pas de multiple commun proche.

23 Les deux seuils, 1,15 et 0,55, et les trois niveaux du tube.

60 La carte de lumière des tubes en panne, à part.

157 Les trois couleurs d'un tube, du cœur au bord.

PARTIE 3 CHAPITRE 8 LE CODE

SUITE

SRC TEXTURE C LIGNES 177 A 182

```
177         // the tube: white at the core, teal at the edge
178         double edge = (double)dx / (half_w + 1);
179         out[y * TEX_SIZE + x] = edge < 0.45 ? glow[0]
180             : edge < 0.8 ? glow[1] : glow[2];
181     }
182 }
```

154 Quatre lettres pour un néon.

SRC TEXTURE C LIGNES 192 A 196

```
192     continue;
193     out[y * TEX_SIZE + x] = mix(out[y * TEX_SIZE + x],
194         glow[1], 0.40 * (1.0 - d) * (1.0 - d));
195 }
196 }
```

SRC TEXTURE C LIGNES 229 A 237

```
229
230 // the wall that carries a strip: sheet metal, with the neon in it
231 static void make_lamp_wall(unsigned int *out, const struct plate *p,
232     const unsigned int *glow)
233 {
234     make_plating(out, p);
235     neon_strip(out, TEX_SIZE / 2, TEX_SIZE / 2, 4, 34, glow);
236 }
237
```

SRC TEXTURE C LIGNES 242 A 248

```
242     make_plating(wall_texture[2], &PARTITION);
243     make_door(wall_texture[3]);
244     make_lamp_wall(wall_texture[4], &HULL, NEON);
245     make_lamp_wall(wall_texture[5], &BULK, ALARM);
246     make_lamp_wall(wall_texture[6], &PARTITION, COLD);
247 }
248
```

SRC TEXTURE C LIGNES 258 A 261

```
258 {
259     make_plating(ceiling_texture, &ROOF);
260     neon_strip(ceiling_texture, TEX_SIZE / 2, TEX_SIZE / 2, 34, 5, NEON);
261 }
```

SRC WORLD C LIGNES 151 A 166

```
151     || x >= (int)strlen(map[y]))
152     return 0;
153     char c = map[y][x];
154     return c == 'T' || c == 'R' || c == 'B' || c == 'F';
155 }
156
157 // a tube at the end of its life. "F" for failing: it holds, it drops, it
158 // comes back. this is not a ripple, it is a wobble that no longer
159 // passes.
160 int lamp_faulty(int x, int y)
161 {
162     if (x < 0 || y < 0 || x >= map_width || y >= map_height
163         || x >= (int)strlen(map[y]))
164         return 0;
165     return map[y][x] == 'F';
166 }
```

SRC WORLD C LIGNES 181 A 190

```
181     return 0;
182     char c = map[y][x];
183     if (c == 'T' || c == 'F')
184         return LAMP_TEXTURE;
185     if (c == 'R')
```


PARTIE 3 CHAPITRE 8 LE CODE

SUITE

```
186         return ALARM_TEXTURE;
187     if (c == 'B')
188         return COLD_TEXTURE;
189     if (c >= '1' && c < '1' + WALL_KINDS)
190         return c - '1';
```

La fenêtre décide de l'image



Figure 9.1. L'image en 640 pixels sur 400, plus fine, avec des blocs de 2 pixels. La carte, dessinée en pixels de l'image, paraît plus petite.

Dans ce chapitre, vous allez laisser la fenêtre décider de la taille de l'image, et adapter le champ de vision à sa forme.

Depuis la deuxième partie, l'image faisait toujours 320 pixels sur 200, agrandie pour remplir la fenêtre. Nous inversons la logique. L'image fait la moitié de la fenêtre, en largeur comme en hauteur, avec un pixel de l'image pour un bloc de 2 pixels sur 2 de la fenêtre. Une fenêtre plus large montre plus de monde, elle n'agrandit plus ce qui était déjà là. La fenêtre s'ouvre à 1280 pixels sur 800, pour une image de 640 sur 400.

Le tampon de pixels `view` n'est plus un tableau de taille fixe. Il est alloué et réalloué à chaque changement de taille de la fenêtre, et sa largeur et sa hauteur deviennent deux variables, `view_width` et `view_height`, que toutes les fonctions de dessin utilisent à la place des anciennes constantes. Une taille minimale de 160 sur 100 est imposée.

Le champ de vision suit la forme de la fenêtre. La longueur du vecteur `plane`, 0,66 pour une fenêtre de rapport 16 sur 10, est multipliée par le rapport de la fenêtre et divisée par 1,6. Une fenêtre plus large voit ainsi plus loin sur les côtés au lieu d'étirer l'image. La fonction `fit_view_to_window` fait cette correction à chaque image.

VOCABULAIRE

Le rapport d'une image est sa largeur divisée par sa hauteur. Un écran de 1280 sur 800 a un rapport de 1,6.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE SCREEN H LIGNES 4 A 24

```
4  #include <X11/Xlib.h>
5
6  // one view pixel is a block of PIXEL by PIXEL pixels: two keeps the
   edges
7  // without costing four times as many rays
8  #define PIXEL      2
9  // the size the window opens at. after that a wider window shows more
   world,
10 // it does not blow up what was already there
11 #define WIN_WIDTH   1280
12 #define WIN_HEIGHT  800
13 // never less than this, whatever the player does to the window
14 #define VIEW_MIN_W  160
15 #define VIEW_MIN_H  100
16
17 // the picture, and its size at this instant. it changes with the window,
   so
18 // it can no longer be an array whose size is known at compile time.
19 extern unsigned int *view;
20 extern int view_width, view_height;
21
22 #define HORIZON      (view_height / 2)
23
24 // XCreateImage wants to know how each row of pixels is padded, in bits.
   this
```

INCLUDE WORLD H LIGNES 41 A 45

```
41 void move_player(struct player *player, double step_x, double step_y);
42 void turn_player(struct player *player, double angle);
43 void fit_view_to_window(struct player *player);
44
45 #endif
```

SRC MAIN C LIGNES 79 A 83

```
79     move_player(&player, step_x, step_y);
80     turn_player(&player, turn);
81     fit_view_to_window(&player);
82     render_floor_and_ceiling(&player);
83     render_walls(&player);
```

SRC RENDER C LIGNES 24 A 29

```
24
25     int bottom = top + height;
26     if (bottom > view_height)
27         bottom = view_height;
28
29     for (; y < bottom; y++) {
```

SRC RENDER C LIGNES 35 A 39

```
35         if (lit < 0.0)
36             lit = 0.0;
37         view[y * view_width + x] = tint(color, lit, GL00M);
38         tex_y += step;
39     }
```

SRC RENDER C LIGNES 57 A 71

```
57     double left_y = player->dir_y - player->plane_y;
58
59     for (int y = HORIZON + 1; y < view_height; y++) {
```

Ligne par ligne

- 8** Un pixel de l'image est un bloc de 2 pixels sur 2 dans la fenêtre.
- 19** Le tampon de pixels, alloué à la taille demandée.

PARTIE 3 CHAPITRE 9 LE CODE

SUITE

```
60      // how far the ground under this row is: a row one pixel below
61      // the horizon is very far, the bottom row is right at our feet,
62      // and the eye is half a wall above the floor
63      double distance = (double)view_height / (2 * y - view_height);
64
65      // where that row starts on the floor, and what one pixel to the
66      // right is worth. the camera plane spans from -plane to +plane,
67      // so the whole row is two planes wide
68      double step_x = distance * 2.0 * player->plane_x / view_width;
69      double step_y = distance * 2.0 * player->plane_y / view_width;
70      double ground_x = player->x + distance * left_x;
71      double ground_y = player->y + distance * left_y;
```

SRC RENDER C LIGNES 74 A 78

```
74      double base = CARRIED * lamp(distance);
75
76      for (int x = 0; x < view_width; x++) {
77          int tex_x = (int)(ground_x * TEX_SIZE) & TEX_MASK;
78          int tex_y = (int)(ground_y * TEX_SIZE) & TEX_MASK;
```

SRC RENDER C LIGNES 82 A 87

```
82          if (light > 1.0)
83              light = 1.0;
84          view[y * view_width + x] = tint(floor_texture[at], light,
GLOOM);
85          view[(view_height - 1 - y) * view_width + x] =
86              tint(ceiling_texture[at], light * CEILING_PART, GLOOM);
87
```

SRC RENDER C LIGNES 205 A 216

```
205 void render_walls(const struct player *player)
206 {
207     for (int x = 0; x < view_width; x++) {
208         // -1 on the left edge of the screen, +1 on the right edge
209         double camera = 2.0 * x / view_width - 1.0;
210         double ray_x = player->dir_x + player->plane_x * camera;
211         double ray_y = player->dir_y + player->plane_y * camera;
212         struct hit hit = cast_ray(player, ray_x, ray_y);
213
214         int height = (int)(view_height / hit.distance);
215         int top = HORIZON - height / 2;
216
```

SRC RENDER C LIGNES 247 A 252

```
247 static void put_pixel(int x, int y, unsigned int color)
248 {
249     if (x >= 0 && x < view_width && y >= 0 && y < view_height)
250         view[y * view_width + x] = color;
251 }
252
```

SRC SCREEN C LIGNES 8 A 14

```
8
9 // every pixel of the game lands here first
10 unsigned int *view;
11 int view_width = WIN_WIDTH / PIXEL;
12 int view_height = WIN_HEIGHT / PIXEL;
13
14 // the window at the size of the picture, in the middle of the screen
```

SRC SCREEN C LIGNES 50 A 73

```
50 static void screen_fit(struct screen *screen)
51 {
52     // the window decides how many rays we cast, and a view
53     // pixel stays a whole number of screen pixels
```

PARTIE 3 CHAPITRE 9 LE CODE

SUITE

```
54     int vw = screen->width / PIXEL;
55     int vh = screen->height / PIXEL;
56     if (vw < VIEW_MIN_W)
57         vw = VIEW_MIN_W;
58     if (vh < VIEW_MIN_H)
59         vh = VIEW_MIN_H;
60     if (vw != view_width || vh != view_height || !view) {
61         unsigned int *bigger = realloc(view, (size_t)vw * vh *
sizeof(*view));
62         if (bigger) {
63             view = bigger;
64             view_width = vw;
65             view_height = vh;
66         }
67     }
68
69     screen->scale = PIXEL;
70     screen->left = (screen->width - view_width * screen->scale) / 2;
71     screen->top = (screen->height - view_height * screen->scale) / 2;
72 }
73
```

54 La largeur de l'image, celle de la fenêtre divisée par 2.

61 Le tampon est réalloué à la nouvelle taille.

206 Le fond n'est effacé que si l'image ne couvre pas toute la fenêtre. Effacer à chaque image coûterait un passage de plus sur chaque pixel.

SRC SCREEN C LIGNES 119 A 123

```
119 // under that there is nothing left to show
120 XSizeHints hints = { .flags = PMinSize,
121     .min_width = view_width, .min_height = view_height };
122 XSetWMNormalHints(screen->display, screen->window, &hints);
123
```

SRC SCREEN C LIGNES 200 A 218

```
200 void screen_present(struct screen *screen)
201 {
202     // we only clear what we do not redraw: clearing it all
203     // first is writing every pixel of every frame twice
204     int scale = screen->scale;
205     int cover_w = view_width * scale, cover_h = view_height * scale;
206     if (screen->left > 0 || screen->top > 0
        || cover_w < screen->width || cover_h < screen->height)
207         for (int i = 0; i < screen->width * screen->height; i++)
208             screen->pixels[i] = WINDOW_BG;
209     // a window narrower than the picture: we draw the columns that fit,
    and
210     // this does not change from one row to the next
211     int visible = (screen->width - screen->left) / scale;
212     if (visible > view_width)
213         visible = view_width;
214     for (int y = 0; y < view_height; y++) {
215         unsigned int *row = view + y * view_width;
216         for (int copy = 0; copy < scale; copy++) {
217             int line = screen->top + y * scale + copy;
```

SRC WORLD C LIGNES 1 A 6

```
1 #include "world.h"
2 #include "texture.h"
3 #include "screen.h"
4 #include "render.h"
5
6 #include <math.h>
```

SRC WORLD C LIGNES 207 A 223

```
207 }
208
209 // the camera plane follows the shape of the window. its length is the
    field
210 // of view: keeping it fixed as the window widens stretches the picture.
211 void fit_view_to_window(struct player *player)
212 {
213     double aspect = (double)view_width / view_height;
```

PARTIE 3 CHAPITRE 9 LE CODE

SUITE

```
214     double length = FIELD_OF_VIEW * aspect / (16.0 / 10.0);
215     double n = hypot(player->plane_x, player->plane_y);
216     if (n <= 0.0)
217         return;
218     player->plane_x = player->plane_x / n * length;
219     player->plane_y = player->plane_y / n * length;
220 }
221
222 // each axis is tested on its own, so a shoulder against a wall keeps
    sliding
223 // instead of stopping the player dead. the margin is the nose: without
    it we
```

214 La longueur du vecteur `plane`, corrigée par le rapport de la fenêtre.

La carte se remplit



Figure 10.1. La carte translucide, avec son cadre. Seules les cases déjà vues sont dessinées, et la porte apparaît en ambre.

Dans ce chapitre, vous allez faire apparaître la carte au fur et à mesure de l'exploration, et la rendre translucide.

Une carte qui montre tout le niveau dès la première seconde ne laisse rien à découvrir. Nous faisons apparaître la carte au fur et à mesure. Le tableau `seen` marque les cases déjà vues. À chaque image, la fonction `remember` marque toutes les cases situées à moins de 4 cases du joueur. Le reste de la carte reste sombre.

La carte est aussi redessinée. Elle reçoit un cadre, qui montre la taille du niveau même quand rien n'a été vu, et elle est translucide. La fonction `blend` mélange la couleur de la carte avec l'image déjà dessinée, dans une proportion de 78 pour cent, et le monde reste visible derrière. Les portes apparaissent en ambre, car une porte dessinée comme un mur ferait des pièces sans lien entre elles. La touche M cache et montre la carte, au relâchement de la touche et non à l'appui, sans quoi une touche maintenue la ferait clignoter.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE RENDER H LIGNES 29 A 40

```
29 #define MAP_WALL    0x6f7b8f
30 #define MAP_FLOOR    0x161a22
31 // a door is not a wall on the map, even shut: drawn as one, the rooms
    look
32 // like squares with nothing joining them.
33 #define MAP_DOOR    0xc6964a
34 #define MAP_FRAME    0x8a94a6
35 #define MAP_UNSEEN  0x0e1116
36 // how thick the frame is, and how much of the world shows through the
    map
37 #define MAP_EDGE    2
38 #define MAP_ALPHA    0.78
39 #define MAP_HEADING  0xe8b04b
40 #define MAP_PLAYER  0xd8534f
```

INCLUDE SCREEN H LIGNES 47 A 51

```
47     int forward, back, left, right, strafe_left, strafe_right, quit;
48     int push;           // a press on space, waiting to be read
49     int map;           // is the map on screen? M takes it away
50 };
51
```

INCLUDE WORLD H LIGNES 24 A 32

```
24 int wall_kind(int x, int y);
25 int is_lamp(int x, int y);
26 // how far the player remembers what has been seen
27 #define SEEN_REACH 4
28
29 int is_seen(int x, int y);
30 void remember(const struct player *player);
31 // and this one no longer holds: it flickers
32 int lamp_faulty(int x, int y);
```

SRC MAIN C LIGNES 46 A 50

```
46     level_marks(&player.x, &player.y, &exit_x, &exit_y,
47                 &player.dir_x, &player.dir_y);
48     struct keys keys = { .map = 1 };
49     double last = now_in_seconds();
50
```

SRC MAIN C LIGNES 62 A 66

```
62     move_doors(elapsed);
63     lamp_flicker(moment);
64     remember(&player);
65
66     // the way out. the level had no end: one walked until one
```

SRC MAIN C LIGNES 83 A 88

```
83     render_floor_and_ceiling(&player);
84     render_walls(&player);
85     if (keys.map)
86         render_map(&player, MAP_CELL, MAP_LEFT, MAP_TOP);
87     screen_present(&screen);
88
```

SRC RENDER C LIGNES 243 A 261

```
243 }
244
245 // the same pixel, but blended with what is already there. `part` says
    how
```


PARTIE 3 CHAPITRE 10 LE CODE

SUITE

```
246 // much of the new colour wins, from 0 to 1.
247 static void blend(int x, int y, unsigned int color, double part)
248 {
249     if (x < 0 || x >= view_width || y < 0 || y >= view_height)
250         return;
251     unsigned int under = view[y * view_width + x];
252     unsigned int out = 0;
253     for (int d = 0; d <= RED_SHIFT; d += GREEN_SHIFT) {
254         double a = (under >> d) & CHANNEL, b = (color >> d) & CHANNEL;
255         out |= (unsigned int)(a + (b - a) * part) << d;
256     }
257     view[y * view_width + x] = out;
258 }
259
260 // one pixel, if it is on the screen at all. the heading line runs off
the
261 // map, and until now it wrote wherever that landed in memory
```

SRC RENDER C LIGNES 269 A 303

```
269 void render_map(const struct player *player, int cell, int left, int top)
270 {
271     // the frame is always there, even over what has not been seen:
without it
272     // nothing says how big the level is
273     int l = map_width * cell, h = map_height * cell;
274     for (int i = -MAP_EDGE; i < l + MAP_EDGE; i++)
275         for (int e = 1; e <= MAP_EDGE; e++) {
276             put_pixel(left + i, top - e, MAP_FRAME);
277             put_pixel(left + i, top + h + e - 1, MAP_FRAME);
278         }
279     for (int j = -MAP_EDGE; j < h + MAP_EDGE; j++)
280         for (int e = 1; e <= MAP_EDGE; e++) {
281             put_pixel(left - e, top + j, MAP_FRAME);
282             put_pixel(left + l + e - 1, top + j, MAP_FRAME);
283         }
284
285     // what we do not know is dark, not missing, and the
286     // whole thing is translucent over the world
287     for (int j = 0; j < h; j++)
288         for (int i = 0; i < l; i++)
289             blend(left + i, top + j, MAP_UNSEEN, MAP_ALPHA);
290
291     for (int y = 0; y < map_height; y++)
292         for (int x = 0; x < map_width; x++) {
293             // what we have not walked past is not on the map yet
294             if (!is_seen(x, y))
295                 continue;
296             unsigned int color = is_door(x, y) ? MAP_DOOR
297                 : is_wall(x, y) ? MAP_WALL : MAP_FLOOR;
298             for (int line = 0; line < cell; line++)
299                 for (int column = 0; column < cell; column++)
300                     blend(left + x * cell + column,
301                         top + y * cell + line, color, MAP_ALPHA);
302         }
303 }
```

SRC SCREEN C LIGNES 188 A 197

```
188     keys->push = 1;
189     break;
190     case XK_m:
191         // on release, not on press: a held key repeats,
192         // and the map used to blink
193         if (!down)
194             keys->map = !keys->map;
195         break;
196     case XK_F11:
197         if (down)
```

247 Le mélange d'une couleur avec le pixel déjà là, dans une proportion donnée.

294 Une case non vue n'est pas dessinée.

193 La carte bascule au relâchement de la touche M.

PARTIE 3 CHAPITRE 10 LE CODE

SUITE

SRC WORLD C LIGNES 17 A 24

```
17 // not an object
18 static double *doors;
19
20 // what we have seen. a map that shows the whole level from the first
  second
21 // leaves nothing to find; this one fills in as one goes.
22 static char *seen;
23
24 // what the level file can hold. a wall is anything that is not floor.
```

22 Un octet par case, 1 quand la case a été vue.

164 Les cases marquées forment un disque de rayon 4 autour du joueur, et non un carré.

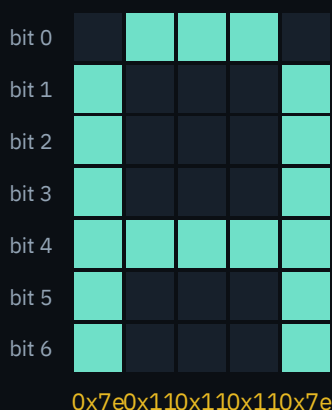
SRC WORLD C LIGNES 63 A 69

```
63 free(doors);
64 doors = calloc((size_t)map_width * map_height, sizeof(*doors));
65 free(seen);
66 seen = calloc((size_t)map_width * map_height, 1);
67 if (!doors || !seen)
68     return 0;
69 }
```

SRC WORLD C LIGNES 141 A 168

```
141     if (doors[i] > 1.0)
142         doors[i] = 1.0;
143     }
144 }
145
146 int is_seen(int x, int y)
147 {
148     if (!seen || x < 0 || y < 0 || x >= map_width || y >= map_height)
149         return 0;
150     return seen[y * map_width + x];
151 }
152
153 // everything a few steps away, and nothing behind a wall: we mark what
  the
154 // light really reaches
155 void remember(const struct player *player)
156 {
157     if (!seen)
158         return;
159     int cx = (int)player->x, cy = (int)player->y;
160     for (int y = cy - SEEN_REACH; y <= cy + SEEN_REACH; y++)
161         for (int x = cx - SEEN_REACH; x <= cx + SEEN_REACH; x++) {
162             if (x < 0 || y < 0 || x >= map_width || y >= map_height)
163                 continue;
164             if ((x - cx) * (x - cx) + (y - cy) * (y - cy) > SEEN_REACH *
SEEN_REACH)
165                 continue;
166             seen[y * map_width + x] = 1;
167         }
168 }
```

Une police en cent quatre-vingts octets



La lettre A, cinq colonnes de sept points.

Chaque colonne est un octet. Le bit 0, le plus à droite dans l'écriture binaire, est le point du haut.

Première colonne, 0x7e = 1111110 en binaire. Lu de droite à gauche, bit 0 en premier, cela donne les points de haut en bas.

Figure 11.1. La lettre A dans la police du jeu. Les cinq octets de la table décrivent les cinq colonnes, et chaque bit allumé est un point.

Dans ce chapitre, vous allez écrire une police de caractères en cent quatre-vingts octets et afficher du texte à l'écran.

Le jeu doit parler au joueur, et il le fait sans bibliothèque. Une police de caractères de 5 points de large sur 7 de haut est rangée dans une table de 36 lettres et chiffres, à raison de 5 octets par caractère, soit 180 octets. Chaque octet décrit une colonne, et chaque bit de l'octet un point de la colonne, le bit 0 en haut. La figure 11.1 montre la lettre A.

La fonction `glyph_of` trouve l'indice d'un caractère dans la table, en convertissant les minuscules en majuscules. La fonction `draw_text` écrit un texte à l'écran, point par point, avec un facteur d'agrandissement, et une ombre d'un pixel sous chaque point pour que le texte reste lisible sur une plaque claire. La fonction `draw_text_centered` centre le texte sur la largeur de l'image.

La même police sert à peindre le nom du studio sur le sol du sas, comme le montre la page suivante. Le fichier `text.c` est le sixième fichier de code du jeu.

VOCABULAIRE

Un **glyphe** est le dessin d'un caractère. La table de la police contient un glyphe par lettre et par chiffre.

EXERCICE

Ajoutez le point d'exclamation à la police. Il faut une trente-septième entrée dans la table et un cas de plus dans `glyph_of`.

Une police en cent quatre-vingts octets, suite

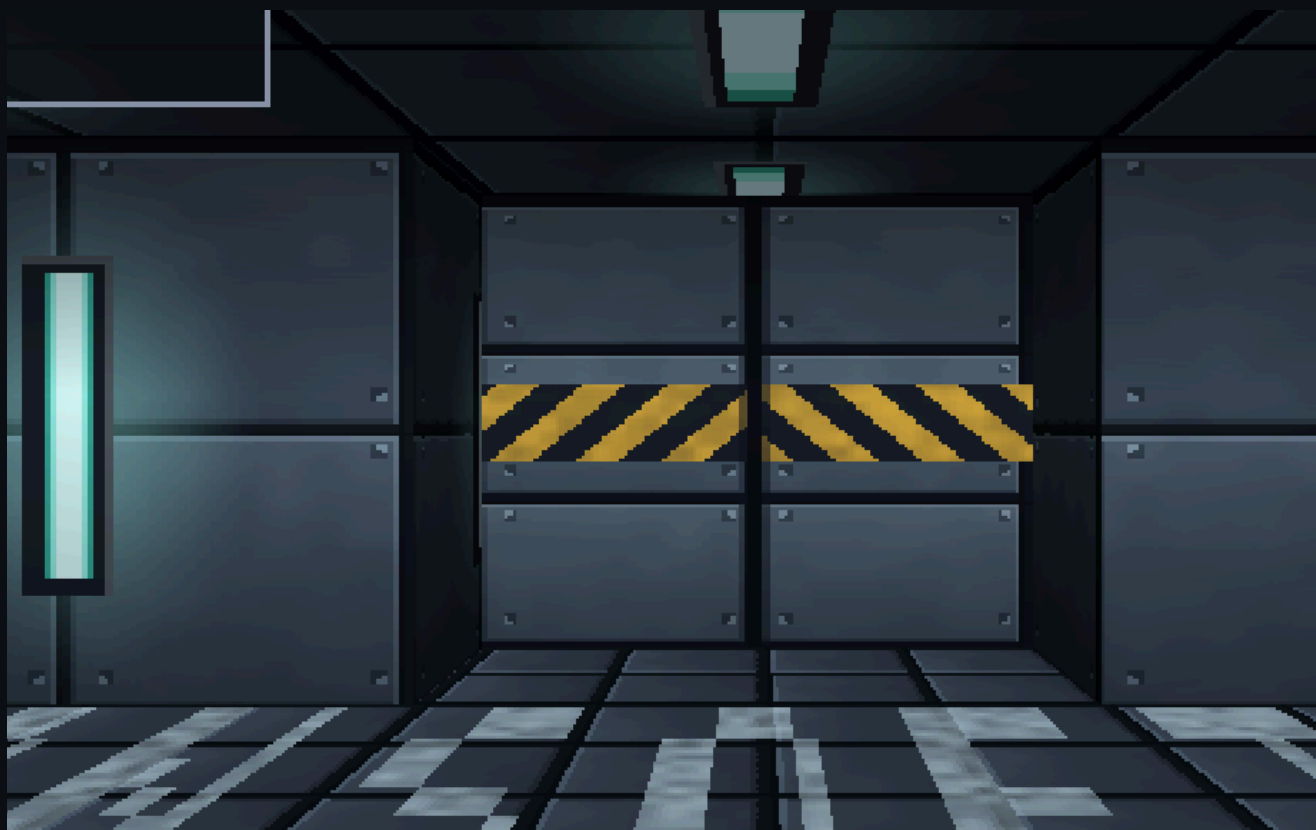


Figure 11.2. Le nom du studio peint sur le sol du sas, en lettres tournées d'un quart de tour, lisibles dans le sens de la marche.

Le nom peint au sol

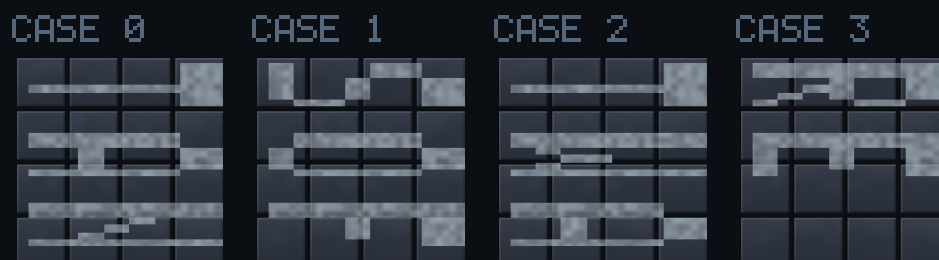


Figure 11.2. Les quatre cases du sol qui portent le nom, telles que le programme les dessine. Les lettres sont tournées d'un quart de tour, et la peinture est usée par endroits.

Le nom du studio est peint sur le sol du sas, à l'endroit où le joueur commence. Il occupe quatre cases alignées dans le sens de la marche, avec trois lettres par case. Chaque lettre est tournée d'un quart de tour, pour se lire dans le sens où l'on avance.

La fonction `stencil_tile` calcule la couleur d'un pixel du sol de l'une de ces cases. Elle détermine d'abord quelle lettre occupe ce pixel, à partir du numéro de la case et de la coordonnée y du pixel. Puis elle convertit la position du pixel dans la lettre en une colonne et une ligne de la police, en tenant compte du quart de tour et d'une marge autour de la lettre. Enfin, si le point de la police est allumé, elle mélange la couleur de la peinture au sol, avec une usure tirée de la fonction `smooth_noise`.

La même police sert donc aux textes affichés à l'écran et à la peinture au sol. Une seule table de 180 octets fournit les deux.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE TEXT H LIGNES 1 A 24

```
1  #ifndef TEXT_H
2  #define TEXT_H
3
4  // we speak to the player, and without a library: a five by seven font,
   one
5  // byte per column, drawn like everything else
6
7  // how long a notice stays on screen, and how high above the bottom
8  #define NOTICE_SECONDS 5.0
9  #define NOTICE_UP      34
10
11 #define GLYPH_W      5
12 #define GLYPH_H      7
13 #define TEXT_SCALE 2          // one font pixel is this many view pixels
14 #define TEXT_COLOR 0xe8e4da
15 #define TEXT_SHADOW 0x101318
16
17 // one dot of the font: is this letter lit there?
18 int glyph_bit(char c, int column, int line);
19 // written from x, y in the view, top left. gives back the width written.
20 int draw_text(int x, int y, const char *text, unsigned int color, int
   scale);
21 // the same, centred on the width of the view
22 void draw_text_centered(int y, const char *text, unsigned int color, int
   scale);
23
24 #endif
```

INCLUDE TEXTURE H LIGNES 58 A 63

```
58 // the same colour, taken under this much neon and this much gloom
59 unsigned int tint(unsigned int color, double lamp, double night);
60 // the painted tile: `cell` says which of the four we are looking at
61 unsigned int stencil_tile(int x, int y, int cell);
62
63 // a wall met on a north-south line keeps this much of its light
```

INCLUDE TEXTURE H LIGNES 74 A 86

```
74 #define COLD_TEXTURE 6          // and the cold white of the machines
75
76 // the name stencilled on the airlock floor: four squares, three letters
77 // each, every letter turned a quarter turn to read the way one walks
78 #define STENCIL      "TANSOFTWARE"
79 #define STENCIL_CELLS 4
80 #define STENCIL_EACH 3          // how many letters to a square
81 #define STENCIL_PAD 10         // the margin around a letter, in pixels
82 #define STENCIL_PAINT 0xb9c6cf
83 #define STENCIL_WEAR 0.72      // the paint is worn, not fresh
84
85 extern unsigned int wall_texture[WALL_KINDS][TEX_SIZE * TEX_SIZE];
86 extern unsigned int floor_texture[TEX_SIZE * TEX_SIZE];
```

INCLUDE WORLD H LIGNES 24 A 31

```
24 int wall_kind(int x, int y);
25 int is_lamp(int x, int y);
26 // where the studio name is painted on the airlock floor
```

PARTIE 3 CHAPITRE 11 LE CODE

SUITE

```
27 #define STENCIL_X 8
28 #define STENCIL_Y 3
29 int stencil_at(int x, int y);
30 // how far the player remembers what has been seen
31 #define SEEN_REACH 4

SRC RENDER C LIGNES 82 A 89
82     if (light > 1.0)
83         light = 1.0;
84     int painted = stencil_at((int)ground_x, (int)ground_y);
85     unsigned int ground = painted >= 0
86         ? stencil_tile(tex_x, tex_y, painted) :
floor_texture[at];
87     view[y * view_width + x] = tint(ground, light, GLOOM);
88     view[(view_height - 1 - y) * view_width + x] =
89         tint(ceiling_texture[at], light * CEILING_PART, GLOOM);
```

SRC TEXT C LIGNES 1 A 85

```
1 #include "text.h"
2
3 #include "screen.h"
4
5 // five columns by seven rows, one byte per column, lowest bit at the
top:
6 // twenty-six letters and ten digits, and nothing more
7 static const unsigned char font[36][GLYPH_W] = {
8     {0x7e,0x11,0x11,0x11,0x7e}, {0x7f,0x49,0x49,0x49,0x36},
{0x3e,0x41,0x41,0x41,0x22}, {0x7f,0x41,0x41,0x41,0x3e}, // A B C D
9     {0x7f,0x49,0x49,0x49,0x41}, {0x7f,0x09,0x09,0x09,0x01},
{0x3e,0x41,0x49,0x49,0x7a}, {0x7f,0x08,0x08,0x08,0x7f}, // E F G H
10    {0x00,0x41,0x7f,0x41,0x00}, {0x20,0x40,0x41,0x3f,0x01},
{0x7f,0x08,0x14,0x22,0x41}, {0x7f,0x40,0x40,0x40,0x40}, // I J K L
11    {0x7f,0x02,0x0c,0x02,0x7f}, {0x7f,0x04,0x08,0x10,0x7f},
{0x3e,0x41,0x41,0x41,0x3e}, {0x7f,0x09,0x09,0x09,0x06}, // M N O P
12    {0x3e,0x41,0x51,0x21,0x5e}, {0x7f,0x09,0x19,0x29,0x46},
{0x46,0x49,0x49,0x49,0x31}, {0x01,0x01,0x7f,0x01,0x01}, // Q R S T
13    {0x3f,0x40,0x40,0x40,0x3f}, {0x1f,0x20,0x40,0x20,0x1f},
{0x7f,0x20,0x18,0x20,0x7f}, {0x63,0x14,0x08,0x14,0x63}, // U V W X
14    {0x07,0x08,0x70,0x08,0x07}, {0x61,0x51,0x49,0x45,0x43},
{0x3e,0x51,0x49,0x45,0x3e}, {0x00,0x42,0x7f,0x40,0x00}, // Y Z 0 1
15    {0x42,0x61,0x51,0x49,0x46}, {0x21,0x41,0x45,0x4b,0x31},
{0x18,0x14,0x12,0x7f,0x10}, {0x27,0x45,0x45,0x45,0x39}, // 2 3 4 5
16    {0x3c,0x4a,0x49,0x49,0x30}, {0x01,0x71,0x09,0x05,0x03},
{0x36,0x49,0x49,0x49,0x36}, {0x06,0x49,0x49,0x29,0x1e}, // 6 7 8 9
17 };
18
19 // where to find a letter in the table. anything that is not in it,
spaces
20 // included, moves on one step without drawing.
21 static int glyph_of(char c)
22 {
23     if (c >= 'a' && c <= 'z')
24         c -= 'a' - 'A';
25     if (c >= 'A' && c <= 'Z')
26         return c - 'A';
27     if (c >= '0' && c <= '9')
28         return c - '0' + 26;
29     return -1;
30 }
31
32 // is this letter lit at that spot? the floor uses it too: the stencilled
33 // name comes out of the same font as the text on screen.
34 int glyph_bit(char c, int column, int line)
35 {
36     int index = glyph_of(c);
37     if (index < 0 || column < 0 || column >= GLYPH_W
```

7 La table, 36 caractères de 5 octets. Les lettres d'abord, puis les chiffres.

8 La lettre A puis la lettre B. La figure 11.1 décode la première.

PARTIE 3 CHAPITRE 11 LE CODE

SUITE

```
38     || line < 0 || line >= GLYPH_H)
39     return 0;
40     return (font[index][column] >> line) & 1;
41 }
42
43 // one font pixel, `size` view pixels wide
44 static void block(int x, int y, int size, unsigned int color)
45 {
46     for (int j = 0; j < size; j++)
47         for (int i = 0; i < size; i++) {
48             int px = x + i, py = y + j;
49             if (px >= 0 && px < view_width && py >= 0 && py <
view_height)
50                 view[py * view_width + px] = color;
51         }
52 }
53
54 int draw_text(int x, int y, const char *text, unsigned int color, int
scale)
55 {
56     int width = 0;
57     for (const char *p = text; *p; p++, width += GLYPH_W + 1) {
58         int index = glyph_of(*p);
59         if (index < 0)
60             continue;
61         for (int column = 0; column < GLYPH_W; column++) {
62             unsigned char bits = font[index][column];
63             for (int line = 0; line < GLYPH_H; line++) {
64                 if (!(bits & (1 << line)))
65                     continue;
66                 int px = x + (width + column) * scale;
67                 int py = y + line * scale;
68                 // a one pixel shadow: without it the text
69                 // vanishes as soon as it crosses a light plate.
70                 block(px + scale, py + scale, scale, TEXT_SHADOW);
71                 block(px, py, scale, color);
72             }
73         }
74     }
75     return width * scale;
76 }
77
78 // the same, centred on the width of the view
79 void draw_text_centered(int y, const char *text, unsigned int color, int
scale)
80 {
81     int width = 0;
82     for (const char *p = text; *p; p++)
83         width += GLYPH_W + 1;
84     draw_text((view_width - width * scale) / 2, y, text, color, scale);
85 }
```

40 Le point de la colonne à la ligne demandée, extrait par un décalage et un masque.

57 Chaque caractère avance de 6 colonnes, les 5 de la lettre et une d'espace.

70 L'ombre, un pixel en bas à droite du point, dessinée avant le point.

SRC TEXTURE C LIGNES 2 A 6

```
2
3 #include "render.h"
4 #include "text.h"
5
6 unsigned int wall_texture[WALL_KINDS][TEX_SIZE * TEX_SIZE];
```

SRC TEXTURE C LIGNES 249 A 271

```
249
250 // the floor: dark tiles, tight, no rivets
251 unsigned int stencil_tile(int x, int y, int cell)
252 {
253     unsigned int ground = floor_texture[y * TEX_SIZE + x];
254     int part = TEX_SIZE / STENCIL_EACH;
255     int letter = cell * STENCIL_EACH + y / part;
```


PARTIE 3 CHAPITRE 11 LE CODE

SUITE

```
256     if (letter >= (int)sizeof(STENCIL) - 1)
257         return ground;
258
259     // a quarter turn: the width of the letter follows the width of the
260     // corridor, its height follows the way one walks
261     int column = (y % part - STENCIL_PAD) * GLYPH_W / (part - 2 *
STENCIL_PAD);
262     int line = (TEX_SIZE - 1 - x - STENCIL_PAD) * GLYPH_H
263         / (TEX_SIZE - 2 * STENCIL_PAD);
264     if (!glyph_bit(STENCIL[letter], column, line))
265         return ground;
266     return mix(ground, STENCIL_PAINT,
267         STENCIL_WEAR * (0.55 + 0.45 * smooth_noise(x, y, 4)));
268 }
269
270 void make_floor_texture(void)
271 {
```

261 Le quart de tour. La colonne de la lettre suit la coordonnée y de la case, et sa ligne suit x.

176 Les quatre cases peintes, alignées dans le sens de la marche à la sortie du sas.

SRC WORLD C LIGNES 168 A 182

```
168 }
169
170 // the studio name, painted on the airlock floor. gives back which of the
171 // four squares we are looking at, or -1 if it is none of them.
172 int stencil_at(int x, int y)
173 {
174     // the squares follow one another across the walk: that is how the
    letters
175     // line up left to right for anyone heading east
176     if (x != STENCIL_X || y < STENCIL_Y || y >= STENCIL_Y +
STENCIL_CELLS)
177         return -1;
178     return y - STENCIL_Y;
179 }
180
181 int is_door(int x, int y)
182 {
```

Le badge, la trappe et un mot au joueur

LA CASE DU BADGE

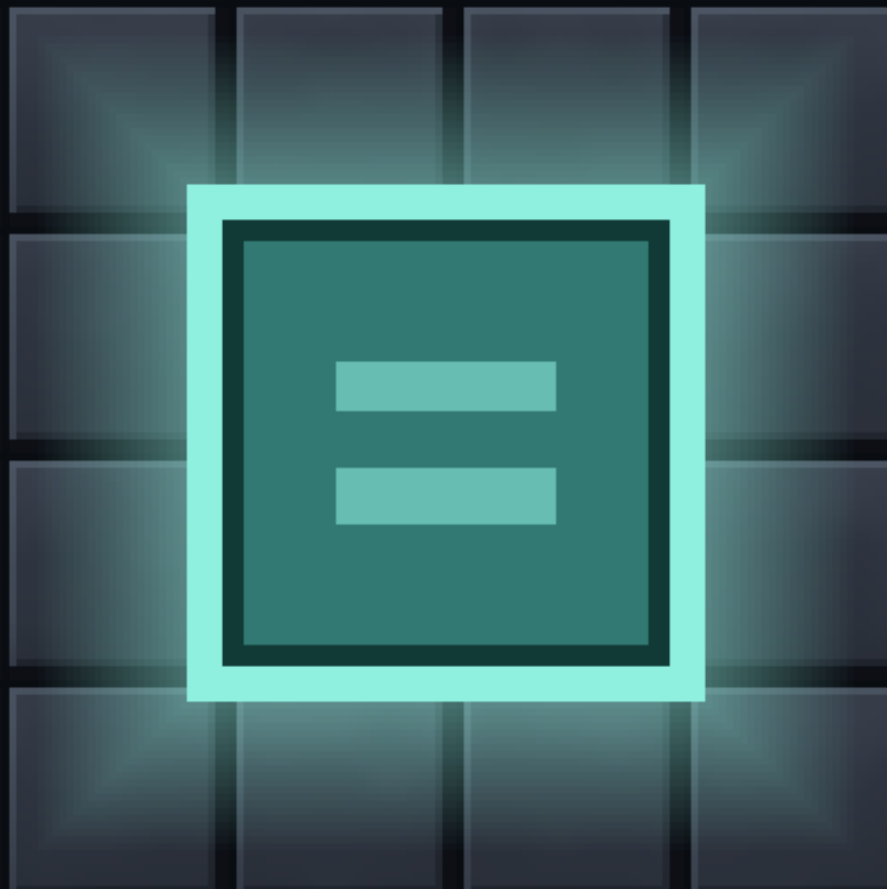


Figure 12.1. La case du sol qui porte le badge. Une plaque avec un bord brillant, une puce au centre et un halo qui déborde sur les dalles.

Dans ce chapitre, vous allez verrouiller la sortie, cacher un badge dans le niveau, et faire parler le jeu au joueur.

Un niveau avec une sortie et rien à chercher est un couloir. Nous verrouillons la sortie et nous cachons la clé dans la soute. La clé est un badge, marqué par la lettre K dans le fichier, dessiné sur le sol comme une plaque lumineuse avec un halo, visible de loin. Le joueur le ramasse en marchant dessus. La sortie, appelée la trappe, ne s'ouvre qu'avec le badge.

Le jeu dit ce qu'il attend. La fonction `say` affiche une phrase en bas de l'écran pendant 5 secondes, avec la police du chapitre précédent. Le premier écran indique les touches. Devant une porte fermée, la fonction `door_ahead` déclenche le message d'ouverture. Le badge ramassé, la trappe verrouillée puis la trappe ouverte ont chacun leur phrase, et le jeu se termine 5 secondes après l'ouverture, le temps de lire.

Les messages sont écrits en anglais et en majuscules, les seules lettres de la police. Un jeu qui dit ce que le joueur peut faire au moment où il peut le faire, et qui se tait le reste du temps, n'a pas besoin de mode d'emploi.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

Ligne par ligne

22 Le message et l'heure à laquelle il disparaît.

```

INCLUDE TEXTURE H LIGNES 60 A 65
60 // the painted tile: `cell` says which of the four we are looking at
61 unsigned int stencil_tile(int x, int y, int cell);
62 // and the tile the badge sleeps on, read where the floor is read
63 unsigned int badge_tile(int x, int y);
64
65 // a wall met on a north-south line keeps this much of its light

INCLUDE TEXTURE H LIGNES 85 A 94
85 #define STENCIL_WEAR 0.72 // the paint is worn, not fresh
86
87 // the badge: a plate set into the tile, it shines, and one goes to get
  it
88 #define BADGE_BODY 0x1d5f5a
89 #define BADGE_EDGE 0x8ff0e0
90 #define BADGE_R 36.0 // half the side of the plate
91 #define BADGE_HALO 22.0 // and how far its glow spills on the tile
92
93 extern unsigned int wall_texture[WALL_KINDS][TEX_SIZE * TEX_SIZE];
94 extern unsigned int floor_texture[TEX_SIZE * TEX_SIZE];

INCLUDE WORLD H LIGNES 28 A 35
28 #define STENCIL_Y 3
29 int stencil_at(int x, int y);
30 int is_badge(int x, int y);
31 int have_badge(void);
32 // gives 1 on the frame the badge is picked up
33 int walk_over(const struct player *player);
34 // how far the player remembers what has been seen
35 #define SEEN_REACH 4

INCLUDE WORLD H LIGNES 50 A 55
50 // from 0 (shut) to 1 (both leaves tucked into the wall)
51 double door_at(int x, int y);
52 // is there a shut door in front of us, within reach?
53 int door_ahead(const struct player *player);
54 void push_door(const struct player *player);
55 void move_doors(double elapsed);

SRC MAIN C LIGNES 10 A 28
10 #include "render.h"
11 #include "light.h"
12 #include "text.h"
13 #include "screen.h"
14 #include "texture.h"
15 #include "world.h"
16
17 // a game says what can be done at the moment it can be done, and holds
  its
18 // tongue the rest of the time.
19 static const char *notice;
20 static double notice_until;
21
22 static void say(const char *line, double moment)
23 {
24     notice = line;
25     notice_until = moment + NOTICE_SECONDS;
26 }
27

```

PARTIE 3 CHAPITRE 12 LE CODE

SUITE

```
28 static double now_in_seconds(void)
```

SRC MAIN C LIGNES 60 A 67

```
60 struct keys keys = { .map = 1 };
61 double last = now_in_seconds();
62 // the first thing one reads, and the only one nobody triggers
63 say("ARROWS TO MOVE M FOR THE MAP", last);
64 // when the hatch opened: we leave time to read it
65 double left_at = 0.0;
66
67 while (!keys.quit) {
```

SRC MAIN C LIGNES 72 A 99

```
72 screen_read_keys(&screen, &keys);
73
74 // a door that opens by itself teaches nothing; a door that
75 // asks for a key and says so does
76 if (door_ahead(&player))
77     say("PRESS SPACE TO OPEN", moment);
78 if (keys.push) {
79     push_door(&player);
80     keys.push = 0;
81 }
82 if (walk_over(&player))
83     say("SECURITY BADGE TAKEN", moment);
84 move_doors(elapsed);
85 lamp_flicker(moment);
86 remember(&player);
87
88 // the way out, and it wants the badge
89 if ((int)player.x == exit_x && (int)player.y == exit_y) {
90     if (!have_badge())
91         say("THE HATCH IS LOCKED", moment);
92     else if (left_at == 0.0) {
93         left_at = moment;
94         say("THE HATCH OPENS", moment);
95     }
96 }
97 if (left_at > 0.0 && moment - left_at > NOTICE_SECONDS)
98     keys.quit = 1;
99
```

SRC MAIN C LIGNES 114 A 120

```
114 if (keys.map)
115     render_map(&player, MAP_CELL, MAP_LEFT, MAP_TOP);
116 if (moment < notice_until)
117     draw_text_centered(view_height - NOTICE_UP, notice,
118         TEXT_COLOR, TEXT_SCALE);
119 screen_present(&screen);
120
```

SRC RENDER C LIGNES 83 A 93

```
83 light = 1.0;
84 int painted = stencil_at((int)ground_x, (int)ground_y);
85 // the badge shines with a light of its own: it is seen
86 // from the far end of the hold
87 int on_badge = is_badge((int)ground_x, (int)ground_y);
88 unsigned int ground = on_badge ? badge_tile(tex_x, tex_y)
89 : painted >= 0 ? stencil_tile(tex_x, tex_y, painted)
90 : floor_texture[at];
91 view[y * view_width + x] = tint(ground, on_badge ? 1.0 :
light, GLOOM);
92 view[(view_height - 1 - y) * view_width + x] =
93     tint(ceiling_texture[at], light * CEILING_PART, GLOOM);
```

SRC TEXTURE C LIGNES 3 A 8

```
3 #include "render.h"
4 #include "text.h"
```

- 63** Le premier message, le seul que rien ne déclenche.
- 76** Une porte fermée devant le joueur déclenche le message d'ouverture.
- 97** Le jeu se termine 5 secondes après l'ouverture de la trappe, le temps de lire.
- 91** Le badge reçoit toute la lumière, il brille de lui-même.

PARTIE 3 CHAPITRE 12 LE CODE

SUITE

```
5
6 #include <math.h>
7
8 unsigned int wall_texture[WALL_KINDS][TEX_SIZE * TEX_SIZE];
```

SRC TEXTURE C LIGNES 251 A 278

```
251
252 // the floor: dark tiles, tight, no rivets
253 // a plate set into the tile: a rim that shines, a chip in the middle,
    and a
254 // glow that spills onto the tiling around it
255 unsigned int badge_tile(int x, int y)
256 {
257     double dx = x - TEX_SIZE / 2.0, dy = y - TEX_SIZE / 2.0;
258     double inside = fabs(dx) > fabs(dy) ? fabs(dx) : fabs(dy);
259     unsigned int ground = floor_texture[y * TEX_SIZE + x];
260
261     if (inside > BADGE_R + BADGE_HALO)
262         return ground;
263     if (inside > BADGE_R)
264         return mix(ground, BADGE_EDGE,
265             0.45 * (1.0 - (inside - BADGE_R) / BADGE_HALO));
266     if (inside > BADGE_R - 5.0)
267         return BADGE_EDGE;
268     if (inside > BADGE_R - 8.0)
269         return mix(BADGE_BODY, 0x000000, 0.4);
270     // the chip: three lines, like on a card
271     if (fabs(dy) < 12.0 && fabs(dx) < 16.0
272         && ((int)(dy + 12.0) / 8) % 2 == 0)
273         return mix(BADGE_EDGE, BADGE_BODY, 0.35);
274     return mix(BADGE_BODY, BADGE_EDGE, 0.18);
275 }
276
277 unsigned int stencil_tile(int x, int y, int cell)
278 {
```

255 Le dessin de la case du badge, en fonction de la distance au centre, le halo, le bord, la puce.

SRC WORLD C LIGNES 113 A 148

```
113 }
114
115 // the door in front of us, if there is a shut one within reach. a few
    steps
116 // ahead, not at arm's length: one pushes a door from where one stands.
117 static int door_in_front(const struct player *player, int *door_x, int
    *door_y)
118 {
119     for (double d = 0.6; d <= DOOR_REACH; d += 0.4) {
120         int x = (int)(player->x + player->dir_x * d);
121         int y = (int)(player->y + player->dir_y * d);
122         if (is_door(x, y)) {
123             *door_x = x;
124             *door_y = y;
125             return door_at(x, y) == 0.0;
126         }
127         if (is_wall(x, y))
128             return 0; // a wall between us: nothing to push
129     }
130     return 0;
131 }
132
133 // a game never asks for a key without saying so: the line shown on
    screen
134 // comes out of this function.
135 int door_ahead(const struct player *player)
136 {
137     int x, y;
138     return door_in_front(player, &x, &y);
```

PARTIE 3 CHAPITRE 12 LE CODE

SUITE

```
139 }
140
141 // we nudge it once: after that the two leaves finish their travel
142 void push_door(const struct player *player)
143 {
144     int x, y;
145     if (door_in_front(player, &x, &y))
146         doors[y * map_width + x] = 0.001;
147 }
148
```

243 La case du badge n'existe plus une fois le badge ramassé.

SRC WORLD C LIGNES 187 A 208

```
187 // the studio name, painted on the airlock floor. gives back which of the
188 // four squares we are looking at, or -1 if it is none of them.
189 // the badge. a way out and nothing else to look for is a corridor, not a
190 // level: the hatch is locked, and what opens it is down in the hold.
191 static int badge_taken;
192
193 int have_badge(void)
194 {
195     return badge_taken;
196 }
197
198 // gives 1 on the frame the badge is picked up, so it can be said
199 int walk_over(const struct player *player)
200 {
201     if (!is_badge((int)player->x, (int)player->y))
202         return 0;
203     badge_taken = 1;
204     return 1;
205 }
206
207 int stencil_at(int x, int y)
208 {
```

SRC WORLD C LIGNES 231 A 245

```
231     char c = map[y][x];
232     return c == 'T' || c == 'R' || c == 'B' || c == 'F';
233 }
234
235 // we pick it up by walking over it: a chest would be a block that eats
236 // the screen, a plate on the floor is seen from far off
237 int is_badge(int x, int y)
238 {
239     if (x < 0 || y < 0 || x >= map_width || y >= map_height
240         || x >= (int)strlen(map[y]))
241         return 0;
242     char c = map[y][x];
243     return c == 'K' && !badge_taken;
244 }
245
```

SRC WORLD C LIGNES 291 A 295

```
291     if (c == '+')
292         return door_at(x, y) < DOOR_WALKABLE;
293     return !(c == '.' || c == 'S' || c == 'E' || c == 'K');
294 }
295
```



Figure 12.2. Le message d'ouverture devant la porte, en bas de l'écran. La partie 3 se termine ici, avec un niveau qui a un début, une clé et une fin.

VOCABULAIRE

Un **message** ou notice est une phrase affichée quelques secondes à l'écran pour dire au joueur ce qu'il peut faire.

EXERCICE

Changez `NOTICE_SECONDS` pour 2 secondes et observez que le message d'ouverture réapparaît tant que le joueur reste devant la porte fermée.

4

QUATRIEME PARTIE

Quelque chose dans le noir

Dans cette quatrième partie, vous allez peupler la station. Des créatures sortiront de planches d'images, verront le joueur, le poursuivront et frapperont. Le joueur répondra avec une barre d'acier, puis avec une arme de poing, ramassera ce qui traîne au sol, mourra, recommencera, et sortira. À la fin, la borne jouera toute seule quand personne ne sera aux touches.

18 CHAPITRES 11 FICHIERS 1545 LIGNES LANGAGE C

Le fichier d'une planche d'images

le fichier, octet par octet, de gauche à droite

TECS 4 lettres	largeur 4 octets	hauteur 4 octets	images 4 octets	pixels AARRGGBB 4 octets chacun
-------------------	---------------------	---------------------	--------------------	------------------------------------

chaque pixel tient dans un entier de 32 bits, l'octet du haut est l'opacité, puis le rouge, le vert et une opacité sous 128 est un pixel transparent, que le jeu ne dessine pas

Figure 1.1. Le format TECS. Quatre lettres, trois entiers, puis les pixels de toutes les images, dans l'ordre de lecture.

Dans ce chapitre, vous allez définir un format de fichier pour les planches d'images et écrire la fonction qui le lit.

Les murs, les sols et les néons sont dessinés par le code. Un corps ne peut pas l'être. Les créatures de cette partie viennent de planches d'images, sculptées et photographiées hors du jeu, puis enregistrées dans un format de fichier propre au projet, le format TECS. Le moteur n'a pas de décodeur d'images, et ce format est le plus simple possible, quatre entiers puis les pixels.

Le fichier commence par les quatre lettres TECS, puis par la largeur, la hauteur et le nombre d'images de la planche, chacun sur 4 octets. Viennent ensuite tous les pixels, image après image, ligne après ligne, chacun sur 4 octets, l'opacité, le rouge, le vert et le bleu. Un pixel dont l'opacité est inférieure à 128 est transparent et ne sera pas dessiné.

La fonction `sprite_load` lit ce fichier, vérifie les quatre lettres et la taille annoncée, et charge les pixels en mémoire. Elle calcule aussi, une fois pour toutes, le plus petit rectangle qui contient tous les pixels opaques de toutes les images. Une planche d'arme couvre tout l'écran mais n'en remplit qu'un quart, et ce rectangle évitera de parcourir les trois quarts vides à chaque image. La fonction `sprite_at` donne un pixel d'une image, ou 0 hors de la planche.

VOCABULAIRE

Une planche ou **sprite** est une suite d'images d'un même objet, ici une créature vue sous plusieurs angles ou une arme dans ses positions successives. **L'opacité** est le quatrième octet d'un pixel. À 255 le pixel est plein, à 0 il est transparent.

Le fichier d'une planche d'images, suite



Figure 1.2. L'image ne change pas encore. Le chargeur de planches est écrit, et rien ne l'utilise.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE SPRITE H LIGNES 1 A 22

```
1  #ifndef SPRITE_H
2  #define SPRITE_H
3
4  // a sprite is a raw file prepared offline: four integers, then the
   pixels.
5  // the engine still has no image decoder, and the art stays a file one
   can
6  // replace without rebuilding.
7  //  "TECS", width, height, how many frames, then 0xAARRGGBB pixels
8
9  struct sprite {
10     int width, height, frames;
11     unsigned int *pixels;        // frames * height * width
12     // where its pixels actually are. a weapon sheet covers the whole
13     // frame and fills a quarter of it: copying all of it reads two
14     // hundred thousand pixels to place sixty thousand. worked out once.
15     int x0, y0, x1, y1;
16 };
17
18 int sprite_load(struct sprite *s, const char *path);
19 // one pixel of one frame, or 0 when it is clear or out of the sheet
20 unsigned int sprite_at(const struct sprite *s, int frame, int x, int y);
21
22 #endif
```

SRC SPRITE C LIGNES 1 A 67

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #include "sprite.h"
6
7  int sprite_load(struct sprite *s, const char *path)
8  {
9      memset(s, 0, sizeof *s);
10     FILE *f = fopen(path, "rb");
11     if (!f)
12         return 0;
13     char magic[4];
14     unsigned int head[3];
15     if (fread(magic, 1, 4, f) != 4 || memcmp(magic, "TECS", 4) != 0
16         || fread(head, sizeof *head, 3, f) != 3) {
17         fclose(f);
18         return 0;
19     }
20     s->width = (int)head[0];
21     s->height = (int)head[1];
22     s->frames = (int)head[2];
23     size_t amount = (size_t)s->width * s->height * s->frames;
24     if (s->width <= 0 || s->height <= 0 || s->frames <= 0 || amount > 64u
25     << 20) {
26         fclose(f);
27         return 0;
28     }
29     s->pixels = malloc(amount * sizeof *s->pixels);
```

Ligne par ligne

- 15** Le rectangle des pixels opaques, calculé au chargement.
- 15** Les quatre lettres du début. Un fichier qui ne les porte pas est refusé.
- 23** Le nombre de pixels de la planche entière.
- 24** Une taille absurde est refusée avant d'allouer quoi que ce soit. La limite est de 64 millions de pixels.

PARTIE 4 CHAPITRE 1 LE CODE

SUITE

```
29     if (!s->pixels || fread(s->pixels, sizeof *s->pixels, amount, f) !=
amount) {
30         free(s->pixels);
31         s->pixels = NULL;
32         fclose(f);
33         return 0;
34     }
35     fclose(f);
36
37     // the box: the smallest rectangle holding everything that is not
38     // clear, across every frame
39     s->x0 = s->width;
40     s->y0 = s->height;
41     s->x1 = 0;
42     s->y1 = 0;
43     for (int n = 0; n < s->frames; n++)
44         for (int y = 0; y < s->height; y++)
45             for (int x = 0; x < s->width; x++)
46                 if (s->pixels[((size_t)n * s->height + y)
47                     * s->width + x] >> 24 >= 128) {
48                     if (x < s->x0) s->x0 = x;
49                     if (y < s->y0) s->y0 = y;
50                     if (x >= s->x1) s->x1 = x + 1;
51                     if (y >= s->y1) s->y1 = y + 1;
52                 }
53     if (s->x1 <= s->x0 || s->y1 <= s->y0) {
54         s->x0 = s->y0 = 0;
55         s->x1 = s->width;
56         s->y1 = s->height;
57     }
58     return 1;
59 }
60
61 unsigned int sprite_at(const struct sprite *s, int frame, int x, int y)
62 {
63     if (!s->pixels || frame < 0 || frame >= s->frames
64         || x < 0 || y < 0 || x >= s->width || y >= s->height)
65         return 0;
66     return s->pixels[((size_t)frame * s->height + y) * s->width + x];
67 }
```

47 L'opacité, l'octet du haut du pixel. Un pixel à moitié opaque ou plus compte.

66 Le pixel demandé. Les images se suivent en mémoire, chacune ligne par ligne.

Une créature dans le couloir

LA PLANCHE DE LA CREATURE 25 IMAGES



Figure 2.1. La planche de la créature. Cinq angles par rangée, de dos à face, quatre rangées pour les pas de la marche, et une rangée pour le corps au sol.

Dans ce chapitre, vous allez placer une créature dans le monde et apprendre à projeter un point du monde sur l'écran.

Une créature est une position dans le niveau, une direction, un état et une planche d'images. Le fichier `thing.c` en tient jusqu'à 36 dans un tableau. Le fichier du niveau les place, la lettre x pour une créature debout et la lettre y pour un corps au sol, et la fonction `things_from_level` les crée au chargement.

Pour dessiner une créature, le programme doit trouver où elle apparaît à l'écran et à quelle taille. La méthode est celle de tous les jeux du genre. Le vecteur qui va du joueur à la créature est exprimé dans le repère de la caméra, c'est-à-dire décomposé en une profondeur, le long de la direction du regard, et un écart, le long du plan de la caméra. Cette décomposition est l'inverse du calcul des rayons, et elle utilise l'inverse de la matrice formée par les vecteurs `plane` et `dir`. La page suivante fait le calcul avec des nombres.

La profondeur joue le rôle de la distance d'un mur. La hauteur de la créature à l'écran est la hauteur de l'image divisée par la profondeur, et sa largeur suit les proportions de la planche. La colonne de son centre est donnée par l'écart divisé par la profondeur, un nombre entre -1 et 1 ramené sur la largeur de l'écran. Une créature dont la profondeur est inférieure à 0,35 est derrière le joueur ou trop près, et n'est pas dessinée.

Les pieds doivent rester au sol. Le sol à cette profondeur se trouve à une demi-hauteur sous l'horizon, et la créature est dessinée depuis ce point vers le haut. Centrée sur l'horizon, une créature moins haute qu'un couloir flotterait. Les pixels transparents de la planche sont sautés, et pour le moment la créature est dessinée par-dessus tout, même derrière un mur. Le chapitre suivant règle ce point.

Un exemple de projection

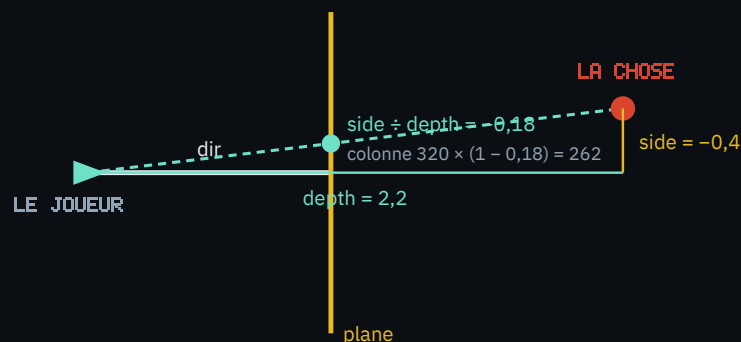


Figure 2.2. Une créature vue de dessus, à la profondeur 2,2 et à l'écart -0,4. Le rayon qui va du joueur à la créature coupe le plan de la caméra à -0,18, et ce nombre donne la colonne de l'écran.

Le joueur regarde dans la direction (1 ; 0), avec un vecteur `plane` de (0 ; 0,66), et une créature se trouve à 2,2 cases devant lui et 0,4 case sur sa gauche. Le vecteur qui va du joueur à la créature vaut donc (2,2 ; -0,4), puisque la gauche du joueur est le sens des y négatifs.

Le programme décompose ce vecteur en deux nombres, la profondeur, le long de la direction, et l'écart latéral, le long du plan de la caméra, avec l'inverse de la matrice formée par `plane` et `dir`. Le déterminant vaut $\text{plane}_x \times \text{dir}_y - \text{dir}_x \times \text{plane}_y = 0 - 0,66 = -0,66$. L'écart vaut $(\text{dir}_y \times rx - \text{dir}_x \times ry) \div \text{déterminant} = 0,4 \div (-0,66) = -0,61$. La profondeur vaut $(-\text{plane}_y \times rx + \text{plane}_x \times ry) \div \text{déterminant} = -1,452 \div (-0,66) = 2,2$.

La colonne du centre de la créature vaut alors la demi-largeur multipliée par 1 + écart ÷ profondeur, soit $320 \times (1 - 0,61 \div 2,2) = 231$ pour une image de 640 pixels, à gauche du centre. Sa hauteur est la hauteur de l'image divisée par la profondeur, $400 \div 2,2 = 181$ pixels, comme un mur à cette distance.

EXERCICE

Refaites le calcul pour une créature en (3 ; 0), droit devant. L'écart vaut 0, la colonne 320, et la hauteur $400 \div 3 = 133$ pixels.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE THING H LIGNES 1 A 34

```

1  #ifndef THING_H_INCLUDED
2  #define THING_H_INCLUDED
3
4  #include "world.h"
5
6  // a thing is a sheet of pixels prepared offline, a place in the world,
   and
7  // four states. the engine still has no image decoder: the sheet is a raw
8  // file one can replace without rebuilding.
9
10 #define THINGS_MAX      36
11 #define THING_HEALTH    100.0
12
13 enum thing_state { THING_IDLE, THING_ALERT, THING_HUNT, THING_STRIKE,
   THING_DEAD };
14
15 struct thing {
16     double x, y;
17     double dir_x, dir_y;
18     enum thing_state state;
19     double since;           // when it entered this state
20     double health;
21     double stride;         // the walk cycle
22     int used;
23 };
24
25 extern struct thing things[THINGS_MAX];
26
27 void things_clear(void);
28 // the level says where they are: "x" one standing, "y" a body.
29 void things_from_level(void);
30 int  thing_add(double x, double y);
31 // drawn after the walls
32 void things_draw(const struct player *player, double now);
33
34 #endif

```

SRC MAIN C LIGNES 13 A 17

```

13 #include "screen.h"
14 #include "texture.h"
15 #include "thing.h"
16 #include "world.h"
17

```

SRC MAIN C LIGNES 61 A 66

```

61     struct keys keys = { .map = 1 };
62     double last = now_in_seconds();
63     things_clear();
64     things_from_level();
65     // the first thing one reads, and the only one nobody triggers
66     say("ARROWS TO MOVE      M FOR THE MAP", last);

```

SRC MAIN C LIGNES 120 A 124

```

120         draw_text_centered(view_height - NOTICE_UP, notice,
121             TEXT_COLOR, TEXT_SCALE);
122     things_draw(&player, moment);
123     screen_present(&screen);
124

```

Ligne par ligne

13 Les cinq états d'une créature. Seuls le premier et le dernier servent dans ce chapitre.

PARTIE 4 CHAPITRE 2 LE CODE

SUITE

SRC THING C LIGNES 1 A 100

```
1  #include <math.h>
2  #include <string.h>
3
4  #include "thing.h"
5  #include "render.h"
6  #include "screen.h"
7  #include "sprite.h"
8  #include "world.h"
9
10 struct thing things[THINGS_MAX];
11
12 static struct sprite sheet;
13
14 static void load_sheets(void)
15 {
16     static int tried;
17     if (!tried++)
18         sprite_load(&sheet, "art/thing.tecs");
19 }
20
21 void things_clear(void)
22 {
23     memset(things, 0, sizeof things);
24 }
25
26 void things_from_level(void)
27 {
28     for (int y = 0; y < map_height; y++)
29         for (int x = 0; x < map_width; x++) {
30             char c = map[y][x];
31             int i = -1;
32             if (c == 'x' || c == 'y')
33                 i = thing_add(x + 0.5, y + 0.5);
34             if (i < 0)
35                 continue;
36             if (c == 'y') {
37                 // a body on the floor: the level places them,
38                 // and they say what happened before us
39                 things[i].state = THING_DEAD;
40                 things[i].health = 0.0;
41             }
42         }
43 }
44
45 int thing_add(double x, double y)
46 {
47     for (int i = 0; i < THINGS_MAX; i++)
48         if (!things[i].used) {
49             things[i] = (struct thing){ .x = x, .y = y, .dir_x = 1.0,
50             .health = THING_HEALTH, .used = 1 };
51             return i;
52         }
53     return -1;
54 }
55
56 void things_draw(const struct player *player, double now)
57 {
58     (void)now;
59     // the camera matrix, inverted once for all of them
60     double det = player->plane_x * player->dir_y - player->dir_x *
player->plane_y;
61     if (fabs(det) < 1e-9)
62         return;
63     double inv = 1.0 / det;
64
65     load_sheets();
```

32 Les deux lettres du fichier qui placent une créature.

60 Le déterminant de la matrice de la caméra, calculé une fois pour toutes les créatures.

PARTIE 4 CHAPITRE 2 LE CODE

SUITE

```
66     for (int n = 0; n < THINGS_MAX; n++) {
67         struct thing *t = &things[n];
68         if (!t->used || !sheet.pixels)
69             continue;
70         double rx = t->x - player->x, ry = t->y - player->y;
71         double side = inv * (player->dir_y * rx - player->dir_x * ry);
72         double depth = inv * (-player->plane_y * rx + player->plane_x *
ry);
73         if (depth < 0.35)
74             continue;
75
76         int middle = (int)((view_width / 2) * (1 + side / depth));
77         // the feet stay on the floor whatever the size. the ground at
78         // that depth is half a height below the horizon: start there
79         // and go up. centring on the horizon floats anything that is
80         // not exactly one corridor tall.
81         double full = view_height / depth;
82         int height = (int)full;
83         int width = height * sheet.width / sheet.height;
84         int top = (int)(HORIZON + full * 0.5) - height;
85         for (int x = middle - width / 2; x < middle + width / 2; x++) {
86             if (x < 0 || x >= view_width)
87                 continue;
88             int sx = (x - (middle - width / 2)) * sheet.width / width;
89             for (int y = top; y < top + height; y++) {
90                 if (y < 0 || y >= view_height)
91                     continue;
92                 int sy = (y - top) * sheet.height / height;
93                 unsigned int c = sprite_at(&sheet, 0, sx, sy);
94                 if ((c >> 24) < 128)
95                     continue;
96                 view[y * view_width + x] = c;
97             }
98         }
99     }
100 }
```

SRC WORLD C LIGNES 291 A 296

```
291     if (c == '+')
292         return door_at(x, y) < DOOR_WALKABLE;
293     return !(c == '.' || c == 'S' || c == 'E' || c == 'K'
294             || c == 'x' || c == 'y');
295 }
296
```

- 71** L'écart latéral de la créature, dans le repère de la caméra.
- 72** Sa profondeur, la distance qui sert à la taille.
- 76** La colonne du centre de la créature.
- 84** Le haut de l'image, calculé depuis le sol pour que les pieds restent posés.
- 94** Un pixel transparent est sauté.



Figure 2.2. La première créature dans le couloir, dessinée à sa taille et à sa place. Elle ne bouge pas encore, et une seule image de la planche est utilisée.

VOCABULAIRE

Le repère de la caméra est le système de coordonnées formé par la direction du regard et le plan de la caméra. Un point y a une profondeur et un écart. **Un déterminant** est le nombre qui permet d'inverser une matrice de deux vecteurs.

Le tampon de profondeur

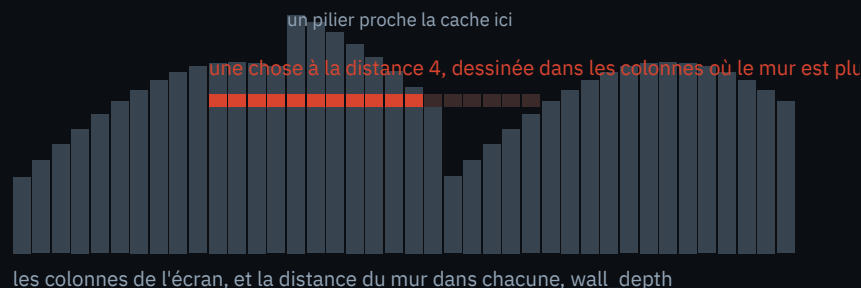


Figure 3.1. Le tampon de profondeur, une distance par colonne de l'écran. Une créature à la distance 4 n'est dessinée que dans les colonnes où le mur est plus loin qu'elle. Un pilier proche la cache sur quelques colonnes.

Dans ce chapitre, vous allez écrire un tampon de profondeur pour que les murs cachent les créatures qui sont derrière eux.

Une créature derrière un mur doit être cachée par le mur. Pour le savoir, le programme conserve, pour chaque colonne de l'écran, la distance du mur que le rayon de cette colonne a touché. Ce tableau s'appelle le tampon de profondeur, `wall_depth`. La fonction `render_walls` le remplit en même temps qu'elle dessine les murs.

Au moment de dessiner une créature, chaque colonne de son image est comparée au tampon. Si la profondeur de la créature est supérieure à la distance du mur dans cette colonne, la colonne n'est pas dessinée, car le mur est devant. Une créature à moitié derrière un pilier est ainsi coupée au bord du pilier, colonne par colonne. Les créatures doivent donc être dessinées après les murs, jamais avant.

VOCABULAIRE

Le tampon de profondeur est le tableau qui conserve, pour chaque colonne, la distance du mur touché par le rayon de cette colonne.

Le tampon de profondeur, suite



Figure 3.2. La créature est correctement cachée par les murs. Ici, rien ne se trouve entre elle et le joueur.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE RENDER H LIGNES 12 A 20

```
12 // how wide the view is: 0.66 against a unit direction is about 66
    degrees
13 #define FIELD_OF_VIEW 0.66
14
15 // how far the wall is, column by column. the walls fill it and the
16 // sprites read it: anything further than the wall in its column is
17 // behind it, and is not drawn. one array answers the whole question.
18 extern double *wall_depth;
19
20 // never divide by less than this: a wall right against the eye
```

INCLUDE THING H LIGNES 29 A 33

```
29 void things_from_level(void);
30 int  thing_add(double x, double y);
31 // drawn after the walls, so the depth buffer is full
32 void things_draw(const struct player *player, double now);
33
```

SRC RENDER C LIGNES 6 A 10

```
6
7 #include <math.h>
8 #include <stdlib.h>
9
10 // one column of the wall, read down one column of the texture
```

SRC RENDER C LIGNES 210 A 229

```
210
211 // one ray per column of the screen, and how far that ray went decides
    how
212 // tall the wall is drawn: near is tall, far is short. the depth buffer
213 // holds one value per column: the walls fill it and the things read it.
214 double *wall_depth;
215
216 void render_walls(const struct player *player)
217 {
218     // it follows the width of the view, which changes with the window
219     static int columns;
220     if (columns != view_width) {
221         double *bigger = realloc(wall_depth,
222             (size_t)view_width * sizeof(*wall_depth));
223         if (bigger) {
224             wall_depth = bigger;
225             columns = view_width;
226         }
227     }
228     for (int x = 0; x < view_width; x++) {
229         // -1 on the left edge of the screen, +1 on the right edge
```

SRC RENDER C LIGNES 232 A 239

```
232     double ray_y = player->dir_y + player->plane_y * camera;
233     struct hit hit = cast_ray(player, ray_x, ray_y);
234     // keep it: this is what tells the things whether they
235     // stand in front of that wall or behind it
236     if (wall_depth)
237         wall_depth[x] = hit.distance;
238
239     int height = (int)(view_height / hit.distance);
```

Ligne par ligne

- 18** Le tampon, une distance par colonne. Les murs l'écrivent, les créatures le lisent.
- 220** Le tampon suit la largeur de l'image, qui change avec la fenêtre.
- 237** La distance du mur est conservée pour chaque colonne.

PARTIE 4 CHAPITRE 3 LE CODE

SUITE

SRC THING C LIGNES 59 A 63

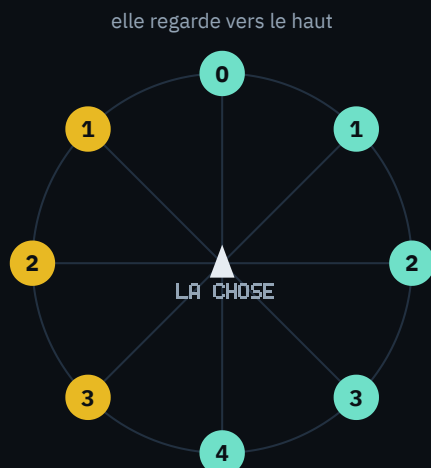
```
59     // the camera matrix, inverted once for all of them
60     double det = player->plane_x * player->dir_y - player->dir_x *
player->plane_y;
61     if (fabs(det) < 1e-9 || !wall_depth)
62         return;
63     double inv = 1.0 / det;
```

86 Une colonne de la créature plus loin que le mur n'est pas dessinée.

SRC THING C LIGNES 84 A 88

```
84     int top = (int)(HORIZON + full * 0.5) - height;
85     for (int x = middle - width / 2; x < middle + width / 2; x++) {
86         if (x < 0 || x >= view_width || depth >= wall_depth[x])
87             continue;
88         int sx = (x - (middle - width / 2)) * sheet.width / width;
```

Cinq angles, trois en miroir



Huit secteurs autour de la chose.

Cinq sont dessinés sur la planche, de face (secteur 4, en bas) à dos (secteur 0).

Les trois autres sont les mêmes images retournées, le secteur 5 est le 3 en miroir

Le secteur est choisi d'après l'angle entre la direction de la chose et la direction du joueur vu depuis la chose.

Figure 4.1. Les huit secteurs autour d'une créature qui regarde vers le haut. Les cinq secteurs verts ont leur image dans la planche. Les trois secteurs jaunes réutilisent l'image du secteur opposé, retournée de gauche à droite.

Dans ce chapitre, vous allez choisir l'image de la créature selon l'angle sous lequel le joueur la voit, et trier les créatures par distance.

Une créature vue de face quand elle approche et de dos quand elle s'éloigne vit dans le monde. Une créature toujours vue de face tourne avec la caméra. La planche contient cinq angles, de dos à face, pour chaque pas de la marche. Les trois angles manquants, ceux du côté gauche, sont obtenus en retournant les images du côté droit. Les jeux de 1993 procédaient ainsi pour tenir sur une disquette.

Le programme calcule l'angle entre la direction de la créature et la direction du joueur vu depuis elle, puis le ramène à l'un de huit secteurs de 45 degrés. Les secteurs 0 à 4 sont dans la planche, et les secteurs 5, 6 et 7 sont les secteurs 3, 2 et 1 en miroir. La rangée de la planche dépend de l'état, la rangée du corps au sol pour une créature morte, et sinon le pas de marche courant. L'image finale est la rangée multipliée par 5 plus le secteur.

Les créatures sont aussi triées par distance avant d'être dessinées, de la plus lointaine à la plus proche. Une créature proche recouvre ainsi une créature lointaine, comme le tampon de profondeur le fait pour les murs.

Cinq angles, trois en miroir, suite

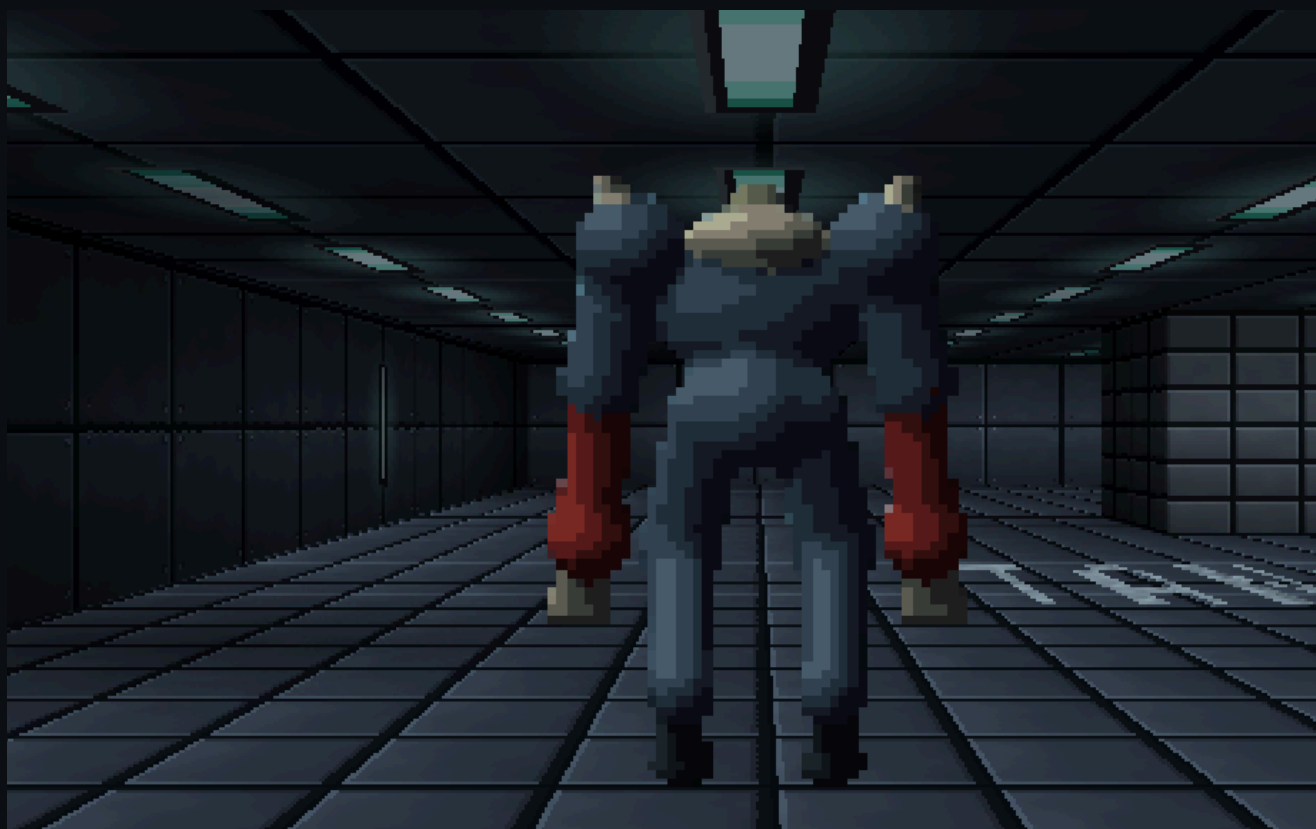


Figure 4.2. La créature vue de dos. Elle regarde vers l'est, et le joueur la voit depuis l'ouest.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

SRC THING C LIGNES 10 A 18

```
10 struct thing things[THINGS_MAX];
11
12 // the sheet, loaded once. five angles per step of the walk, four steps:
the
13 // three missing angles come back mirrored, the way games did in 1993 to
fit
14 // on a floppy.
15 #define ANGLES 5
16 #define WALK_STEPS 4
17 static struct sprite sheet;
18
```

SRC THING C LIGNES 68 A 90

```
68 double inv = 1.0 / det;
69
70 // farthest first, so a near one covers a far one
71 int order[THINGS_MAX], count = 0;
72 for (int i = 0; i < THINGS_MAX; i++)
73     if (things[i].used)
74         order[count++] = i;
75 for (int a = 0; a < count; a++)
76     for (int b = a + 1; b < count; b++) {
77         struct thing *p = &things[order[a]], *q = &things[order[b]];
78         double da = (p->x - player->x) * (p->x - player->x)
79             + (p->y - player->y) * (p->y - player->y);
80         double db = (q->x - player->x) * (q->x - player->x)
81             + (q->y - player->y) * (q->y - player->y);
82         if (db > da) { int t = order[a]; order[a] = order[b];
order[b] = t; }
83     }
84
85 load_sheets();
86 for (int n = 0; n < count; n++) {
87     struct thing *t = &things[order[n]];
88     if (!sheet.pixels)
89         continue;
90     double rx = t->x - player->x, ry = t->y - player->y;
```

SRC THING C LIGNES 103 A 139

```
103 int width = height * sheet.width / sheet.height;
104 int top = (int)(HORIZON + full * 0.5) - height;
105
106 // which angle we see of it: between the way it faces and the way
107 // we look at it. head on when it comes, from behind when it
108 // leaves. that is what makes it live in the world instead of
109 // turning with the camera.
110 double to_x = -rx, to_y = -ry;
111 double n2 = sqrt(to_x * to_x + to_y * to_y);
112 if (n2 > 0.0) { to_x /= n2; to_y /= n2; }
113 double cosa = t->dir_x * to_x + t->dir_y * to_y;
114 double sina = t->dir_x * to_y - t->dir_y * to_x;
115 double angle = atan2(sina, cosa); // -pi to pi
116 int sector = (int)floor((angle + M_PI) / (2 * M_PI) * 8.0 + 0.5)
% 8;
117 int mirror = 0;
118 if (sector > 4) { // the three missing
```

Ligne par ligne

- 15** Cinq angles par pas de marche dans la planche.
- 82** Le tri par distance, la plus lointaine d'abord.
- 115** L'angle entre la direction de la créature et celle du joueur vu depuis elle, entre $-\pi$ et π .
- 116** L'angle ramené à un secteur de 0 à 7.
- 118** Les secteurs 5, 6 et 7 deviennent 3, 2 et 1, en miroir.

PARTIE 4 CHAPITRE 4 LE CODE

SUITE

```
119         sector = 8 - sector;
120         mirror = 1;
121     }
122     int row = t->state == THING_DEAD ? WALK_STEPS
123             : ((int)(t->stride * 2.0) % WALK_STEPS);
124     int frame = row * ANGLES + sector;
125     if (frame >= sheet.frames)
126         frame = sector % sheet.frames;
127     for (int x = middle - width / 2; x < middle + width / 2; x++) {
128         if (x < 0 || x >= view_width || depth >= wall_depth[x])
129             continue;
130         int sx = (x - (middle - width / 2)) * sheet.width / width;
131         if (mirror)
132             sx = sheet.width - 1 - sx;
133         for (int y = top; y < top + height; y++) {
134             if (y < 0 || y >= view_height)
135                 continue;
136             int sy = (y - top) * sheet.height / height;
137             unsigned int c = sprite_at(&sheet, frame, sx, sy);
138             if ((c >> 24) < 128)
139                 continue;
```

124 L'image de la planche, par rangée et par secteur.

132 Le miroir, la colonne de gauche devient la colonne de droite.

Elle vous voit, et elle vient

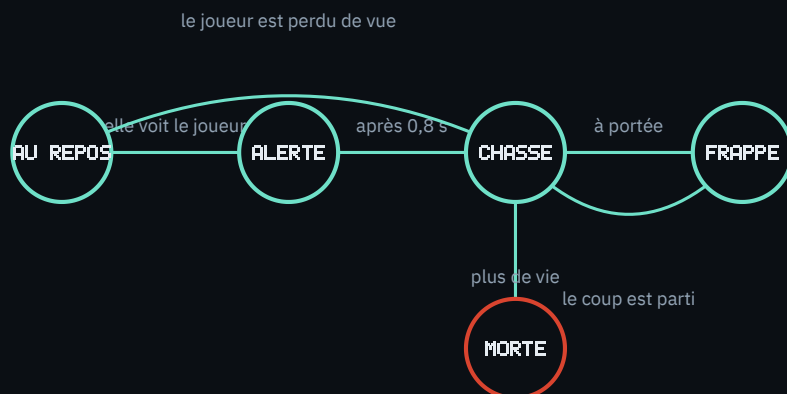


Figure 5.1. Les états d'une créature et les passages de l'un à l'autre. Une créature frappée passe aussi de l'alerte à la chasse, et une créature qui n'a plus de vie meurt dans n'importe quel état.

Dans ce chapitre, vous allez donner un comportement aux créatures avec une machine à états, la vue, la poursuite et l'attaque.

Une créature vit selon une machine à états, dont la figure 5.1 donne le schéma. Au repos, elle attend. Quand elle voit le joueur, elle passe en alerte pendant 0,8 seconde, une pause qui laisse au joueur le temps de comprendre. Puis elle chasse, en marchant vers le dernier endroit où elle a vu le joueur. À portée, elle frappe. Le coup part au bout de 0,45 seconde, et non aussitôt, ce qui laisse le temps de reculer. Après le coup, elle attend 1,45 seconde avant de pouvoir frapper de nouveau.

La fonction `sees` dit si une créature voit le joueur. Elle parcourt la ligne droite entre les deux, par petits pas, et s'arrête au premier mur. Une créature qui arrive là où elle a vu le joueur pour la dernière fois sans le retrouver retourne au repos. Couper la ligne de vue sert donc à quelque chose, et fermer une porte devient une tactique.

La fonction `step_towards` fait marcher une créature vers un point. Elle tourne progressivement, sans pivoter sur place, avance à 1,9 case par seconde, et teste chaque axe séparément pour glisser le long des murs, comme le joueur. En chasse, elle ne marche pas en ligne droite. Elle ondule autour de sa cible, chacune avec sa propre phase, pour que deux créatures ne se balancent pas de la même façon. Une créature qui vient droit sur le joueur se joue en reculant. Une créature qui ondule doit être suivie.

Elle s'arrête à 1,55 case du joueur. Plus près, une créature aussi haute que le couloir remplirait tout l'écran et les coups viendraient de nulle part. Chaque coup retire 9 points de vie, et la fonction `things_update` renvoie la somme des dégâts infligés pendant l'image.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE THING H LIGNES 9 A 26

```

9
10 #define THINGS_MAX      36
11 // how close it gets before it swings. under a metre and a half a thing
  as
12 // tall as the corridor fills the whole screen and one sees nothing at
  all:
13 // not it, not the room, not where the next blow comes from. it stops at
14 // arm's length, and the bar is longer than its arm.
15 #define THING_REACH      1.55
16 #define THING_SPEED      1.9
17 #define THING_SIGHT      11.0
18 #define THING_HEALTH     100.0
19 #define THING_DAMAGE      9.0
20 // it does not hit without pause. without this gap two of them against a
  wall
21 // take a hundred points off in two and a half seconds: one does not die
  of a
22 // mistake, one dies of having walked in. there has to be time to see the
  blow
23 // start, to step back, and to answer it.
24 #define THING_GAP        1.45
25
26 enum thing_state { THING_IDLE, THING_ALERT, THING_HUNT, THING_STRIKE,
  THING_DEAD };

```

INCLUDE THING H LIGNES 33 A 43

```

33 double health;
34 double stride;          // the walk cycle
35 double last_seen_x, last_seen_y;
36 double next_strike;     // not before that moment
37 // it does not walk in a straight line and it does not shrug off a
  hit.
38 // a thing that comes at you and never flinches is played by walking
39 // backwards; one that weaves and staggers has to be played.
40 double weave;           // its own phase, so they do not all sway
  alike
41 double stagger;         // it just took one: it shows
42 int used;
43 };

```

INCLUDE THING H LIGNES 49 A 54

```

49 void things_from_level(void);
50 int thing_add(double x, double y);
51 // their whole life: look, walk, strike. returns the damage dealt this
  frame
52 double things_update(const struct player *player, double elapsed, double
  now);
53 // drawn after the walls, so the depth buffer is full
54 void things_draw(const struct player *player, double now);

```

SRC MAIN C LIGNES 85 A 89

```

85 if (walk_over(&player))
86     say("SECURITY BADGE TAKEN", moment);
87 things_update(&player, elapsed, moment);
88 move_doors(elapsed);
89 lamp_flicker(moment);

```

Ligne par ligne

- 15** La distance à laquelle une créature s'arrête et frappe.
- 24** Le temps minimal entre deux coups d'une même créature.

PARTIE 4 CHAPITRE 5 LE CODE

SUITE

SRC THING C LIGNES 53 A 194

```
53     if (!things[i].used) {
54         things[i] = (struct thing){ .x = x, .y = y, .dir_x = 1.0,
55             .health = THING_HEALTH, .used = 1,
56             .weave = i * 1.7 };
57         return i;
58     }
59     return -1;
60 }
61
62 // can it see him? a straight line through the grid, and the first wall
63 // stops
64 // it. this is the same walk the rays do, kept simple on purpose
65 static int sees(const struct thing *t, const struct player *p)
66 {
67     double dx = p->x - t->x, dy = p->y - t->y;
68     double d = sqrt(dx * dx + dy * dy);
69     if (d > THING_SIGHT)
70         return 0;
71     int steps = (int)(d * 8.0) + 1;
72     for (int i = 1; i < steps; i++) {
73         double f = (double)i / steps;
74         if (is_wall((int)(t->x + dx * f), (int)(t->y + dy * f)))
75             return 0;
76     }
77     return 1;
78 }
79
80 static void step_towards(struct thing *t, double tx, double ty, double
81     elapsed)
82 {
83     double dx = tx - t->x, dy = ty - t->y;
84     double d = sqrt(dx * dx + dy * dy);
85     if (d < 0.001)
86         return;
87     dx /= d; dy /= d;
88     // it turns, it does not snap: a thing that pivots on the spot is a
89     // cursor
90     double turn = 6.0 * elapsed;
91     t->dir_x += (dx - t->dir_x) * turn;
92     t->dir_y += (dy - t->dir_y) * turn;
93     double n = sqrt(t->dir_x * t->dir_x + t->dir_y * t->dir_y);
94     if (n > 0.0) { t->dir_x /= n; t->dir_y /= n; }
95
96     double move = THING_SPEED * elapsed;
97     // each axis on its own, so a shoulder on a wall keeps sliding
98     if (!is_wall((int)(t->x + t->dir_x * move * 1.6), (int)t->y))
99         t->x += t->dir_x * move;
100     if (!is_wall((int)t->x, (int)(t->y + t->dir_y * move * 1.6)))
101         t->y += t->dir_y * move;
102     t->stride += move * 1.7;
103 }
104
105 double things_update(const struct player *player, double elapsed, double
106     now)
107 {
108     double damage = 0.0;
109     for (int i = 0; i < THINGS_MAX; i++) {
110         struct thing *t = &things[i];
111         if (!t->used || t->state == THING_DEAD)
112             continue;
113         double dx = player->x - t->x, dy = player->y - t->y;
114         double distance = sqrt(dx * dx + dy * dy);
115         int visible = sees(t, player);
116         if (visible) {
117             t->last_seen_x = player->x;
118             t->last_seen_y = player->y;
```

64 La ligne de vue, parcourue par pas d'un huitième de case.

88 La direction tourne progressivement vers la cible.

PARTIE 4 CHAPITRE 5 LE CODE

SUITE

```
115     }
116
117     switch (t->state) {
118     case THING_IDLE:
119         // it does not charge the moment it sees you. it
120         // wakes, it shrieks, and then it comes: that second
121         // of warning turns a scare into a fight
122         if (visible) {
123             t->state = THING_ALERT;
124             t->since = now;
125         }
126         break;
127     case THING_ALERT:
128         if (now - t->since > 0.8) {
129             t->state = THING_HUNT;
130             t->since = now;
131         }
132         break;
133     case THING_HUNT: {
134         // it weaves as it closes. straight at you it is played
135         // by walking backwards; across, it has to be followed.
136         // the offset is worked out here and thrown away: writing
137         // it into last_seen would pile up frame after frame.
138         double bx = t->last_seen_x, by = t->last_seen_y;
139         if (now > t->stagger) {
140             double margin = distance > 3.0 ? 1.1 : distance * 0.35;
141             double ox = -(player->y - t->y), oy = player->x - t->x;
142             double n = sqrt(ox * ox + oy * oy);
143             if (n > 0.001) {
144                 double k = margin * sin(now * 2.3 + t->weave);
145                 bx += ox / n * k;
146                 by += oy / n * k;
147             }
148         }
149         // it stops at arm's length. without that it walks into
150         // the player, drops under the nearest drawing distance,
151         // and the blows come out of nowhere.
152         if (distance > THING_REACH * 0.85 && now > t->stagger)
153             step_towards(t, bx, by, elapsed);
154         if (distance < THING_REACH && visible && now > t->
>next_strike) {
155             t->state = THING_STRIKE;
156             t->since = now;
157             // it tells you it is coming. a quarter of a
158             // second of posture before it lands: that is
159             // the window to step back.
160         } else if (!visible
161             && fabs(t->x - t->last_seen_x) < 0.4
162             && fabs(t->y - t->last_seen_y) < 0.4) {
163             // it reached where you were and you are not
164             // there: it loses you. breaking the line of
165             // sight has to work, or the doors teach nothing
166             t->state = THING_IDLE;
167             t->since = now;
168         }
169         break;
170     }
171     case THING_STRIKE:
172         // the blow lands at the end of the swing, not at the start:
173         // that quarter second is the window to step back
174         if (now - t->since > 0.45) {
175             if (distance < THING_REACH + 0.3)
176                 damage += THING_DAMAGE;
177             t->state = THING_HUNT;
178             t->since = now;
```

- 118** Au repos, la seule chose qui compte est de voir le joueur.
- 144** L'ondulation autour de la cible, avec une phase propre à chaque créature.
- 174** Le coup part au bout de 0,45 seconde. Avant, le joueur peut reculer.

PARTIE 4 CHAPITRE 5 LE CODE

SUITE

```
179             t->next_strike = now + THING_GAP;
180         }
181         break;
182     default:
183         break;
184     }
185 }
186 return damage;
187 }
188
189 // -----
190 drawing
191 // drawing a body from code was tried and dropped: distance fields give a
192 // shape that reads in ascii and a smear on screen. a sheet metal pattern
193 // can be computed, a body cannot. so we sculpt a sheet and project it.
194 void things_draw(const struct player *player, double now)
```

179 Pas d'autre coup avant
1,45 seconde.



Figure 5.2. La créature a vu le joueur. Elle s'est retournée et elle vient, en ondulant dans le couloir.

VOCABULAIRE

Une **machine à états** est une façon d'organiser un comportement en quelques états distincts, avec des règles de passage de l'un à l'autre.

EXERCICE

Portez **THING_GAP** à 3 secondes et observez que le combat devient facile. Ce nombre est celui qui règle la difficulté.

Une barre d'acier

LA PLANCHE DE LA BARRE 4 IMAGES

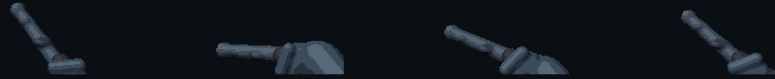


Figure 6.1. Les quatre images de la barre, au repos, pendant le coup et sur le retour. Chaque image couvre tout le cadre du jeu.

Dans ce chapitre, vous allez mettre une arme dans les mains du joueur et l'afficher à l'écran.

Le joueur doit pouvoir répondre. Sa première arme est une barre d'acier arrachée à une cloison. Le fichier `fight.c` tient la vie du joueur et le moment de son dernier coup. La touche Contrôle déclenche `fight_strike`, qui refuse un coup moins de 0,62 seconde après le précédent. Dans ce chapitre, le coup ne touche encore rien.

Ce que le joueur tient est toujours à l'écran. C'est ce qui fait une vue à la première personne, et sans cela le joueur ne sait même pas qu'il peut frapper. La barre et le gant qui la tient viennent d'une planche de quatre images, la barre au repos, le coup, et deux positions de retour. La planche a été photographiée avec le champ de vision exact du jeu et couvre tout le cadre, ce qui garde la perspective juste quelle que soit la taille de la fenêtre.

La fonction `draw_sheet` ne parcourt que le rectangle utile de la planche, calculé au chargement, et non les trois quarts vides. Elle balance légèrement l'arme quand le joueur marche, et l'assombrit selon la lumière de la case où il se trouve. Collée telle quelle, l'arme brillerait comme un autocollant dans un couloir noir.

Une barre d'acier, suite

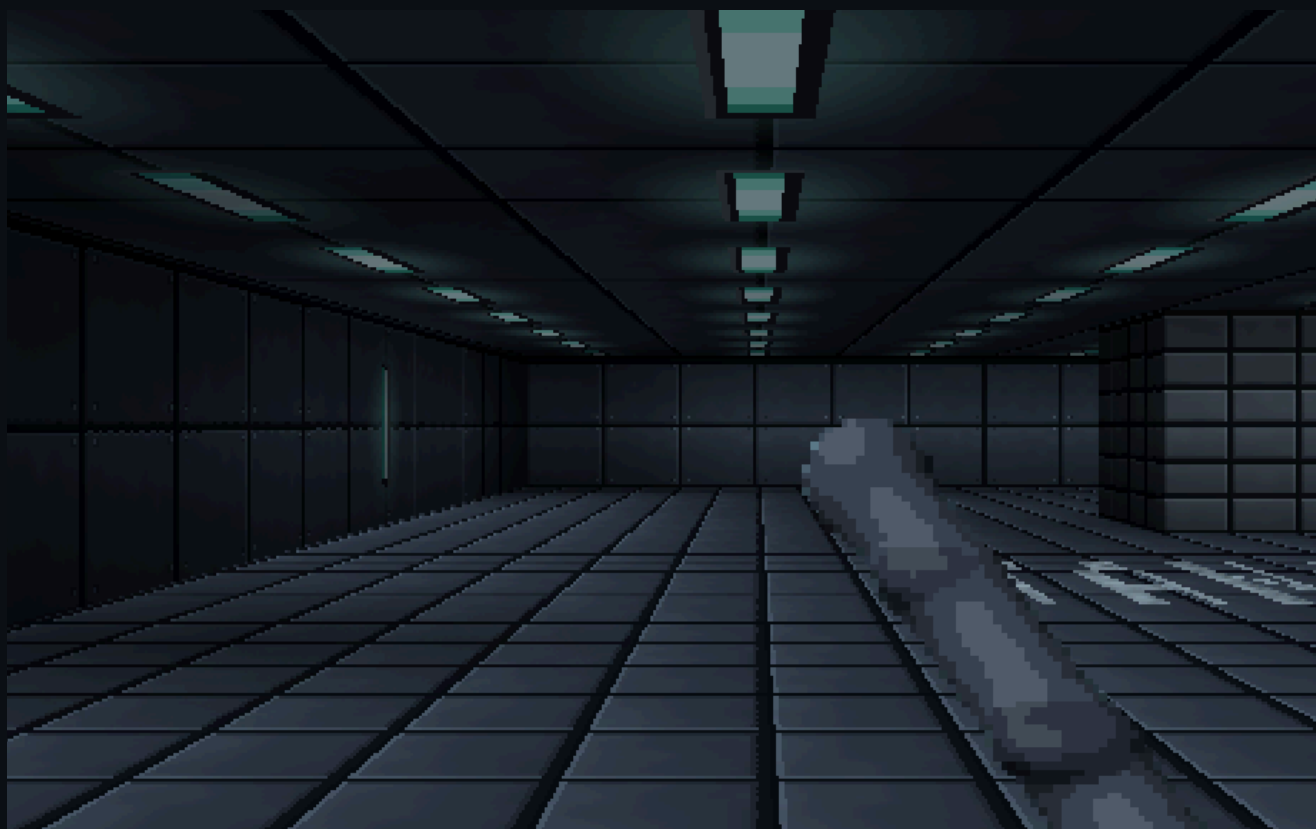


Figure 6.2. La barre d'acier et le gant, en bas à droite de l'image. La touche Contrôle la fait frapper.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

Ligne par ligne

14 Le temps minimal entre deux coups de barre.

INCLUDE FIGHT H LIGNES 1 A 32

```

1  #ifndef FIGHT_H
2  #define FIGHT_H
3
4  #include "world.h"
5
6  // what the player can do about it. a steel bar torn off a bulkhead
   first:
7  // hands would need flesh, and flesh done badly ruins everything around
   it.
8  // a bar is drawn with the same words as the walls, and it says by itself
9  // what it is for.
10 #define PLAYER_HEALTH    100.0
11 #define PIPE_REACH        2.05
12 #define PIPE_DAMAGE       34.0
13 #define PIPE_ARC          0.55      // how wide the swing is, in cosine
14 #define PIPE_COOLDOWN     0.62
15 #define PIPE_SWING        0.34      // how long the swing takes to pass
16
17 struct fight {
18     double health;
19     double last_hit;      // when the player last swung
20 };
21
22 extern struct fight fight;
23
24 void fight_reset(void);
25 // returns 1 when something was hit, so the caller can say so
26 int  fight_strike(double now);
27 // what one holds is always on screen. that is what makes a first person
28 // view: without it the player does not even know he can swing. "moving"
29 // drives the walking sway, "now" the animation of the blow.
30 void fight_draw(const struct player *player, double now, int moving);
31
32 #endif

```

INCLUDE SCREEN H LIGNES 48 A 52

```

48     int push;           // a press on space, waiting to be read
49     int map;            // is the map on screen? M takes it away
50     int hit;            // a press on ctrl: swing
51 };
52

```

SRC FIGHT C LIGNES 1 A 102

```

1  #include <math.h>
2  #include <string.h>
3
4  #include "fight.h"
5  #include "light.h"
6  #include "render.h"
7  #include "screen.h"
8  #include "sprite.h"
9
10 struct fight fight;
11
12 void fight_reset(void)
13 {

```

PARTIE 4 CHAPITRE 6 LE CODE

SUITE

```
14     memset(&fight, 0, sizeof fight);
15     fight.health = PLAYER_HEALTH;
16 }
17
18 // the swing is a cone, and that is the whole point of a bar: it forgives
19 // the aim and charges you the distance. you have to walk up to the thing
20 // you do not know yet.
21 int fight_strike(double now)
22 {
23     if (now - fight.last_hit < PIPE_COOLDOWN)
24         return 0;
25     fight.last_hit = now;
26     return 0;
27 }
28
29 // -----
30 drawing
31 // both weapons are sheets, sculpted and photographed with the exact
32 // field
33 // of view of the game, glove included. a held weapon is a volume seen in
34 // perspective, and that does not work out when drawn flat.
35 static void draw_sheet(struct sprite *p, int frame, double lit, int
36 moving)
37 {
38     if (!p->pixels)
39         return;
40     static double sway;
41     sway += moving ? 0.10 : -sway * 0.10;
42     double scale = view_height / 400.0;
43     int bx = (int)(sin(sway) * 6 * scale);
44     int by = (int)(fabs(cos(sway)) * 5 * scale);
45
46     // walk the box only. the sheet covers the whole frame so the
47     // perspective lands right, but it fills a quarter of it: without
48     // this bound we read two hundred and fifty thousand pixels to lay
49     // down sixty thousand, and the game loses ten frames a second.
50     int ya = p->y0 * view_height / p->height + by;
51     int yb = (p->y1 * view_height + p->height - 1) / p->height + by + 1;
52     int xa = p->x0 * view_width / p->width + bx;
53     int xb = (p->x1 * view_width + p->width - 1) / p->width + bx + 1;
54     if (ya < 0) ya = 0;
55     if (xa < 0) xa = 0;
56     if (yb > view_height) yb = view_height;
57     if (xb > view_width) xb = view_width;
58     for (int y = ya; y < yb; y++) {
59         int sy = (y - by) * p->height / view_height;
60         if (sy < 0 || sy >= p->height)
61             continue;
62         for (int x = xa; x < xb; x++) {
63             int sx = (x - bx) * p->width / view_width;
64             unsigned int c = sprite_at(p, frame, sx, sy);
65             if ((c >> 24) < 128)
66                 continue;
67             // it takes the light of the room you stand in.
68             // rendered lit and pasted as is, it glowed like a
69             // sticker in a black corridor.
70             int r = (int)((c >> 16) & 0xff) * lit;
71             int g = (int)((c >> 8) & 0xff) * lit;
72             int b = (int)((c & 0xff) * lit);
73             view[y * view_width + x] =
74                 (unsigned int)((r << 16) | (g << 8) | b);
75         }
76     }
77 }
```

- 38** Le balancement de la marche, qui s'amortit à l'arrêt.
- 47** Le rectangle utile de la planche, converti aux dimensions de l'image.
- 67** L'arme prend la lumière de la case du joueur.

PARTIE 4 CHAPITRE 6 LE CODE

SUITE

```
75
76 static struct sprite pipe_sheet;
77
78 static void load_sheets(void)
79 {
80     static int tried;
81     if (!tried++)
82         sprite_load(&pipe_sheet, "art/pipe.tecs");
83 }
84
85 static void draw_pipe(double lit, double now, int moving)
86 {
87     // four frames: at rest, the blow, and two on the way back
88     double since_hit = now - fight.last_hit;
89     int frame = since_hit < PIPE_SWING * 0.40 ? 1
90               : since_hit < PIPE_SWING * 0.70 ? 2
91               : since_hit < PIPE_SWING ? 3 : 0;
92     draw_sheet(&pipe_sheet, frame, lit, moving);
93 }
94
95 void fight_draw(const struct player *player, double now, int moving)
96 {
97     load_sheets();
98     double lit = 0.34 + 0.62 * lit_at(player->x, player->y);
99     if (lit > 1.0)
100         lit = 1.0;
101     draw_pipe(lit, now, moving);
102 }
```

89 L'image de la planche
selon le temps écoulé
depuis le coup.

179 Les deux touches Contrôle
déclenchent le coup.

SRC MAIN C LIGNES 8 A 15

```
8 #include <unistd.h>
9
10 #include "fight.h"
11 #include "light.h"
12 #include "render.h"
13 #include "screen.h"
14 #include "text.h"
15 #include "texture.h"
```

SRC MAIN C LIGNES 64 A 68

```
64 things_clear();
65 things_from_level();
66 fight_reset();
67 // the first thing one reads, and the only one nobody triggers
68 say("ARROWS TO MOVE M FOR THE MAP", last);
```

SRC MAIN C LIGNES 87 A 94

```
87 if (walk_over(&player))
88     say("SECURITY BADGE TAKEN", moment);
89 if (keys.hit) {
90     fight_strike(moment);
91     keys.hit = 0;
92 }
93 things_update(&player, elapsed, moment);
94 move_doors(elapsed);
```

SRC MAIN C LIGNES 128 A 133

```
128 TEXT_COLOR, TEXT_SCALE);
129 things_draw(&player, moment);
130 fight_draw(&player, moment, keys.forward || keys.back
131           || keys.strafe_left || keys.strafe_right);
132 screen_present(&screen);
133
```

SRC SCREEN C LIGNES 176 A 181

```
176 switch (key) {
177     case XK_Escape: keys->quit = down; break;
178     case XK_Control_L:
179     case XK_Control_R: if (down) keys->hit = 1; break;
```

PARTIE 4 CHAPITRE 6 LE CODE

SUITE

```
180         case XK_Up:      keys->forward = down; break;
181         case XK_Down:    keys->back = down; break;
```

Le coup est un cône

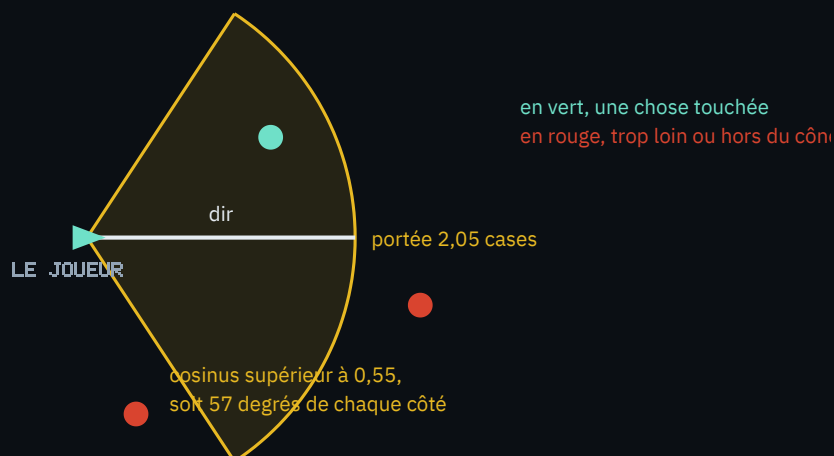


Figure 7.1. Le cône de la barre, vu de dessus. Une créature est touchée si elle se trouve à moins de 2,05 cases et si le cosinus de l'angle entre la direction du joueur et la créature dépasse 0,55.

Dans ce chapitre, vous allez faire toucher la barre, avec un cône défini par un produit scalaire.

La barre touche maintenant. La fonction `fight_strike` cherche la créature la plus proche dans un cône devant le joueur, jusqu'à 2,05 cases. Le cône est défini par un cosinus. Le produit scalaire entre la direction du joueur et le vecteur vers la créature, divisé par la distance, doit dépasser 0,55, ce qui correspond à 57 degrés de chaque côté. Une barre pardonne la visée et fait payer la distance, il faut s'approcher.

La créature touchée perd 34 points de vie. Elle s'arrête pendant un quart de seconde, ce qui donne son rythme au corps à corps, et une créature au repos qui reçoit un coup passe en alerte. Pendant 0,09 seconde, elle est dessinée délavée, une seule image blanchie qui dit que le coup a porté. Sans ce retour visuel, le joueur ne sait pas si sa barre a touché.

Les créatures reçoivent aussi la lumière de la case où elles se trouvent, avec un plancher plus haut que celui de la salle. Une créature aussi sombre que le mur derrière elle ne serait vue qu'au moment où elle frappe, et ce n'est pas de la tension, c'est de l'injustice.

VOCABULAIRE

Le **produit scalaire** de deux vecteurs de longueur 1 est le cosinus de l'angle entre eux. Il vaut 1 quand ils pointent dans le même sens et 0 quand ils sont perpendiculaires.

Le coup est un cône, suite

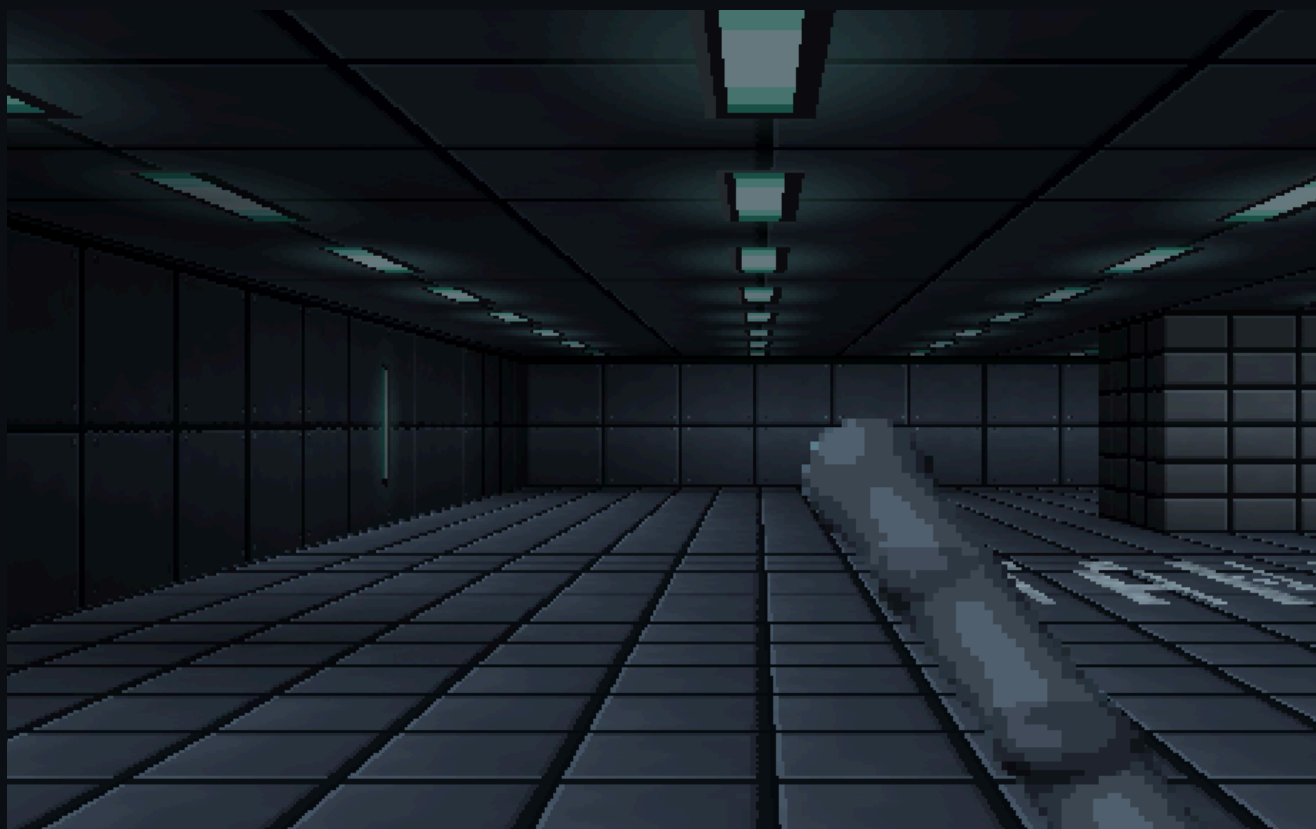


Figure 7.2. La barre frappe. Le blanc d'une image dit que le coup a porté.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE FIGHT H LIGNES 24 A 28

```
24 void fight_reset(void);
25 // returns 1 when something was hit, so the caller can say so
26 int fight_strike(const struct player *player, double now);
27 // what one holds is always on screen. that is what makes a first person
28 // view: without it the player does not even know he can swing. "moving"
```

INCLUDE THING H LIGNES 40 A 46

```
40 double weave; // its own phase, so they do not all sway
    alike
41 double stagger; // it just took one: it shows
42 // when it was hit, and nothing else. using "since" for this washed
    it
43 // out at every change of state: up close it was pale all the time.
44 double hurt_at;
45 int used;
46 };
```

SRC FIGHT C LIGNES 7 A 11

```
7 #include "screen.h"
8 #include "sprite.h"
9 #include "thing.h"
10
11 struct fight fight;
```

SRC FIGHT C LIGNES 20 A 60

```
20 // the aim and charges you the distance. you have to walk up to the thing
21 // you do not know yet.
22 int fight_strike(const struct player *player, double now)
23 {
24     if (now - fight.last_hit < PIPE_COOLDOWN)
25         return 0;
26     fight.last_hit = now;
27     struct thing *best = NULL;
28     double nearest = PIPE_REACH;
29     for (int i = 0; i < THINGS_MAX; i++) {
30         struct thing *t = &things[i];
31         if (!t->used || t->state == THING_DEAD)
32             continue;
33         double dx = t->x - player->x, dy = t->y - player->y;
34         double d = sqrt(dx * dx + dy * dy);
35         if (d > nearest || d < 0.001)
36             continue;
37         // the bar sweeps wide, and that is its whole point
38         if ((dx * player->dir_x + dy * player->dir_y) / d < PIPE_ARC)
39             continue;
40         nearest = d;
41         best = t;
42     }
43     if (!best)
44         return 0;
45     best->health -= PIPE_DAMAGE;
46     // it takes the blow: a quarter second where it stops coming, and
47     // that is what gives melee its rhythm
48     best->stagger = now + 0.26;
49     best->hurt_at = now;
```

Ligne par ligne

- 38** Le cône, un produit scalaire comparé au cosinus de l'angle limite.
- 48** La créature touchée s'arrête un quart de seconde.
- 49** L'instant du coup, pour l'image blanchie.

PARTIE 4 CHAPITRE 7 LE CODE

SUITE

```
50     if (best->health <= 0.0) {
51         best->state = THING_DEAD;
52     } else {
53         // being hit wakes it, even if it had not seen you
54         if (best->state == THING_IDLE)
55             best->state = THING_ALERT;
56         best->since = now;
57     }
58     return 1;
59 }
60
```

264 La lumière de la créature, jamais sous 46 pour cent.

267 L'image blanchie dure 0,09 seconde.

287 Chaque composante est tirée à mi-chemin du blanc.

SRC MAIN C LIGNES 88 A 92

```
88     say("SECURITY BADGE TAKEN", moment);
89     if (keys.hit) {
90         fight_strike(&player, moment);
91         keys.hit = 0;
92     }
```

SRC THING C LIGNES 3 A 7

```
3
4 #include "thing.h"
5 #include "light.h"
6 #include "render.h"
7 #include "screen.h"
```

SRC THING C LIGNES 258 A 269

```
258     if (frame >= sheet.frames)
259         frame = sector % sheet.frames;
260
261     // its floor of light is higher than the room's. a beast as dark
262     // as the wall behind it is only seen as it swings, and that is
263     // not tension, it is unfair.
264     double light = 0.46 + 0.54 * lit_at(t->x, t->y);
265     // it just took a hit: one washed-out frame, and one knows the
266     // shot landed
267     int hit = t->state != THING_DEAD && now - t->hurt_at < 0.09;
268     for (int x = middle - width / 2; x < middle + width / 2; x++) {
269         if (x < 0 || x >= view_width || depth >= wall_depth[x])
```

SRC THING C LIGNES 279 A 293

```
279         if ((c >> 24) < 128)
280             continue;
281         // it takes the light of the square it stands on,
282         // or it glows in the dark like a sticker
283         int r = (int)(((c >> 16) & 0xff) * light);
284         int g = (int)(((c >> 8) & 0xff) * light);
285         int b = (int)((c & 0xff) * light);
286         if (hit) {
287             r += (255 - r) / 2; g += (255 - g) / 2;
288             b += (255 - b) / 2;
289         }
290         view[y * view_width + x] =
291             (unsigned int)((r << 16) | (g << 8) | b);
292     }
293 }
```

Elle rend les coups

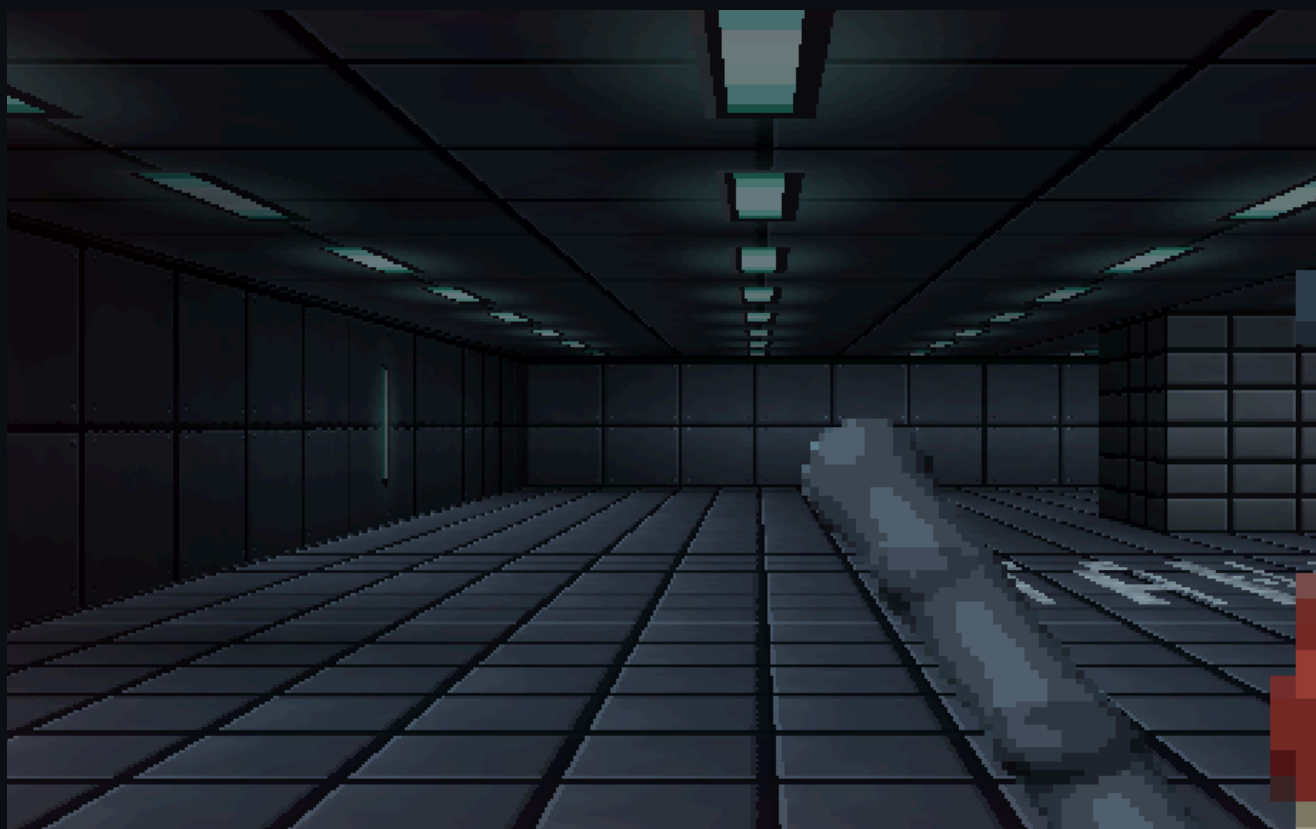


Figure 8.1. Le joueur vient de recevoir un coup. Les bords de l'écran rougissent.

Dans ce chapitre, vous allez faire subir les dégâts au joueur, afficher qu'il est touché, et lui permettre de recommencer après sa mort.

Les dégâts calculés par `things_update` atteignent maintenant le joueur. La fonction `fight_take` retire les points de vie et allume pendant 0,45 seconde un rougeoiement sur les bords de l'écran. Les bords et non le centre, car c'est là que l'œil le remarque sans perdre la salle de vue.

Un joueur mort ne doit pas relancer le programme. Un joueur renvoyé au terminal pour réessayer ne réessaie pas. Quand la vie tombe à zéro, le message change après 1,5 seconde pour proposer la touche Espace, qui recharge le niveau, remplace le joueur et les créatures, et rend la vie. Un mort ne bouge plus et ne pousse plus les portes. La touche Espace est lue une seule fois par image et partagée entre l'ouverture des portes et la reprise, sinon l'un des deux usages ne la verrait jamais.

Le nom du fichier du niveau peut aussi venir de la variable d'environnement `TEC_LEVEL`, ce qui permet d'essayer un autre niveau sans recompiler, comme pour les planches d'images.

EXERCICE

Lancez le jeu avec `TEC_LEVEL=autre.txt ./game` après avoir écrit un petit niveau de quelques lignes avec un S, un x et des murs.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE FIGHT H LIGNES 18 A 22

```
18     double health;
19     double last_hit;      // when the player last swung
20     double hurt;          // how long the screen still shows he was hit
21 };
22
```

INCLUDE FIGHT H LIGNES 26 A 31

```
26 // returns 1 when something was hit, so the caller can say so
27 int  fight_strike(const struct player *player, double now);
28 void fight_take(double damage, double now);
29 void fight_step(double elapsed);
30 // what one holds is always on screen. that is what makes a first person
31 // view: without it the player does not even know he can swing. "moving"
```

SRC FIGHT C LIGNES 15 A 35

```
15     memset(&fight, 0, sizeof fight);
16     fight.health = PLAYER_HEALTH;
17 }
18
19 void fight_take(double damage, double now)
20 {
21     (void)now;
22     if (damage <= 0.0)
23         return;
24     fight.health -= damage;
25     fight.hurt = 0.45;
26     if (fight.health < 0.0)
27         fight.health = 0.0;
28 }
29
30 void fight_step(double elapsed)
31 {
32     if (fight.hurt > 0.0)
33         fight.hurt -= elapsed;
34 }
35
```

SRC FIGHT C LIGNES 144 A 169

```
144 void fight_draw(const struct player *player, double now, int moving)
145 {
146     int w = view_width, h = view_height;
147
148     // being hit: the edges go dark red. not the middle, the edges,
because
149     // that is where the eye notices without losing sight of the room
150     if (fight.hurt > 0.0) {
151         double k = fight.hurt / 0.45;
152         for (int y = 0; y < h; y++)
153             for (int x = 0; x < w; x++) {
154                 double ex = fabs(2.0 * x / w - 1.0);
155                 double ey = fabs(2.0 * y / h - 1.0);
156                 double edge = fmax(ex, ey);
157                 if (edge < 0.55)
158                     continue;
159                 double f = (edge - 0.55) / 0.45 * k;
160                 unsigned int c = view[y * w + x];
```

Ligne par ligne

- 25** La durée du rougeoiement des bords.
- 156** La distance au centre de l'écran, entre 0 au milieu et 1 au bord. Sous 0,55, rien ne change.

PARTIE 4 CHAPITRE 8 LE CODE

SUITE

```
161         int r = (int)(((c >> 16) & 0xff) * (1 - f) + 190 * f);
162         int g = (int)(((c >> 8) & 0xff) * (1 - f * 0.9));
163         int b = (int)((c & 0xff) * (1 - f * 0.9));
164         view[y * w + x] = (unsigned int)((r << 16) | (g << 8) |
b);
165     }
166 }
167
168 load_sheets();
169 double lit = 0.34 + 0.62 * lit_at(player->x, player->y);
```

42 Le niveau peut être choisi sans recompiler.

88 La touche Espace, lue une fois et partagée.

SRC MAIN C LIGNES 5 A 9

```
5 #include <X11/Xlib.h>
6 #include <X11/Xutil.h>
7 #include <stdlib.h>
8 #include <time.h>
9 #include <unistd.h>
```

SRC MAIN C LIGNES 38 A 47

```
38 int main(void)
39 {
40     // the level can be swapped without a rebuild: that is already the
41     // rule for the sprite sheets, and it holds for the map
42     const char *level_file = getenv("TEC_LEVEL");
43     if (!level_file)
44         level_file = LEVEL_FILE;
45     if (!load_level(level_file))
46         return 1;
47 }
```

SRC MAIN C LIGNES 71 A 75

```
71 things_from_level();
72 fight_reset();
73 double died_at = 0.0;
74 // the first thing one reads, and the only one nobody triggers
75 say("ARROWS TO MOVE      M FOR THE MAP", last);
```

SRC MAIN C LIGNES 82 A 149

```
82 screen_read_keys(&screen, &keys);
83
84 // the key is read once and shared. it opens a door and it
85 // restarts after a death: if each side clears it on its own,
86 // the last one never sees it. death is checked first,
87 // because a dead man pushes nothing.
88 int push_key = keys.push;
89 keys.push = 0;
90 int death_push = fight.health <= 0.0 && push_key;
91 if (fight.health <= 0.0)
92     push_key = 0;
93
94 // a door that opens by itself teaches nothing; a door that
95 // asks for a key and says so does
96 if (door_ahead(&player) && fight.health > 0.0)
97     say("PRESS SPACE TO OPEN", moment);
98 if (push_key)
99     push_door(&player);
100 if (walk_over(&player))
101     say("SECURITY BADGE TAKEN", moment);
102 if (keys.hit) {
103     if (fight.health > 0.0)
104         fight_strike(&player, moment);
105     keys.hit = 0;
106 }
107 fight_take(things_update(&player, elapsed, moment), moment);
108 fight_step(elapsed);
109 move_doors(elapsed);
110 lamp_flicker(moment);
111 remember(&player);
112 }
```

PARTIE 4 CHAPITRE 8 LE CODE

SUITE

```
113         // dying must not mean starting the program again. a player
114         // sent back to a terminal to retry does not retry.
115         if (fight.health <= 0.0) {
116             if (!died_at)
117                 died_at = moment;
118             say(moment - died_at > 1.5 ? "SPACE TO TRY AGAIN" : "YOU
DIED",
119                 moment);
120             if (moment - died_at > 1.5 && death_push) {
121                 died_at = 0.0;
122                 load_level(level_file);
123                 level_marks(&player.x, &player.y, &exit_x, &exit_y,
124                     &player.dir_x, &player.dir_y);
125                 things_clear();
126                 things_from_level();
127                 fight_reset();
128                 say("AGAIN", moment);
129             }
130         }
131
132         // the way out, and it wants the badge
133         if ((int)player.x == exit_x && (int)player.y == exit_y) {
134             if (!have_badge())
135                 say("THE HATCH IS LOCKED", moment);
136             else
137                 say("THE HATCH OPENS", moment);
138         }
139
140         // every move is scaled by the time the last frame took, so the
game
141         // runs at the same speed whatever the machine is doing
142         int alive = fight.health > 0.0;
143         double forward = alive
144             ? (keys.forward - keys.back) * WALK_SPEED * elapsed : 0.0;
145         double sideways = alive
146             ? (keys.strafe_right - keys.strafe_left) * WALK_SPEED *
elapsed : 0.0;
147         double turn = (alive ? (keys.right - keys.left) * TURN_SPEED :
0.0) * elapsed;
148
149         double step_x = player.dir_x * forward + player.plane_x *
sideways;
```

118 Le message de la mort, puis la proposition de reprise.

142 Un mort ne se déplace plus.

Une barre et un nombre



Figure 9.1. La barre de vie en bas à gauche et le rappel de la touche d'attaque en bas à droite.

Dans ce chapitre, vous allez écrire l'affichage tête haute, une barre de vie et le rappel des touches, mesurés en proportion de l'écran.

L'affichage tête haute, ou HUD, est réduit au strict nécessaire, une barre de vie en bas à gauche et la touche d'attaque en bas à droite. Un HUD chargé cesse d'être lu au bout de deux minutes. La barre change de couleur avant de changer de longueur, verte au-dessus de 55 pour cent, ambre au-dessus de 25 pour cent, rouge en dessous, et elle bat quand la vie est basse. Une barre rouge immobile se fond dans le décor, une barre qui bat jamais.

Les éléments du HUD sont mesurés en unités de HUD et non en pixels. Une unité vaut la largeur de l'image divisée par 640. À taille fixe, la carte mangeait le tiers d'une petite fenêtre et n'était qu'un timbre sur une grande. La carte, les messages et le texte suivent maintenant la largeur de l'image. Enfin, la boucle principale enseigne le coup après 7 secondes, dans le sas, où il n'y a rien à frapper. L'outil vient avant le danger.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE FIGHT H LIGNES 32 A 37

```
32 // drives the walking sway, "now" the animation of the blow.
33 void fight_draw(const struct player *player, double now, int moving);
34 // health, rounds, both weapons and the key that swings
35 void fight_hud(double now);
36
37 #endif
```

INCLUDE RENDER H LIGNES 23 A 30

```
23 #define VERY_FAR      1e30
24
25 // the overlay is measured in hud units, and one hud unit is the view
  width
26 // divided by this. at 640 view pixels across it is worth one pixel,
  which
27 // is exactly what the numbers below used to mean.
28 #define HUD_DIVISOR 640
29 #define MAP_CELL    3
30 #define MAP_LEFT    4
```

SRC FIGHT C LIGNES 7 A 11

```
7 #include "screen.h"
8 #include "sprite.h"
9 #include "text.h"
10 #include "thing.h"
11
```

SRC FIGHT C LIGNES 173 A 218

```
173     draw_pipe(lit, now, moving);
174 }
175
176 // ----- the
  hud
177 // one bar and one number at the bottom, nothing else. a hud with health,
178 // rounds, a compass and a minimap stops being read after two minutes.
179 // health changes colour before it changes length, so it warns early.
180 void fight_hud(double now)
181 {
182     int unit = view_width / HUD_DIVISOR;
183     if (unit < 1)
184         unit = 1;
185     int margin = 10 * unit, tall = 8 * unit, wide = 110 * unit;
186     int y = view_height - margin - tall;
187     double part = fight.health / PLAYER_HEALTH;
188     if (part < 0.0)
189         part = 0.0;
190     unsigned int filled = part > 0.55 ? 0x6fd8c0
191         : part > 0.25 ? 0xd8b04a : 0xc4423a;
192     // under a quarter it beats. a still red bar sinks into the scenery
193     // after ten seconds, a pulsing one never does.
194     if (part <= 0.25) {
195         double k = 0.62 + 0.38 * sin(now * 9.0);
196         filled = (unsigned int)((int)(((filled >> 16) & 0xff) * k) << 16
197             | (int)(((filled >> 8) & 0xff) * k) << 8
198             | (int)((filled & 0xff) * k));
199     }
200     for (int i = 0; i < tall; i++)
```

Ligne par ligne

- 28** Une unité de HUD est la largeur de l'image divisée par 640.
- 190** Les trois couleurs de la barre, selon la vie restante.
- 195** Le battement de la barre sous 25 pour cent.

PARTIE 4 CHAPITRE 9 LE CODE

SUITE

```
201         for (int j = 0; j < wide; j++) {
202             int px = margin + j, py = y + i;
203             if (px >= view_width || py >= view_height || px < 0 || py <
0)
204                 continue;
205             int edge = i < unit || i >= tall - unit
|| j < unit || j >= wide - unit;
206             view[py * view_width + px] = edge ? 0x2b3440
207                 : (j < (int)(wide * part) ? filled : 0x141a22);
208         }
209         // the key that swings, at all times. a player who missed the
210         // teaching line has no way left to know how to strike.
211         int t2 = unit * 2;
212         int line_h = (GLYPH_H + 3) * t2;
213         int bottom = view_height - margin;
214         const char *attack = "CTRL ATTACK";
215         int l3 = (int)strlen(attack) * (GLYPH_W + 1) * unit;
216         draw_text(view_width - margin - l3, bottom - line_h, attack,
0x6a7b8c, unit);
217     }
218 }
```

87 La leçon du coup, 7 secondes après le début.

178 La carte dessinée en unités de HUD.

SRC MAIN C LIGNES 72 A 81

```
72     fight_reset();
73     double died_at = 0.0;
74     // the first two things to learn, and they are learned in the quiet.
75     // the swing is taught in the airlock, where there is nothing to hit:
76     // the tool comes before the danger, always.
77     say("ARROWS TO MOVE      M FOR THE MAP", last);
78     double lesson_at = last;
79     int lesson = 0;
80
81     while (!keys.quit) {
```

SRC MAIN C LIGNES 85 A 92

```
85
86     screen_read_keys(&screen, &keys);
87     if (!lesson && moment - lesson_at > 7.0) {
88         say("CTRL SWINGS WHAT YOU ARE HOLDING", moment);
89         lesson = 1;
90     }
91
92     // the key is read once and shared. it opens a door and it
```

SRC MAIN C LIGNES 163 A 184

```
163     render_floor_and_ceiling(&player);
164     render_walls(&player);
165     // the overlay is a share of the view, not a pixel count. at
166     // fixed sizes the map ate a third of a small window and was
167     // a stamp on a wide one: measure it against the width.
168     int hud = view_width / HUD_DIVISOR;
169     if (hud < 1)
170         hud = 1;
171     // after the walls, never before: the depth buffer has to be
172     // filled for them to know what hides them
173     things_draw(&player, moment);
174     fight_draw(&player, moment, keys.forward || keys.back
|| keys.strafe_left || keys.strafe_right);
175     fight_hud(moment);
176     if (keys.map)
177         render_map(&player, hud * MAP_CELL, hud * MAP_LEFT,
178             hud * MAP_TOP);
179     if (moment < notice_until)
180         draw_text_centered(view_height - hud * NOTICE_UP, notice,
181             TEXT_COLOR, hud * TEXT_SCALE);
182     screen_present(&screen);
183
184 }
```

Elle tient sur le sol

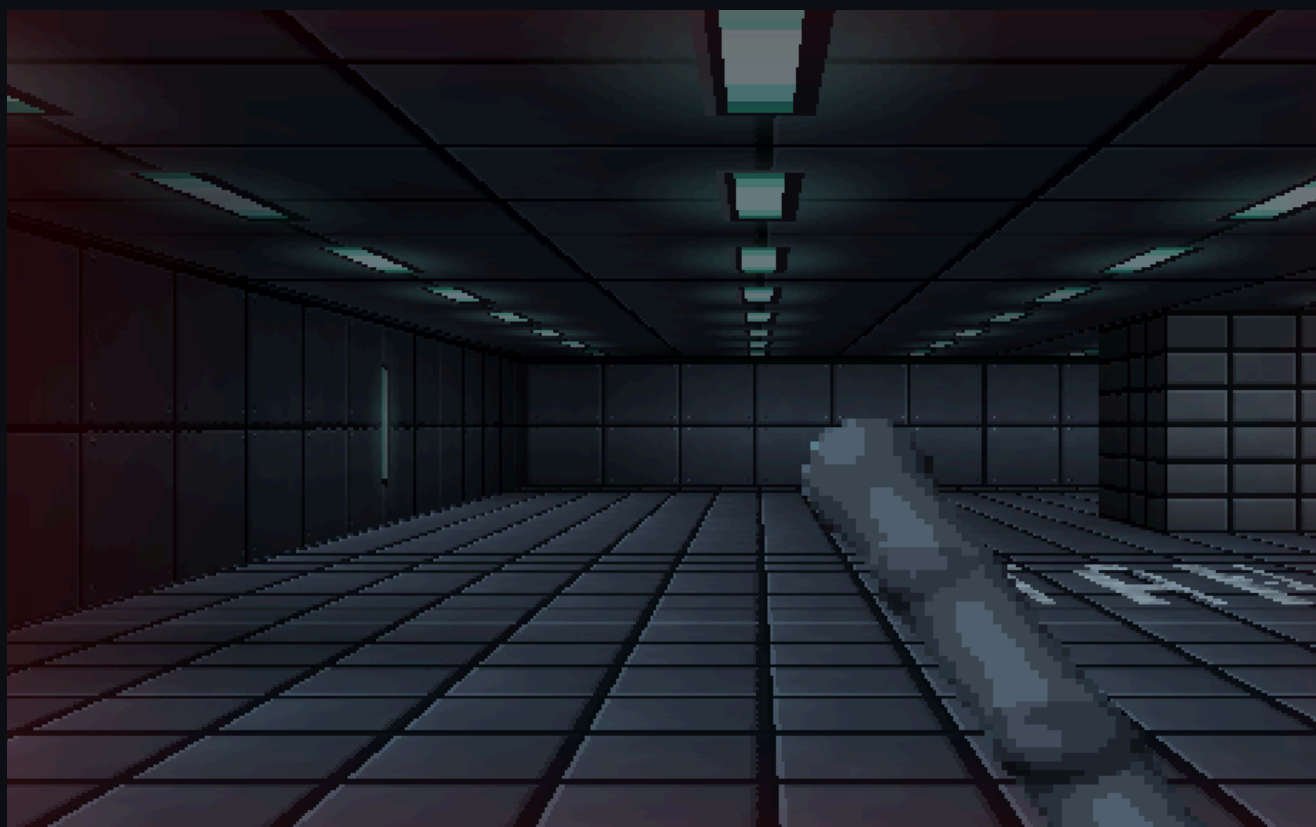


Figure 10.1. L'ombre sous les pieds de la créature. Elle est posée sur le sol, et non collée sur l'image.

Dans ce chapitre, vous allez poser les créatures sur le sol avec une ombre de contact.

Une créature dessinée au bon pixel peut encore sembler flotter. L'œil cherche l'endroit où elle touche le sol et ne le trouve pas. Nous ajoutons une ombre de contact, une ellipse sombre sous les pieds, dont la taille suit celle de la créature et qui respecte le tampon de profondeur. Elle assombrit le sol de 55 pour cent au centre et de moins en moins vers le bord.

Pendant qu'elle prépare son coup, la créature est dessinée un dixième plus grande. Ce léger agrandissement se lit comme un élan, et il prévient le joueur.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

SRC THING C LIGNES 233 A 240

```
233 // not exactly one corridor tall.
234 double full = view_height / depth;
235 // while it winds up it throws itself forward: a tenth bigger,
236 // and that reads as a lunge
237 double lunge = t->state == THING_STRIKE ? 1.10 : 1.0;
238 int height = (int)(full * lunge);
239 int width = height * sheet.width / sheet.height;
240 int top = (int)(HORIZON + full * 0.5) - height;
```

SRC THING C LIGNES 261 A 291

```
261 if (frame >= sheet.frames)
262     frame = sector % sheet.frames;
263
264 // the contact shadow, and it is what puts it on the floor.
265 // without it a thing drawn on the right pixel still looks like
266 // it flies: the eye hunts for where it meets the ground and
267 // does not find it.
268 int ground = (int)(HORIZON + full * 0.5);
269 int half_x = width / 3, half_y = (int)(full * 0.075) + 1;
270 for (int y = ground - half_y; y <= ground + half_y; y++) {
271     if (y < 0 || y >= view_height)
272         continue;
273     for (int x = middle - half_x; x <= middle + half_x; x++) {
274         if (x < 0 || x >= view_width || depth >= wall_depth[x])
275             continue;
276         double ox = (double)(x - middle) / half_x;
277         double oy = (double)(y - ground) / half_y;
278         double r2 = ox * ox + oy * oy;
279         if (r2 > 1.0)
280             continue;
281         double k = (1.0 - r2) * 0.55;
282         unsigned int c = view[y * view_width + x];
283         int r = (int)(((c >> 16) & 0xff) * (1 - k));
284         int g = (int)(((c >> 8) & 0xff) * (1 - k));
285         int b = (int)((c & 0xff) * (1 - k));
286         view[y * view_width + x] =
287             (unsigned int)((r << 16) | (g << 8) | b);
288     }
289 }
290
291 // its floor of light is higher than the room's. a beast as dark
```

Ligne par ligne

- 237** L'élan, un dixième de plus pendant la frappe.
- 269** Les demi-axes de l'ellipse de l'ombre, liés à la taille de la créature.
- 281** L'assombrissement, maximal au centre et nul au bord de l'ellipse.

Un cri porte

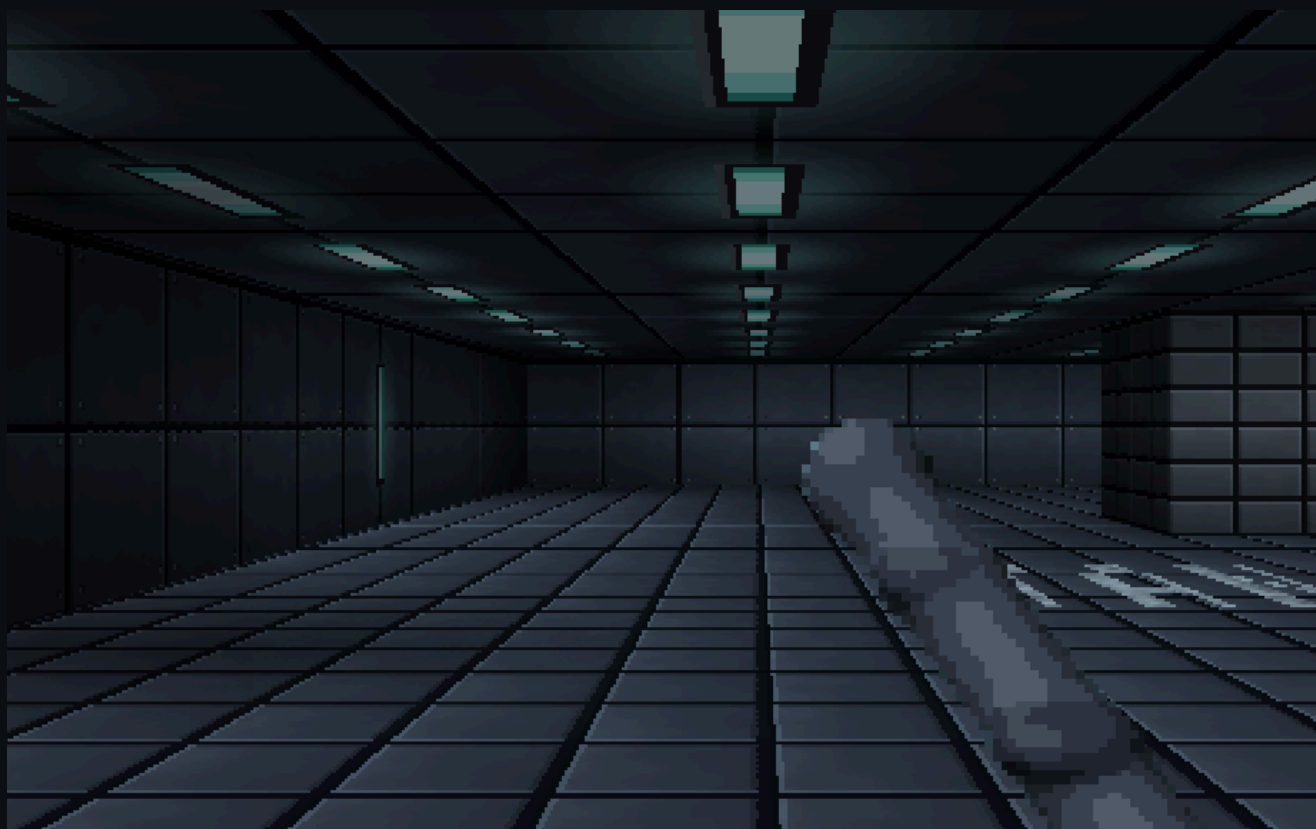


Figure 11.1. L'image ne change pas. Le cri est une règle du jeu, pas un dessin.

Dans ce chapitre, vous allez faire crier les créatures pour qu'elles réveillent leurs voisines, et adoucir le passage des portes.

Une créature qui se réveille appelle les autres. La fonction `things_hear` réveille toutes les créatures au repos situées à moins de 7 cases d'un point. Une créature ne crie qu'une fois par réveil, grâce au drapeau `silent`, qui se remet à zéro quand elle retourne au repos. Un combat n'est ainsi jamais contre une seule créature, et le joueur apprend à réfléchir avant d'être vu.

Nous corrigeons aussi un détail du déplacement. Une porte d'une case de large avec une marge de 0,2 case ne laisse que 0,6 case utile, et le joueur accrochait le cadre une fois sur deux. Une fois dans une case de porte, la marche est ramenée doucement vers le milieu de la case, dans le sens perpendiculaire au passage.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE THING H LIGNES 44 A 48

```
44     double hurt_at;
45     int used;
46     int silent;           // it has cried once: it does not cry twice
47 };
48
```

INCLUDE THING H LIGNES 57 A 62

```
57 // drawn after the walls, so the depth buffer is full
58 void things_draw(const struct player *player, double now);
59 // a cry carries: everything asleep within reach wakes up
60 void things_hear(double x, double y, double reach, double now);
61
62 #endif
```

SRC THING C LIGNES 124 A 132

```
124         t->state = THING_ALERT;
125         t->since = now;
126         if (!t->silent) {
127             // it calls, and the neighbours get up
128             t->silent = 1;
129             things_hear(t->x, t->y, 7.0, now);
130         }
131     }
132     break;
```

SRC THING C LIGNES 172 A 176

```
172         t->state = THING_IDLE;
173         t->since = now;
174         t->silent = 0;
175     }
176     break;
```

SRC THING C LIGNES 330 A 348

```
330     }
331 }
332
333 // a cry carries. one waking thing calls others: that is what makes a
334 // fight
335 // never be against a single one, and what makes one think before being
336 // seen.
337 void things_hear(double x, double y, double reach, double now)
338 {
339     for (int i = 0; i < THINGS_MAX; i++) {
340         struct thing *t = &things[i];
341         if (!t->used || t->state != THING_IDLE)
342             continue;
343         if (hypot(t->x - x, t->y - y) > reach)
344             continue;
345         t->state = THING_ALERT;
346         t->since = now;
347         t->last_seen_x = x;
348         t->last_seen_y = y;
349     }
350 }
```

SRC WORLD C LIGNES 321 A 337

```
321     if (!is_wall((int)player->x, (int)(player->y + step_y + nose_y)))
322         player->y += step_y;
323
```

Ligne par ligne

129 Le cri, à 7 cases à la ronde.

339 Seules les créatures au repos entendent le cri.

PARTIE 4 CHAPITRE 11 LE CODE

SUITE

```
324 // doors are taken without snagging. a one metre gap with a twenty
325 // centimetre nose leaves sixty usable, so you scrape the frame every
326 // other time and blame the controls. once inside a door cell the
327 // walk is pulled gently back to its middle.
328 int cx = (int)player->x, cy = (int)player->y;
329 if (!is_door(cx, cy))
330     return;
331 double magnet = 0.12;
332 if (is_wall(cx - 1, cy) && is_wall(cx + 1, cy))
333     player->x += (cx + 0.5 - player->x) * magnet;
334 else if (is_wall(cx, cy - 1) && is_wall(cx, cy + 1))
335     player->y += (cy + 0.5 - player->y) * magnet;
336 }
337
```

331 La force de l'aimant vers le milieu de la porte, 12 pour cent de l'écart par image.

Ce qui traîne au sol



Figure 12.1. Les trois cases d'objets, dessinées par la fonction `loot_tile` avec deux couleurs chacune. La forme est celle du badge.

Dans ce chapitre, vous allez placer des objets à ramasser au sol, avec une couleur qui les distingue de loin.

Le badge de la troisième partie devient un cas parmi quatre. Le fichier du niveau place une arme avec la lettre G, des munitions avec la lettre A et des soins avec la lettre M. Le type énuméré `loot` nomme ces objets, la fonction `loot_at` lit la grille, et la fonction `loot_take` ramasse ce qui se trouve sous le joueur. Un tableau `taken` mémorise ce qui a été pris, sans quoi le joueur se servirait sans fin en repassant.

Les quatre objets sont dessinés par la même fonction `loot_tile`, celle du badge, avec deux couleurs en argument. La couleur dit de loin ce que c'est, ambre pour les munitions, vert pour les soins. Un objet se ramasse en marchant dessus, sans arrêt et sans panneau, et les yeux restent sur la salle. Les soins rendent 40 points de vie.

Ce qui traîne au sol, suite

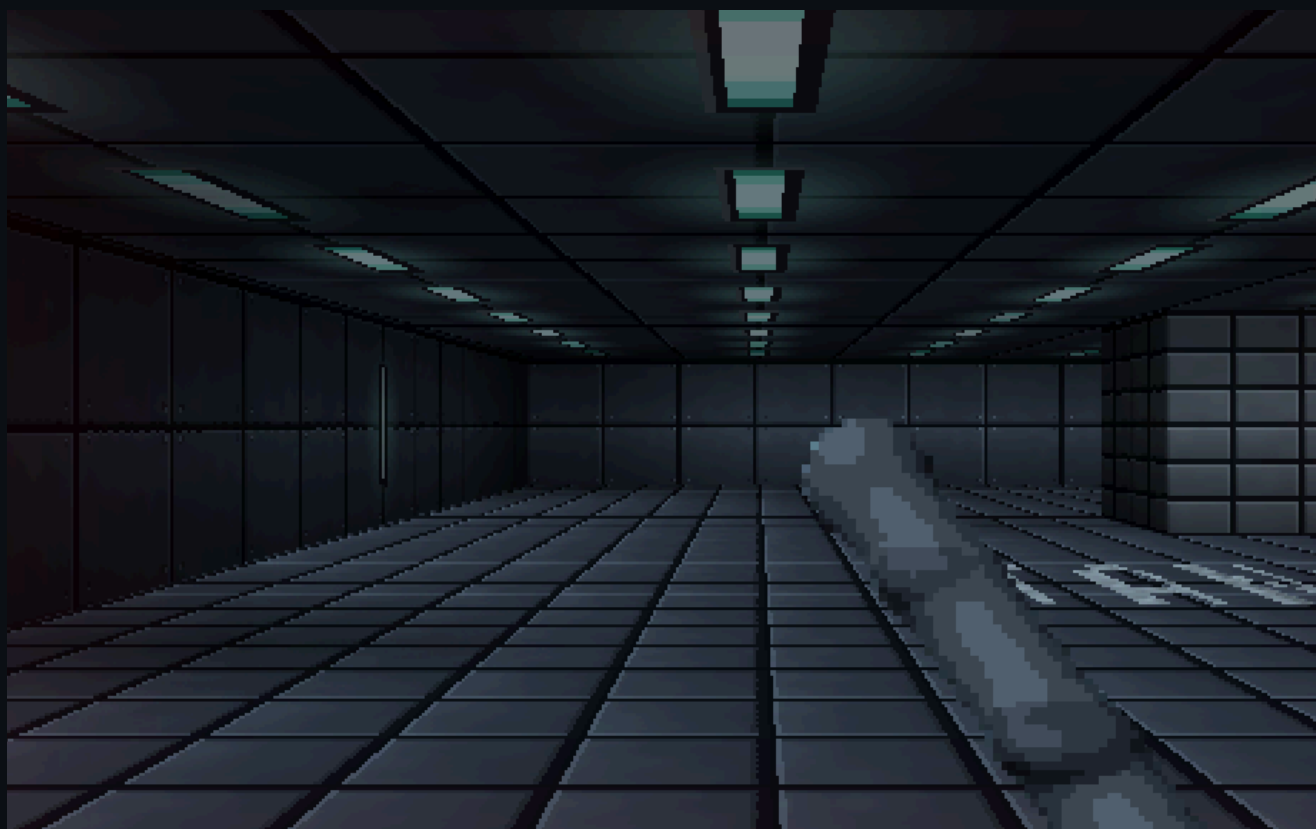


Figure 12.2. Les trois objets au sol, l'arme, les munitions en ambre et les soins en vert. Ils brillent de leur propre lumière.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

Ligne par ligne

31 Les objets que le sol peut porter.

INCLUDE FIGHT H LIGNES 14 A 18

```
14 #define PIPE_COOLDOWN 0.62
15 #define PIPE_SWING 0.34 // how long the swing takes to pass
16 #define MED_PACK 40.0
17
18 struct fight {
```

INCLUDE FIGHT H LIGNES 35 A 40

```
35 // health, rounds, both weapons and the key that swings
36 void fight_hud(double now);
37 // patch up
38 void fight_heal(double amount);
39
40 #endif
```

INCLUDE TEXTURE H LIGNES 60 A 67

```
60 // the painted tile: `cell` says which of the four we are looking at
61 unsigned int stencil_tile(int x, int y, int cell);
62 // and the tile an item sleeps on, read where the floor is read. one
63 // function for all four: what changes is the colour, and the colour is
64 // what says which one it is from across the room.
65 unsigned int loot_tile(int x, int y, unsigned int body, unsigned int
    edge);
66
67 // a wall met on a north-south line keeps this much of its light
```

INCLUDE TEXTURE H LIGNES 90 A 99

```
90 #define BADGE_BODY 0xd5f5a
91 #define BADGE_EDGE 0x8ff0e0
92 #define GUN_BODY 0x2b3140
93 #define GUN_EDGE 0xccd8e6
94 #define AMMO_BODY 0x4a3210
95 #define AMMO_EDGE 0xf0b44c
96 #define MED_BODY 0x123a2c
97 #define MED_EDGE 0x86f0b4
98 #define BADGE_R 36.0 // half the side of the plate
99 #define BADGE_HALO 22.0 // and how far its glow spills on the tile
```

INCLUDE WORLD H LIGNES 28 A 35

```
28 #define STENCIL_Y 3
29 int stencil_at(int x, int y);
30 // what lies on the floor to be picked up, and what is left of it
31 enum loot { LOOT_NONE, LOOT_GUN, LOOT_AMMO, LOOT_MED, LOOT_BADGE };
32 enum loot loot_at(int x, int y);
33 enum loot loot_take(const struct player *player);
34 int is_badge(int x, int y);
35 int have_badge(void);
```

SRC FIGHT C LIGNES 16 A 26

```
16 memset(&fight, 0, sizeof fight);
17 fight.health = PLAYER_HEALTH;
18 }
19
20 void fight_heal(double amount)
21 {
22     fight.health += amount;
23     if (fight.health > PLAYER_HEALTH)
24         fight.health = PLAYER_HEALTH;
```

PARTIE 4 CHAPITRE 12 LE CODE

SUITE

```
25 }  
26
```

SRC MAIN C LIGNES 106 A 127

```
106     if (push_key)  
107         push_door(&player);  
108     // picked up by walking over it, no stop and no panel  
109     switch (loot_take(&player)) {  
110     case LOOT_GUN:  
111         say("SIDEARM IT STILL FEEDS", moment);  
112         break;  
113     case LOOT_AMMO:  
114         say("ROUNDS", moment);  
115         break;  
116     case LOOT_MED:  
117         fight_heal(MED_PACK);  
118         say("PATCHED UP", moment);  
119         break;  
120     case LOOT_BADGE:  
121         say("SECURITY BADGE TAKEN", moment);  
122         break;  
123     default:  
124         break;  
125     }  
126     if (keys.hit) {  
127         if (fight.health > 0.0)
```

109 Ce qui vient d'être ramassé, et le message qui va avec.

255 La case du badge, généralisée à deux couleurs.

SRC RENDER C LIGNES 84 A 105

```
84         light = 1.0;  
85         int painted = stencil_at((int)ground_x, (int)ground_y);  
86         // what can be picked up glows on its own,  
87         // in its own colour: amber for rounds,  
88         // green for a patch up. you know from  
89         // across the room whether it is worth it.  
90         enum loot item = loot_at((int)ground_x, (int)ground_y);  
91         unsigned int body = BADGE_BODY, edge = BADGE_EDGE;  
92         switch (item) {  
93         case LOOT_GUN: body = GUN_BODY; edge = GUN_EDGE; break;  
94         case LOOT_AMMO: body = AMMO_BODY; edge = AMMO_EDGE; break;  
95         case LOOT_MED: body = MED_BODY; edge = MED_EDGE; break;  
96         default: break;  
97         }  
98         unsigned int ground = item != LOOT_NONE  
99             ? loot_tile(tex_x, tex_y, body, edge)  
100             : painted >= 0 ? stencil_tile(tex_x, tex_y, painted)  
101             : floor_texture[at];  
102         view[y * view_width + x] =  
103             tint(ground, item != LOOT_NONE ? 1.0 : light, GLOOM);  
104         view[(view_height - 1 - y) * view_width + x] =  
105             tint(ceiling_texture[at], light * CEILING_PART, GLOOM);
```

SRC TEXTURE C LIGNES 253 A 257

```
253 // a plate set into the tile: a rim that shines, a chip in the middle,  
    and a  
254 // glow that spills onto the tiling around it  
255 unsigned int loot_tile(int x, int y, unsigned int body, unsigned int  
    edge)  
256 {  
257     double dx = x - TEX_SIZE / 2.0, dy = y - TEX_SIZE / 2.0;
```

SRC TEXTURE C LIGNES 262 A 276

```
262     return ground;  
263     if (inside > BADGE_R)  
264         return mix(ground, edge,  
265             0.45 * (1.0 - (inside - BADGE_R) / BADGE_HALO));  
266     if (inside > BADGE_R - 5.0)  
267         return edge;  
268     if (inside > BADGE_R - 8.0)
```

PARTIE 4 CHAPITRE 12 LE CODE

SUITE

```
269         return mix(body, 0x000000, 0.4);
270     // the chip: three lines, like on a card
271     if (fabs(dy) < 12.0 && fabs(dx) < 16.0
272         && ((int)(dy + 12.0) / 8) % 2 == 0)
273         return mix(edge, body, 0.35);
274     return mix(body, edge, 0.18);
275 }
276
```

26 Un octet par case, 1 quand l'objet a été ramassé.

241 Les quatre lettres du fichier.

SRC WORLD C LIGNES 21 A 28

```
21 // leaves nothing to find; this one fills in as one goes.
22 static char *seen;
23
24 // what has already been picked up: one cell, one byte. without it you
25 // walk back over it and help yourself forever.
26 static char *taken;
27
28 // what the level file can hold. a wall is anything that is not floor.
```

SRC WORLD C LIGNES 69 A 75

```
69     free(seen);
70     seen = calloc((size_t)map_width * map_height, 1);
71     free(taken);
72     taken = calloc((size_t)map_width * map_height, 1);
73     if (!doors || !seen || !taken)
74         return 0;
75 }
```

SRC WORLD C LIGNES 230 A 265

```
230 }
231
232 // you pick it up by walking over it. a chest to open is a panel that
233 // the screen and stops the walk; a plate on the floor is seen from far,
234 // taken without stopping, and the eyes stay on the room.
235 enum loot loot_at(int x, int y)
236 {
237     if (x < 0 || y < 0 || x >= map_width || y >= map_height
238         || (taken && taken[y * map_width + x]))
239         return LOOT_NONE;
240     switch (map[y][x]) {
241     case 'K': return LOOT_BADGE;
242     case 'G': return LOOT_GUN;
243     case 'A': return LOOT_AMMO;
244     case 'M': return LOOT_MED;
245     default: return LOOT_NONE;
246     }
247 }
248
249 enum loot loot_take(const struct player *player)
250 {
251     int x = (int)player->x, y = (int)player->y;
252     enum loot l = loot_at(x, y);
253     if (l == LOOT_NONE)
254         return LOOT_NONE;
255     taken[y * map_width + x] = 1;
256     if (l == LOOT_BADGE)
257         badge_taken = 1;
258     return l;
259 }
260
261 int is_badge(int x, int y)
262 {
263     return loot_at(x, y) == LOOT_BADGE;
264 }
265
```

PARTIE 4 CHAPITRE 12 LE CODE

SUITE

SRC WORLD C LIGNES 312 A 317

```
312         return door_at(x, y) < DOOR_WALKABLE;
313     return !(c == '.' || c == 'S' || c == 'E' || c == 'K'
314             || c == 'x' || c == 'y' || c == 'G'
315             || c == 'A' || c == 'M');
316 }
317
```

Une arme de poing, et elle coûte

LA PLANCHE DE L'ARME 4 IMAGES

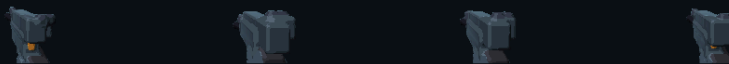


Figure 13.1. Les quatre images de l'arme de poing, au repos, au départ du coup, et sur le retour.

Dans ce chapitre, vous allez ajouter une arme de poing, ses munitions, et une visée qui touche ou manque.

L'arme de poing tue à distance et coûte des munitions. Elle change la distance à laquelle on joue, pas seulement les dégâts. Elle est ramassée vide de sens tant qu'il n'y a pas de munitions, et les munitions sont ailleurs. Chaque coup retire un projectile, inflige 60 points, et se répète au plus tous les 0,42 seconde. Un coup de feu porte à 10 cases et réveille tout ce qui dort à cette distance. On gagne un combat et on en réveille deux.

Un projectile touche ou manque, jamais à peu près. Le programme mesure l'écart entre le projectile et le corps, et non un angle. Un cône de cinq degrés pardonnerait cinquante pixels à bout portant et trois mètres à l'autre bout de la salle. La créature est touchée si son écart latéral est inférieur à 0,36 case et si elle est devant le premier mur, car le projectile s'arrête au mur. Sans cette vérification, il traversait les cloisons.

Les touches 1 et 2 choisissent l'arme. Le jeu revient à la barre quand le chargeur est vide, mais un joueur qui ne choisit jamais n'apprend pas qu'il tient deux choses. Le HUD montre les deux armes et le nombre de projectiles. La ligne de l'arme n'existe pas tant qu'elle n'a pas été trouvée. Grisée, elle disait au joueur qu'il avait un pistolet, et il cherchait la touche pour le sortir.

VOCABULAIRE

L'écart latéral d'une créature est sa distance à la ligne de visée, mesurée perpendiculairement à la direction du joueur.

Une arme de poing, et elle coûte, suite

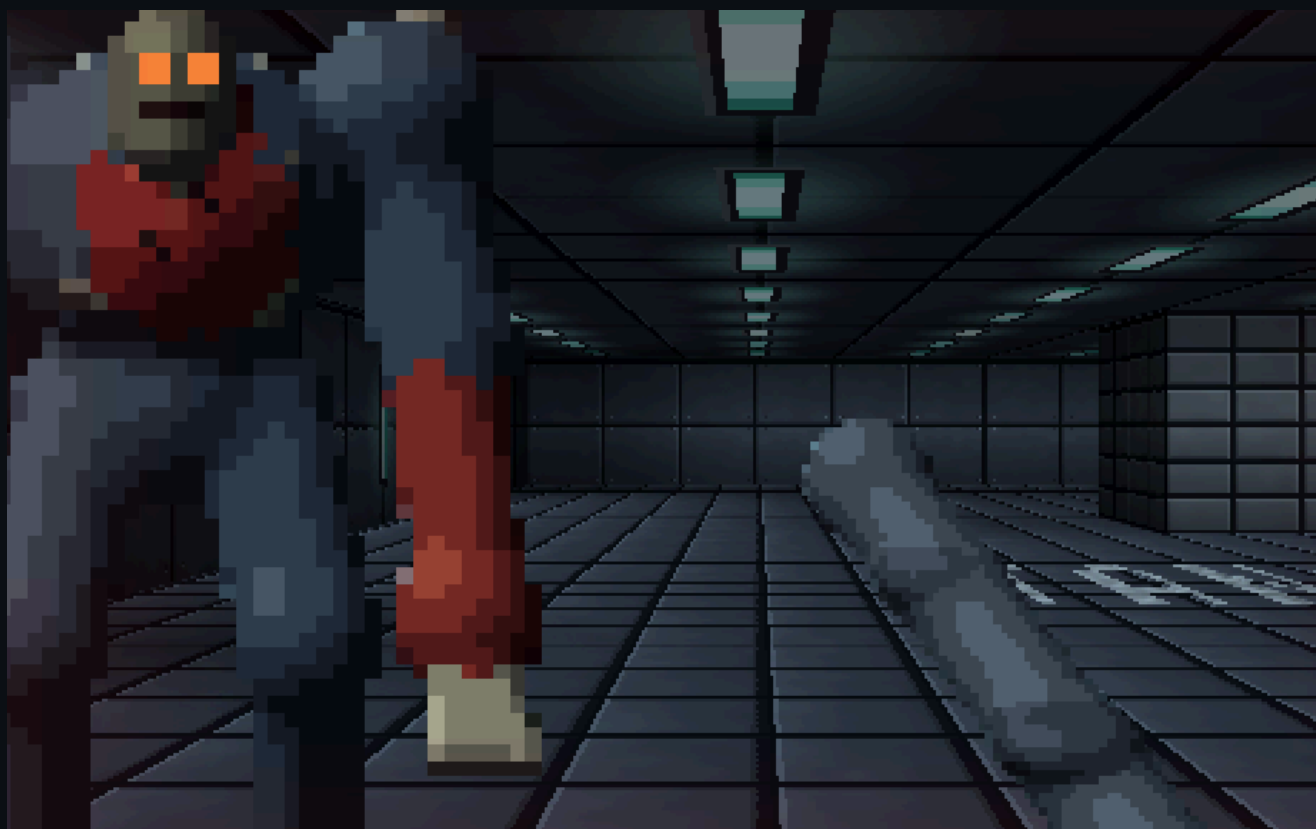


Figure 13.2. Le HUD montre les deux armes. La barre est sortie, l'arme n'a pas encore été trouvée.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE FIGHT H LIGNES 14 A 34

```
14 #define PIPE_COOLDOWN 0.62
15 #define PIPE_SWING 0.34 // how long the swing takes to pass
16
17 // and what one finds later. a sidearm kills at a distance and costs
  rounds:
18 // it changes the range one plays at, not only the damage. it comes
  empty,
19 // and the rounds are somewhere else.
20 #define GUN_DAMAGE 60.0
21 #define GUN_COOLDOWN 0.42
22 #define GUN_RANGE 14.0
23 #define AMMO_PACK 10 // what one crate holds
24 #define MED_PACK 40.0
25
26 struct fight {
27     double health;
28     int ammo;
29     int has_gun;
30     int chosen; // 1 the bar, 2 the sidearm, 0 let the game
  pick
31     double last_hit; // when the player last swung
32     double flash; // how long the muzzle still lights the room
33     double hurt; // how long the screen still shows he was hit
34 };
```

INCLUDE FIGHT H LIGNES 47 A 52

```
47 // health, rounds, both weapons and the key that swings
48 void fight_hud(double now);
49 // is the sidearm out
50 int fight_gun_out(void);
51 // patch up
52 void fight_heal(double amount);
```

INCLUDE SCREEN H LIGNES 49 A 53

```
49 int map; // is the map on screen? M takes it away
50 int hit; // a press on ctrl: swing
51 int weapon; // 1 or 2: the bar or the sidearm
52 };
53
```

SRC FIGHT C LIGNES 1 A 4

```
1 #include <math.h>
2 #include <stdio.h>
3 #include <string.h>
4
```

SRC FIGHT C LIGNES 17 A 28

```
17 memset(&fight, 0, sizeof fight);
18 fight.health = PLAYER_HEALTH;
19 }
20
21 // what is in hand, decided in one place. the player's own pick comes
22 // first, and the gun only comes out when it can fire: an empty weapon in
23 // hand is a trap, not a choice.
24 int fight_gun_out(void)
25 {
26     return fight.has_gun && fight.ammo > 0 && fight.chosen != 1;
```

Ligne par ligne

- 20** Les dégâts d'un projectile.
- 26** L'arme n'est en main que si elle a été trouvée, chargée, et non écartée par le joueur.

PARTIE 4 CHAPITRE 13 LE CODE

SUITE

```
27 }
28
```

SRC FIGHT C LIGNES 47 A 52

```
47 void fight_step(double elapsed)
48 {
49     if (fight.flash > 0.0)
50         fight.flash -= elapsed;
51     if (fight.hurt > 0.0)
52         fight.hurt -= elapsed;
```

SRC FIGHT C LIGNES 58 A 82

```
58 int fight_strike(const struct player *player, double now)
59 {
60     double wait = fight_gun_out() ? GUN_COOLDOWN : PIPE_COOLDOWN;
61     if (now - fight.last_hit < wait)
62         return 0;
63     fight.last_hit = now;
64     int gun = fight_gun_out();
65     if (gun) {
66         fight.ammo--;
67         fight.flash = 0.08;
68         // a gunshot carries. that is what makes rounds cost more
69         // than their count: you win one fight and wake two.
70         things_hear(player->x, player->y, 10.0, now);
71     }
72
73     // the bullet stops at the first wall. without this line it went
74     // through partitions, and since nothing showed the impact, the
75     // player was right to think everything was a hit.
76     double wall = wall_depth ? wall_depth[view_width / 2] : 1e9;
77     struct thing *best = NULL;
78     double nearest = gun ? GUN_RANGE : PIPE_REACH;
79     if (gun && nearest > wall)
80         nearest = wall;
81     for (int i = 0; i < THINGS_MAX; i++) {
82         struct thing *t = &things[i];
```

SRC FIGHT C LIGNES 87 A 105

```
87     if (d > nearest || d < 0.001)
88         continue;
89     if (gun) {
90         // a bullet misses or lands, never "about right". a
91         // five degree cone forgives fifty pixels point
92         // blank and three metres across a room: measure
93         // the round to the body, not the angle.
94         double ahead = dx * player->dir_x + dy * player->dir_y;
95         if (ahead <= 0.0)
96             continue;
97         double across = fabs(dx * -player->dir_y
98                             + dy * player->dir_x);
99         if (across > 0.36)
100             continue;
101     }
102     // the bar sweeps wide, and that is its whole point
103     if (!gun && (dx * player->dir_x + dy * player->dir_y) / d <
PIPE_ARC)
104         continue;
105     nearest = d;
```

SRC FIGHT C LIGNES 108 A 115

```
108     if (!best)
109         return 0;
110     best->health -= gun ? GUN_DAMAGE : PIPE_DAMAGE;
111     // it takes the blow: a quarter second where it stops coming, and
112     // that is what gives melee its rhythm
113     best->stagger = now + (gun ? 0.16 : 0.26);
114     best->hurt_at = now;
115     if (best->health <= 0.0) {
```

70 Un coup de feu porte à 10 cases.

76 La distance du mur au centre de l'écran, où le projectile s'arrête.

97 L'écart latéral de la créature par rapport à la ligne de visée.

99 Un projectile qui passe à plus de 0,36 case du corps manque.

PARTIE 4 CHAPITRE 13 LE CODE

SUITE

SRC FIGHT C LIGNES 128 A 133

```
128 // of view of the game, glove included. a held weapon is a volume seen in
129 // perspective, and that does not work out when drawn flat.
130 static void draw_sheet(struct sprite *p, int frame, double lit,
131                       int moving, int flash)
132 {
133     if (!p->pixels)
```

SRC FIGHT C LIGNES 166 A 174

```
166     int g = (int)(((c >> 8) & 0xff) * lit);
167     int b = (int)((c & 0xff) * lit);
168     if (flash) {
169         r += (255 - r) * 3 / 5;
170         g += (255 - g) * 5 / 9;
171         b += (255 - b) * 2 / 5;
172     }
173     view[y * view_width + x] =
174         (unsigned int)((r << 16) | (g << 8) | b);
```

SRC FIGHT C LIGNES 177 A 189

```
177 }
178
179 static struct sprite pipe_sheet, gun_sheet;
180
181 static void load_sheets(void)
182 {
183     static int tried;
184     if (!tried++) {
185         sprite_load(&pipe_sheet, "art/pipe.tecs");
186         sprite_load(&gun_sheet, "art/gun.tecs");
187     }
188 }
189
```

SRC FIGHT C LIGNES 195 A 207

```
195     : since_hit < PIPE_SWING * 0.70 ? 2
196     : since_hit < PIPE_SWING ? 3 : 0;
197     draw_sheet(&pipe_sheet, frame, lit, moving, 0);
198 }
199
200 static void draw_gun(double lit, double now, int moving)
201 {
202     double since_hit = now - fight.last_hit;
203     int frame = since_hit < 0.07 ? 1 : since_hit < 0.15 ? 2
204               : since_hit < 0.26 ? 3 : 0;
205     draw_sheet(&gun_sheet, frame, lit, moving, fight.flash > 0.0);
206 }
207
```

SRC FIGHT C LIGNES 234 A 241

```
234     if (lit > 1.0)
235         lit = 1.0;
236     if (fight_gun_out())
237         draw_gun(lit, now, moving);
238     else
239         draw_pipe(lit, now, moving);
240 }
241
```

SRC FIGHT C LIGNES 274 A 299

```
274         : (j < (int)(wide * part) ? filled : 0x141a22);
275     }
276     // both weapons, their key, and the key that swings, at all times. a
277     // player who missed the teaching line has no way left to know how
278     // to strike. three lines bottom right answer that once and for all.
```

PARTIE 4 CHAPITRE 13 LE CODE

SUITE

```
279     int t2 = unit * 2;
280     int line_h = (GLYPH_H + 3) * t2;
281     int bottom = view_height - margin;
282     char line[28];
283     int gun = fight_gun_out();
284
285     snprintf(line, sizeof line, "1 PIPE");
286     int l1 = (int)strlen(line) * (GLYPH_W + 1) * t2;
287     draw_text(view_width - margin - l1, bottom - line_h * 3, line,
288             gun ? 0x55606e : 0xe8e4da, t2);
289     // the second line does not exist until the gun is found. greyed out
290     // it still read as "you have a pistol", and the player hunted for
291     // the key to draw it. what you do not own is not shown at all.
292     if (fight.has_gun) {
293         snprintf(line, sizeof line, "2 GUN %d", fight.ammo);
294         int l2 = (int)strlen(line) * (GLYPH_W + 1) * t2;
295         draw_text(view_width - margin - l2, bottom - line_h * 2, line,
296             gun ? 0xf0c060 : 0x55606e, t2);
297     }
298     const char *attack = "CTRL  ATTACK";
299     int l3 = (int)strlen(attack) * (GLYPH_W + 1) * unit;
```

292 La ligne de l'arme n'est dessinée que si l'arme a été trouvée.

184 Les touches 1 et 2, sur le clavier et sur le pavé numérique.

SRC MAIN C LIGNES 109 A 118

```
109     switch (loot_take(&player)) {
110     case LOOT_GUN:
111         fight.has_gun = 1;
112         fight.ammo += AMMO_PACK;
113         say("SIDEARM  IT STILL FEEDS", moment);
114         break;
115     case LOOT_AMMO:
116         fight.ammo += AMMO_PACK;
117         say("ROUNDS", moment);
118         break;
```

SRC MAIN C LIGNES 126 A 141

```
126         default:
127             break;
128     }
129     // 1 and 2 pick what is held. asking for the gun with no
130     // rounds does nothing, and the hud already says why.
131     if (keys.weapon) {
132         if (keys.weapon == 1) {
133             fight.chosen = 1;
134             say("PIPE", moment);
135         } else if (fight.has_gun) {
136             fight.chosen = 2;
137             say(fight.ammo ? "SIDEARM" : "SIDEARM  EMPTY", moment);
138         }
139         keys.weapon = 0;
140     }
141     if (keys.hit) {
```

SRC SCREEN C LIGNES 178 A 187

```
178     case XK_Control_L:
179     case XK_Control_R: if (down) keys->hit = 1; break;
180     // you choose your weapon, you do not put up with
181     // it. the game falls back to the bar on an empty
182     // magazine, which is right, but on its own the
183     // player never learns he holds two things.
184     case XK_1: case XK_KP_1: if (down) keys->weapon = 1; break;
185     case XK_2: case XK_KP_2: if (down) keys->weapon = 2; break;
186     case XK_Up:     keys->forward = down; break;
187     case XK_Down:   keys->back = down; break;
```

Où le projectile est allé

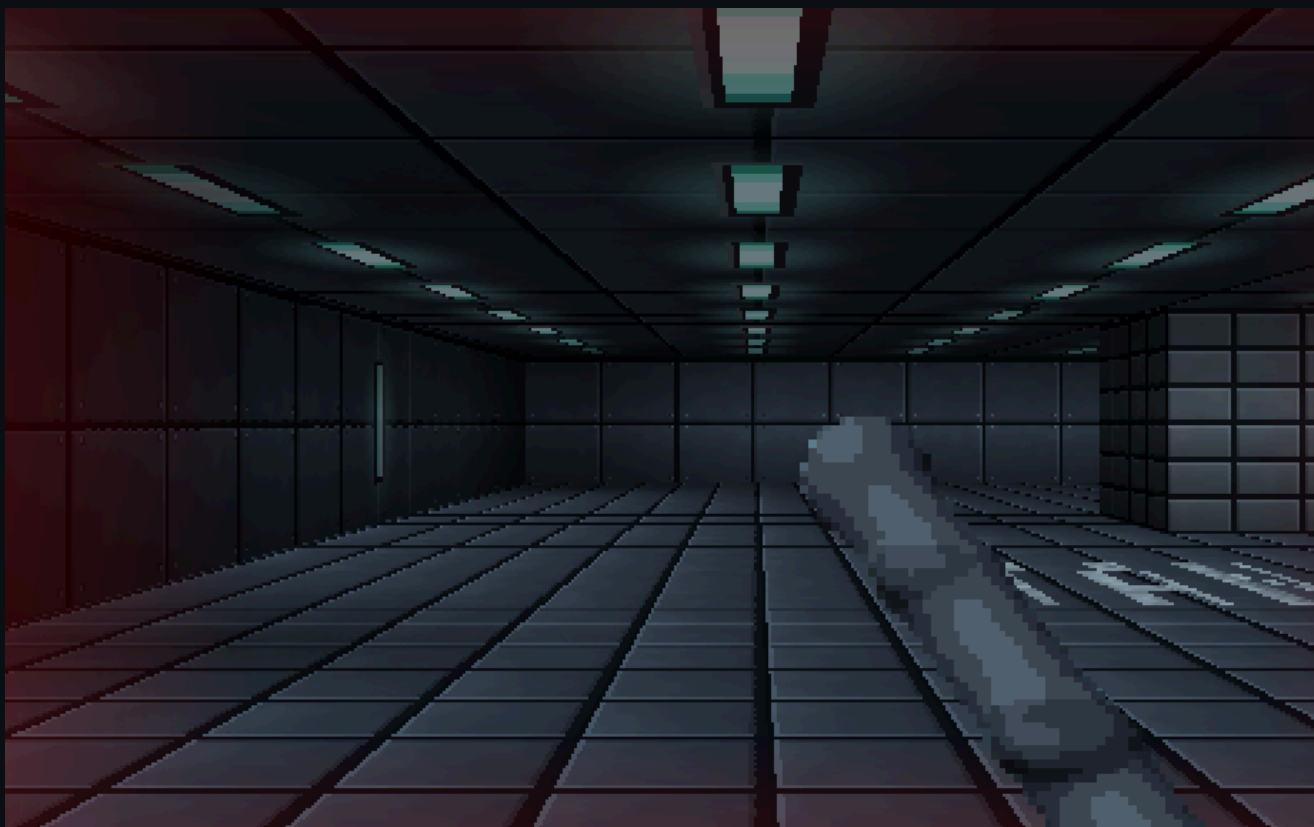


Figure 14.1. Le viseur n'apparaît qu'avec l'arme. Ici la barre est en main, et le centre de l'écran reste libre.

Dans ce chapitre, vous allez montrer où le projectile est allé, avec un éclair, un impact et un viseur.

Sans point d'impact, on tire dans le noir. Rien ne dit si le mur, la créature ou rien du tout a été touché. Nous ajoutons trois retours visuels. L'éclair de la bouche du canon éclaire toute la salle pendant une image, d'autant plus que le mur est proche. L'impact est une tache claire au centre de l'écran, large de près et petite de loin, qui dure un dixième de seconde, sur la créature ou sur le mur selon ce qui a été touché. Une barre qui coupe l'air n'allume rien, car un éclat au milieu de l'écran sur un coup manqué se lirait comme un coup de feu.

Le viseur, quatre traits autour d'un trou, n'apparaît qu'avec l'arme de poing sortie. Dessiné en permanence, il promettrait une visée qu'une barre n'a pas. Chaque trait porte son ombre, sans quoi un viseur d'une seule couleur disparaît sur un mur de la même valeur. Il s'écarte et passe à l'ambre pendant un sixième de seconde quand le coup porte.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE FIGHT H LIGNES 32 A 41

```
32     double flash;           // how long the muzzle still lights the room
33     double hurt;           // how long the screen still shows he was hit
34     // where the bullet landed. with no point of impact one shoots in the
35     // dark: nothing says whether the wall, the beast, or nothing at all
36     // was hit.
37     double spark;
38     double spark_depth;
39     double hitmark;         // the blow landed: the sight says so for a
moment
40 };
41
```

SRC FIGHT C LIGNES 51 A 58

```
51     if (fight.hurt > 0.0)
52         fight.hurt -= elapsed;
53     if (fight.spark > 0.0)
54         fight.spark -= elapsed;
55     if (fight.hitmark > 0.0)
56         fight.hitmark -= elapsed;
57 }
58
```

SRC FIGHT C LIGNES 110 A 129

```
110     best = t;
111 }
112 // the spark means one thing with a gun and another with a bar. a
113 // round always lands somewhere, on the beast or on the wall, and
114 // the impact says where. a bar cutting air lights nothing at all:
115 // a burst mid screen on a miss reads as gunfire.
116 if (gun) {
117     fight.spark = 0.11;
118     fight.spark_depth = best ? nearest
119         : (wall < GUN_RANGE ? wall : GUN_RANGE);
120 } else if (best) {
121     fight.spark = 0.07;
122     fight.spark_depth = nearest;
123 }
124 if (!best)
125     return 0;
126 best->health -= gun ? GUN_DAMAGE : PIPE_DAMAGE;
127 fight.hitmark = 0.16;
128 // it takes the blow: a quarter second where it stops coming, and
129 // that is what gives melee its rhythm
```

SRC FIGHT C LIGNES 227 A 248

```
227     int w = view_width, h = view_height;
228
229     // the muzzle flash lights the room. one frame of white over
230     // everything, falling off with the distance the wall is at: without
231     // it the shot is a sound and a hole, with it the corridor exists
232     if (fight.flash > 0.0 && wall_depth)
233         for (int x = 0; x < w; x++) {
234             double k = 0.55 / (1.0 + wall_depth[x] * 0.8);
235             for (int y = 0; y < h; y++) {
236                 unsigned int c = view[y * w + x];
```

Ligne par ligne

118 La distance de l'impact, celle de la créature touchée ou celle du mur.

234 L'éclair, plus fort dans les colonnes où le mur est proche.

PARTIE 4 CHAPITRE 14 LE CODE

SUITE

```
237         int r = (int)(((c >> 16) & 0xff) + 255 * k);
238         int g = (int)(((c >> 8) & 0xff) + 245 * k);
239         int b = (int)((c & 0xff) + 220 * k);
240         view[y * w + x] = (unsigned int)
241             (((r > 255 ? 255 : r) << 16)
242              | ((g > 255 ? 255 : g) << 8)
243              | (b > 255 ? 255 : b));
244     }
245 }
246
247 // being hit: the edges go dark red. not the middle, the edges,
because
248 // that is where the eye notices without losing sight of the room
```

291 Les quatre traits du viseur, seulement avec l'arme de poing.

308 Le rayon de l'impact, inversement proportionnel à sa distance.

SRC FIGHT C LIGNES 273 A 332

```
273     else
274         draw_pipe(lit, now, moving);
275
276     // the sight only shows with the gun out. drawn all the time it
277     // promises an aim a steel bar does not have, and you think you hold
278     // a pistol before finding one. four strokes around a hole, and only
279     // when there is a round to place.
280     int cx = view_width / 2, cy = HORIZON;
281     int u = view_width / HUD_DIVISOR;
282     if (u < 1)
283         u = 1;
284     // every stroke carries its shadow. a sight of one colour vanishes
285     // the moment it lands on a wall of the same value, which is what a
286     // grey corridor is made of. it also spreads and turns amber for a
287     // sixth of a second on a hit: the only immediate word that it told.
288     int landed = fight.hitmark > 0.0;
289     unsigned int stroke = landed ? 0xf0c060 : 0xd2e2ee;
290     int across = landed ? 2 * u : 0;
291     for (int k = 3 * u + across; fight_gun_out() && k <= 8 * u + across;
k++)
292         for (int side = 0; side < 4; side++) {
293             int dx = side == 0 ? -1 : side == 1 ? 1 : 0;
294             int dy = side == 2 ? -1 : side == 3 ? 1 : 0;
295             int x = cx + dx * k, y = cy + dy * k;
296             if (x < 1 || y < 1 || x >= view_width - 1 || y >= view_height
- 1)
297                 continue;
298             // the shadow first, across the stroke, then the stroke
299             view[(y + (dx ? 1 : 0)) * view_width + x + (dy ? 1 : 0)] =
0x0a0e14;
300             view[y * view_width + x] = stroke;
301         }
302
303     // and the impact where the round stopped: wide up close, a dot far
304     // away, and it lasts a tenth of a second
305     if (fight.spark > 0.0) {
306         double d = fight.spark_depth < 0.5 ? 0.5 : fight.spark_depth;
307         // iron on flesh makes a short spark, not a burst
308         int r = (int)((fight_gun_out() ? 14 : 7) * u / d);
309         if (r < u)
310             r = u;
311         if (r > 16 * u)
312             r = 16 * u;
313         double k = fight.spark / 0.11;
314         for (int y = cy - r; y <= cy + r; y++)
315             for (int x = cx - r; x <= cx + r; x++) {
316                 if (x < 0 || y < 0 || x >= view_width || y >=
view_height)
```

PARTIE 4 CHAPITRE 14 LE CODE

SUITE

```
317             continue;
318             double dx = (double)(x - cx) / r, dy = (double)(y - cy) /
r;
319             double q = dx * dx + dy * dy;
320             if (q > 1.0)
321                 continue;
322             double f2 = (1.0 - q) * k;
323             unsigned int c = view[y * view_width + x];
324             int rr = (int)(((c >> 16) & 0xff) + (255 - ((c >> 16) &
0xff)) * f2);
325             int gg = (int)(((c >> 8) & 0xff) + (226 - ((c >> 8) &
0xff)) * f2);
326             int bb = (int)((c & 0xff) + (150 - (int)(c & 0xff)) * f2
* 0.7);
327             view[y * view_width + x] =
328                 (unsigned int)((rr << 16) | (gg << 8) | bb);
329         }
330     }
331 }
332
```

La lourde

LA PLANCHE DE LA LOURDE 25 IMAGES



Figure 15.1. La planche de la lourde, la même sculpture que la créature ordinaire, dans une matière rouillée et un peu plus grande.

Dans ce chapitre, vous allez ajouter une seconde créature plus résistante, sans dupliquer le code de la première.

Une seconde créature apparaît, la lourde, marquée par la lettre X. Trois cents points de vie, soit neuf coups de barre, et personne ne reste immobile pendant neuf coups. Elle rend l'arme de poing nécessaire, et c'est pourquoi elle n'apparaît qu'après elle dans le niveau. Elle frappe plus fort, marche un peu plus lentement, s'arrête un peu plus loin et mesure un cinquième de plus.

Les deux créatures partagent le même code. Quatre fonctions, `reach_of`, `damage_of`, `speed_of` et `size_of`, lisent le drapeau `tough` et renvoient la bonne valeur. Lire ce drapeau à un seul endroit plutôt que de disperser le test dans la boucle est ce qui permettra à une troisième créature d'arriver sans rien casser. La lourde a sa propre planche, la même sculpture dans une matière rouillée.

La lourde, suite



Figure 15.2. La lourde au fond de la salle, plus grande et plus sombre que la créature ordinaire.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE THING H LIGNES 23 A 34

```
23 // start, to step back, and to answer it.
24 #define THING_GAP      1.45
25 // the heavy one. three hundred points is nine swings of the bar, and
  nobody
26 // stands still for nine swings. it is what makes the sidearm necessary,
  and
27 // that is why it only shows up after it.
28 #define TOUGH_HEALTH    300.0
29 #define TOUGH_DAMAGE    14.0
30 #define TOUGH_SPEED     1.62
31 #define TOUGH_REACH     1.75
32 #define TOUGH_SIZE      1.22
33
34 enum thing_state { THING_IDLE, THING_ALERT, THING_HUNT, THING_STRIKE,
  THING_DEAD };
```

INCLUDE THING H LIGNES 53 A 64

```
53     int used;
54     int silent;           // it has cried once: it does not cry twice
55     int tough;           // the heavy one, the one the bar cannot
  finish
56 };
57
58 extern struct thing things[THINGS_MAX];
59
60 void things_clear(void);
61 // the level says where they are. "x" one standing, "y" a body, "X" a
  heavy.
62 // placing them by hand in main.c would be two truths for one fact.
63 void things_from_level(void);
64 int  thing_add(double x, double y);
```

SRC FIGHT C LIGNES 101 A 106

```
101         double across = fabs(dx * -player->dir_y
102             + dy * player->dir_x);
103         double radius = t->tough ? 0.46 : 0.36;
104         if (across > radius)
105             continue;
106     }
```

SRC THING C LIGNES 16 A 51

```
16 #define ANGLES      5
17 #define WALK_STEPS  4
18 static struct sprite sheet, sheet_tough;
19
20 static void load_sheets(void)
21 {
22     static int tried;
23     if (!tried++) {
24         sprite_load(&sheet, "art/thing.tecs");
25         sprite_load(&sheet_tough, "art/heavy.tecs");
26     }
27 }
28
29 // two beasts, one piece of code. what tells them apart is four numbers,
  and
30 // reading them here rather than scattering the test through the loop is
  what
31 // lets a third arrive without breaking anything.
```

Ligne par ligne

- 28** Neuf coups de barre, ou cinq projectiles.
- 103** Un corps plus large est plus facile à toucher.

PARTIE 4 CHAPITRE 15 LE CODE

SUITE

```
32 static double reach_of(const struct thing *t)
33 {
34     return t->tough ? TOUGH_REACH : THING_REACH;
35 }
36
37 static double damage_of(const struct thing *t)
38 {
39     return t->tough ? TOUGH_DAMAGE : THING_DAMAGE;
40 }
41
42 static double speed_of(const struct thing *t)
43 {
44     return t->tough ? TOUGH_SPEED : THING_SPEED;
45 }
46
47 static double size_of(const struct thing *t)
48 {
49     return t->tough ? TOUGH_SIZE : 1.0;
50 }
51
```

32 La portée selon la créature, lue à un seul endroit.

73 La lettre X du fichier crée une lourde.

256 La planche de la lourde, si elle a pu être chargée.

SRC THING C LIGNES 61 A 65

```
61     char c = map[y][x];
62     int i = -1;
63     if (c == 'x' || c == 'y' || c == 'X')
64         i = thing_add(x + 0.5, y + 0.5);
65     if (i < 0)
```

SRC THING C LIGNES 70 A 77

```
70         things[i].state = THING_DEAD;
71         things[i].health = 0.0;
72     }
73     if (c == 'X') {
74         things[i].tough = 1;
75         things[i].health = TOUGH_HEALTH;
76     }
77 }
```

SRC THING C LIGNES 121 A 125

```
121     if (n > 0.0) { t->dir_x /= n; t->dir_y /= n; }
122
123     double move = speed_of(t) * elapsed;
124     // each axis on its own, so a shoulder on a wall keeps sliding
125     if (!is_wall((int)(t->x + t->dir_x * move * 1.6), (int)t->y))
```

SRC THING C LIGNES 185 A 191

```
185         // the player, drops under the nearest drawing distance,
186         // and the blows come out of nowhere.
187         if (distance > reach_of(t) * 0.85 && now > t->stagger)
188             step_towards(t, bx, by, elapsed);
189         if (distance < reach_of(t) && visible && now > t->
190             >next_strike) {
191             t->state = THING_STRIKE;
192             t->since = now;
```

SRC THING C LIGNES 209 A 214

```
209         // that quarter second is the window to step back
210         if (now - t->since > 0.45) {
211             if (distance < reach_of(t) + 0.3)
212                 damage += damage_of(t);
213             t->state = THING_HUNT;
214             t->since = now;
```

SRC THING C LIGNES 254 A 260

```
254     for (int n = 0; n < count; n++) {
255         struct thing *t = &things[order[n]];
256         const struct sprite *f = t->tough && sheet_tough.pixels
257             ? &sheet_tough : &sheet;
258         if (!f->pixels)
```

PARTIE 4 CHAPITRE 15 LE CODE

SUITE

```
259         continue;
260         double rx = t->x - player->x, ry = t->y - player->y;
```

SRC THING C LIGNES 273 A 278

```
273         // and that reads as a lunge
274         double lunge = t->state == THING_STRIKE ? 1.10 : 1.0;
275         int height = (int)(full * size_of(t) * lunge);
276         int width = height * f->width / f->height;
277         int top = (int)(HORIZON + full * 0.5) - height;
278
```

SRC THING C LIGNES 296 A 301

```
296         : ((int)(t->stride * 2.0) % WALK_STEPS);
297         int frame = row * ANGLES + sector;
298         if (frame >= f->frames)
299             frame = sector % f->frames;
300
301         // the contact shadow, and it is what puts it on the floor.
```

SRC THING C LIGNES 336 A 347

```
336         if (x < 0 || x >= view_width || depth >= wall_depth[x])
337             continue;
338         int sx = (x - (middle - width / 2)) * f->width / width;
339         if (mirror)
340             sx = f->width - 1 - sx;
341         for (int y = top; y < top + height; y++) {
342             if (y < 0 || y >= view_height)
343                 continue;
344             int sy = (y - top) * f->height / height;
345             unsigned int c = sprite_at(f, frame, sx, sy);
346             if ((c >> 24) < 128)
347                 continue;
```

SRC WORLD C LIGNES 312 A 316

```
312         return door_at(x, y) < DOOR_WALKABLE;
313         return !(c == '.' || c == 'S' || c == 'E' || c == 'K'
314                || c == 'x' || c == 'y' || c == 'X' || c == 'G'
315                || c == 'A' || c == 'M');
316     }
```

Mesurer avant de dire que c'est lent

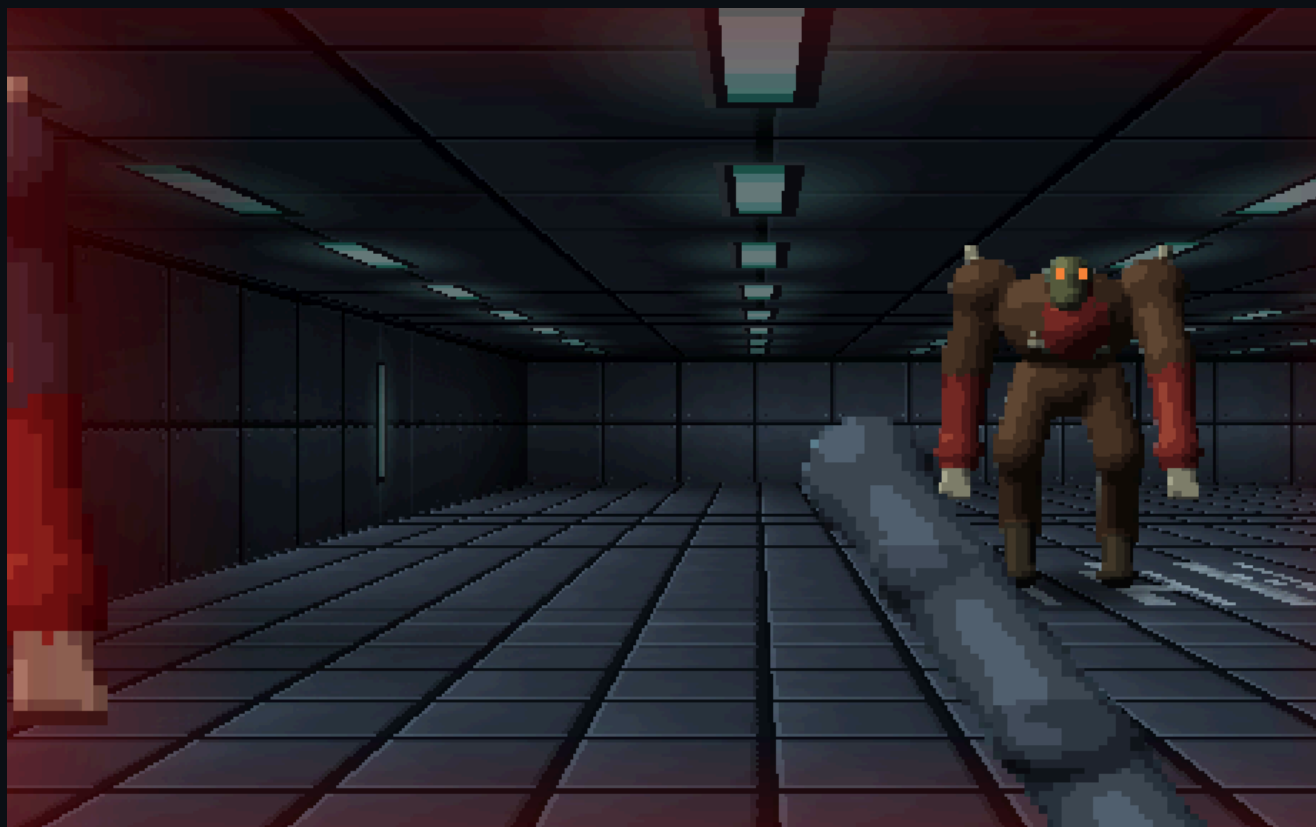


Figure 16.1. L'image est la même, et le jeu retrouve sa vitesse. Rien de ce chapitre ne se voit, tout se sent.

Dans ce chapitre, vous allez mesurer ce qui ralentit le jeu et le corriger, avec une table de lumière et deux autres optimisations.

Le jeu s'est mis à ralentir, et ce chapitre le mesure avant de le corriger. Trois causes apparaissent, toutes dans le dessin du sol, qui traite cent mille pixels par image. La fonction `tint` faisait trois multiplications et trois conversions par pixel. La fonction `lit_at` était appelée pour chaque pixel alors que la lumière ne change que d'une case à l'autre. Et chaque lecture du niveau appelait `strlen`, parce que les lignes du fichier n'ont pas toutes la même longueur.

La première correction est une table de lumière, comme en 1993. Pour 64 niveaux de lumière et 256 valeurs de composante, la table donne la composante éclairée, en 48 kilooctets calculés une fois au démarrage. La fonction `tinted` ne fait plus que trois lectures dans la table. La seconde correction lit la lumière tous les 8 pixels et glisse entre les deux valeurs. L'œil voit la même rampe. La troisième complète les lignes du niveau avec des dièses au chargement, pour que la grille soit rectangulaire en mémoire, et tous les appels à `strlen` disparaissent.

Une quatrième correction touche l'affichage. La seconde ligne de chaque bloc de pixels est la copie de la première, et le programme les calculait toutes les deux. La première est construite, et la seconde est copiée avec `memcpy`, bien plus vite. La règle de ce chapitre vaut pour tout programme. On mesure, on trouve la cause, on corrige la cause, et on mesure de nouveau.

VOCABULAIRE

Une table de correspondance est un tableau calculé une fois qui remplace un calcul répété par une simple lecture.

Mesurer avant de dire que c'est lent, suite

EXERCICE

Mesurez la différence. Compilez avec et sans la table de lumière, et comptez les images par seconde avec une horloge dans la boucle principale.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE TEXTURE H LIGNES 58 A 69

```
58 // the same colour, taken under this much neon and this much gloom
59 unsigned int tint(unsigned int color, double lamp, double night);
60
61 // the light table, the way it was done in 1993. tinting a colour is
  three
62 // multiplies and three casts, and a floor of a hundred thousand pixels
  asks
63 // for that eight hundred thousand times a frame. doom computed none of
  it:
64 // it read a table. one table per channel, and the pixel only reads.
65 #define LIGHT_LEVELS 64
66 void make_light_table(void);
67 unsigned int tinted(unsigned int color, double lamp);
68 // the painted tile: `cell` says which of the four we are looking at
69 unsigned int stencil_tile(int x, int y, int cell);
```

SRC MAIN C LIGNES 52 A 56

```
52 // the textures cost nothing to keep and everything to draw: once,
  here
53 make_wall_textures();
54 make_light_table();
55 light_map();
56 make_floor_texture();
```

SRC RENDER C LIGNES 36 A 40

```
36 if (lit < 0.0)
37     lit = 0.0;
38     view[y * view_width + x] = tinted(color, lit);
39     tex_y += step;
40 }
```

SRC RENDER C LIGNES 75 A 124

```
75 double base = CARRIED * lamp(distance);
76
77 // light does not change from pixel to pixel, it changes
78 // from cell to cell. asking for it a hundred thousand
79 // times a frame cost half the game. sample every eight
80 // pixels and slide between: the eye sees the same ramp.
81 int glow_step = 8;
82 double glow = base + lit_at(ground_x, ground_y);
83 double glow_next = glow, glow_slope = 0.0;
84 // and what lies on the floor is only asked at a cell
85 // change: one whole tile carries the same thing
86 int cell_x = -99999, cell_y = -99999, painted = -1;
87 enum loot item = LOOT_NONE;
88 unsigned int body = BADGE_BODY, edge = BADGE_EDGE;
89
90 for (int x = 0; x < view_width; x++) {
91     if (x % glow_step == 0) {
92         glow = glow_next;
93         double far_x = ground_x + step_x * glow_step;
94         double far_y = ground_y + step_y * glow_step;
95         glow_next = base + lit_at(far_x, far_y);
96         glow_slope = (glow_next - glow) / glow_step;
97     } else {
98         glow += glow_slope;
99     }
100     double light = glow > 1.0 ? 1.0 : glow;
101 }
```

Ligne par ligne

- 65** Le nombre de niveaux de lumière de la table.
- 81** La lumière est lue tous les 8 pixels.
- 98** Entre deux lectures, la lumière glisse en ligne droite.

PARTIE 4 CHAPITRE 16 LE CODE

SUITE

```
102         int tex_x = (int)(ground_x * TEX_SIZE) & TEX_MASK;
103         int tex_y = (int)(ground_y * TEX_SIZE) & TEX_MASK;
104         int at = tex_y * TEX_SIZE + tex_x;
105
106         if ((int)ground_x != cell_x || (int)ground_y != cell_y) {
107             cell_x = (int)ground_x;
108             cell_y = (int)ground_y;
109             painted = stencil_at(cell_x, cell_y);
110             // what can be picked up glows on its own,
111             // in its own colour: amber for rounds,
112             // green for a patch up. you know from
113             // across the room whether it is worth it.
114             item = loot_at(cell_x, cell_y);
115             body = BADGE_BODY;
116             edge = BADGE_EDGE;
117             switch (item) {
118                 case LOOT_GUN:  body = GUN_BODY;  edge = GUN_EDGE;
119                 case LOOT_AMMO: body = AMMO_BODY; edge = AMMO_EDGE;
120                 case LOOT_MED:  body = MED_BODY;  edge = MED_EDGE;
121                 default: break;
122             }
123         }
124         unsigned int ground = item != LOOT_NONE
```

106 Ce que porte le sol n'est demandé qu'au changement de case.

SRC RENDER C LIGNES 127 A 133

```
127         : floor_texture[at];
128         view[y * view_width + x] =
129             tinted(ground, item != LOOT_NONE ? 1.0 : light);
130         view[(view_height - 1 - y) * view_width + x] =
131             tinted(ceiling_texture[at], light * CEILING_PART);
132
133         ground_x += step_x;
```

SRC SCREEN C LIGNES 6 A 10

```
6  #include <stdio.h>
7  #include <stdlib.h>
8  #include <string.h>
9
10 // every pixel of the game lands here first
```

SRC SCREEN C LIGNES 228 A 253

```
228     if (visible > view_width)
229         visible = view_width;
230     // the second row is a copy of the first. we grew every pixel twice,
231     // once per row, a million writes a frame. the bottom row is the top
232     // row, so build it once and copy the block: memcpy is far faster.
233     for (int y = 0; y < view_height; y++) {
234         unsigned int *row = view + y * view_width;
235         int line = screen->top + y * scale;
236         if (line >= screen->height)
237             continue;
238         unsigned int *out = screen->pixels + line * screen->width
239             + screen->left;
240         for (int x = 0; x < visible; x++) {
241             unsigned int color = row[x];
242             for (int again = 0; again < scale; again++)
243                 *out++ = color;
244         }
245         for (int copy = 1; copy < scale; copy++) {
246             int twin = line + copy;
247             if (twin >= screen->height)
248                 break;
```

PARTIE 4 CHAPITRE 16 LE CODE

SUITE

```
249         memcpy(screen->pixels + twin * screen->width + screen->left,
250                screen->pixels + line * screen->width + screen->left,
251                (size_t)visible * scale * sizeof(*screen->pixels));
252     }
253 }
```

SRC TEXTURE C LIGNES 1 A 3

```
1 #include "light.h"
2 #include "texture.h"
3
```

SRC TEXTURE C LIGNES 68 A 106

```
68     }
69     return out;
70 }
71
72 // three channels, sixty-four levels, two hundred and fifty-six values:
73 // forty-eight kilobytes, laid down once and for all at startup
74 static unsigned char light_table[3][LIGHT_LEVELS][CHANNEL + 1];
75
76 void make_light_table(void)
77 {
78     for (int level = 0; level < LIGHT_LEVELS; level++) {
79         double lamp = (double)level / (LIGHT_LEVELS - 1);
80         for (int channel = 0; channel < 3; channel++) {
81             int d = channel * GREEN_SHIFT;
82             double part = lamp * ((LIGHT_LAMP >> d) & CHANNEL)
83                 + GLOOM * ((LIGHT_NIGHT >> d) & CHANNEL);
84             for (int v = 0; v <= CHANNEL; v++) {
85                 double out = v * part * (1.0 / CHANNEL);
86                 light_table[channel][level][v] =
87                     (unsigned char)(out > CHANNEL ? CHANNEL : out);
88             }
89         }
90     }
91 }
92
93 unsigned int tinted(unsigned int color, double lamp)
94 {
95     int level = (int)(lamp * (LIGHT_LEVELS - 1) + 0.5);
96     if (level < 0)
97         level = 0;
98     if (level >= LIGHT_LEVELS)
99         level = LIGHT_LEVELS - 1;
100     return ((unsigned int)light_table[2][level][(color >> RED_SHIFT) &
CHANNEL]
101            << RED_SHIFT)
102        | ((unsigned int)light_table[1][level][(color >> GREEN_SHIFT) &
CHANNEL]
103         << GREEN_SHIFT)
104        | light_table[0][level][color & CHANNEL];
105 }
106
```

SRC WORLD C LIGNES 64 A 84

```
64     }
65     fclose(f);
66
67     // a rectangular map has to be one in memory too. the file's lines
68     // are not all the same length, and every reader made up for it with
69     // a strlen. on the floor that runs once per pixel, which is half
70     // the frame. pad the rows on load and every strlen goes away.
71     for (int y = 0; y < map_height; y++) {
72         int length = (int)strlen(map[y]);
```

249 La seconde ligne du bloc est copiée, pas recalculée.

74 La table, 3 canaux, 64 niveaux, 256 valeurs, 48 kilooctets.

93 Trois lectures dans la table, et rien d'autre.

PARTIE 4 CHAPITRE 16 LE CODE

SUITE

```
73         if (length >= map_width)
74             continue;
75         char *square = realloc(map[y], (size_t)map_width + 1);
76         if (!square)
77             return 0;
78         memset(square + length, '#', (size_t)map_width - length);
79         square[map_width] = '\0';
80         map[y] = square;
81     }
82
83     if (map_height > 0) {
84         free(doors);
```

78 Les lignes courtes sont complétées avec des murs.

SRC WORLD C LIGNES 230 A 234

```
230 int is_door(int x, int y)
231 {
232     if (x < 0 || y < 0 || x >= map_width || y >= map_height)
233         return 0;
234     return map[y][x] == '+';
```

SRC WORLD C LIGNES 239 A 243

```
239 int is_lamp(int x, int y)
240 {
241     if (x < 0 || y < 0 || x >= map_width || y >= map_height)
242         return 0;
243     char c = map[y][x];
```

SRC WORLD C LIGNES 283 A 287

```
283 int lamp_faulty(int x, int y)
284 {
285     if (x < 0 || y < 0 || x >= map_width || y >= map_height)
286         return 0;
287     return map[y][x] == 'F';
```

SRC WORLD C LIGNES 300 A 304

```
300 int wall_kind(int x, int y)
301 {
302     if (x < 0 || y < 0 || x >= map_width || y >= map_height)
303         return 0;
304     char c = map[y][x];
```

SRC WORLD C LIGNES 316 A 320

```
316 int is_wall(int x, int y)
317 {
318     if (x < 0 || x >= map_width || y < 0 || y >= map_height)
319         return 1;
320     // the file carries more than walls and floor: where the player
```

La sortie

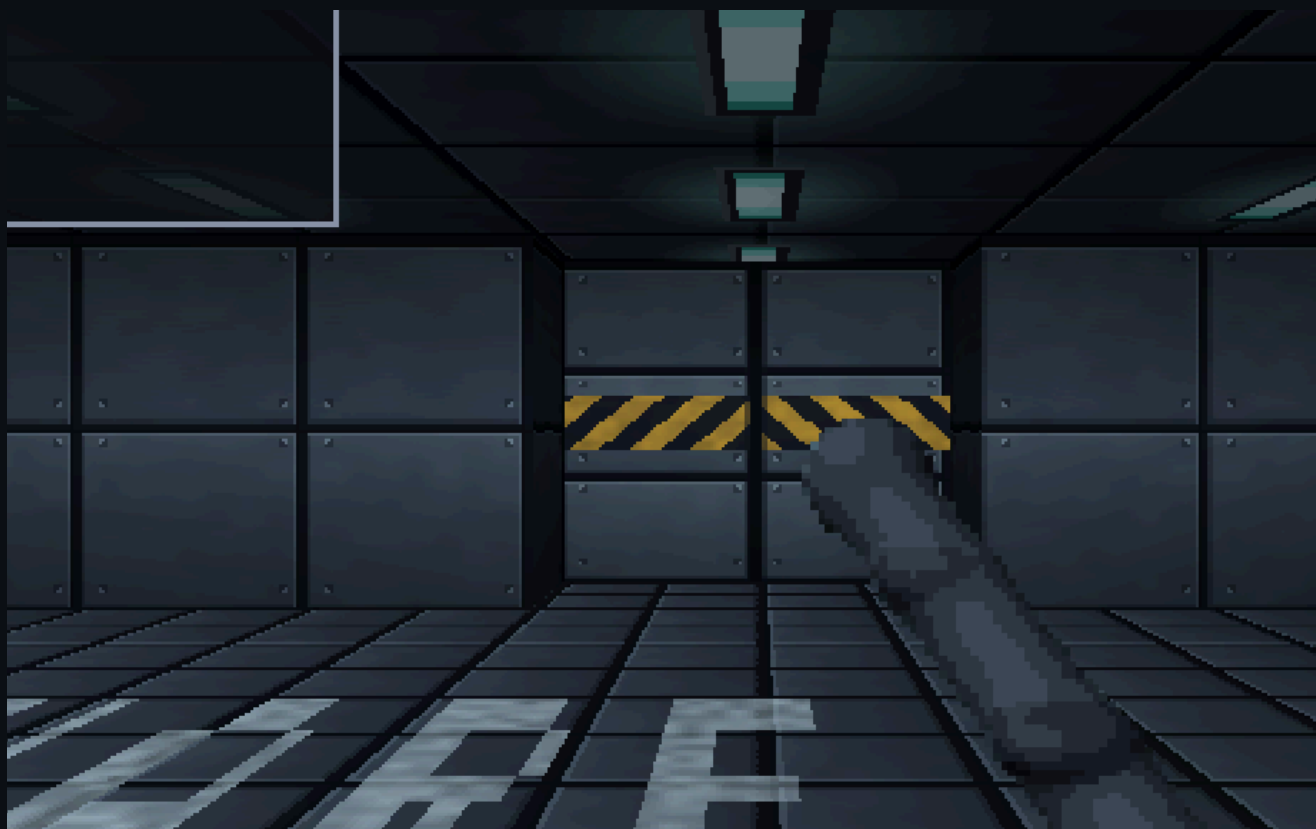


Figure 17.1. Le sas au départ. L'écran de fin n'apparaît qu'à la trappe, une fois le badge trouvé.

Dans ce chapitre, vous allez écrire la fin du niveau, avec l'image qui s'assombrit et le compte des créatures abattues.

Le niveau a maintenant une vraie fin. Quand le joueur atteint la trappe avec le badge, l'image s'assombrit sur 2,5 secondes, jusqu'à 82 pour cent et non jusqu'au noir, car la salle est encore là derrière les mots. Deux lignes s'écrivent au milieu de l'écran, la première dit que le joueur est sorti, la seconde compte les créatures abattues. Ce compte dit que c'était sa partie et non une partie.

Dès que la trappe s'ouvre, le joueur ne bouge plus et ne prend plus de dégâts. La fonction `fight_dim` assombrit toute l'image d'une quantité donnée, et le compteur `put_down` est incrémenté à chaque créature abattue.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE FIGHT H LIGNES 38 A 42

```
38     double spark_depth;
39     double hitmark;           // the blow landed: the sight says so for a
    moment
40     int put_down;             // how many of them we have put down
41 };
42
```

INCLUDE FIGHT H LIGNES 58 A 64

```
58 // patch up
59 void fight_heal(double amount);
60 // the level is over: the whole view goes dark, by this much, and the
61 // words are written on top. the edges alone are for taking a hit.
62 void fight_dim(double amount);
63
64 #endif
```

INCLUDE TEXT H LIGNES 8 A 13

```
8  #define NOTICE_SECONDS 5.0
9  #define NOTICE_UP      34
10 // the two lines of the ending, this far above and below the middle
11 #define ENDING_GAP      10
12
13 #define GLYPH_W         5
```

INCLUDE WORLD H LIGNES 49 A 58

```
49 #define DOOR_REACH      2.6
50
51 // the way out: once the hatch opens the view goes dark over this many
52 // seconds, down to this much, and the words stay. not to black: the room
53 // is still there, behind the words
54 #define ENDING_FADE      2.5
55 #define ENDING_DARK      0.82
56
57 int is_door(int x, int y);
58 // from 0 (shut) to 1 (both leaves tucked into the wall)
```

SRC FIGHT C LIGNES 133 A 137

```
133     if (best->health <= 0.0) {
134         best->state = THING_DEAD;
135         fight.put_down++;
136     } else {
137         // being hit wakes it, even if it had not seen you
```

SRC FIGHT C LIGNES 393 A 406

```
393     draw_text(view_width - margin - 13, bottom - line_h, attack,
0x6a7b8c, unit);
394 }
395
396 void fight_dim(double amount)
397 {
398     double keep = 1.0 - amount;
399     for (int i = 0; i < view_width * view_height; i++) {
400         unsigned int c = view[i];
401         int r = (int)((c >> RED_SHIFT) & CHANNEL) * keep;
402         int g = (int)((c >> GREEN_SHIFT) & CHANNEL) * keep;
403         int b = (int)((c & CHANNEL) * keep);
404         view[i] = (unsigned int)((r << RED_SHIFT) | (g << GREEN_SHIFT) |
b);
```

Ligne par ligne

- 54** La durée de l'assombrissement.
- 55** L'image garde 18 pour cent de sa lumière.
- 135** Le compte des créatures abattues.

PARTIE 4 CHAPITRE 17 LE CODE

SUITE

```
405     }
406 }
```

SRC MAIN C LIGNES 5 A 9

```
5  #include <X11/Xlib.h>
6  #include <X11/Xutil.h>
7  #include <stdio.h>
8  #include <stdlib.h>
9  #include <time.h>
```

SRC MAIN C LIGNES 74 A 78

```
74     fight_reset();
75     double died_at = 0.0;
76     double won_at = 0.0;
77     // the first two things to learn, and they are learned in the quiet.
78     // the swing is taught in the airlock, where there is nothing to hit:
```

SRC MAIN C LIGNES 147 A 153

```
147         keys.hit = 0;
148     }
149     double damage = things_update(&player, elapsed, moment);
150     if (!won_at)
151         fight_take(damage, moment);
152     fight_step(elapsed);
153     move_doors(elapsed);
```

SRC MAIN C LIGNES 178 A 191

```
178         if (!have_badge())
179             say("THE HATCH IS LOCKED", moment);
180         else if (!won_at)
181             won_at = moment;
182     }
183
184     // every move is scaled by the time the last frame took, so the
game
185     // runs at the same speed whatever the machine is doing
186     int alive = fight.health > 0.0;
187     // the hatch is open: the man is out, and he stops here
188     if (won_at)
189         alive = 0;
190     double forward = alive
191         ? (keys.forward - keys.back) * WALK_SPEED * elapsed : 0.0;
```

SRC MAIN C LIGNES 220 A 235

```
220         draw_text_centered(view_height - hud * NOTICE_UP, notice,
221             TEXT_COLOR, hud * TEXT_SCALE);
222     // the end: the view goes dark, and the count says it was
223     // our game and not a game
224     if (won_at) {
225         double gone = (moment - won_at) / ENDING_FADE * ENDING_DARK;
226         fight_dim(gone > ENDING_DARK ? ENDING_DARK : gone);
227         draw_text_centered(view_height / 2 - hud * ENDING_GAP,
228             "YOU MADE IT OUT", TEXT_COLOR, hud * TEXT_SCALE);
229         char count[32];
230         snprintf(count, sizeof count, "%d PUT DOWN", fight.put_down);
231         draw_text_centered(view_height / 2 + hud * ENDING_GAP,
232             count, TEXT_COLOR, hud * TEXT_SCALE);
233     }
234     screen_present(&screen);
235
```

188 La trappe ouverte, le joueur s'arrête.

230 La seconde ligne de la fin, avec le compte.

Personne aux touches



Figure 18.1. Le niveau de la quatrième partie et la route de la borne, en trait fin, depuis le départ. Les créatures rencontrées en chemin sont combattues, puis la route reprend.

Dans ce chapitre, vous allez faire jouer le niveau tout seul, comme une borne d'arcade, en écrivant dans les touches à la place d'une main.

Une borne d'arcade joue toute seule quand personne n'a mis de pièce. Après 12 secondes sans touche, le niveau se joue lui-même. Le fichier `demo.c` lit une route dans un fichier, un point par ligne, et la suit. La fonction `demo_drive` ne touche pas au joueur, elle écrit dans la structure `keys` les mêmes touches qu'une main appuierait. Le reste du programme ne sait pas qui joue.

La fonction `bearing` donne l'angle entre le regard et le prochain point, et les flèches gauche et droite l'annulent. Le joueur avance quand le point est dans un cône devant lui. Un point atteint, ou un point sur lequel la borne ne gagne rien pendant 3 secondes, cède la place au suivant. Une créature vivante en vue prend le pas sur la route. La borne se tourne vers elle, s'approche avec la barre ou reste à distance avec l'arme, et frappe quand elle est dans l'axe. Une porte devant elle est poussée. La première touche rend la main au joueur.

VOCABULAIRE

Le mode borne ou mode démonstration est le comportement d'un jeu d'arcade qui se joue seul pour attirer les passants.

Personne aux touches, suite

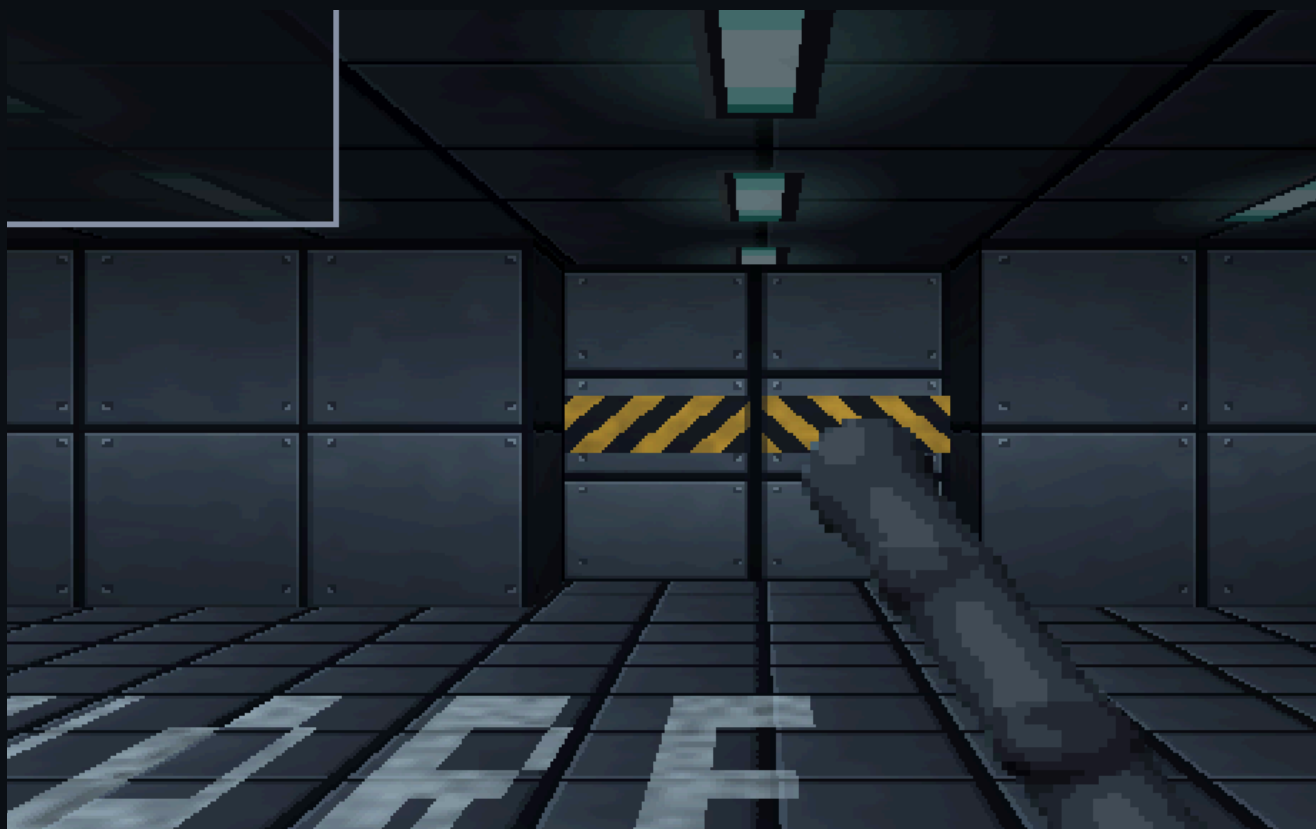


Figure 18.2. Le sas, quelques secondes avant que la borne ne prenne les touches.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE DEMO H LIGNES 1 A 36

```

1  #ifndef DEMO_H
2  #define DEMO_H
3
4  #include "screen.h"
5  #include "world.h"
6
7  // the level plays itself, the way an arcade cabinet does when nobody has
8  // put a coin in. it walks a route written in a file, one point per line,
9  // and it fights what gets in the way with whatever is in hand.
10
11 #define ROUTE_FILE      "levels/keep.route"
12 #define ROUTE_MAX      64
13
14 // no key for this long, and the cabinet takes over. any key gives it
   back
15 #define DEMO_AFTER      12.0
16 // close enough to a point to go for the next one, and how long it keeps
17 // trying a point it gains nothing on before giving that point up
18 #define DEMO_ARRIVED    0.45
19 #define DEMO_PATIENCE   3.0
20 // it turns until the point is this close to straight ahead, and it walks
21 // as soon as the point is inside this cone
22 #define DEMO_TURN_TOL   0.06
23 #define DEMO_WALK_CONE  0.35
24 // how far it notices something alive, how wide a body is to aim at, and
25 // how close it walks in with the bar in hand
26 #define DEMO_ENGAGE     7.0
27 #define DEMO_BODY       0.30
28 #define DEMO_STANDOFF   1.6
29
30 // how many points the file gave
31 int  demo_load(const char *path);
32 void demo_reset(void);
33 // reads the world, writes the keys: the same keys a hand would press
34 void demo_drive(const struct player *player, struct keys *keys, double
   now);
35
36 #endif

```

INCLUDE SCREEN H LIGNES 50 A 54

```

50  int hit;           // a press on ctrl: swing
51  int weapon;        // 1 or 2: the bar or the sidearm
52  int any;           // somebody is at the keys
53 };
54

```

SRC DEMO C LIGNES 1 A 129

```

1  #include <math.h>
2  #include <stdio.h>
3
4  #include "demo.h"
5  #include "fight.h"
6  #include "thing.h"
7

```

Ligne par ligne

15

Le délai sans touche avant que la borne ne prenne la main.

PARTIE 4 CHAPITRE 18 LE CODE

SUITE

```
8 static double route_x[ROUTE_MAX], route_y[ROUTE_MAX];
9 static int points, leg;
10 // how close it has come to the point so far, and when it last got closer
11 static double best_yet, better_at;
12
13 void demo_reset(void)
14 {
15     leg = 0;
16     best_yet = -1.0;
17     better_at = 0.0;
18 }
19
20 int demo_load(const char *path)
21 {
22     FILE *f = fopen(path, "r");
23     if (!f)
24         return 0;
25     points = 0;
26     double x, y;
27     while (points < ROUTE_MAX && fscanf(f, "%lf %lf", &x, &y) == 2) {
28         route_x[points] = x;
29         route_y[points] = y;
30         points++;
31     }
32     fclose(f);
33     demo_reset();
34     return points;
35 }
36
37 // the angle from where the eyes point to where the target is. positive
38 // is
39 // to the right, the way the rotation goes, and it is what the arrow keys
40 // have to undo
41 static double bearing(const struct player *p, double x, double y)
42 {
43     double dx = x - p->x, dy = y - p->y;
44     return atan2(p->dir_x * dy - p->dir_y * dx, p->dir_x * dx + p->dir_y
45 * dy);
46 }
47
48 // a straight line to it, with no wall in between
49 static int clear_to(const struct player *p, double x, double y)
50 {
51     double dx = x - p->x, dy = y - p->y;
52     double d = hypot(dx, dy);
53     for (double t = 0.3; t < d; t += 0.3)
54         if (is_wall((int)(p->x + dx / d * t), (int)(p->y + dy / d * t)))
55             return 0;
56     return 1;
57 }
58
59 static const struct thing *nearest_live(const struct player *p)
60 {
61     const struct thing *best = 0;
62     double nearest = DEMO_ENGAGE;
63     for (int i = 0; i < THINGS_MAX; i++) {
64         const struct thing *t = &things[i];
65         if (!t->used || t->state == THING_DEAD)
66             continue;
67         double d = hypot(t->x - p->x, t->y - p->y);
68         if (d < nearest && clear_to(p, t->x, t->y)) {
69             nearest = d;
70             best = t;
71         }
72     }
73 }
```

40 L'angle entre le regard et un point, positif vers la droite.

PARTIE 4 CHAPITRE 18 LE CODE

SUITE

```
71     return best;
72 }
73
74 static void next_point(double now)
75 {
76     leg++;
77     best_yet = -1.0;
78     better_at = now;
79 }
80
81 // face it. in reach, swing; with the bar, walk in first
82 static void fight_it(const struct player *p, struct keys *keys,
83                     const struct thing *t, double now)
84 {
85     double turn = bearing(p, t->x, t->y);
86     double d = hypot(t->x - p->x, t->y - p->y);
87     double aim = atan2(DEMO_BODY, d);
88     keys->right = turn > aim;
89     keys->left = turn < -aim;
90     double reach = fight_gun_out() ? GUN_RANGE : PIPE_REACH;
91     double stop = fight_gun_out() ? GUN_RANGE : DEMO_STANDOFF;
92     if (fabs(turn) < aim && d < reach)
93         keys->hit = 1;
94     keys->forward = fabs(turn) < DEMO_WALK_CONE && d > stop;
95     // a fight is not time lost on the route
96     better_at = now;
97 }
98
99 // nothing in the way: turn to the next point, and walk when it is ahead
100 static void follow_route(const struct player *p, struct keys *keys,
101                          double now)
102 {
103     double turn = bearing(p, route_x[leg], route_y[leg]);
104     double d = hypot(route_x[leg] - p->x, route_y[leg] - p->y);
105     keys->right = turn > DEMO_TURN_TOL;
106     keys->left = turn < -DEMO_TURN_TOL;
107     keys->forward = fabs(turn) < DEMO_WALK_CONE;
108     if (d < DEMO_ARRIVED) {
109         next_point(now);
110     } else if (best_yet < 0.0 || d < best_yet - DEMO_ARRIVED / 4) {
111         best_yet = d;
112         better_at = now;
113     } else if (now - better_at > DEMO_PATIENCE) {
114         next_point(now);
115     }
116 }
117
118 void demo_drive(const struct player *p, struct keys *keys, double now)
119 {
120     keys->forward = keys->back = keys->left = keys->right = 0;
121     if (leg >= points)
122         return;
123     const struct thing *t = nearest_live(p);
124     if (t)
125         fight_it(p, keys, t, now);
126     else
127         follow_route(p, keys, now);
128     if (door_ahead(p))
129         keys->push = 1;
```

104 La borne appuie sur la flèche droite tant que le point est à droite.

112 Un point sur lequel la borne ne progresse plus est abandonné après 3 secondes.

123 Une créature en vue passe avant la route.

SRC MAIN C LIGNES 10 A 14

```
10 #include <unistd.h>
11
12 #include "demo.h"
13 #include "fight.h"
14 #include "light.h"
```

PARTIE 4 CHAPITRE 18 LE CODE

SUITE

SRC MAIN C LIGNES 74 A 83

```
74     things_from_level();
75     fight_reset();
76     demo_load(ROUTE_FILE);
77     double died_at = 0.0;
78     double won_at = 0.0;
79     // when the keys were last touched, and whether the cabinet is
playing
80     double last_key = last;
81     int cabinet = 0;
82     // the first two things to learn, and they are learned in the quiet.
83     // the swing is taught in the airlock, where there is nothing to hit:
```

104 La borne joue si personne n'a touché une touche depuis 12 secondes.

176 N'importe quelle touche signale qu'une main est là.

SRC MAIN C LIGNES 93 A 109

```
93
94     screen_read_keys(&screen, &keys);
95     if (keys.any) {
96         keys.any = 0;
97         last_key = moment;
98         // the hand is back: the cabinet lets go of what it held
99         if (cabinet)
100             keys.forward = keys.back = keys.left = keys.right = 0;
101         cabinet = 0;
102     }
103     // nobody at the keys for a while: the level plays itself
104     if (moment - last_key > DEMO_AFTER && fight.health > 0.0 &&
!won_at) {
105         demo_drive(&player, &keys, moment);
106         cabinet = 1;
107     }
108     if (!lesson && moment - lesson_at > 7.0) {
109         say("CTRL SWINGS WHAT YOU ARE HOLDING", moment);
```

SRC MAIN C LIGNES 188 A 192

```
188         things_from_level();
189         fight_reset();
190         demo_reset();
191         say("AGAIN", moment);
192     }
```

SRC SCREEN C LIGNES 173 A 178

```
173     if (event.type == KeyPress || event.type == KeyRelease) {
174         int down = event.type == KeyPress;
175         if (down)
176             keys->any = 1;
177         KeySym key = XkbKeycodeToKeysym(screen->display,
178             event.xkey.keycode, 0, 0);
```

4



QUATRIEME PARTIE SUITE

Le son, l'histoire et la fin

Dans cette seconde moitié de la quatrième partie, vous allez terminer le jeu. Le son sera calculé comme les textures, sans aucun fichier. La station racontera ce qui s'est passé pendant que le joueur marche, une porte bloquée montrera un combat qu'il ne peut pas rejoindre, les survivants parleront, et la chose du pont quatre attendra au fond de la soute.

12 CHAPITRES 14 FICHIERS 2603 LIGNES LANGAGE C

Le son est calculé lui aussi



Figure 1.1. Les quatre formes de base du synthétiseur, calculées avec les formules de `sound.c`. La dent de scie et le carré sont les oscillateurs. L'enveloppe monte vite et descend lentement. La dent de scie filtrée a perdu ses angles, c'est-à-dire ses aigus.

Dans ce chapitre, vous allez calculer le son du jeu, la musique et les bruitages, avec un oscillateur, une enveloppe, un filtre et une horloge, sans aucun fichier.

Le jeu n'a pas de fichier sonore. Comme les textures, le son est calculé par le programme, à partir de quatre pièces, un oscillateur, une enveloppe, un filtre et une horloge. Le nouveau fichier `sound.c` produit 44 100 échantillons par seconde, c'est-à-dire 44 100 nombres entre -1 et 1 qui décrivent la pression de l'air, et les remet à la carte son par la bibliothèque ALSA. Le calcul se fait dans un second fil d'exécution, pour que la boucle du jeu n'attende jamais la carte son.

Un oscillateur produit une onde qui se répète, une dent de scie ou un carré, dont la fréquence est la hauteur de la note. Une enveloppe fait monter puis descendre le volume d'un son, avec une attaque et une décroissance. Un filtre passe-bas atténue les aigus au-dessus d'une fréquence de coupure, et un filtre passe-bande ne garde qu'une bande de fréquences. La figure 1.1 montre ces quatre formes. La fonction `drive` écrase l'onde comme un amplificateur poussé trop fort, avec la fonction `tanh`, ce qui donne le son d'une guitare saturée.

La musique est faite de quatre couches dans la même tonalité et au même tempo, qui s'empilent sans arrangement. Le souffle de la station, deux dents de scie légèrement désaccordées et fortement filtrées. La pulsation, une basse en croches dont le filtre s'ouvre à chaque note. Le métal, une grosse caisse et une caisse claire faites de bruit filtré. Et le riff, une onde riche, saturée, passée dans un simulateur d'ampli. Le jeu ne lance jamais un nouveau morceau, il monte et descend ces couches. La page suivante calcule les notes.

Les bruitages sont des sons courts calculés de la même façon, un tir, un coup, une porte qui glisse sur son rail, un pas sur la tôle, une explosion, un cri. Les cris viennent d'une gorge simulée, un bourdonnement passé dans trois résonances, comme une voix humaine dont on aurait écarté les résonances. La fonction `fill` mélange la musique, jusqu'à huit bruitages en cours, et un écho, puis écrit les échantillons. La distance d'un bruitage le rend plus sourd et plus faible, comme dans une vraie salle.

Le son est calculé lui aussi, suite

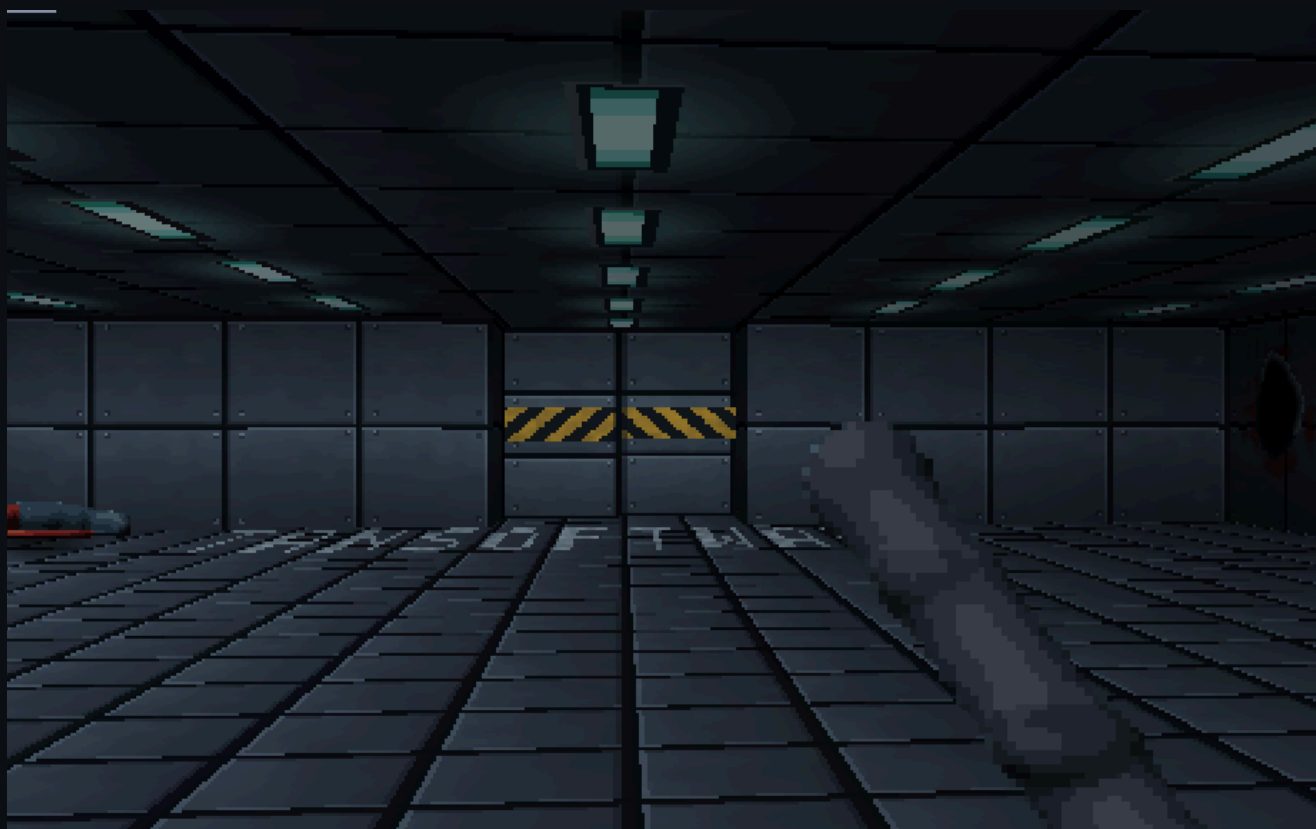


Figure 1.2. L'image ne change pas. Le son, lui, commence. Une porte qui s'ouvre fait maintenant du bruit.

VOCABULAIRE

Un échantillon est un nombre qui décrit la pression de l'air à un instant. Il y en a 44 100 par seconde. **Un fil d'exécution** ou thread est une seconde suite d'instructions qui s'exécute en même temps que la boucle du jeu. Un verrou, le mutex, empêche les deux de modifier les mêmes données au même moment.

EXERCICE

Changez **BPM** pour 120 et écoutez le riff ralentir. Puis remettez 162, le tempo qui fait la tension.

Les notes, en nombres

NOTE	DEMI TONS	CALCUL	FREQUENCE
mi	0	$41,20 \times 2^{0/12}$	41,2 Hz
fa	1	$41,20 \times 2^{1/12}$	43,6 Hz
sol	3	$41,20 \times 2^{3/12}$	49 Hz
la	5	$41,20 \times 2^{5/12}$	55 Hz
si	7	$41,20 \times 2^{7/12}$	61,7 Hz
do	8	$41,20 \times 2^{8/12}$	65,4 Hz
ré	10	$41,20 \times 2^{10/12}$	73,4 Hz
mi	12	$41,20 \times 2^{12/12}$	82,4 Hz

Tableau 1.1. Les huit degrés de la gamme phrygienne sur mi, et leur fréquence. Chaque demi-ton multiplie la fréquence par 1,0595, et le douzième retombe sur le double de la note de base.

Un son est une vibration, et sa hauteur est le nombre de vibrations par seconde, en hertz. La note de base de la musique du jeu est un mi grave, à 41,2 hertz. Toutes les autres notes s'en déduisent par une seule règle. Monter d'une octave double la fréquence, et une octave compte douze demi-tons. Monter d'un demi-ton multiplie donc la fréquence par la racine douzième de 2, soit 1,0595.

La gamme du jeu est la gamme phrygienne, choisie parce qu'elle sonne comme une menace. Ses huit degrés sont à 0, 1, 3, 5, 7, 8, 10 et 12 demi-tons de la note de base. Le tableau 1.1 donne leur fréquence. Le programme calcule chaque note avec la fonction `pow`, $41,2 \times 2$ élevé à la puissance du nombre de demi-tons divisé par 12.

Le tempo est de 162 battements par minute, soit un temps toutes les 0,37 seconde et une double croche toutes les 0,093 seconde. Le riff de la figure 1.2 avance d'une note par double croche, et tous les sons du jeu se calent sur cette horloge.

EXERCICE

Calculez la fréquence du la, à 5 demi-tons au-dessus du mi. Vous devez trouver $41,2 \times 1,335 = 55$ hertz, le la grave d'une basse.

Le riff

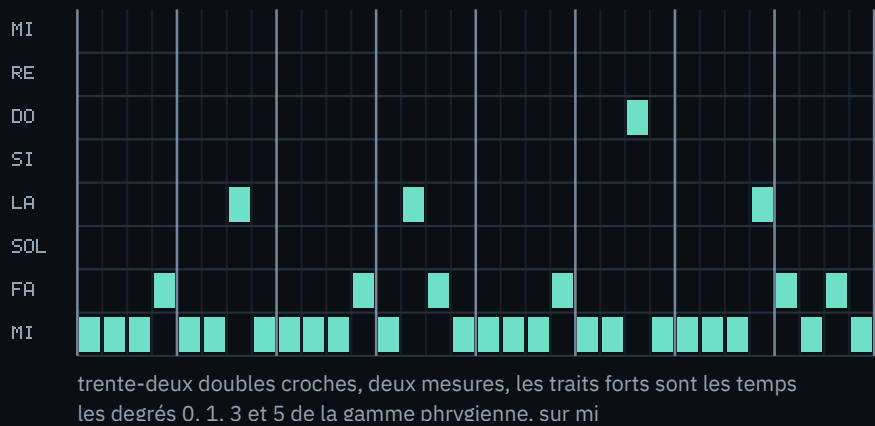


Figure 1.2. Le riff de la couche de guitare, une note par double croche sur deux mesures. Les cases pleines sont les notes jouées, sur quatre degrés de la gamme.

La couche de guitare joue un riff de trente-deux doubles croches, deux mesures qui se répètent. Le tableau **RIFF** donne, pour chaque double croche, le degré de la gamme à jouer, et le tableau **SCALE** convertit ce degré en demi-tons. La figure 1.2 montre le riff comme sur un rouleau de piano mécanique.

À chaque double croche, une enveloppe très courte imite le coup de médiator et une enveloppe plus longue tient la note. Le tout passe dans la saturation, puis dans le simulateur d'ampli, un passe-bas à 3 800 hertz, un coupe-bas à 95 hertz et un peu de présence à 2 100 hertz. La grosse caisse et la caisse claire suivent les tableaux **KICK** et **SNARE**, seize cases chacun.

Le tout est calé sur une seule horloge, le temps écoulé depuis le démarrage du mélangeur. La double croche courante est ce temps divisé par la durée d'une double croche, et sa position dans le tableau est le reste de la division par 32.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE SOUND H LIGNES 1 A 45

```

1  #ifndef SOUND_H
2  #define SOUND_H
3
4  // the sound is calculated, like the textures. there is no file and no
   sample,
5  // only oscillators, an envelope, a resonant filter and a sequencer
   clock.
6  // also takes the frames we hand it and does nothing else.
7  #define SOUND_RATE      44100
8  // the output volume is low on purpose, because a game left running in
   the
9  // background must not make the room jump. 0.35 leaves room for the
   gunshots,
10 // which are the only sounds that should crack.
11 #define SOUND_MASTER    0.35
12 #define SOUND_CHUNK     512      // frames per write: short enough to
   stay in time
13
14 // the music is made of four layers in the same key and the same tempo,
   so
15 // they stack without any arranging. the game only turns them up and
   down, it
16 // never starts a new piece.
17 enum layer {
18     LAYER_BREATH,      // the station idling. on from the third door
19     LAYER_PULSE,       // a bass in eighths, when something is awake
20     LAYER_METAL,       // struck noise, when it is a fight
21     LAYER_DRIVE,      // the riff. this one is the fight itself
22     LAYER_COUNT
23 };
24
25 enum sfx {
26     SFX_SHOT, SFX_PUNCH, SFX_IMPACT, SFX_DOOR, SFX_PICKUP,
27     SFX_SHRIEK, SFX_GROWL, SFX_HURT, SFX_DIE,
28     // what blows up makes a sound, but the shake itself makes none. a
   shake
29     // is the consequence of something, and that something is what makes
   the
30     // noise.
31     SFX_BLAST, SFX_STEP, SFX_COUNT
32 };
33
34 int  sound_open(void);
35 void sound_close(void);
36 // where the layer should sit, from 0 to 1. it slides there, it does not
   jump.
37 void sound_layer(enum layer which, double target);
38 // plays a sound once. distance makes it duller and quieter, the way a
   room
39 // does.
40 void sound_play(enum sfx which, double distance);
41 // turns the sound off or on, with the N key during the game
42 void sound_mute(int on);
43 int  sound_muted(void);
44
45 #endif

```

Ligne par ligne

- 7** Le nombre d'échantillons par seconde, celui d'un disque compact.
- 17** Les quatre couches de la musique.

PARTIE 4 CHAPITRE 1 LE CODE

SUITE

SRC MAIN C LIGNES 15 A 19

```
15 #include "render.h"
16 #include "screen.h"
17 #include "sound.h"
18 #include "text.h"
19 #include "texture.h"
```

14 Le tempo, en battements par minute.

21 Le filtre passe-bas. Deux nombres d'état, et une fréquence de coupure.

SRC MAIN C LIGNES 72 A 76

```
72 struct keys keys = { .map = 1 };
73 double last = now_in_seconds();
74 sound_open();
75 things_clear();
76 things_from_level();
```

SRC MAIN C LIGNES 127 A 134

```
127 if (door_ahead(&player) && fight.health > 0.0)
128     say("PRESS SPACE TO OPEN", moment);
129 if (push_key) {
130     push_door(&player);
131     sound_play(SFX_D00R, 0.0);
132 }
133 // picked up by walking over it, no stop and no panel
134 switch (loot_take(&player)) {
```

SRC SOUND C LIGNES 1 A 412

```
1 #include <alsa/asoundlib.h>
2 #include <math.h>
3 #include <pthread.h>
4 #include <stdlib.h>
5 #include <string.h>
6
7 #include "sound.h"
8
9 // ----- the
  parts
10 // everything below is made of four pieces: an oscillator, an envelope, a
11 // filter and a clock. a guitar tone is a rich wave, hard clipping, and a
12 // narrow band: none of those three is the string.
13
14 #define BPM      162.0
15 #define BEAT     (60.0 / BPM)
16 #define SIXTEEN  (BEAT / 4.0)
17 #define ROOT     41.20          // e1, low enough to feel it
18
19 struct filter { double low, band; };
20
21 static double lowpass(struct filter *f, double in, double cut, double q)
22 {
23     double g = 2.0 * sin(M_PI * (cut < 20000 ? cut : 20000) /
  SOUND_RATE);
24     f->low += g * f->band;
25     double high = in - f->low - q * f->band;
26     f->band += g * high;
27     return f->low;
28 }
29
30 static double bandpass(struct filter *f, double in, double cut, double q)
31 {
32     double g = 2.0 * sin(M_PI * (cut < 20000 ? cut : 20000) /
  SOUND_RATE);
33     f->low += g * f->band;
34     double high = in - f->low - q * f->band;
35     f->band += g * high;
36     return f->band;
```

PARTIE 4 CHAPITRE 1 LE CODE

SUITE

```
37 }
38
39 static double saw(double phase) { return 2.0 * (phase - floor(phase +
0.5)); }
40 static double square(double phase) { return phase - floor(phase) < 0.5 ?
1.0 : -1.0; }
41
42 static unsigned int seed = 1;
43 static double noise(void)
44 {
45     seed = seed * 1103515245u + 12345u;
46     return (double)((seed >> 9) & 0xffff) / 32768.0 - 1.0;
47 }
48
49 static double envelope(double t, double attack, double decay)
50 {
51     if (t < 0.0)
52         return 0.0;
53     if (t < attack)
54         return t / attack;
55     double x = (t - attack) / decay;
56     return x < 1.0 ? pow(1.0 - x, 1.4) : 0.0;
57 }
58
59 // an amplifier runs out of headroom at some point, and tanh has exactly
that
60 // shape
61 static double drive(double x, double gain) { return tanh(x * gain) /
tanh(gain); }
62
63 // ----- the
voices
64 // a mouth is a buzz pushed through three resonances. move them apart and
the
65 // throat stops being a human one, which is what we want for the thing.
66 struct throat { struct filter a, b, c; double phase; };
67
68 static double voice(struct throat *v, double t, double pitch,
69                     double f1, double f2, double f3, double growl)
70 {
71     v->phase += pitch * (1.0 + 0.04 * sin(2 * M_PI * 5.0 * t)) /
SOUND_RATE;
72     double p = v->phase - floor(v->phase);
73     double pulse = p < 0.35 ? sin(M_PI * p / 0.35)
74                     : -0.25 * (1.0 - (p - 0.35) / 0.65);
75     double source = pulse + growl * noise() * 0.5;
76     return (bandpass(&v->a, source, f1, 0.08)
77           + bandpass(&v->b, source, f2, 0.10) * 0.7
78           + bandpass(&v->c, source, f3, 0.14) * 0.4) * 0.5;
79 }
80
81 // ----- the
state
82 #define VOICES 8
83
84 struct shot {
85     enum sfx kind;
86     double start; // when it began, in seconds since the mixer
started
87     double distance;
88     int busy;
89 };
90
91 static struct mixer {
92     snd_pcm_t *pcm;
93     pthread_t thread;
94     pthread_mutex_t lock;
```

39 La dent de scie, une droite qui monte de -1 à 1 puis retombe.

49 L'enveloppe, une montée linéaire puis une descente adoucie.

61 La saturation. La tangente hyperbolique a exactement la forme d'un ampli à bout de course.

68 La gorge, un bourdonnement dans trois résonances.

PARTIE 4 CHAPITRE 1 LE CODE

SUITE

```
95     int running;
96     int muted;
97     double clock;                // seconds since the mixer started
98     double layer[LAYER_COUNT];  // where each layer is now
99     double target[LAYER_COUNT];  // where it is going
100    struct shot shots[VOICES];
101    struct filter cab1, cab2, cabhi, bass, pad, hit, room, sfx;
102    struct filter reach_of, rail, blast;
103    struct throat mouth[2];
104    double ph_pad, ph_pad2, ph_bass, ph_riff, ph_riff2;
105    double echo[SOUND_RATE / 2];
106    int echo_at;
107 } snd;
108
109 void sound_layer(enum layer which, double target)
110 {
111     if (!snd.running || which < 0 || which >= LAYER_COUNT)
112         return;
113     pthread_mutex_lock(&snd.lock);
114     snd.target[which] = target < 0.0 ? 0.0 : (target > 1.0 ? 1.0 :
target);
115     pthread_mutex_unlock(&snd.lock);
116 }
117
118 void sound_mute(int on)
119 {
120     if (!snd.running)
121         return;
122     pthread_mutex_lock(&snd.lock);
123     snd.muted = on;
124     pthread_mutex_unlock(&snd.lock);
125 }
126
127 int sound_muted(void) { return snd.muted; }
128
129 void sound_play(enum sfx which, double distance)
130 {
131     if (!snd.running)
132         return;
133     pthread_mutex_lock(&snd.lock);
134     for (int i = 0; i < VOICES; i++)
135         if (!snd.shots[i].busy) {
136             snd.shots[i] = (struct shot){ which, snd.clock, distance, 1
};
137             break;
138         }
139     pthread_mutex_unlock(&snd.lock);
140 }
141
142 // ----- one
shots
143 static double one_shot(struct shot *s, double t)
144 {
145     switch (s->kind) {
146     case SFX_SHOT: {
147         double e1 = envelope(t, 0.0004, 0.05), e2 = envelope(t, 0.001,
0.14);
148         return bandpass(&snd.sfx, noise(), 2600, 0.5) * e1 * 1.4
+ lowpass(&snd.hit, noise(), 420 + 900 * e2, 0.6) * e2;
149     }
150     case SFX_PUNCH: {
151         double e = envelope(t, 0.004, 0.10);
152         return lowpass(&snd.sfx, noise(), 300 + 500 * e, 0.9) * e * 1.2;
153     }
154     case SFX_IMPACT: {
155         double e = envelope(t, 0.0005, 0.12);
```

PARTIE 4 CHAPITRE 1 LE CODE

SUITE

```
157         return bandpass(&snd.sfxf, noise(), 1500 + 3000 * e, 0.4) * e
158             + sin(2 * M_PI * 320 * t) * envelope(t, 0.001, 0.35) * 0.25;
159     }
160     case SFX_D00R: {
161         // this is a door that slides, not a filter that whistles. a
station
162         // door is a rail that rubs, which is a narrow steady band, a
mass
163         // that moves, which is low steady noise, and a knock at the end.
none
164         // of the three sweeps, and that is what makes them bearable.
double walked = t < 0.10 ? t / 0.10
165             : (t < 0.95 ? 1.0 : fmax(0.0, (1.15 - t) / 0.20));
166         double sfx = bandpass(&snd.rail, noise(), 1100, 0.9) * walked *
0.30
167             + lowpass(&snd.reach_of, noise(), 170, 0.5) * walked * 0.50;
168         if (t > 1.05) {
169             double e = envelope(t - 1.05, 0.001, 0.22);
170             sfx += (sin(2 * M_PI * 88 * (t - 1.05)) * 0.7
171                 + noise() * 0.3) * e * 0.8;
172         }
173     }
174     return sfx;
175 }
176 case SFX_STEP: {
177     // a step on sheet metal. without it the character slides as if
on
178     // ice, because the sound of the step is what gives him weight.
it has
179     // to stay under everything else, almost inaudible on its own.
double e = envelope(t, 0.002, 0.085);
180     return (lowpass(&snd.reach_of, noise(), 240, 0.4) * 0.8
181         + bandpass(&snd.rail, noise(), 2600, 1.4) * 0.12) * e * 0.55;
182 }
183 case SFX_BLAST: {
184     // this one is close, so we hear the crack, then the blast, then
the
185     // debris
186     double crack = envelope(t, 0.0008, 0.09);
187     double blast = envelope(t, 0.006, 1.10);
188     double debris = envelope(t, 0.12, 1.60);
189     return bandpass(&snd.rail, noise(), 2200, 0.5) * crack * 0.85
190         + lowpass(&snd.blast, noise(), 70 + 150 * blast, 0.35)
191             * blast * 1.05
192         + sin(2 * M_PI * (50 + 34 * blast) * t) * blast * 0.45
193         + bandpass(&snd.reach_of, noise(), 900, 1.3) * debris * 0.16;
194 }
195 case SFX_PICKUP: {
196     double e = envelope(t, 0.002, 0.22);
197     return (sin(2 * M_PI * 880 * t) + sin(2 * M_PI * 1320 * t) * 0.5)
* e * 0.3;
198 }
199 case SFX_SHRIEK:
200     return voice(&snd.mouth[0], t, 210 + 320 * envelope(t, 0.05,
1.4),
201         720, 1600 + 900 * t, 3200, 0.7) * envelope(t, 0.02, 1.5);
202 case SFX_GROWL:
203     return voice(&snd.mouth[1], t, 78 + 22 * sin(2 * M_PI * 0.5 * t),
420 + 120 * t, 1100, 2600, 0.35) * envelope(t, 0.25, 2.3) *
0.8;
204 case SFX_HURT: {
205     double e = envelope(t, 0.003, 0.30);
206     return (sin(2 * M_PI * 140 * t) * 0.6 + noise() * 0.4) * e * 0.7;
```

PARTIE 4 CHAPITRE 1 LE CODE

SUITE

```
209     }
210     case SFX_DIE:
211         return voice(&snd.mouth[1], t, 120 - 60 * t, 500, 900, 2200, 0.5)
212             * envelope(t, 0.02, 1.8);
213     default:
214         return 0.0;
215     }
216 }
217
218 static double shot_length(enum sfx kind)
219 {
220     switch (kind) {
221         case SFX_SHRIEK: return 1.6;
222         case SFX_GROWL:  return 2.6;
223         case SFX_DIE:    return 1.9;
224         case SFX_DOOR:   return 1.4;
225         case SFX_STEP:   return 0.16;
226         case SFX_BLAST:  return 2.4;
227         default:         return 0.6;
228     }
229 }
230
231 // ----- the
music
232 static const int RIFF[32] = {
233     0,0,0,1, 0,0,3,0, 0,0,0,1, 0,3,1,0,
234     0,0,0,1, 0,0,5,0, 0,0,0,3, 1,0,1,0,
235 };
236 // the phrygian scale, because it is the one that sounds like a threat
237 static const double SCALE[8] = { 0, 1, 3, 5, 7, 8, 10, 12 };
238 static const int KICK[16] = { 1,0,1,0, 0,1,0,0, 1,0,1,0, 0,1,0,1 };
239 static const int SNARE[16] = { 0,0,0,0, 1,0,0,0, 0,0,0,0, 1,0,0,0 };
240 static const int PULSE[8] = { 0, 0, 3, 0, 4, 0, 2, 1 };
241
242 static double music(double t)
243 {
244     int six = (int)(t / SIXTEEN);
245     double in_six = t - six * SIXTEEN;
246     double out = 0.0;
247
248     // the breath is two detuned saws, heavily filtered, and it sounds
like
249     // the station idling. we stay above 20 hz, because below that a saw
250     // becomes a flutter that wears you down. e2 is low enough to sit
under
251     // everything and high enough to be a sound.
252     if (snd.layer[LAYER_BREATH] > 0.001) {
253         double f = ROOT * 2.0;
254         snd.ph_pad += f / SOUND_RATE;
255         snd.ph_pad2 += f * 1.004 / SOUND_RATE;
256         double pad = (saw(snd.ph_pad) + saw(snd.ph_pad2)) * 0.5;
257         // the sweep is wide and slow, because a drone that never moves
is
258         // just noise
259         double cut = 210 + 150 * sin(2 * M_PI * t / 17.0);
260         out += lowpass(&snd.pad, pad, cut, 1.0)
261             * 0.16 * snd.layer[LAYER_BREATH];
262     }
263
264     // the pulse plays eighth notes, and its filter opens as it heats up
265     if (snd.layer[LAYER_PULSE] > 0.001) {
266         int step = (int)(t / (BEAT / 2.0));
267         double since = t - step * (BEAT / 2.0);
268         double f = ROOT * 2.0 * pow(2.0, SCALE[PULSE[step % 8]] / 12.0);
```

PARTIE 4 CHAPITRE 1 LE CODE

SUITE

```
269         snd.ph_bass += f / SOUND_RATE;
270         double e = envelope(since, 0.004, 0.22);
271         out += lowpass(&snd.bass, square(snd.ph_bass) * e,
272             300 + 800 * e, 0.7) * 0.5 * snd.layer[LAYER_PULSE];
273     }
274
275     // the metal layer is struck noise and a kick. it is not a drum kit,
it is
276     // a bulkhead.
277     if (snd.layer[LAYER_METAL] > 0.001) {
278         double m = 0.0;
279         if (KICK[six % 16]) {
280             double e = envelope(in_six, 0.001, 0.13);
281             m += drive(sin(2 * M_PI * (52 + 55 * e) * in_six) * e, 2.2) *
0.85;
282         }
283         if (SNARE[six % 16]) {
284             double e = envelope(in_six, 0.0008, 0.17);
285             m += bandpass(&snd.hit, noise(), 1900, 0.5) * e * 0.45;
286         }
287         double e = envelope(in_six, 0.0004, 0.035);
288         m += noise() * e * ((six % 4 == 0) ? 0.16 : 0.09);
289         out += m * snd.layer[LAYER_METAL];
290     }
291
292     // the drive layer is the riff, a rich wave that is clipped hard and
sent
293     // through a cabinet
294     if (snd.layer[LAYER_DRIVE] > 0.001) {
295         double f = ROOT * pow(2.0, SCALE[RIFF[six % 32]] / 12.0);
296         snd.ph_riff += f / SOUND_RATE;
297         snd.ph_riff2 += f * 2.0 * 1.003 / SOUND_RATE;
298         double gate = envelope(in_six, 0.0015, SIXTEEN * 0.92);
299         double pick = envelope(in_six, 0.0004, 0.008) * 0.35;
300         double amp = drive((saw(snd.ph_riff) * 0.8
301             + saw(snd.ph_riff2) * 0.45 + pick) * gate, 9.0);
302         double cab = lowpass(&snd.cab1, amp, 3800, 0.9);
303         cab -= lowpass(&snd.cab2, cab, 95, 0.9);
304         cab += bandpass(&snd.cabhi, amp, 2100, 0.6) * 0.25;
305         out += cab * 0.30 * snd.layer[LAYER_DRIVE];
306     }
307     return out;
308 }
309
310 // ----- the
mixer
311 static void fill(short *buffer, int frames)
312 {
313     pthread_mutex_lock(&snd.lock);
314     for (int i = 0; i < LAYER_COUNT; i++) {
315         // a layer slides in over about a second, because a sudden jump
sounds
316         // like a fault
317         double d = snd.target[i] - snd.layer[i];
318         double step = (double)frames / SOUND_RATE / 1.0;
319         if (fabs(d) <= step)
320             snd.layer[i] = snd.target[i];
321         else
322             snd.layer[i] += d > 0 ? step : -step;
323     }
324     for (int n = 0; n < frames; n++) {
325         double t = snd.clock + n / (double)SOUND_RATE;
326         double mix = music(t);
327         for (int i = 0; i < VOICES; i++) {
328             struct shot *s = &snd.shots[i];
329             if (!s->busy)
330                 continue;
331             double age = t - s->start;
```

295 La note du riff, la note de base multipliée par 2 à la puissance des demi-tons sur 12.

311 Le mélangeur. Il fait glisser les couches, additionne la musique et les bruitages, ajoute l'écho et écrit les échantillons.

PARTIE 4 CHAPITRE 1 LE CODE

SUITE

```
332         if (age > shot_length(s->kind)) {
333             s->busy = 0;
334             continue;
335         }
336         // distance takes the top off the sound and turns it down,
the way
337         // a room does
338         double far = 1.0 / (1.0 + s->distance * s->distance * 0.05);
339         mix += one_shot(s, age) * far;
340     }
341     double back = snd.echo[snd.echo_at];
342     snd.echo[snd.echo_at] = mix + back * 0.24;
343     snd.echo_at = (snd.echo_at + 1) % (int)(SIXTEEN * 3 *
SOUND_RATE);
344     double wet = mix + lowpass(&snd.room, back, 3000, 0.8) * 0.2;
345     wet *= SOUND_MASTER;
346     if (snd.muted)
347         wet = 0.0;
348     if (wet > 1.0) wet = 1.0;
349     if (wet < -1.0) wet = -1.0;
350     short v = (short)(wet * 30000);
351     buffer[n * 2] = v;
352     buffer[n * 2 + 1] = v;
353 }
354 snd.clock += frames / (double)SOUND_RATE;
355 pthread_mutex_unlock(&snd.lock);
356 }
357
358 // the thread receives the mixer it feeds, because that is the one thing
a
359 // thread start routine is allowed to receive
360 static void *run(void *state)
361 {
362     struct mixer *m = state;
363     short buffer[SOUND_CHUNK * 2];
364     while (m->running) {
365         fill(buffer, SOUND_CHUNK);
366         snd_pcm_sframes_t wrote = snd_pcm_wrotei(m->pcm, buffer,
SOUND_CHUNK);
367         if (wrote < 0)
368             snd_pcm_recover(m->pcm, (int)wrote, 1);
369     }
370     return NULL;
371 }
372
373 int sound_open(void)
374 {
375     memset(&snd, 0, sizeof snd);
376     // we create the lock first, before any return, because a game
started
377     // without sound still locks this mutex the first time a sound is
asked
378     // for.
379     pthread_mutex_init(&snd.lock, NULL);
380     // a test run must never make noise on the machine, so a game
launched for
381     // a screenshot stays silent
382     if (getenv("TEC_SILENT"))
383         return 0;
384     if (snd_pcm_open(&snd.pcm, "default", SND_PCM_STREAM_PLAYBACK, 0) <
0)
385         return 0;
386     if (snd_pcm_set_params(snd.pcm, SND_PCM_FORMAT_S16_LE,
SND_PCM_ACCESS_RW_INTERLEAVED, 2, SOUND_RATE, 1, 120000) < 0)
387     {
388         snd_pcm_close(snd.pcm);
389         snd.pcm = NULL;
```

338 L'atténuation d'un
bruitage avec la distance.

PARTIE 4 CHAPITRE 1 LE CODE

SUITE

```
390         return 0;
391     }
392     snd.running = 1;
393     if (pthread_create(&snd.thread, NULL, run, &snd) != 0) {
394         snd.running = 0;
395         snd_pcm_close(snd.pcm);
396         snd.pcm = NULL;
397         return 0;
398     }
399     return 1;
400 }
401
402 void sound_close(void)
403 {
404     if (!snd.running)
405         return;
406     snd.running = 0;
407     pthread_join(snd.thread, NULL);
408     if (snd.pcm) {
409         snd_pcm_drop(snd.pcm);
410         snd_pcm_close(snd.pcm);
411     }
412 }
```

393 Le fil d'exécution du son, qui alimente la carte sans jamais bloquer le jeu.

Le rectangle autour d'une planche

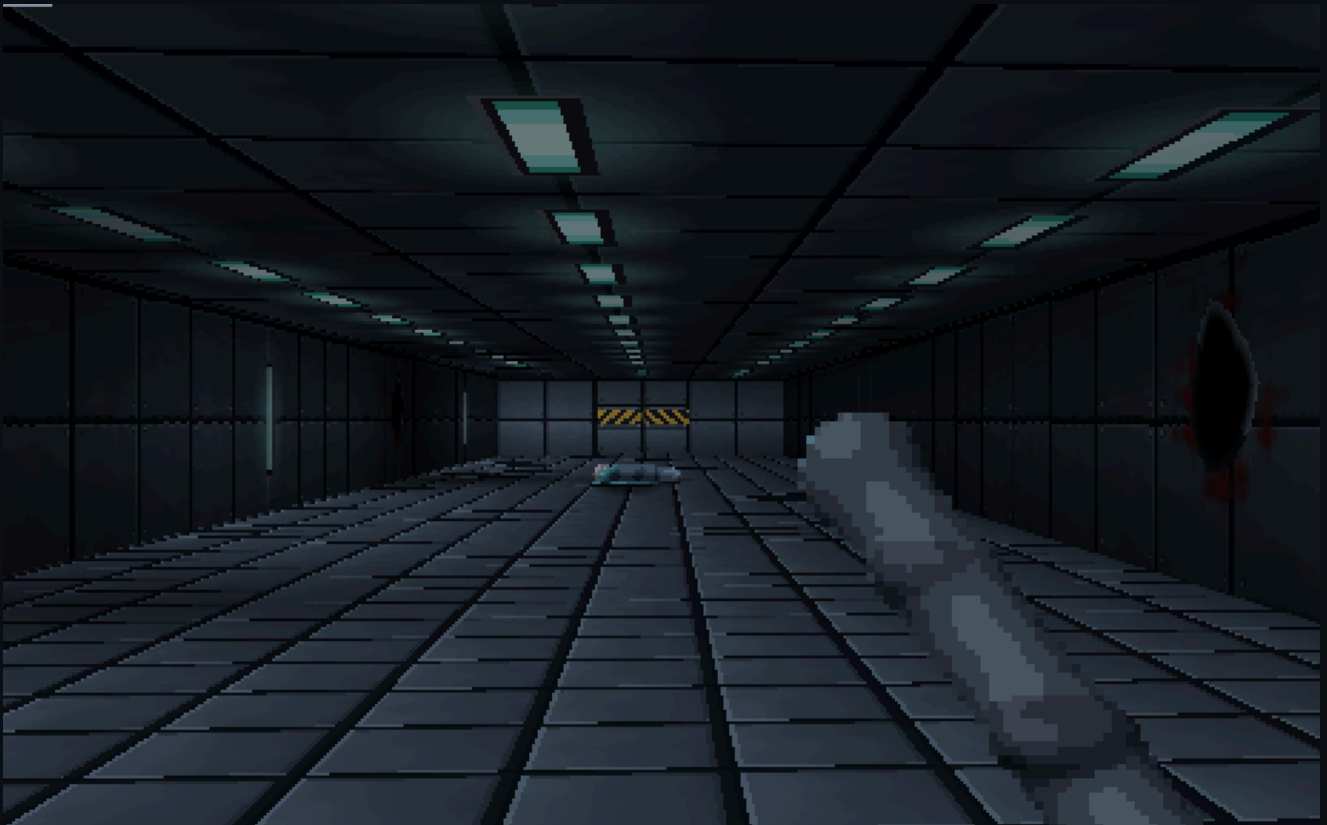


Figure 2.1. Un survivant allongé au sol, dessiné avec la planche de l'équipage. Ses pixels transparents sont ignorés par la fonction `sprite_solid`.

Dans ce chapitre, vous allez ranger le test d'opacité des planches dans une fonction.

Un court rangement avant la suite. Le test qui décide si un pixel d'une planche est opaque, l'octet du haut supérieur ou égal à 128, était écrit à trois endroits. Il devient la fonction `sprite_solid`, avec deux constantes nommées, le décalage de l'opacité et le seuil. Une fonction `sprite_free` libère la mémoire d'une planche.

Ranger un test dans une fonction n'est pas seulement une question de propreté. Le jour où le seuil change, il ne change qu'à un seul endroit, et le nom de la fonction dit ce que le test signifie.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE SPRITE H LIGNES 17 A 28

```
17
18 int sprite_load(struct sprite *s, const char *path);
19 void sprite_free(struct sprite *s);
20 // gives one pixel of one frame, or 0 when it is transparent or outside
  the
21 // box
22 unsigned int sprite_at(const struct sprite *s, int frame, int x, int y);
23 // a pixel is drawn when its alpha, which is the top byte, is at least
  half
24 #define ALPHA_SHIFT 24
25 #define ALPHA_OPAQUE 128
26 int sprite_solid(unsigned int pixel);
27
28 #endif
```

SRC SPRITE C LIGNES 44 A 49

```
44     for (int y = 0; y < s->height; y++)
45         for (int x = 0; x < s->width; x++)
46             if (sprite_solid(s->pixels[((size_t)n * s->height + y)
47                                     * s->width + x])) {
48                 if (x < s->x0) s->x0 = x;
49                 if (y < s->y0) s->y0 = y;
```

SRC SPRITE C LIGNES 59 A 68

```
59 }
60
61 void sprite_free(struct sprite *s)
62 {
63     free(s->pixels);
64     memset(s, 0, sizeof *s);
65 }
66
67 unsigned int sprite_at(const struct sprite *s, int frame, int x, int y)
68 {
```

SRC SPRITE C LIGNES 72 A 78

```
72     return s->pixels[((size_t)frame * s->height + y) * s->width + x];
73 }
74
75 int sprite_solid(unsigned int pixel)
76 {
77     return (pixel >> ALPHA_SHIFT) >= ALPHA_OPAQUE;
78 }
```

Ligne par ligne

- 25** Le seuil d'opacité, la moitié de 255.
- 77** Le test, écrit une seule fois.

La tôle arrachée

LA TOLE ARRACHEE



LA CLOISON DU FOND



Figure 3.1. La plaque arrachée du sas et la cloison épaisse de la dernière salle, telles que le programme les dessine.

Dans ce chapitre, vous allez dessiner une plaque arrachée et ce qui a giclé dessus, en retirant de la matière à une plaque ordinaire.

Le sas ne dit pas qu'il s'est passé quelque chose avec un texte. Il le dit avec ses murs. Une nouvelle texture montre une plaque arrachée, le noir derrière elle, un bord de métal replié qui accroche la lumière, et ce qui a giclé dessus. C'est la première chose que le joueur voit.

La fonction `make_torn` part de la plaque ordinaire et lui retire de la matière. Le métal se déchire en lignes droites, pas en cercles. Sept rayons de longueur aléatoire, tirés par la fonction `noise`, sont reliés par des segments, et tout ce qui est à l'intérieur de ce contour est un trou. Le bord replié est clair du côté de la lumière et sombre de l'autre. Les éclaboussures partent du trou et coulent vers le bas, parce que le sang coule, avec deux bruits lissés, l'un pour la forme et l'autre pour les coulures.

Une neuvième texture, la cloison épaisse de la dernière salle, presque noire, prépare la fin. Le joueur doit reconnaître la dernière salle à son mur avant de savoir pourquoi.

La tôle arrachée, suite

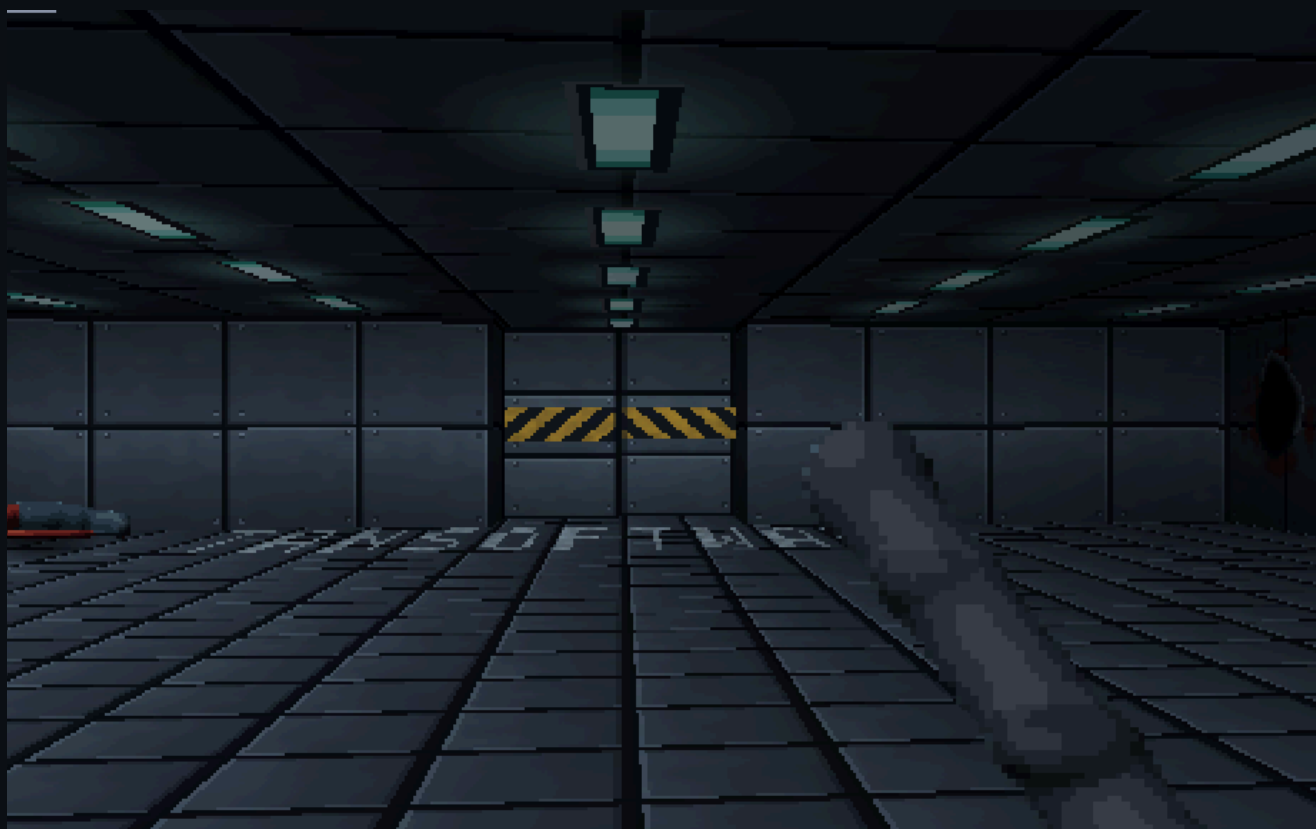


Figure 3.2. Le sas, avec la tôle arrachée sur le mur de gauche. Le joueur se réveille devant elle.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE TEXTURE H LIGNES 78 A 107

```

78 // several kinds of wall, so one room is not the next: a panel size and
   four
79 // colours, still drawn in code
80 #define WALL_KINDS 9
81 #define DOOR_TEXTURE 3           // which of the nine is the door
82 // three colours of strip, one per zone: the gangway in service, the
   machine
83 // room, and the hold on its emergency lighting
84 #define LAMP_TEXTURE 4           // the ordinary strip, teal
85 #define ALARM_TEXTURE 5          // the emergency one, amber
86 #define COLD_TEXTURE 6           // and the cold white of the machines
87 // the torn plating. the airlock does not say that something happened
   with a
88 // text, it says it with its walls, a plate ripped off, the dark behind
   it,
89 // and what splashed on it. it is the first thing the player sees.
90 #define TORN_TEXTURE 7
91 #define VAULT_TEXTURE 8           // the thick bulkhead of the last room
92 #define BLOOD_DARK 0x3a0f0e
93 #define BLOOD_WET 0x6e1c16
94 #define TORN_EDGE 0x8f9aa8       // the metal curled up along the tear
95
96 // the name stencilled on the airlock floor: four squares, three letters
97 // each, every letter turned a quarter turn to read the way one walks
98 #define STENCIL "TANSOFTWARE"
99 // three tiles, for a geometric reason. with four, the middle of the band
100 // falls half a cell off the player's line and the name looks shifted to
   the
101 // left. with three, the middle falls exactly on the waking cell, and the
   word
102 // is centred in the frame.
103 #define STENCIL_CELLS 3
104 #define STENCIL_EACH 4           // how many letters to a square
105 #define STENCIL_PAD 7           // the margin around a letter, in pixels
106 #define STENCIL_PAINT 0xb9c6cf
107 #define STENCIL_WEAR 0.72       // the paint is worn, not fresh

```

SRC TEXTURE C LIGNES 16 A 20

```

16 double noise(int x, int y)
17 {
18     unsigned int n = (unsigned int)x * 374761393u + (unsigned int)y *
668265263u;
19     n = (n ^ (n >> 13)) * 1274126177u;
20     return (double)((n >> 16) & 0xffff) / 65535.0;

```

SRC TEXTURE C LIGNES 187 A 196

```

187     .seam = 0x090b10, .across = 4, .down = 4, .rivets = 0,
188 };
189 // the bulkhead of the last room is thick, almost black, and it gives
   nothing
190 // back. you have to recognise the last room by its wall before you know
   why.
191 static const struct plate VAULT = {
192     .color = 0x2a2b33, .light = 0x5d5f6e, .dark = 0x0a0a0e,
193     .seam = 0x050507, .across = 1, .down = 3, .rivets = 1,
194 };

```

PARTIE 4 CHAPITRE 3 LE CODE

SUITE

```
195 static const struct plate R00F = {
196     .color = 0x232833, .light = 0x525d6d, .dark = 0x080a0e,
SRC TEXTURE C LIGNES 281 A 331
281 }
282
283 // a plate ripped off, and what splashed on it. the tear is a hole in the
284 // plating, with a curled edge that catches the light, the dark behind
    it, and
285 // splashes that run downwards because blood runs. none of it is a decal,
    it
286 // is the same plate as everywhere else with material taken away.
287 static void make_torn(unsigned int *out, const struct plate *p)
288 {
289     make_plating(out, p);
290     for (int y = 0; y < TEX_SIZE; y++)
291         for (int x = 0; x < TEX_SIZE; x++) {
292             // metal tears in straight lines, not in circles. a circle
293             // softened with noise gives an ink stain, while torn plating
294             // leaves facets and angles, so we draw seven spokes at
random and
295             // join them with segments, and that is all.
296             double dx = x - 74.0, dy = y - 52.0;
297             double d = sqrt(dx * dx + dy * dy);
298             double a = (atan2(dy, dx) + 3.14159265) / 6.2831853 * 7.0;
299             int whole = (int)a;
300             double f = a - whole;
301             double r0 = 18.0 + 16.0 * noise(whole, 3);
302             double r1 = 18.0 + 16.0 * noise((whole + 1) % 7, 3);
303             double border = r0 + (r1 - r0) * f;
304             if (d < border) {
305                 // the curled edge is light on the lit side and dark on
the
306                 // other, like a real fold in sheet metal
307                 double fold = (border - d) / 7.0;
308                 double toward_light = (-dx - dy) / (d + 1.0);
309                 out[y * TEX_SIZE + x] = fold < 1.0
                    ? mix(toward_light > 0.0 ? TORN_EDGE : 0x1b2027,
310                        0x05070a, fold)
311                        : 0x05070a;
312                 continue;
313             }
314             double r = d;
315             // the splashes start at the hole and run down
316             double s = smooth_noise(x / 2, y, 7);
317             double reach = (r - 26.0) / 42.0;
318             double run_down = smooth_noise(x, y / 6, 5);
319             double stain = (1.0 - reach) * (s * 0.7 + run_down * 0.6) -
320             0.42;
321             if (reach < 1.0 && stain > 0.0) {
322                 unsigned int blood = mix(BLOOD_DARK, BLOOD_WET,
323                     fmin(1.0, stain * 2.4));
324                 out[y * TEX_SIZE + x] = mix(out[y * TEX_SIZE + x],
325                     blood, fmin(0.92, stain * 3.0));
326             }
327         }
328 }
329
330 void make_wall_textures(void)
331 {
```

SRC TEXTURE C LIGNES 337 A 342

```
337     make_lamp_wall(wall_texture[5], &BULK, ALARM);
338     make_lamp_wall(wall_texture[6], &PARTITION, COLD);
339     make_torn(wall_texture[7], &HULL);
340     make_plating(wall_texture[8], &VAULT);
```

298 L'angle autour du centre du trou, ramené entre 0 et 7, pour choisir le rayon.

303 Le contour du trou entre deux rayons, un segment de droite.

308 Le côté du bord replié qui fait face à la lumière, en haut à gauche.

320 Les éclaboussures, plus fortes près du trou, avec des coulures verticales.

PARTIE 4 CHAPITRE 3 LE CODE

SUITE

```
341 }  
342
```

Trois sortes de portes



Figure 4.1. La porte bloquée, ouverte au tiers. À travers la fente, un combat que le joueur ne peut pas rejoindre.

Dans ce chapitre, vous allez écrire trois sortes de portes, et faire répondre une porte à celui qui la pousse.

Le niveau a maintenant trois sortes de portes, et deux d'entre elles disent quelque chose. Le signe plus est la porte ordinaire, elle s'ouvre. La lettre L est une porte scellée, qui reste fermée tant que le système des portes tient. La lettre J est une porte bloquée, qui s'ouvre au tiers et se coince, si bien qu'on voit à travers sans passer. La lettre D est la porte du coffre, qui ne s'ouvre qu'avec le badge.

La fonction `push_door` renvoie maintenant ce qui s'est passé, un type énuméré `push`, et la boucle principale choisit le message. Une porte bloquée a une limite d'ouverture, donnée par `door_limit`, et la fonction `move_doors` s'arrête à cette limite. Quand le système des portes cède, la fonction `world_release_seals` ouvre toutes les portes scellées du pont en même temps. Le joueur n'obtient pas une permission, il entend le pont s'ouvrir.

Le nom du studio au sol passe de quatre cases à trois, pour une raison géométrique. Avec quatre cases, le milieu du mot tombait une demi-case à côté de la ligne du joueur, et le mot paraissait décalé. Avec trois, il est centré sur la case où il se réveille.

VOCABULAIRE

Un **type énuméré** est une liste de valeurs nommées. Le type `push` nomme les cinq réponses possibles d'une porte.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE WORLD_H LIGNES 1 A 7

```
1  #ifndef WORLD_H
2
3  // how long the hatch takes to open, which is how long the corridor has
   to be
4  // held
5  #define HATCH_SECONDS 20.0
6  #define WORLD_H
7
```

INCLUDE WORLD_H LIGNES 28 A 35

```
28 int wall_kind(int x, int y);
29 int is_lamp(int x, int y);
30 // where the name is painted on the airlock floor. it takes four floor
   cells,
31 // from (7,4) to (7,7), right in front of the player, who wakes up at
   (6,6).
32 #define STENCIL_X 9
33 #define STENCIL_Y 5
34 int stencil_at(int x, int y);
35 int is_badge(int x, int y);
```

INCLUDE WORLD_H LIGNES 39 A 45

```
39
40 int is_seen(int x, int y);
41 // how many cells have been seen so far. the map stays off screen while
   it has
42 // nothing to show.
43 int seen_count(void);
44 void remember(const struct player *player);
45 // and this one no longer holds: it flickers
```

INCLUDE WORLD_H LIGNES 59 A 86

```
59 #define ENDING_DARK    0.82
60
61 // three kinds of door, and two of them tell you something. "+" simply
   opens.
62 // "L" stays sealed while the door system holds, and it is the system
   that
63 // will give. "J" is jammed, so it opens a third of the way and sticks,
   and
64 // you see through it without getting through.
65 #define DOOR_JAMMED_MAX 0.46
66
67 enum push { PUSH_NOTHING, PUSH_OPENS, PUSH_JAMMED, PUSH_SEALED,
   PUSH_LOCKED };
68
69 // the door system gives way, and everything that was sealed can open
70 void world_release_seals(void);
71 int  world_seals_released(void);
72 // what lies on the floor to be picked up, and what is left of it
73 enum loot { LOOT_NONE, LOOT_GUN, LOOT_AMMO, LOOT_MED, LOOT_BADGE };
74 enum loot loot_at(int x, int y);
75 enum loot loot_take(const struct player *player);
76 // where the thing at the bottom stands, in the last room
77 int is_door(int x, int y);
78 // how far this door is able to open
79 double door_limit(int x, int y);
80 // from 0 (shut) to 1 (both leaves tucked into the wall)
81 double door_at(int x, int y);
```

Ligne par ligne

- 65** La porte bloquée s'ouvre jusqu'à 0,46, moins que les 0,8 nécessaires pour passer.
- 67** Les cinq réponses d'une porte poussée.

PARTIE 4 CHAPITRE 4 LE CODE

SUITE

```
82 // is there a shut door in front of us, within reach?
83 int door_ahead(const struct player *player);
84 enum push push_door(const struct player *player);
85 void move_doors(double elapsed);
86 void move_player(struct player *player, double step_x, double step_y);
```

SRC WORLD C LIGNES 25 A 47

```
25 // walk back over it and help yourself forever.
26 static char *taken;
27
28 // whether the door system has given way
29 static int seals_released;
30
31 // when the system gives way, the doors do not offer anything, they open.
32 // while it holds, a sealed door says nothing and does not answer, and
  when it
33 // gives, every sealed door on the deck slides open by itself. the player
  is
34 // not granted a permission, he hears the deck open.
35 void world_release_seals(void)
36 {
37     seals_released = 1;
38     if (!doors)
39         return;
40     for (int y = 0; y < map_height; y++)
41         for (int x = 0; x < (int)strlen(map[y]); x++)
42             if (map[y][x] == 'L' && doors[y * map_width + x] == 0.0)
43                 doors[y * map_width + x] = 0.001;
44 }
45 int world_seals_released(void) { return seals_released; }
46
47 // what the level file can hold. a wall is anything that is not floor.
```

SRC WORLD C LIGNES 109 A 113

```
109     if (!doors || !seen || !taken)
110         return 0;
111     seals_released = 0;
112 }
113 return map_height > 0;
```

SRC WORLD C LIGNES 179 A 211

```
179 {
180     int x, y;
181     if (!door_in_front(player, &x, &y))
182         return 0;
183     return !(map[y][x] == 'L' && !seals_released);
184 }
185
186 // we nudge the door once, and after that the two leaves finish their
  travel
187 // on their own. the door also answers, because what it does is what the
188 // player learns.
189 enum push push_door(const struct player *player)
190 {
191     int x, y;
192     if (!door_in_front(player, &x, &y))
193         return PUSH_NOTHING;
194     char c = map[y][x];
195     if (c == 'L' && !seals_released)
196         return PUSH_SEALED;
197     if (c == 'D' && !have_badge())
198         return PUSH_LOCKED;
199     if (doors[y * map_width + x] > 0.0)
200         return PUSH_NOTHING;
201     doors[y * map_width + x] = 0.001;
202     return c == 'J' ? PUSH_JAMMED : PUSH_OPENS;
```

35 Le système cède. Toutes les portes scellées se mettent à s'ouvrir.

202 La porte bloquée répond qu'elle est bloquée, la porte ordinaire qu'elle s'ouvre.

PARTIE 4 CHAPITRE 4 LE CODE

SUITE

```
203 }
204
205 double door_limit(int x, int y)
206 {
207     if (x < 0 || y < 0 || x >= map_width || y >= map_height)
208         return 1.0;
209     return map[y][x] == 'J' ? DOOR_JAMMED_MAX : 1.0;
210 }
211
```

219 Chaque porte s'ouvre jusqu'à sa propre limite.

SRC WORLD C LIGNES 216 A 237

```
216     if (!doors)
217         return;
218     for (int i = 0; i < map_width * map_height; i++) {
219         double max = door_limit(i % map_width, i / map_width);
220         if (doors[i] > 0.0 && doors[i] < max) {
221             doors[i] += elapsed / DOOR_SECONDS;
222             if (doors[i] > max)
223                 doors[i] = max;
224         }
225     }
226 }
227
228 int seen_count(void)
229 {
230     int n = 0;
231     for (int y = 0; y < map_height; y++)
232         for (int x = 0; x < map_width; x++)
233             if (is_seen(x, y))
234                 n++;
235     return n;
236 }
237
```

SRC WORLD C LIGNES 284 A 289

```
284     if (x < 0 || y < 0 || x >= map_width || y >= map_height)
285         return 0;
286     char c = map[y][x];
287     return c == '+' || c == 'J' || c == 'L' || c == 'D';
288 }
289
```

SRC WORLD C LIGNES 303 A 307

```
303 enum loot loot_at(int x, int y)
304 {
305     if (x < 0 || y < 0 || x >= map_width || y >= map_height || (taken &&
306         taken[y * map_width + x]))
307         return LOOT_NONE;
308     switch (map[y][x]) {
309
```

SRC WORLD C LIGNES 373 A 385

```
373         // starts, and where the way out is. only a wall stops anyone.
374         char c = map[y][x];
375         // every door can be walked through, not only the ordinary one,
376         // because a sealed door that opens and stays a wall would end the level
377         // without a word. the jammed one never opens far enough to pass, and it is its
378         // limit that forbids it, not an exception written here.
379         if (is_door(x, y))
380             return door_at(x, y) < DOOR_WALKABLE;
381         return !(c == '.' || c == 'S' || c == 'E' || c == 'K' || c == 'G'
382             || c == 'A' || c == 'M' || c == 'x' || c == 'y' || c == 'w'
383             || c == 'Z' || c == 'X' || c == 'n' || c == 'h');
384     }
385
```

La carte est un radar



Figure 5.1. Le radar dans le coin, centré sur le joueur, en bas de la porte.

Dans ce chapitre, vous allez transformer la carte en radar centré sur le joueur.

La carte montrait tout le niveau, dans un cadre fixe. Un niveau de 56 cases sur 42 en faisait un grand rectangle qui mangeait l'image. La carte devient un radar centré sur le joueur, qui montre 11 cases de chaque côté et 8 vers le haut et le bas. Le joueur est toujours au centre, et la carte glisse sous lui quand il avance.

Le changement tient dans la fonction `render_map`. Les coordonnées de la case dessinée sont maintenant relatives à la case du joueur, et le point du joueur est dessiné au centre du cadre. Le reste, le cadre, les cases vues et les portes en ambre, ne change pas.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE RENDER H LIGNES 28 A 33

```
28 #define HUD_DIVISOR 640
29 #define MAP_CELL 3
30 // how many cells the map window shows on each side of the player
31 #define MAP_WINDOW 11
32 #define MAP_LEFT 4
33 #define MAP_TOP 4
```

SRC RENDER C LIGNES 1 A 4

```
1 #include <stdlib.h>
2
3 #include "render.h"
4 #include "light.h"
```

SRC RENDER C LIGNES 257 A 261

```
257 static int columns;
258 if (columns != view_width) {
259     double *bigger = realloc(wall_depth, (size_t)view_width *
sizeof(*wall_depth));
260     if (bigger) {
261         wall_depth = bigger;
```

SRC RENDER C LIGNES 331 A 338

```
331 void render_map(const struct player *player, int cell, int left, int top)
332 {
333     int half_x = MAP_WINDOW, half_y = MAP_WINDOW * 3 / 4;
334     int l = (2 * half_x + 1) * cell, h = (2 * half_y + 1) * cell;
335     int cx = (int)player->x, cy = (int)player->y;
336
337     for (int i = -MAP_EDGE; i < l + MAP_EDGE; i++)
338         for (int e = 1; e <= MAP_EDGE; e++) {
```

SRC RENDER C LIGNES 352 A 358

```
352         blend(left + i, top + j, MAP_UNSEEN, MAP_ALPHA);
353
354     for (int dy = -half_y; dy <= half_y; dy++)
355         for (int dx = -half_x; dx <= half_x; dx++) {
356             int x = cx + dx, y = cy + dy;
357             if (!is_seen(x, y))
358                 continue;
```

SRC RENDER C LIGNES 361 A 372

```
361         for (int line = 0; line < cell; line++)
362             for (int column = 0; column < cell; column++)
363                 blend(left + (dx + half_x) * cell + column,
top + (dy + half_y) * cell + line,
365                     color, MAP_ALPHA);
366     }
367
368     // where we stand, and which way we look
369     int dot_x = left + half_x * cell + cell / 2;
370     int dot_y = top + half_y * cell + cell / 2;
371     for (int step = MAP_DOT + 1; step < MAP_ARROW * cell; step++)
372         put_pixel(dot_x + (int)(player->dir_x * step),
```

Ligne par ligne

- 31** Le nombre de cases montrées de chaque côté du joueur.
- 335** La case du joueur, au centre du radar.
- 369** Le point du joueur, toujours au centre du cadre.

L'histoire, les bandes noires et le journal

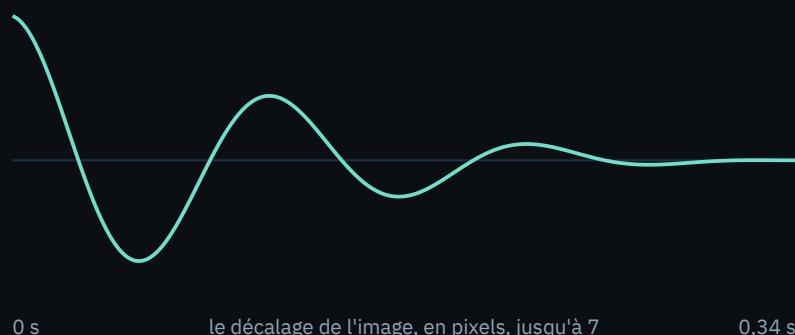


Figure 6.1. Le décalage de l'image pendant une secousse de 0,34 seconde, calculé par la fonction `offset_of`. L'amplitude, 7 pixels au plus, s'éteint avec le carré du temps restant, et le cosinus fait vibrer l'image 55 fois par seconde.

Dans ce chapitre, vous allez écrire l'histoire du jeu avec des bandes noires, des secousses, une alarme et un journal, sans jamais prendre les commandes au joueur.

L'ouverture du jeu n'est pas une scène qui prend les commandes. La station tremble parce qu'elle est frappée, le journal défile parce qu'il s'écrit, et la couleur change parce que l'alarme est allumée. Tout se passe pendant que le joueur marche. Le nouveau fichier `story.c` tient ces effets, et il ne dessine rien lui-même dans le monde. Il donne des nombres, et l'affichage les applique.

Deux bandes noires en haut et en bas de l'image disent au joueur où il en est. Tant qu'elles sont là, il regarde. Quand elles partent, il joue. Aucun panneau n'a besoin de dire qu'une scène commence, le cadre le dit, comme au cinéma. Cinq lignes de texte s'affichent au milieu, une idée par ligne, et la dernière le fait marcher. La touche Espace saute le film. Les jambes sont rendues quand les bandes commencent à partir, pas quand elles sont parties, et ce chevauchement donne l'impression que le jeu prend le relais du film.

Une secousse est un accent, jamais un état. Elle est courte, elle décroît, et elle suit un seul axe. La fonction `story_hit` déclenche un grondement puis, une demi-seconde plus tard, la secousse, parce que cet ordre transforme un sursaut en impact. La figure 6.1 montre le décalage de l'image pendant une secousse, un cosinus rapide dont l'amplitude s'éteint avec le carré du temps restant. Deux secousses au début, et c'est tout. Répétée, une secousse devient insupportable au bout de vingt secondes.

L'alarme est une source de lumière, pas un filtre. Elle éclaire le tiers supérieur de l'image en rouge, avec une montée rapide et une descente lente, et laisse le reste tranquille. La poussière est dans le monde et non sur la vitre. Chaque grain a une position au sol autour du point d'impact et une hauteur, il est projeté comme une créature, et il reste où il est tombé quand le joueur se retourne. La mort devient un écran entier. Le monde se fige, perd sa couleur, vire au rouge, les bandes reviennent, et ce qu'il faut faire est écrit en grand.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE STORY_H LIGNES 1 A 85

```

1  #ifndef STORY_H
2  #define STORY_H
3
4  #include "world.h"
5
6  // the opening is not a cutscene, and nothing is taken away from the
   player.
7  // the station shakes because it is being hit, the log scrolls because it
   is
8  // being written, and the colour changes because the alarm is on. all of
   it
9  // runs while he walks.
10
11 // how long the sequence in the airlock lasts before the room goes quiet
12 #define STORY_OPENING 26.0
13 // how far the picture gets thrown at the worst of a hit, in view pixels
14 #define SHAKE_MAX 7
15 // the log is written one character at a time, at this speed
16 #define LOG_SPEED 34.0
17 #define LOG_LINES 6
18 #define LOG_RIGHT 4
19 #define LOG_TOP 4
20
21 // ----- the film
22 // two black bars, and the player knows at once where he stands. while
   they
23 // are on screen, he watches; when they leave, he plays. no panel has to
   say
24 // "cutscene": the frame says it, the way films always have.
25 #define BARS_MAX 0.155 // how much of the view one bar takes
26 #define BARS_IN 0.9 // how long they take to come down
27 #define BARS_OUT 1.2 // and to leave, a little slower
28 #define FILM_LINE 3.2 // how long one line stays on screen
29
30 void story_begin(double now);
31 // how tall a bar is, as a share of the view. it is 0 while we are
   playing.
32 double storyBars(void);
33 // tells whether the player can walk, or whether he is only watching
34 int story_control(void);
35 // how fast the camera drifts on its own during the opening, in radians
   per
36 // second
37 double story_drift(void);
38 // pressing space during the film skips it
39 void story_skip(double now);
40 // at the end the bars come back and the game closes on its own. the
   closing
41 // lines also say how many he put down.
42 void story_score(int amount);
43 void story_finish(double now);
44 // when we start over, the opening film does not play again, because it
   has
45 // already been seen
46 void story_restart(double now);

```

Ligne par ligne

- 14** Le décalage maximal de l'image pendant une secousse, en pixels.
- 25** Chaque bande noire prend 15,5 pour cent de la hauteur.

PARTIE 4 CHAPITRE 6 LE CODE

SUITE

```
47 // death is a whole screen, not a line at the bottom. the world freezes,
    loses
48 // its colour and turns red, the bars come back, and what to do next is
49 // written in large letters. without that the player does not even know
    that
50 // he lost.
51 void story_death(double now);
52 int story_dead(void);
53 void story_draw_death(void);
54 int story_finished(void);
55 void story_draw_film(void);
56 // advances the film and the log. when the player opens a door, the alarm
    ends
57 // sooner.
58 void story_update(double now, int doors_opened);
59 // the clock is handed over once per frame, so that nothing else has to
    carry
60 // it around
61 void story_tick(double now);
62 // how far the picture is shifted this frame, in view pixels. axis 0 is
    across
63 // and axis 1 is up and down.
64 int story_shake(int axis);
65 // the alarm lights the top of the view and the edges. x and y say which
    pixel
66 // we are colouring.
67 unsigned int story_grade_at(unsigned int color, int y);
68 // where the hit landed. the dust rises at that spot, not in front of the
69 // camera.
70 void story_shake_here(double x, double y);
71 // something hits the station now, at this spot. we play a rumble half a
72 // second before and then a shake that fades, because that order is what
    makes
73 // it feel like an impact.
74 void story_hit(double now, double force, double x, double y);
75 // the blast puts a warm glow over everything, and then it falls back
76 void story_blast(double now);
77 void story_draw_blast(void);
78 void story_draw_dust(const struct player *player, double now);
79 // the game tells us whether a notice is already on screen, and the log
    stays
80 // quiet while it is
81 void story_notice(int visible);
82 void story_draw_log(void);
83 int story_opening_over(void);
84
85 #endif
```

INCLUDE TEXT H LIGNES 13 A 21

```
13 #define GLYPH_W    5
14 #define GLYPH_H    7
15
16 // draws the log in the corner. the line being written is lit and the
    lines
17 // already read are not, and nothing else separates them.
18 #define LOG_FRESH  0x9fe8d8
19 #define LOG_OLD    0x3d5163
20 #define TEXT_SCALE 2 // one font pixel is this many view pixels
21 #define TEXT_COLOR 0xe8e4da
```

SRC STORY C LIGNES 1 A 494

```
1 #include <math.h>
2 #include <stdio.h>
3 #include <string.h>
```


PARTIE 4 CHAPITRE 6 LE CODE

SUITE

```
4
5 #include "story.h"
6 #include "render.h"
7 #include "render.h"
8 #include "screen.h"
9 #include "sound.h"
10 #include "text.h"
11
12 // ----- the
rules
13 // a shake is an accent, never a state: short, decaying, along one axis.
text
14 // is read where the eye already is, low and centre, one line at a time.
an
15 // alarm is a source, not a filter: it lights the edges, the rest keeps
its colour.
16
17 // the log tells what happens, it does not explain. what to do is said by
the
18 // situation, never by the ticker.
19 static const char *LOG[] = {
20     "KEEP STATION DECK 3",
21     "HULL BREACH SECTIONS 7 12 14",
22     "CREW CHANNEL NO REPLY",
23     "QUARANTINE ENFORCED",
24     "EVACUATION DENIED",
25 };
26 #define LOG_COUNT ((int)(sizeof LOG / sizeof *LOG))
27
28 // ----- the film
29 // what the bars are for: they open a space where the game is allowed to
30 // speak, and the player knows without being told that he does not have
the
31 // keys meanwhile. five lines, one idea each, and the last one sets him
walking.
32 static const char *OPENING[] = {
33     "KEEP STATION DECK THREE",
34     "THE CREW STOPPED ANSWERING AT 0400",
35     "QUARANTINE CAME DOWN AN HOUR LATER",
36     "YOU WOKE UP ALONE IN THE AIRLOCK",
37     "SOMETHING ELSE WOKE UP TOO",
38 };
39 #define OPENING_COUNT ((int)(sizeof OPENING / sizeof *OPENING))
40
41 // the second line counts what was done. closing lines that say the same
thing
42 // to everyone reward nobody, so this one gives the player the number of
his
43 // own game, which is the only trace that will be left of it.
44 static char tally[48] = "NOTHING ELSE IS BREATHING DOWN HERE";
45 static const char *CLOSING[] = {
46     "DECK THREE IS QUIET",
47     tally,
48     "TANSOFTWARE",
49 };
50 #define CLOSING_COUNT ((int)(sizeof CLOSING / sizeof *CLOSING))
51
52 enum film { FILM_IN, FILM_TEXT, FILM_OUT, FILM_PLAY,
53     FILM_END_IN, FILM_END, FILM_DONE, FILM_DEAD };
54 static enum film film;
55 static double film_at;
56
57 static double started;
58 static int line_shown = -1;
59 static double line_at; // when the current line appeared
60 static double alarm; // how much the beacon is on, 0 to 1
```

32 Les cinq lignes du film d'ouverture.

52 Les états du film, de l'entrée des bandes à leur retour à la fin.

PARTIE 4 CHAPITRE 6 LE CODE

SUITE

```
61 static int quiet;                // the opening is over
62
63 // one hit at a time. it has a direction and a start, and it dies out on
  its
64 // own.
65 static double hit_at;             // when the shake itself starts
66 static double hit_len = 0.34;    // and how long it lasts
67 static double rumble_at;         // when the warning sound was played
68 static double hit_dx, hit_dy;
69 static int hits_done;
70 static double next_hit;
71 static double dust_until;
72 static double dust_x, dust_y;
73
74 // the caller does not carry the clock, so we keep the last one we were
  given
75 static double last_now;
76
77 void story_begin(double now)
78 {
79     started = now;
80     line_at = now;
81     line_shown = -1;              // the log waits for the end of the
  film
82     film = FILM_IN;
83     film_at = now;
84     // two hits at the start, and that is all. a shake is an accent of
  the
85     // opening, and if you repeat it, it becomes a state that is
  unbearable
86     // after twenty seconds. the player has to walk in peace ten seconds
  after
87     // being scared.
88     next_hit = now + 2.0;
89 }
90
91 void story_shake_here(double x, double y) { dust_x = x; dust_y = y; }
92
93 void story_hit(double now, double force, double x, double y)
94 {
95     rumble_at = now;
96     // a shake makes no sound of its own. what makes a sound is what
  happens,
97     // a blast or a shot, and the shake is only its consequence on
  screen. it
98     // lands at once when the cause is close, and the half second of
  warning
99     // is only for a distant hit.
100     hit_at = now + (force > 1.5 ? 0.0 : 0.5);
101     // a big shake lasts longer, it is not only wider, because a blast
  that
102     // stops after a third of a second looks like a hiccup, not like a
  blast
103     hit_len = 0.34 + 0.55 * fmax(0.0, force - 1.0);
104     double angle = now * 1.7;
105     hit_dx = cos(angle) * force;
106     hit_dy = sin(angle) * 0.6 * force;
107     dust_x = x;
108     dust_y = y;
109     dust_until = now + 1.4;
110 }
111
112 void story_update(double now, int doors_opened)
113 {
114     double t = now - started;
115
116     if (line_shown < 0 && film >= FILM_OUT) {
117         line_shown = 0;
```

100 La secousse vient une demi-seconde après le grondement, sauf pour un coup tout proche.

PARTIE 4 CHAPITRE 6 LE CODE

SUITE

```
118         line_at = now;
119     }
120     // we show one line at a time, and it stays long enough to be read
121     if (line_shown >= 0 && line_shown < LOG_COUNT - 1
122         && now - line_at > 5.0) {
123         line_shown++;
124         line_at = now;
125     }
126
127     if (!quiet) {
128         // the rumble comes first and then the blow, because that half
second
129         // is what turns a jolt into an impact
130         if (hits_done < 2 && now > next_hit) {
131             rumble_at = now;
132             hit_at = now + 0.55;
133             double angle = 1.1 + hits_done * 2.3;
134             // the second hit is weaker than the first, because it is
moving
135             // away
136             double force = hits_done ? 0.55 : 1.0;
137             hit_dx = cos(angle) * force;
138             hit_dy = sin(angle) * 0.6 * force;
139             next_hit = now + 6.0;
140             hits_done++;
141         }
142         if (rumble_at > 0.0 && now > hit_at && now < hit_at + 0.05)
143             dust_until = now + 1.4;
144         // the beacon turns, with a sharp rise and a slow fall rather
than a
145         // sine. it also fades on its own after the opening, whether he
moves
146         // or not, because an alarm that waits for the player is a
switch.
147         double phase = fmod(t, 2.6) / 2.6;
148         double swell = phase < 0.12 ? phase / 0.12
149             : pow(1.0 - (phase - 0.12) / 0.88, 2.2);
150         double left_over = t < 18.0 ? 1.0 : fmax(0.0, (24.0 - t) / 6.0);
151         alarm = swell * left_over;
152         // the door only makes the end come sooner, it is not the switch
of
153         // the storm
154         if (doors_opened > 0 || t > 24.0)
155             quiet = 1;
156         } else if (alarm > 0.001) {
157             alarm *= 0.94;
158         }
159     }
160
161     // ----- the film
162     static void film_next(enum film next, double now)
163     {
164         film = next;
165         film_at = now;
166     }
167
168     static void film_step(double now)
169     {
170         double age = now - film_at;
171         switch (film) {
172             case FILM_IN:     if (age > BARS_IN) film_next(FILM_TEXT, now);
break;
173             case FILM_TEXT:   if (age > FILM_LINE * OPENING_COUNT)
174                 film_next(FILM_OUT, now);
175                 break;
```

148 La rotation du gyrophare, une montée rapide et une descente lente.

PARTIE 4 CHAPITRE 6 LE CODE

SUITE

```
176     case FILM_OUT:    if (age > BARS_OUT) film_next(FILM_PLAY, now);
                        break;
177     case FILM_END_IN: if (age > BARS_IN) film_next(FILM_END, now); break;
178     case FILM_END:    if (age > FILM_LINE * CLOSING_COUNT)
179                     film_next(FILM_DONE, now);
180                     break;
181     case FILM_DEAD:   break;
182     default: break;
183 }
184 }
185
186 double storyBars(void)
187 {
188     double age = last_now - film_at;
189     switch (film) {
190     case FILM_IN:      return BARS_MAX * fmin(1.0, age / BARS_IN);
191     case FILM_TEXT:    return BARS_MAX;
192     case FILM_OUT:     return BARS_MAX * fmax(0.0, 1.0 - age / BARS_OUT);
193     case FILM_END_IN:  return BARS_MAX * fmin(1.0, age / BARS_IN);
194     case FILM_END:     return BARS_MAX;
195     case FILM_DONE:    return BARS_MAX;
196     case FILM_DEAD:    return BARS_MAX * fmin(1.0, age / BARS_IN);
197     default:          return 0.0;
198     }
199 }
200
201 // the legs are given back when the bars start to leave, not when they
202 // are
203 // gone. the player walks while they retreat, and that overlap is what
204 // makes
205 // it feel like the game takes over from the film.
206 int story_control(void)
207 {
208     return film == FILM_OUT || film == FILM_PLAY;
209 }
210
211 void story_death(double now)
212 {
213     if (film != FILM_DEAD)
214         film_next(FILM_DEAD, now);
215 }
216
217 int story_dead(void) { return film == FILM_DEAD; }
218
219 // the world freezes, loses its colour and turns red. a death screen that
220 // leaves the picture as it is does not say that we lost, it says that we
221 // are
222 // waiting. colour draining away is the fastest signal there is.
223 void story_draw_death(void)
224 {
225     if (film != FILM_DEAD)
226         return;
227     double k = fmin(1.0, (last_now - film_at) / 1.1);
228     for (int i = 0; i < view_width * view_height; i++) {
229         unsigned int c = view[i];
230         int r = (c >> 16) & 0xff, g = (c >> 8) & 0xff, b = c & 0xff;
231         int grey = (r * 30 + g * 59 + b * 11) / 100;
232         r = (int)((r * (1 - k) + grey * k) * (1.0 - 0.38 * k) + 34 * k);
233         g = (int)((g * (1 - k) + grey * k) * (1.0 - 0.72 * k));
234         b = (int)((b * (1 - k) + grey * k) * (1.0 - 0.72 * k));
235         view[i] = (unsigned int)(((r > 255 ? 255 : r) << 16)
236                                 | ((g > 255 ? 255 : g) << 8) | (b > 255 ? 255 : b));
237     }
238     int e = view_width > 700 ? 6 : 4;
```

206 Le joueur a ses jambes dès que les bandes commencent à partir.

228 La luminance d'un pixel, pour l'écran de mort qui perd sa couleur.

PARTIE 4 CHAPITRE 6 LE CODE

SUITE

```
236     draw_text_centered(view_height / 2 - GLYPH_H * e, "YOU DIED",
237         0xc4423a, e);
238     if (last_now - film_at > 1.3) {
239         draw_text_centered(view_height / 2 + GLYPH_H * e / 2,
240             "SPACE TO TRY AGAIN", 0xc8bcb4, e / 2);
241         draw_text_centered(view_height / 2 + GLYPH_H * e / 2 + GLYPH_H *
242             e,
243             "ESC FOR THE MENU", 0x6a5f5a, e / 3);
244     }
245 }
246 // during the text the camera sweeps the room very slowly, because a
247 // still
248 // shot looks like a dead picture and a moving one looks like film
249 double story_drift(void)
250 {
251     if (film != FILM_IN && film != FILM_TEXT)
252         return 0.0;
253     return 0.085 * sin((last_now - started) * 0.31);
254 }
255 void story_skip(double now)
256 {
257     if (film == FILM_IN || film == FILM_TEXT)
258         film_next(FILM_OUT, now);
259 }
260
261 void story_score(int amount)
262 {
263     if (amount > 0)
264         snprintf(tally, sizeof tally, "%d OF THEM WILL NOT GET UP",
265             amount);
266 }
267
268 void story_finish(double now)
269 {
270     if (film < FILM_END_IN)
271         film_next(FILM_END_IN, now);
272 }
273
274 int story_finished(void) { return film == FILM_DONE; }
275
276 // the opening film does not play again at every death, because the first
277 // viewing sets the place and the third one would make you close the
278 // game. the
279 // legs are given back at once and the station is put back the way it
280 // was.
281 void story_restart(double now)
282 {
283     film = FILM_PLAY;
284     film_at = now;
285     started = now;
286     line_shown = 0;
287     line_at = now;
288     quiet = 1;
289     alarm = 0.0;
290     hit_at = 0.0;
291     rumble_at = 0.0;
292     hits_done = 2;
293     dust_until = 0.0;
294 }
295
296 void story_draw_film(void)
297 {
298     // we open on black, with one second of fade before the first
299     // picture. it
```

PARTIE 4 CHAPITRE 6 LE CODE

SUITE

```
297     // says that it begins, instead of saying that the window just
opened, and
298     // it costs nothing.
299     double opening = (last_now - started) / 0.9;
300     if (opening < 1.0) {
301         if (opening < 0.0)
302             opening = 0.0;
303         for (int i = 0; i < view_width * view_height; i++) {
304             unsigned int c = view[i];
305             view[i] = ((int)(((c >> 16) & 0xff) * opening) << 16)
306                 | ((int)(((c >> 8) & 0xff) * opening) << 8)
307                 | (int)((c & 0xff) * opening);
308         }
309     }
310     double part = storyBars();
311     if (part <= 0.0)
312         return;
313     int tall = (int)(view_height * part);
314     for (int y = 0; y < tall; y++)
315         for (int x = 0; x < view_width; x++) {
316             view[y * view_width + x] = 0x000000;
317             view[(view_height - 1 - y) * view_width + x] = 0x000000;
318         }
319
320     const char **lines = NULL;
321     int amount = 0;
322     double age = last_now - film_at;
323     if (film == FILM_TEXT) { lines = OPENING; amount = OPENING_COUNT; }
324     else if (film == FILM_END) { lines = CLOSING; amount = CLOSING_COUNT; }
325 }
326
327 if (!lines)
328     return;
329
330 int n = (int)(age / FILM_LINE);
331 if (n >= amount)
332     return;
333
334 // each line rises, holds, and fades, so it never blinks
335 double within = age - n * FILM_LINE;
336 double k = within < 0.5 ? within / 0.5
337     : (within > FILM_LINE - 0.7 ? (FILM_LINE - within) / 0.7 : 1.0);
338 if (k < 0.0)
339     k = 0.0;
340 int scale = view_width > 700 ? 3 : 2;
341 unsigned int c = 0xd6e8e2;
342 int r = (int)(((c >> 16) & 0xff) * k), g = (int)(((c >> 8) & 0xff) *
k);
343 int b = (int)((c & 0xff) * k);
344 draw_text_centered(view_height / 2 - scale * 4, lines[n],
345     (unsigned int)((r << 16) | (g << 8) | b), scale);
346 if (film == FILM_TEXT && n == 0 && within > 1.6)
347     draw_text_centered((int)(view_height * (1.0 - BARS_MAX) - 14),
348         "SPACE TO SKIP", 0x3a4a52, 1);
349 }
350
351 // the blast lights up and then dies. a shake without light looks like a
352 // display fault, while with the warm glow through the gap you know that
353 // something blew up, and where.
354 static double blast_until;
355
356 void story_blast(double now) { blast_until = now + 0.55; }
357
358 void story_draw_blast(void)
359 {
360     if (last_now > blast_until)
361         return;
```

PARTIE 4 CHAPITRE 6 LE CODE

SUITE

```
358     // a blast is a slap, not a sepia filter. it goes up to white and
    falls
359     // back fast, because spread over a second it looks like a colour
    setting
360     // that somebody forgot to remove.
361     double k = (blast_until - last_now) / 0.55;
362     k = k * k * 1.15;
363     if (k > 1.0)
364         k = 1.0;
365     for (int y = 0; y < view_height; y++)
366         for (int x = 0; x < view_width; x++) {
367             unsigned int c = view[y * view_width + x];
368             int r = (int)((c >> 16) & 0xff) + (255 - ((c >> 16) & 0xff))
    * k);
369             int g = (int)((c >> 8) & 0xff) + (238 - ((c >> 8) & 0xff)) *
    k);
370             int b = (int)((c & 0xff) + (206 - (int)(c & 0xff)) * k *
    0.9);
371             view[y * view_width + x] = (unsigned int)
372                 (((r > 255 ? 255 : r) << 16)
373                  | ((g > 255 ? 255 : g) << 8)
374                  | (b > 255 ? 255 : (b < 0 ? 0 : b)));
375         }
376     }
377
378     static double age_of(double now) { return now - hit_at; }
379
380     static int offset_of(double axis, double now)
381     {
382         if (hit_at <= 0.0)
383             return 0;
384         double age = age_of(now);
385         if (age < 0.0 || age > hit_len)
386             return 0;
387         double left = 1.0 - age / hit_len;
388         return (int)(SHAKE_MAX * axis * left * left * cos(age * 55.0));
389     }
390
391     void story_tick(double now) { last_now = now; film_step(now); }
392     int story_shake(int axis) { return offset_of(axis ? hit_dy : hit_dx,
    last_now); }
393
394     int story_opening_over(void) { return quiet; }
395
396     // the alarm lights the top of the view and the edges, and leaves the
    rest
397     // alone, because a wash over everything is a filter and it shows
398     unsigned int story_grade_at(unsigned int color, int y)
399     {
400         if (alarm < 0.02)
401             return color;
402         // only the top third and only the far edges, because an alarm lights
    what
403         // faces it, it does not repaint the room
404         double up = 1.0 - (double)y / (view_height * 0.30);
405         if (up < 0.0)
406             up = 0.0;
407         // there is no vignette, because darkening or tinting the edges of
    the
408         // screen is the most recognisable filter there is, and the eye reads
    it
409         // as an effect, not as light. the alarm only lights the ceiling.
410         double k = alarm * up * 0.42;
411         if (k < 0.01)
412             return color;
413         int r = (color >> 16) & 0xff, g = (color >> 8) & 0xff, b = color &
    0xff;
414         r = (int)(r + (255 - r) * k * 0.45);
```

388 Le décalage de la figure 6.1.

404 L'alarme n'éclaire que le tiers supérieur de l'image.

PARTIE 4 CHAPITRE 6 LE CODE

SUITE

```
415     g = (int)(g * (1.0 - k * 0.30));
416     b = (int)(b * (1.0 - k * 0.38));
417     return (unsigned int)((r << 16) | (g << 8) | b);
418 }
419
420 // the dust is in the world, not on the glass. each grain has a position
    on
421 // the floor and a height, it is projected like a sprite, and it stays
    where
422 // it fell even when you turn around.
423 #define GRAINS 80
424
425 void story_draw_dust(const struct player *player, double now)
426 {
427     if (now > dust_until || !wall_depth)
428         return;
429     double age = 1.4 - (dust_until - now);
430     double det = player->plane_x * player->dir_y - player->dir_x *
player->plane_y;
431     if (fabs(det) < 1e-9)
432         return;
433     double inv = 1.0 / det;
434
435     for (int i = 0; i < GRAINS; i++) {
436         // the position is drawn once and for all, around the point where
the
437         // hit landed, so it depends neither on the screen nor on the
camera
438         unsigned int hsh = (unsigned int)(i * 2654435761u) ^ (unsigned
int)dust_x;
439         double a = (double)((hsh >> 5) % 6283) / 1000.0;
440         double r = 1.0 + (double)((hsh >> 11) % 500) / 100.0;
441         double wx = dust_x + cos(a) * r, wy = dust_y + sin(a) * r;
442         // it falls from the ceiling, which is at 1.0, while the eye is
at 0.5
443         double speed_of = 0.9 + (double)((hsh >> 19) % 60) / 100.0;
444         double height_left = 1.0 - age * age * speed_of;
445         if (height_left < 0.02)
446             continue;
447
448         double rx = wx - player->x, ry = wy - player->y;
449         double side = inv * (player->dir_y * rx - player->dir_x * ry);
450         double depth = inv * (-player->plane_y * rx + player->plane_x *
ry);
451         if (depth < 0.3)
452             continue;
453         int x = (int)((view_width / 2) * (1 + side / depth));
454         if (x < 0 || x >= view_width || depth >= wall_depth[x])
455             continue;
456         int y = (int)(HORIZON + (0.5 - height_left) * view_height /
depth);
457         if (y < 0 || y >= view_height)
458             continue;
459         int size_of = depth < 2.5 ? 2 : 1;
460         for (int dy = 0; dy < size_of; dy++)
461             for (int dx = 0; dx < size_of; dx++)
462                 if (x + dx < view_width && y + dy < view_height)
463                     view[(y + dy) * view_width + x + dx] = 0x7d8794;
464     }
465 }
466
467 // one line, large, low and centred, and it fades. that is where the eye
468 // already is.
469 static int notice_visible;
470 void story_notice(int visible) { notice_visible = visible; }
471
472 void story_draw_log(void)
```

444 Un grain de poussière tombe du plafond avec une accélération.

PARTIE 4 CHAPITRE 6 LE CODE

SUITE

```
473 {
474     // we show one text at a time, and the notice of the game comes
    before the
475     // log
476     if (notice_visible || line_shown < 0 || line_shown >= LOG_COUNT)
477         return;
478     double age = last_now - line_at;
479     if (age > 4.6)
480         return;
481     double k = age < 0.35 ? age / 0.35
482               : (age > 3.8 ? fmax(0.0, (4.6 - age) / 0.8) : 1.0);
483     int scale = view_width > 700 ? 3 : 2;
484     // the text sits just above where the weapon will be, not in a corner
485     int y = (int)(view_height * 0.70);
486     unsigned int base = 0x000000, fresh = 0xbfe9dd;
487     int r = (int)((((fresh >> 16) & 0xff) * k + ((base >> 16) & 0xff) * (1
- k)));
488     int g = (int)((((fresh >> 8) & 0xff) * k + ((base >> 8) & 0xff) * (1 -
k)));
489     int b = (int)((fresh & 0xff) * k + (base & 0xff) * (1 - k));
490     if (r + g + b < 24)
491         return;
492     draw_text_centered(y, LOG[line_shown],
493         (unsigned int)((r << 16) | (g << 8) | b), scale);
494 }
```



Figure 6.2. L'explosion derrière la porte bloquée. L'image est secouée et lavée de blanc, et la poussière tombe du plafond.

VOCABULAIRE

Le journal ou log est la suite de lignes que la station écrit pendant l'ouverture. Il raconte, il n'explique pas.

La souris tourne la tête

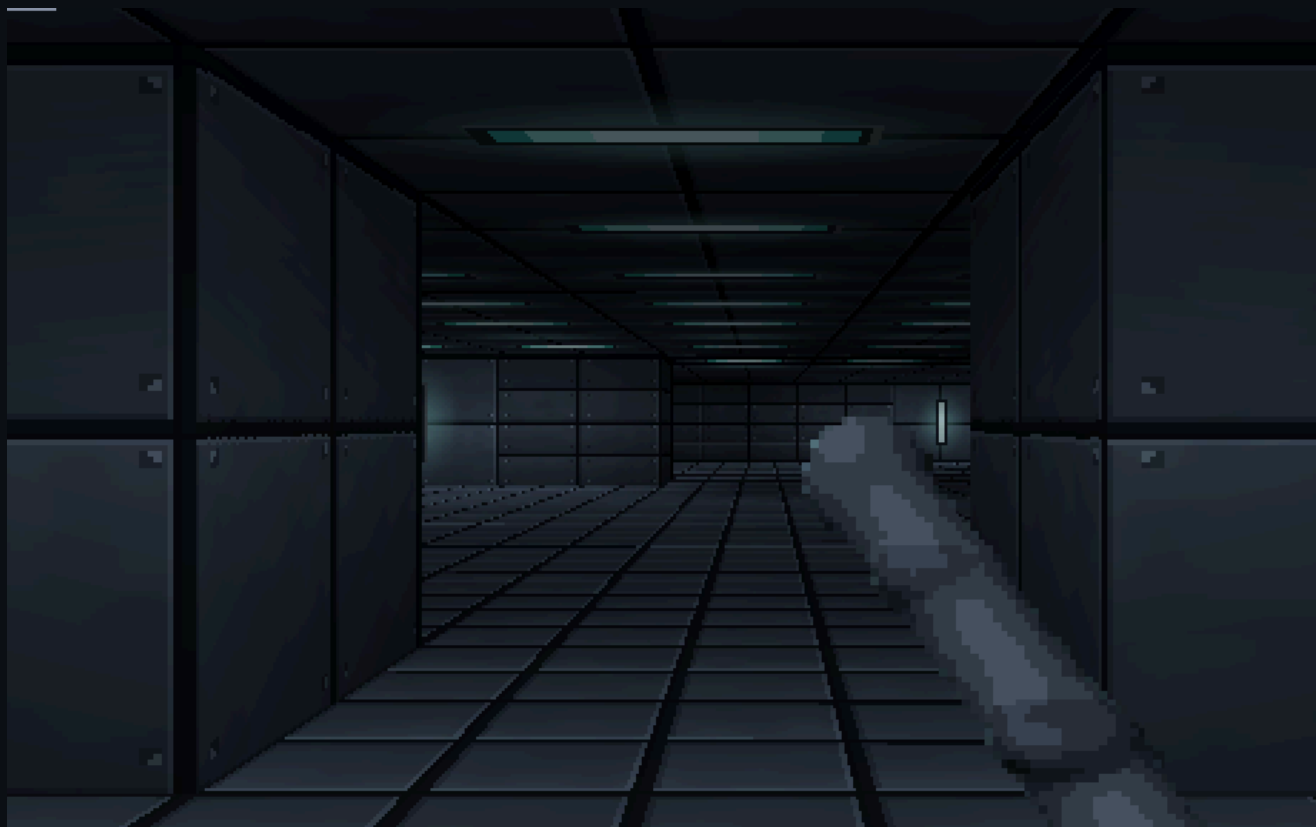


Figure 7.1. Le couloir vu en tournant la tête à la souris. Le pointeur est caché et ramené au centre à chaque image.

Dans ce chapitre, vous allez tourner la tête à la souris, cacher le pointeur, et préparer le menu.

Une vue à la première personne tournée avec les flèches se joue comme en 1992. La souris tourne maintenant la tête. À chaque image, le programme lit de combien le pointeur a bougé, puis le ramène au centre de la fenêtre avec `XWarpPointer`. Le mouvement provoqué par ce retour au centre est ignoré, sinon il serait lu comme un mouvement du joueur. Le jeu ne prend la souris qu'au premier clic dans la fenêtre, cache le pointeur avec un curseur vide, et la rend quand la fenêtre perd le focus ou change de taille.

La taille des pixels de l'image ne dépend plus de la fenêtre. Avec un bloc fixe de 2 pixels, une petite fenêtre calculait très peu de rayons et une grande une image très fine, et le jeu avait un aspect différent à chaque taille. Le programme vise maintenant une hauteur d'image de 400 pixels, et le bloc est celui qui s'en approche le plus, entre 1 et 4. La touche Échap n'arrête plus le jeu, elle demande le menu, écrit au chapitre 12, et la touche N coupe le son. Les touches W, Z et S s'ajoutent aux flèches, et la secousse de l'histoire est appliquée au moment de la copie vers la fenêtre.

VOCABULAIRE

Le focus est la fenêtre qui reçoit le clavier et la souris. Quand le joueur clique ailleurs, le jeu perd le focus et rend la souris.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE SCREEN H LIGNES 4 A 13

```
4  #include <X11/Xlib.h>
5
6  // the size of a view pixel should not depend on the window. with a fixed
7  // block of two, a small window renders very few rays and a wide one
   renders a
8  // very fine picture, so the game looks different at every size. instead
   we
9  // aim for a rendered height, and the block is whatever gets us closest
   to it.
10 #define VIEW_TARGET_H 400
11 #define PIXEL_MAX      4
12 // the size the window opens at. after that a wider window shows more
   world,
13 // it does not blow up what was already there
```

INCLUDE SCREEN H LIGNES 26 A 34

```
26
27 // XCreateImage wants to know how each row of pixels is padded, in bits.
   this
28 // is not the colour depth: 32 is what a modern display expects
29 #define SCANLINE_PAD 32
30
31 // how many radians one pixel of mouse movement is worth
32 #define MOUSE_SPEED 0.0032
33
34 #define GAME_NAME "The Keep"
```

INCLUDE SCREEN H LIGNES 48 A 70

```
48     int scale, left, top;
49     int full;                // are we filling the screen?
50     // the mouse turns the head, because a first person view turned with
   the
51     // arrow keys plays like 1992. every frame we read how far the
   pointer
52     // moved, then we pull it back to the centre. the game only takes the
53     // mouse on the first click.
54     int grab;                // is the mouse ours
55     int warped;              // it has been put back once already
56     Cursor blank;            // and it is not drawn while playing
57 };
58
59 struct keys {
60     int forward, back, left, right, strafe_left, strafe_right, quit;
61     int push;                // a press on space, waiting to be read
62     int hit;                 // a press on ctrl: fists first, then the gun
63     int mute;                // a press on n: the sound goes away
64     int map;                 // is the map on screen? M takes it away
65     int menu;                // escape: the menu opens or closes
66     int validate;           // entree
67     int weapon;              // 1 or 2: the bar or the gun
68     double look;             // how far the mouse moved sideways, in radians
69     int any;                 // somebody is at the keys
70 };
```

INCLUDE SCREEN H LIGNES 75 A 81

```
75 void screen_toggle_fullscreen(struct screen *screen);
76 void screen_read_keys(struct screen *screen, struct keys *keys);
```

Ligne par ligne

- 10** La hauteur d'image visée. Le bloc de pixels est choisi pour s'en approcher.
- 32** Combien de radians vaut un pixel de mouvement de la souris.

PARTIE 4 CHAPITRE 7 LE CODE

SUITE

```
77 // gives the mouse back to the desktop. the menu does it, and so does
    losing
78 // focus.
79 void screen_release_mouse(struct screen *screen);
80 void screen_present(struct screen *screen);
81
```

155 Le curseur vide, un dessin de 8 pixels sur 8 sans un seul point.

SRC SCREEN C LIGNES 1 A 4

```
1 #include "screen.h"
2 #include "story.h"
3
4 #include <X11/XKLib.h>
```

SRC SCREEN C LIGNES 11 A 16

```
11 // every pixel of the game lands here first
12 unsigned int *view;
13 int view_width = WIN_WIDTH / (WIN_HEIGHT / VIEW_TARGET_H);
14 int view_height = WIN_HEIGHT / (WIN_HEIGHT / VIEW_TARGET_H);
15
16 // the window at the size of the picture, in the middle of the screen
```

SRC SCREEN C LIGNES 52 A 65

```
52 static void screen_fit(struct screen *screen)
53 {
54     // the window decides how many rays we cast, and a view pixel stays a
55     // whole number of screen pixels. the block follows the window, so
    that a
56     // pixel looks the same size whatever the window does.
57     int scale = screen->height / VIEW_TARGET_H;
58     if (scale < 1)
59         scale = 1;
60     if (scale > PIXEL_MAX)
61         scale = PIXEL_MAX;
62     int vw = screen->width / scale;
63     int vh = screen->height / scale;
64     if (vw < VIEW_MIN_W)
65         vw = VIEW_MIN_W;
```

SRC SCREEN C LIGNES 75 A 79

```
75     }
76
77     screen->scale = scale;
78     screen->left = (screen->width - view_width * screen->scale) / 2;
79     screen->top = (screen->height - view_height * screen->scale) / 2;
```

SRC SCREEN C LIGNES 143 A 161

```
143 // it the picture stays in the corner when the window grows
144 XSelectInput(screen->display, screen->window,
145     KeyPressMask | KeyReleaseMask | StructureNotifyMask
146     | PointerMotionMask | ButtonPressMask | FocusChangeMask);
147
148 // we use an empty pointer, because a desktop cross in the middle of
    a
149 // corridor breaks everything, and the mouse no longer points at
    anything
150 // anyway
151 char nothing[8] = { 0 };
152 Pixmap empty = XCreateBitmapFromData(screen->display, screen->window,
153     nothing, 8, 8);
154 XColor black = { 0 };
155 screen->blank = XCreatePixmapCursor(screen->display, empty, empty,
156     &black, &black, 0, 0);
157 XFreePixmap(screen->display, empty);
158 screen->warped = 0;
159 screen->grab = 0;
160
161 screen_windowed(screen);
```

PARTIE 4 CHAPITRE 7 LE CODE

SUITE

SRC SCREEN C LIGNES 175 A 192

```
175 }
176
177 // gives the mouse back, so the pointer is visible again and free to
    leave the
178 // window
179 void screen_release_mouse(struct screen *screen)
180 {
181     if (!screen->grab)
182         return;
183     screen->grab = 0;
184     screen->warped = 0;
185     XUndefineCursor(screen->display, screen->window);
186 }
187
188 void screen_read_keys(struct screen *screen, struct keys *keys)
189 {
190     keys->look = 0.0;
191     while (XPending(screen->display)) {
192         XEvent event;
```

214 Le premier clic prend la souris.

230 Le premier mouvement après la prise est ignoré, il est dû au retour au centre.

232 Le pointeur est ramené au centre de la fenêtre.

SRC SCREEN C LIGNES 200 A 237

```
200     if (event.type == ConfigureNotify &&
201         (event.xconfigure.width != screen->width ||
202          event.xconfigure.height != screen->height)) {
203         // we give the mouse back as soon as the window changes size,
204         // because while you drag an edge, the window manager and the
    game
205         // both pull the pointer and the view goes anywhere. a click
    in
206         // the window takes it again.
207         screen_release_mouse(screen);
208         screen_resize(screen, event.xconfigure.width,
209                       event.xconfigure.height);
210     }
211
212     // on a click in the window we take the mouse and hide the
    pointer.
213     // the menu and a loss of focus give it back.
214     if (event.type == ButtonPress && !screen->grab) {
215         screen->grab = 1;
216         screen->warped = 0;
217         XDefineCursor(screen->display, screen->window, screen-
    >blank);
218     }
219     if (event.type == FocusOut)
220         screen_release_mouse(screen);
221
222     // the pointer is pulled back to the centre after every move, and
    we
223     // throw away the move we just caused, otherwise the return to
    the
224     // centre would be read as a move by the player
225     if (event.type == MotionNotify && screen->grab) {
226         int mx = screen->width / 2, my = screen->height / 2;
227         int dx = event.xmotion.x - mx;
228         if (event.xmotion.x == mx && event.xmotion.y == my)
229             continue;
230         if (screen->warped)
231             keys->look += dx * MOUSE_SPEED;
232         XWarpPointer(screen->display, None, screen->window,
233                     0, 0, 0, 0, mx, my);
234         screen->warped = 1;
235     }
236
237     if (event.type == KeyPress || event.type == KeyRelease) {
```

PARTIE 4 CHAPITRE 7 LE CODE

SUITE

SRC SCREEN C LIGNES 242 A 264

```
242         event.xkey.keycode, 0, 0);
243     switch (key) {
244         // escape opens the menu, it does not quit, because a game
        whose
245         // only exit is closing the window has no pause, no restart,
        and
246         // no reminder of the keys
247         case XK_Escape: if (down) keys->menu = 1; break;
248         case XK_Return:
249         case XK_KP_Enter: if (down) keys->validate = 1; break;
250         case XK_Control_L:
251         case XK_Control_R: if (down) keys->hit = 1; break;
252         case XK_n:
253         case XK_N: if (down) keys->mute = 1; break;
254         // you choose your weapon yourself. the game falls back to
        the bar
255         // on an empty magazine, which is right, but on its own the
        player
256         // would never learn that he holds two things.
257         case XK_1: case XK_KP_1: if (down) keys->weapon = 1; break;
258         case XK_2: case XK_KP_2: if (down) keys->weapon = 2; break;
259         case XK_w:
260         case XK_z:
261         case XK_Up:     keys->forward = down; break;
262         case XK_s:
263         case XK_Down:   keys->back = down; break;
264         case XK_Left:   keys->left = down; break;
```

247 Échap demande le menu au lieu de quitter.

298 La secousse de l'histoire décale la copie de l'image vers la fenêtre.

SRC SCREEN C LIGNES 274 A 278

```
274     case XK_m:
275         // on release, not on press: a held key repeats,
276         // and the map would blink
277         if (!down)
278             keys->map = !keys->map;
```

SRC SCREEN C LIGNES 293 A 306

```
293     // first is writing every pixel of every frame twice
294     int scale = screen->scale;
295     // a hit throws the picture, and the alarm takes the colour out of
        it.
296     // both are applied here, on the way to the window, because the
        renderer
297     // knows nothing about the story and the story draws nothing itself.
298     int jolt_x = story_shake(0) * scale, jolt_y = story_shake(1) * scale;
299     // a shake leaves a band that is not copied over, and that band keeps
        the
300     // previous frame, a strip of corridor frozen at the edge. so we also
301     // clear the window when the picture is shifted, not only when it is
302     // smaller.
303     int cover_w = view_width * scale, cover_h = view_height * scale;
304     if (screen->left > 0 || screen->top > 0 || jolt_x || jolt_y
305         || cover_w < screen->width || cover_h < screen->height)
306         for (int i = 0; i < screen->width * screen->height; i++)
```

SRC SCREEN C LIGNES 311 A 348

```
311     if (visible > view_width)
312         visible = view_width;
313     // the shake shifts the copy, so it can leave the window. the rows
        are
314     // bounded and the columns have to be too, so we compute once per
        frame
```

PARTIE 4 CHAPITRE 7 LE CODE

SUITE

```
315     // the first and last column that fall inside, instead of testing
every
316     // pixel.
317     int base = screen->left + jolt_x;
318     int first = base < 0 ? (-base + scale - 1) / scale : 0;
319     int last = visible;
320     if (base + last * scale > screen->width)
321         last = (screen->width - base) / scale;
322     // the second row is a copy of the first. enlarging every pixel once
per
323     // row cost a million writes per frame, but the bottom row of a block
is
324     // the same as its top row, so we build it once and copy it whole.
325     for (int y = 0; y < view_height && first < last; y++) {
326         unsigned int *row = view + y * view_width;
327         int line = screen->top + y * scale + jolt_y;
328         if (line < 0 || line >= screen->height)
329             continue;
330         unsigned int *out = screen->pixels + line * screen->width
331             + base + first * scale;
332         for (int x = first; x < last; x++) {
333             unsigned int color = story_grade_at(row[x], y);
334             for (int again = 0; again < scale; again++)
335                 *out++ = color;
336         }
337         for (int copy = 1; copy < scale; copy++) {
338             int twin = line + copy;
339             if (twin < 0 || twin >= screen->height)
340                 break;
341             memcpy(screen->pixels + twin * screen->width
342                 + base + first * scale,
343                 screen->pixels + line * screen->width
344                 + base + first * scale,
345                 (size_t)(last - first) * scale
346                 * sizeof(*screen->pixels));
347         }
348     }
```


L'équipage, les rôdeurs et la chose du fond

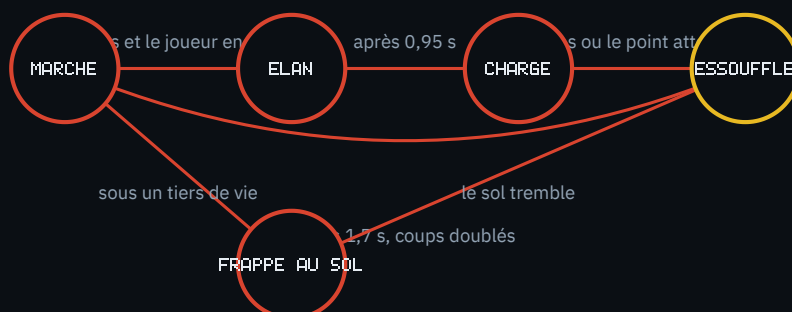


Figure 8.1. Le motif de la chose du fond. Elle marche, annonce sa charge pendant 0,95 seconde, charge en ligne droite trois fois plus vite qu'elle ne marche, puis souffle 1,7 seconde. Pendant qu'elle souffle, les coups comptent double.

Dans ce chapitre, vous allez peupler le niveau de survivants, de rôdeurs et d'un boss qui a un motif à apprendre.

Le niveau se peuple de rôles nouveaux. Les survivants, marqués par la lettre n, sont allongés au sol, respirent encore, et ont trois phrases à dire, dont la dernière ne sort jamais entière. On ne les voit jamais debout, le jeu commence après. Les hommes armés, la lettre h, tiennent le fond du pont derrière la porte bloquée, tirent, et perdent. Ils ne viennent jamais vers le joueur, car la scène est faite pour être regardée. Les rôdeurs, la lettre w, vont et viennent au fond du couloir vu par la fente, ondulent comme s'ils esquivaient, et ne viennent jamais non plus.

La chose du fond, la lettre Z, est le boss, et ce qui en fait un boss est un motif. Une créature qui marche vers le joueur et encaisse les coups n'est qu'une barre de vie plus longue. Celle-ci annonce sa charge, charge, puis souffle. La figure 8.1 montre son motif. Pendant qu'elle souffle, elle prend le double de dégâts, et c'est tout le contrat du combat. On encaisse la charge ou on l'esquive, et on est payé juste après. Sous un tiers de sa vie, elle frappe le sol au lieu des jambes, et le sol tremble.

Une créature teste maintenant son épaule contre les murs, et non son centre, sans quoi elle marchait jusqu'à ce que la moitié de son corps soit dessinée dans la tôle. Elle regarde parfois ailleurs que là où elle marche, avec un vecteur de regard séparé de sa direction, car écrire le regard dans la direction la ferait marcher de ce côté à l'image suivante. Chaque état sonne, un cri au réveil, un grognement à l'abandon, un souffle avant le coup, et le coup lui-même.

La barre de vie du boss n'apparaît que lorsqu'il a vu le joueur. Montrée dès le début, elle annoncerait le combat avant la salle. Sa mort est une question séparée de sa vie à zéro, sans quoi la barre resterait à l'écran, vide, après le dernier coup.

Les planches de l'équipage et du boss

L EQUIPAGE



Figure 8.2. La planche de l'équipage, l'homme qui tire sur la deuxième rangée.

Deux planches nouvelles rejoignent celles de la créature et de la lourde. La planche de l'équipage sert aux hommes armés et aux survivants au sol. Ses cinq angles et sa rangée de corps au sol sont ceux des créatures, et sa deuxième rangée montre l'homme qui tire, choisie pendant 0,16 seconde après chaque tir.

LE BOSS



Figure 8.3. La planche du boss, la même sculpture en chair.

La planche du boss est la même sculpture que la créature, dans une matière de chair, et elle est dessinée une fois et demie plus grande. Ses yeux sont les seuls pixels saturés de l'écran, et le joueur les voit avant de voir la forme.

Le liseré des créatures prend la couleur du néon de la zone où elles se trouvent, vert dans les coursives, blanc froid aux machines et ambre dans la soute, pour qu'elles appartiennent à la salle au lieu d'y être collées. La fonction `zone_tint` lit la texture des murs voisins pour le savoir.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE THING H LIGNES 9 A 23

```

9
10 #define THINGS_MAX      36
11 #define NEVER            1e9      // a moment that does not come
12 #define THING_W          24      // how wide it is drawn, in its own
    pixels
13 #define THING_H          40
14 // how close it comes. under a metre and a half, a thing as tall as the
15 // corridor fills the whole screen and you cannot see anything any more,
    not
16 // it, not the room, not where the next blow comes from. so it stops at
    arm's
17 // length, and the bar is longer than its arm.
18 #define THING_REACH      1.55     // how close it has to be to hit you
19 #define THING_WINDUP     0.45     // how long the blow takes to come down
20 #define BOSS_WINDUP      0.62
21 #define STRIKE_SLACK     0.3      // a little further than its reach
    still lands
22 #define THING_SPEED      1.9
23 #define THING_SIGHT      11.0

```

INCLUDE THING H LIGNES 29 A 35

```

29 // start, to step back, and to answer it.
30 #define THING_GAP        1.45
31 // how far its shoulder stays from a wall. tested on its centre alone it
    walks
32 // until half of its body is drawn inside the plating
33 #define THING_SHOULDER  0.3
34 // the heavy one. three hundred points is nine swings of the bar, and
    nobody
35 // stands still for nine swings. it is what makes the sidearm necessary,
    and

```

INCLUDE THING H LIGNES 41 A 77

```

41 #define TOUGH_SIZE      1.22
42
43 #define BOSS_HEALTH     540.0
44 #define BOSS_DAMAGE     20.0
45 #define BOSS_SPEED      1.45
46 #define BOSS_REACH      2.25
47 #define BOSS_SIZE        1.55     // its size, in corridor heights
48 // these are the only saturated pixels on the screen, so you see them
    before
49 // you see the shape
50 #define THING_EYE        0xff5a3c
51
52 // the rim takes the colour of the neon of the zone it stands in, so that
    it
53 // belongs to the room instead of looking pasted on top of it
54 unsigned int zone_tint(double x, double y);
55
56 enum thing_state { THING_IDLE, THING_ALERT, THING_HUNT, THING_STRIKE,
    THING_DEAD };
57
58 // the thing at the bottom has a pattern, and that is what makes it a
    boss. a
59 // beast that just walks at you and soaks up hits is only a longer health
    bar.

```

Ligne par ligne

- 33** La distance de l'épaule au centre, testée contre les murs.
- 43** La vie du boss, neuf projectiles pendant qu'il souffle, dix-huit sinon.

PARTIE 4 CHAPITRE 8 LE CODE

SUITE

```
60 // this one announces its charge, charges, and then rests, so the player
61 // learns to step aside and to hit while it rests.
62 enum boss_move {
63     BOSS_WALK,      // it walks, like the others
64     BOSS_WIND,      // it stops and roars: the announcement, and it lasts
65     BOSS_CHARGE,    // it rushes in a straight line, three times faster
66     BOSS_REST,      // it missed and it is winded: hits count double here
67     BOSS_SLAM       // under a third of its health it hits the floor
68 };
69 #define BOSS_WIND_TIME    0.95
70 #define BOSS_CHARGE_TIME  1.30
71 #define BOSS_REST_TIME    1.70
72 #define BOSS_WALK_TIME    3.20
73 #define BOSS_CHARGE_HIT   26.0
74 #define BOSS_SLAM_REACH   5.0
75 #define BOSS_SLAM_HIT     18.0
76
77 struct thing {
```

62 Les cinq temps du motif du boss.

INCLUDE THING H LIGNES 81 A 87

```
81     double since;      // when it entered this state
82     double health;
83     double stride;      // the walk cycle, 0 to 1
84     double last_seen_x, last_seen_y;
85     double next_strike; // not before this moment
86     // it does not walk in a straight line and it does not shrug off a
    hit.
87     // a thing that comes at you and never flinches is played by walking
```

INCLUDE THING H LIGNES 93 A 125

```
93     double hurt_at;
94     int used;
95     int silent;        // it has been seen already: it does not
    shriek twice
96     int boss;          // the one at the bottom of the hold
97     enum boss_move move; // and where it is in its pattern
98     double move_at;
99     int tough;         // the heavy one, the one after the sidearm
100    // the survivors lie on the floor, they talk, and at some point they
    stop.
101    // it is the only way to have a crew without ever showing it alive,
102    // because we always arrive too late and we hear it.
103    int crew;
104    int line;           // how many lines it still has
105    double next_line;
106    // these are the ones you see through the jammed door. they come and
    go at
107    // the far end of the corridor and they never come here, because they
    are
108    // not there to fight, they are there so that you understand what is
109    // waiting outside.
110    int roam;
111    double roam_x, roam_y; // the point they start from
112    double roam_to;        // and how far they stray from it
113    double roam_wob;       // and how far they drift sideways
114    int leaving;          // it is done, it leaves by the north
    corridor
115    // where it looks, when that is not where it walks. if we wrote the
    gaze
```

PARTIE 4 CHAPITRE 8 LE CODE

SUITE

```
116     // into dir, it would steer the beast, and at the next step it would
walk
117     // that way.
118     double look_x, look_y;
119     // the armed crew, which you see only once. they hold the far end of
the
120     // deck, they fire, and they lose. they never come towards us,
because the
121     // scene is there to be watched, not to be played.
122     int crewman;
123     double fire_until;
124 };
125
```

INCLUDE THING H LIGNES 135 A 166

```
135 // drawn after the walls, so the depth buffer is full
136 void things_draw(const struct player *player, double now);
137 // gives the health of the boss from 0 to 1 once it is awake, or -1
before
138 // that
139 double boss_health(void);
140 // tells whether the boss is down. it is a separate question, otherwise
the
141 // bar would stay on screen.
142 int boss_down(void);
143 // how much one hit is worth on it right now. it is double while the boss
144 // rests.
145 double thing_weak(const struct thing *t);
146 // tells whether the boss is winded right now, because that is the moment
to
147 // hit it
148 int boss_resting(void);
149 // a noise carries far. after a shot or a cry, everything asleep within
reach
150 // gets up, and that is why you think twice before firing.
151 void things_hear(double x, double y, double reach, double now);
152 // gives what a survivor still has to say, or NULL when he has nothing
left
153 const char *crew_speak(const struct player *player, double now);
154 // after the blast, the beasts at the far end leave
155 void things_roam_away(void);
156 // during the scene one of them falls, and that is the one we hear die
157 void things_roam_kill(void);
158 // the beasts close in on the crew, weaving as they come
159 void things_roam_advance(void);
160 // when a round passes close, the beast flashes white for an instant
161 void things_roam_graze(double now);
162 // makes a crewman fire (0 or 1), and makes a crewman fall
163 void crew_fire(int which, double now);
164 void crew_fall(int which);
165
166 #endif
```

SRC THING C LIGNES 6 A 36

```
6 #include "render.h"
7 #include "screen.h"
8 #include "sound.h"
9 #include "sprite.h"
10 #include "story.h"
11 #include "texture.h"
12 #include "world.h"
13
14 struct thing things[THINGS_MAX];
15
16 // the sheet is loaded once. it has five angles per walking step and four
```

PARTIE 4 CHAPITRE 8 LE CODE

SUITE

```
17 // steps, and the three missing angles are taken in the mirror, the way
    games
18 // did in 1993 to fit on a floppy.
19 #define ANGLES 5
20 #define WALK_STEPS 4
21 // five angles, four steps, and one more row for the body on the floor,
22 // because a dead thing that stays standing cancels the only feedback the
    game
23 // gives when a shot lands
24 #define POSES (WALK_STEPS + 1)
25 static struct sprite sheet, sheet_tough, sheet_boss, sheet_crew;
26
27 static void load_sheet(void)
28 {
29     static int tried;
30     if (!tried++) {
31         sprite_load(&sheet, "art/thing.tecs");
32         sprite_load(&sheet_tough, "art/heavy.tecs");
33         sprite_load(&sheet_boss, "art/boss.tecs");
34         sprite_load(&sheet_crew, "art/crew.tecs");
35     }
36 }
```

65

La couleur du néon de la zone, lue dans les murs voisins.

SRC THING C LIGNES 41 A 77

```
41 static double reach_of(const struct thing *t)
42 {
43     return t->boss ? BOSS_REACH : t->tough ? TOUGH_REACH : THING_REACH;
44 }
45
46 static double damage_of(const struct thing *t)
47 {
48     return t->boss ? BOSS_DAMAGE : t->tough ? TOUGH_DAMAGE :
    THING_DAMAGE;
49 }
50
51 static double speed_of(const struct thing *t)
52 {
53     // the ones in the battle move fast, because it is a melee, not a
    patrol
54     return t->boss ? BOSS_SPEED : t->tough ? TOUGH_SPEED
55         : t->roam ? 3.1 : THING_SPEED;
56 }
57
58 static double size_of(const struct thing *t)
59 {
60     return t->boss ? BOSS_SIZE : t->tough ? TOUGH_SIZE : 1.0;
61 }
62
63 // the neon is turquoise in service, cold white at the machines and amber
    in
64 // the hold. the level file already says which one, so we only read it.
65 unsigned int zone_tint(double x, double y)
66 {
67     for (int dy = -2; dy <= 2; dy++)
68         for (int dx = -2; dx <= 2; dx++) {
69             int k = wall_kind((int)x + dx, (int)y + dy);
70             if (k == ALARM_TEXTURE)
71                 return 0xf0a53c;
72             if (k == COLD_TEXTURE)
73                 return 0xcfe4ff;
74         }
75     return NEON_TUBE;
76 }
77
```

PARTIE 4 CHAPITRE 8 LE CODE

SUITE

SRC THING C LIGNES 87 A 92

```
87     char c = map[y][x];
88     int i = -1;
89     if (c == 'x' || c == 'y' || c == 'w' || c == 'z'
90         || c == 'X' || c == 'n' || c == 'h')
91         i = thing_add(x + 0.5, y + 0.5);
92     if (i < 0)
```

99 Un rôdeur, avec son point d'attache et son amplitude de va-et-vient.

172 L'épaule, dans le sens de la marche.

SRC THING C LIGNES 97 A 123

```
97     things[i].state = THING_DEAD;
98     things[i].health = 0.0;
99     } else if (c == 'w') {
100         things[i].roam = 1;
101         things[i].roam_x = x + 0.5;
102         things[i].roam_y = y + 0.5;
103         things[i].roam_to = 1.2 + (x % 3) * 0.5;
104         things[i].roam_wob = 0.9;
105     } else if (c == 'Z') {
106         things[i].boss = 1;
107         things[i].health = BOSS_HEALTH;
108     } else if (c == 'X') {
109         things[i].tough = 1;
110         things[i].health = TOUGH_HEALTH;
111     } else if (c == 'h') {
112         things[i].crewman = 1;
113         things[i].dir_x = -1.0;
114         things[i].dir_y = 0.0;
115     } else if (c == 'n') {
116         // he lies on the floor, still breathing, and he has two
or
117         // three lines left before he stops
118         things[i].crew = 1;
119         things[i].state = THING_DEAD;
120         things[i].health = 0.0;
121         things[i].line = 1;
122     }
123 }
```

SRC THING C LIGNES 168 A 178

```
168
169     double move = speed_of(t) * elapsed;
170     // each axis on its own, so a shoulder on a wall keeps sliding, and
the
171     // shoulder is what we test, not the centre
172     double side_x = t->dir_x > 0.0 ? THING_SHOULDER : -THING_SHOULDER;
173     double side_y = t->dir_y > 0.0 ? THING_SHOULDER : -THING_SHOULDER;
174     if (!is_wall((int)(t->x + t->dir_x * move + side_x), (int)t->y))
175         t->x += t->dir_x * move;
176     if (!is_wall((int)t->x, (int)(t->y + t->dir_y * move + side_y)))
177         t->y += t->dir_y * move;
178     t->stride += move * 1.7;
```

SRC THING C LIGNES 188 A 303

```
188     double dx = player->x - t->x, dy = player->y - t->y;
189     double distance = sqrt(dx * dx + dy * dy);
190     // they fight, they do not stroll, and they will never see us.
the
191     // sideways back and forth is what makes it look like the beast
192     // dodges, because a beast that moves in a straight line only
looks
193     // like a beast walking.
194     if (t->roam) {
195         double but = t->roam_x
196             + t->roam_to * sin(now * 2.4 + t->roam_x);
197         double side = t->roam_y
198             + t->roam_wob * sin(now * 3.3 + t->roam_x * 3.0);
199         step_towards(t, but, side, elapsed);
```

PARTIE 4 CHAPITRE 8 LE CODE

SUITE

```
200          // they also look at what they attack, not at us, because a
    beast
201          // that steps back while facing the camera looks like a
    figurine
202          // posing. the gaze is kept next to the walking direction,
    never
203          // inside it, or it would steer the beast.
204          if (t->roam_wob > 0.0) {
205              t->look_x = 1.0;
206              t->look_y = 0.0;
207          } else {
208              t->look_x = t->look_y = 0.0;
209          }
210          // it has reached the corridor, so it is out and we remove it
211          if (t->leaving
212              && hypot(t->x - t->roam_x, t->y - t->roam_y) < 0.8)
213              t->used = 0;
214          continue;
215      }
216      // the crew holds its position and looks west. if they saw us
    through
217      // the gap they would leave their post and come for us, and the
    scene
218      // would fall apart.
219      if (t->crewman)
220          continue;
221      int visible = sees(t, player);
222      if (visible) {
223          t->last_seen_x = player->x;
224          t->last_seen_y = player->y;
225      }
226
227      // the pattern of the boss comes before everything else, because
    while
228      // it is in one of its beats it does nothing else
229      if (t->boss && t->state != THING_IDLE && t->state != THING_DEAD)
    {
230          double age = now - t->move_at;
231          switch (t->move) {
232              case BOSS_WIND:
233                  if (age > BOSS_WIND_TIME) {
234                      t->move = BOSS_CHARGE;
235                      t->move_at = now;
236                      // it aims where you are now and it will not correct
    its
237                      // course, and that is what makes the charge
    something you
238                      // can dodge
239                      t->last_seen_x = player->x;
240                      t->last_seen_y = player->y;
241                  }
242                  continue;
243              case BOSS_CHARGE: {
244                  double vx = t->last_seen_x - t->x;
245                  double vy = t->last_seen_y - t->y;
246                  double d = sqrt(vx * vx + vy * vy);
247                  if (d > 0.05) {
248                      double row = BOSS_SPEED * 3.1 * elapsed;
249                      if (!is_wall((int)t->x + vx / d * row * 1.6),
    (int)t->y))
250                          t->x += vx / d * row;
251                      if (!is_wall((int)t->x, (int)t->y + vy / d * row *
    1.6)))
252                          t->y += vy / d * row;
253                      t->dir_x = vx / d;
254                      t->dir_y = vy / d;
255                      t->stride += row * 2.2;
```

232 L'annonce de la charge. Le boss vise la position du joueur à cet instant et ne corrigera plus.

248 La charge, trois fois plus vite que la marche.

PARTIE 4 CHAPITRE 8 LE CODE

SUITE

```
256         }
257         if (distance < BOSS_REACH && now > t->next_strike) {
258             damage += BOSS_CHARGE_HIT;
259             t->next_strike = now + 1.0;
260             sound_play(SFX_HURT, 0.0);
261         }
262         if (age > BOSS_CHARGE_TIME || d < 0.05) {
263             t->move = BOSS_REST;
264             t->move_at = now;
265         }
266         continue;
267     }
268     case BOSS_REST:
269         // it is winded and it stands still. that is the window,
and
270         // it lasts long enough to be taken.
271         if (age > BOSS_REST_TIME) {
272             t->move = BOSS_WALK;
273             t->move_at = now;
274         }
275         continue;
276     case BOSS_SLAM:
277         if (age > BOSS_WIND_TIME) {
278             if (distance < BOSS_SLAM_REACH && visible)
279                 damage += BOSS_SLAM_HIT;
280             sound_play(SFX_IMPACT, 0.0);
281             // the floor takes the blow, so the picture does too,
282             // because a blow that size that shakes nothing has
no
283             // weight
284             story_hit(now, 1.3, t->x, t->y);
285             t->move = BOSS_REST;
286             t->move_at = now;
287         }
288         continue;
289     case BOSS_WALK:
290     default:
291         if (age > BOSS_WALK_TIME && visible && distance < 12.0) {
292             // under a third of its health it changes register,
and it
293             // goes for the floor instead of the legs
294             int bottom = t->health < BOSS_HEALTH / 3.0;
295             t->move = (bottom && distance < BOSS_SLAM_REACH)
296                 ? BOSS_SLAM : BOSS_WIND;
297             t->move_at = now;
298             sound_play(SFX_GROWL, distance);
299         }
300         break;
301     }
302 }
303
```

268 Le souffle, la fenêtre pour frapper.

SRC THING C LIGNES 311 A 315

```
311         t->since = now;
312         if (!t->silent) {
313             sound_play(SFX_SHRIEK, distance);
314             t->silent = 1;
315             things_hear(t->x, t->y, 7.0, now);
```

SRC THING C LIGNES 329 A 333

```
329         // it into last_seen would pile up frame after frame.
330         double bx = t->last_seen_x, by = t->last_seen_y;
331         if (now > t->stagger && !t->boss) {
332             double margin = distance > 3.0 ? 1.1 : distance * 0.35;
```

PARTIE 4 CHAPITRE 8 LE CODE

SUITE

```
333             double ox = -(player->y - t->y), oy = player->x - t->x;
```

SRC THING C LIGNES 347 A 355

```
347             t->state = THING_STRIKE;
348             t->since = now;
349             // it announces its blow with a quarter second of noise
and
350             // posture before it lands. that is the window to step
back,
351             // and without it you would lose health without ever
knowing
352             // where it came from.
353             sound_play(SFX_PUNCH, distance);
354         } else if (!visible
355             && fabs(t->x - t->last_seen_x) < 0.4
```

SRC THING C LIGNES 361 A 376

```
361             t->since = now;
362             t->silent = 0;
363             sound_play(SFX_GROWL, distance);
364         }
365         break;
366     }
367     case THING_STRIKE:
368         // the blow takes a moment to come down, and that moment is
the
369         // time the player has to step back
370         if (now - t->since > (t->boss ? BOSS_WINDUP : THING_WINDUP))
{
371             if (distance < reach_of(t) + STRIKE_SLACK) {
372                 damage += damage_of(t);
373                 sound_play(SFX_HURT, 0.0);
374             }
375             t->state = THING_HUNT;
376             t->since = now;
```

SRC THING C LIGNES 413 A 423

```
413     }
414
415     load_sheet();
416     for (int n = 0; n < count; n++) {
417         struct thing *t = &things[order[n]];
418         const struct sprite *f = t->boss && sheet_boss.pixels ?
&sheet_boss
419             : t->tough && sheet_tough.pixels ? &sheet_tough
420             : (t->crewman || t->crew) && sheet_crew.pixels ? &sheet_crew
421             : &sheet;
422         if (!f->pixels)
423             continue;
```

SRC THING C LIGNES 448 A 465

```
448         double n2 = sqrt(to_x * to_x + to_y * to_y);
449         if (n2 > 0.0) { to_x /= n2; to_y /= n2; }
450         // what it looks at, which is not always where it walks
451         double fx = t->look_x || t->look_y ? t->look_x : t->dir_x;
452         double fy = t->look_x || t->look_y ? t->look_y : t->dir_y;
453         double cosa = fx * to_x + fy * to_y;
454         double sina = fx * to_y - fy * to_x;
455         double angle = atan2(sina, cosa);           // -pi a pi
456         int sector = (int)floor((angle + M_PI) / (2 * M_PI) * 8.0 + 0.5)
% 8;
457         int mirror = 0;
458         if (sector > 4) {                             // the three missing
ones
459             sector = 8 - sector;
460             mirror = 1;
461         }
```

PARTIE 4 CHAPITRE 8 LE CODE

SUITE

```
462         int row = t->state == THING_DEAD ? WALK_STEPS
463             : t->crewman ? (now < t->fire_until ? 1 : 0)
464             : ((int)(t->stride * 2.0) % WALK_STEPS);
465         int frame = row * ANGLES + sector;
```

535 La vie du boss entre 0 et 1, ou -1 tant qu'il n'a pas vu le joueur.

SRC THING C LIGNES 512 A 516

```
512         int sy = (y - top) * f->height / height;
513         unsigned int c = sprite_at(f, frame, sx, sy);
514         if (!sprite_solid(c))
515             continue;
516         // it takes the light of the square it stands on,
```

SRC THING C LIGNES 530 A 569

```
530     }
531
532     // the boss bar only shows once the boss has seen us. shown from the
533     // start it
534     // would announce the fight before the room, while shown when it wakes,
535     // it
536     // becomes the start of the fight.
537     double boss_health(void)
538     {
539         for (int i = 0; i < THINGS_MAX; i++) {
540             struct thing *t = &things[i];
541             if (t->used && t->boss && t->state != THING_IDLE
542                 && t->state != THING_DEAD)
543                 return t->health / BOSS_HEALTH;
544         }
545         return -1.0;
546     }
547
548     // its death is a separate question. if we read dead as health at zero,
549     // the
550     // bar would stay on screen, empty, after the last blow, still saying
551     // that
552     // there is something to kill.
553     int boss_down(void)
554     {
555         for (int i = 0; i < THINGS_MAX; i++)
556             if (things[i].used && things[i].boss
557                 && things[i].state == THING_DEAD)
558                 return 1;
559         return 0;
560     }
561
562     // while it rests, it takes double damage. that is the whole contract of
563     // the
564     // fight, you take the charge or you dodge it, and you get paid right
565     // after. a
566     // noise also carries far, and that is what makes shooting expensive,
567     // because
568     // the gun saves time but the price is the room next door waking up.
569     void things_hear(double x, double y, double reach, double now)
570     {
571         for (int i = 0; i < THINGS_MAX; i++) {
572             struct thing *t = &things[i];
573             if (!t->used || t->roam || t->crewman || t->crew
574                 || t->state != THING_IDLE)
575                 continue;
576             if (hypot(t->x - x, t->y - y) > reach)
```

SRC THING C LIGNES 575 A 723

```
575     }
576 }
577
578 int boss_resting(void)
579 {
580     for (int i = 0; i < THINGS_MAX; i++)
```

PARTIE 4 CHAPITRE 8 LE CODE

SUITE

```
581         if (things[i].used && things[i].boss
582             && things[i].state != THING_DEAD
583             && things[i].move == BOSS_REST)
584             return 1;
585     return 0;
586 }
587
588 double thing_weak(const struct thing *t)
589 {
590     return t->boss && t->move == BOSS_REST ? 2.0 : 1.0;
591 }
592
593 // ----- the
survivors
594 // three lines, and the last one never comes out whole. we never see them
595 // standing: the game starts afterwards.
596 static const char *LINES[][3] = {
597     { "DO NOT GO LEFT", "IT CAME THROUGH THE WALL", "TELL THEM WE" },
598     { "TOOK THE BADGE DOWN TO THE HOLD", "IT IS STILL DOWN THERE", "..."}
599 },
600 { "SEVENTEEN OF US CAME UP HERE", "I AM THE SEVENTEENTH", "" },
601 };
602 #define LINES_COUNT ((int)(sizeof LINES / sizeof *LINES))
603
604 const char *crew_speak(const struct player *player, double now)
605 {
606     int rank = 0;
607     for (int i = 0; i < THINGS_MAX; i++) {
608         struct thing *t = &things[i];
609         if (!t->used || !t->crew)
610             continue;
611         int mine = rank++ % LINES_COUNT;
612         if (t->line > 3 || now < t->next_line)
613             continue;
614         if (hypot(t->x - player->x, t->y - player->y) > 2.8)
615             continue;
616         const char *word = LINES[mine][t->line - 1];
617         t->line++;
618         t->next_line = now + 4.2;
619         if (t->line > 3) {
620             // he stops halfway through, and that is all we will ever
know
621             sound_play(SFX_DIE, 1.0);
622             t->next_line = now + NEVER;
623         }
624         return word && *word ? word : NULL;
625     }
626     return NULL;
627 }
628 // one of them falls during the scene. we hear someone die behind the
door,
629 // and if nobody fell on screen the sound would be just a sound. the one
that
630 // falls is the closest to us, so it is the biggest in the gap.
631 void things_roam_kill(void)
632 {
633     struct thing *pick = NULL;
634     double nearest = VERY_FAR;
635     for (int i = 0; i < THINGS_MAX; i++) {
636         struct thing *t = &things[i];
637         if (!t->used || !t->roam || t->state == THING_DEAD)
638             continue;
639         if (t->x < nearest) {
640             nearest = t->x;
641             pick = t;
642         }
643     }
644 }
```

590 Les coups comptent double pendant le souffle.

596 Les phrases des survivants, trois par survivant, la dernière jamais entière.

PARTIE 4 CHAPITRE 8 LE CODE

SUITE

```
643     }
644     if (pick) {
645         pick->state = THING_DEAD;
646         pick->health = 0.0;
647     }
648 }
649
650 // after the blast, the ones seen through the gap leave. during the scene
651 // they
652 // close in on the crew instead of pacing.
653 void things_roam_advance(void)
654 {
655     for (int i = 0; i < THINGS_MAX; i++)
656         if (things[i].used && things[i].roam
657             && things[i].state != THING_DEAD) {
658             things[i].roam_x += 4.5;
659             things[i].roam_to = 0.5;
660         }
661 }
662 // the two crewmen, in the order they appear in the level. the scene
663 // calls
664 // them by number, because that is easier to read than a search by
665 // position,
666 // and there will never be three of them.
667 static struct thing *crewman_at(int which)
668 {
669     int rank = 0;
670     for (int i = 0; i < THINGS_MAX; i++)
671         if (things[i].used && things[i].crewman && rank++ == which)
672             return &things[i];
673     return NULL;
674 }
675 // when a round passes close, the beast flashes white for an instant. it
676 // is
677 // the detail that says the shots land somewhere, and that the scene is
678 // not a
679 // backdrop.
680 void things_roam_graze(double now)
681 {
682     static int swell;
683     int rank = 0;
684     for (int i = 0; i < THINGS_MAX; i++) {
685         struct thing *t = &things[i];
686         if (!t->used || !t->roam || t->state == THING_DEAD)
687             continue;
688         if (rank++ == swell % 3)
689             t->hurt_at = now;
690     }
691     swell++;
692 }
693 void crew_fire(int which, double now)
694 {
695     struct thing *t = crewman_at(which);
696     if (!t || t->state == THING_DEAD)
697         return;
698     t->fire_until = now + 0.16;
699     sound_play(SFX_SHOT, 8.0);
700 }
701 void crew_fall(int which)
702 {
703     struct thing *t = crewman_at(which);
704     if (!t)
705         return;
706     t->state = THING_DEAD;
```

PARTIE 4 CHAPITRE 8 LE CODE

SUITE

```
706     t->health = 0.0;
707 }
708
709 void things_roam_away(void)
710 {
711     // they leave by a real exit, and then they no longer exist, because
712     a
713     // beast treading against a bulkhead in full view is not frightening
714     any
715     // more. the level gives them a corridor to the north, outside what
716     the
717     // gap lets us see, they take it, and once inside they are removed.
718     for (int i = 0; i < THINGS_MAX; i++)
719         if (things[i].used && things[i].roam) {
720             things[i].roam_x = 38.5;
721             things[i].roam_y = 1.5;
722             things[i].roam_to = 0.0;
723             things[i].roam_wob = 0.0;
724             things[i].leaving = 1;
725         }
```



Figure 8.2. La chose du pont quatre dans la soute, avec sa barre de vie en haut de l'écran. Elle vient de voir le joueur.

Le combat s'entend et se sent



Figure 9.1. Un tir sur une créature, avec l'éclair de la bouche du canon et le viseur ambré qui dit que le coup a porté.

Dans ce chapitre, vous allez donner un son à chaque coup et faire commencer l'arme de poing vide.

Chaque coup produit maintenant un son. Un tir, un coup de barre, l'impact sur la créature, sa mort, et la chute du boss qui fait trembler tout le pont. L'arme de poing commence vide. On peut encore frapper avec la crosse de près, et les munitions sont quelque part dans la soute. Le chargeur qui se vide est annoncé une fois, et la barre revient d'elle-même en main.

Les dégâts d'un coup sont multipliés par la faiblesse de la cible, la fonction `thing_weak` du chapitre précédent, qui vaut 2 pendant que le boss souffle. Le corps du boss est plus large à viser, avec un écart latéral admis de 0,62 case.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

Ligne par ligne

26 L'arme est trouvée vide.

INCLUDE FIGHT H LIGNES 21 A 28

```
21 #define GUN_COOLDOWN    0.42
22 #define GUN_RANGE       14.0
23 #define GUNSHOT_CARRIES 10.0    // how far a shot is heard, in cells
24 // the gun starts empty. up close you can still hit with the stock, and
    the
25 // rounds are somewhere in the hold.
26 #define AMMO_START      0
27 #define AMMO_PACK       10      // what one crate holds
28 #define MED_PACK        40.0
```

INCLUDE FIGHT H LIGNES 32 A 39

```
32     int ammo;
33     int has_gun;
34     int put_down;
35     int chosen;          // 1 the bar, 2 the gun, 0: let the game pick
36     double last_hit;     // when the player last swung
37     double flash;        // how long the muzzle flash still lights the
    room
38     double hurt;         // how long the screen still shows he was hit
39     // where the bullet landed. with no point of impact one shoots in the
```

INCLUDE FIGHT H LIGNES 42 A 50

```
42     double spark;
43     double spark_depth;
44     // the magazine has just run dry. the game says so once, and the hand
    goes
45     // back to the bar on its own.
46     int emptied;
47     // the blow has landed, and the sight shows it for a fraction of a
    second.
48     double hitmark;
49 };
50
```

INCLUDE FIGHT H LIGNES 54 A 62

```
54 // returns 1 when something was hit, so the caller can say so
55 int  fight_strike(const struct player *player, double now);
56 void fight_take(double damage);
57 void fight_step(double elapsed);
58 // what one holds is always on screen. that is what makes a first person
59 // view: without it the player does not even know he can swing. "moving"
60 // says whether the weapon sways with the walk, "now" times the swing.
61 void fight_draw(const struct player *player, double now, int moving);
62 // health, rounds, both weapons and the key that swings
```

SRC FIGHT C LIGNES 7 A 13

```
7  #include "render.h"
8  #include "screen.h"
9  #include "sound.h"
10 #include "sprite.h"
11 #include "story.h"
12 #include "text.h"
13 #include "thing.h"
```

PARTIE 4 CHAPITRE 9 LE CODE

SUITE

SRC FIGHT C LIGNES 19 A 24

```
19     memset(&fight, 0, sizeof fight);
20     fight.health = PLAYER_HEALTH;
21     fight.ammo = AMMO_START;
22     fight.has_gun = 0;        // we start with a bar, not with a gun
23 }
24
```

76 Le dernier projectile vient de partir, le jeu le dira une fois.

SRC FIGHT C LIGNES 38 A 42

```
38 }
39
40 void fight_take(double damage)
41 {
42     if (damage <= 0.0)
```

133 Les dégâts, doublés pendant le souffle du boss.

SRC FIGHT C LIGNES 56 A 61

```
56     if (fight.hitmark > 0.0)
57         fight.hitmark -= elapsed;
58     if (fight.hurt > 0.0)
59         fight.hurt -= elapsed;
60 }
61
```

SRC FIGHT C LIGNES 73 A 88

```
73     fight.ammo--;
74     fight.flash = 0.08;
75     sound_play(SFX_SHOT, 0.0);
76     fight.emptyed = fight.ammo == 0;
77     // a gunshot carries. that is what makes rounds cost more
78     // than their count: you win one fight and wake two.
79     things_hear(player->x, player->y, GUNSHOT_CARRIES, now);
80 } else {
81     sound_play(SFX_PUNCH, 0.0);
82 }
83
84     // the round stops at the first wall, because a bullet does not go
through
85     // partitions, and the impact shows where it stopped
86     double wall = wall_depth ? wall_depth[view_width / 2] : VERY_FAR;
87     struct thing *best = NULL;
88     double nearest = gun ? GUN_RANGE : PIPE_REACH;
```

SRC FIGHT C LIGNES 107 A 116

```
107     double across = fabs(dx * -player->dir_y
108         + dy * player->dir_x);
109     double radius = t->boss ? 0.62 : t->tough ? 0.46 : 0.36;
110     if (across > radius)
111         continue;
112     } else if ((dx * player->dir_x + dy * player->dir_y) / d <
PIPE_ARC) {
113         // the bar sweeps wide, and that is its whole point
114         continue;
115     }
116     nearest = d;
```

SRC FIGHT C LIGNES 131 A 148

```
131     if (!best)
132         return 0;
133     best->health -= (gun ? GUN_DAMAGE : PIPE_DAMAGE) * thing_weak(best);
134     fight.hitmark = 0.16;
135     // it takes the blow: a quarter second where it stops coming, and
136     // that is what gives melee its rhythm
137     best->stagger = now + (gun ? 0.16 : 0.26);
138     best->hurt_at = now;
139     sound_play(SFX_IMPACT, nearest);
140     if (best->health <= 0.0) {
141         best->state = THING_DEAD;
```

PARTIE 4 CHAPITRE 9 LE CODE

SUITE

```
142         fight.put_down++;
143         sound_play(SFX_DIE, nearest);
144         // the boss falls with all its weight, and the whole deck feels
it
145         if (best->boss)
146             story_hit(now, 1.8, best->x, best->y);
147     } else {
148         // being hit wakes it, even if it had not seen you
```

145 La chute du boss secoue tout le pont.

SRC FIGHT C LIGNES 159 A 163

```
159 // perspective, and that does not work out when drawn flat.
160 static void draw_sheet(struct sprite *p, int frame, double lit,
161                       int moving, int flash)
162 {
163     if (!p->pixels)
```

SRC FIGHT C LIGNES 188 A 192

```
188     int sx = (x - bx) * p->width / view_width;
189     unsigned int c = sprite_at(p, frame, sx, sy);
190     if (!sprite_solid(c))
191         continue;
192     // it takes the light of the room you stand in.
```

SRC MAIN C LIGNES 171 A 175

```
171     double damage = things_update(&player, elapsed, moment);
172     if (!won_at)
173         fight_take(damage);
174     fight_step(elapsed);
175     move_doors(elapsed);
```

L'histoire est déclenchée par des lieux

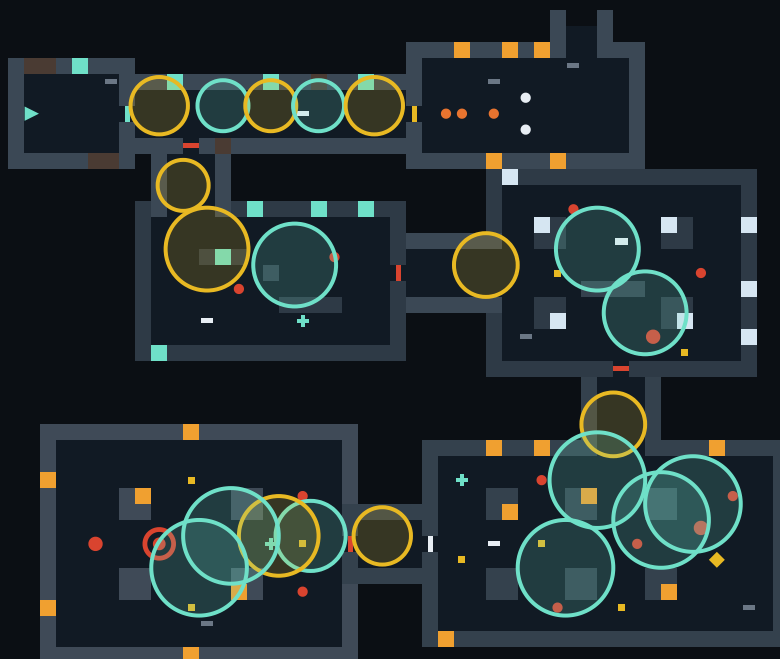


Figure 10.1. Les lieux de la table de `trigger.c` sur le plan du niveau. En jaune, ceux qui écrivent un texte. En vert, ceux qui ne font qu'un son ou un réveil.

Dans ce chapitre, vous allez déclencher l'histoire par des lieux, avec une table qui dit ce qui se passe et où.

L'histoire n'avance pas avec l'horloge, elle avance avec les pas du joueur. Le nouveau fichier `trigger.c` contient une table de lieux, chacun avec une case, un rayon, un texte à écrire ou non, un son ou non, et une action. Quand le joueur entre dans le rayon, le lieu se déclenche une seule fois, car une scène qui se joue deux fois n'est plus une scène. Le joueur croit avoir provoqué ce qui arrive, et c'est vrai.

L'ordre de la table est l'ordre du jeu. Le couloir, où la station continue de se disloquer autour du joueur. L'atelier, où la première créature se tient en pleine lumière, seule, pour apprendre à se battre sur un cas propre. La salle des machines, où elles viennent l'une après l'autre. La soute, en lumière ambrée, où ce qui dormait se lève en trois temps. Et le fond, la cloison épaisse, le rugissement, puis la chose elle-même. L'action de réveil ne réveille que la créature la plus proche du point visé, parce qu'une salle qui se lève d'un coup ressemble à un interrupteur.

L'histoire est déclenchée par des lieux, suite



Figure 10.2. L'atelier, où la première créature attend en pleine lumière. Le texte du lieu vient de s'écrire.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE TRIGGER H LIGNES 1 A 15

```
1  #ifndef TRIGGER_H
2  #define TRIGGER_H
3
4  #include "world.h"
5
6  // the story is set off by places, not by the clock. the player crosses a
   line
7  // and the scene starts, so he thinks it happened because he went there,
   and
8  // that is true. nothing takes the keys away from him, everything plays
   in
9  // front of him while he walks.
10
11 void triggers_reset(void);
12 // called every frame. it gives the line to show, or NULL when there is
   none.
13 const char *triggers_step(const struct player *player, double now);
14
15 #endif
```

SRC TRIGGER C LIGNES 1 A 127

```
1  #include <math.h>
2  #include <string.h>
3
4  #include "trigger.h"
5  #include "sound.h"
6  #include "story.h"
7  #include "thing.h"
8  #include "world.h"
9
10 // a cell, a radius, and what happens when you walk in. each one plays
   only
11 // once, because a scene that plays twice is not a scene any more.
12 enum act { ACT_NONE, ACT_WAKE, ACT_SHAKE, ACT_BLAST };
13
14 struct trigger {
15     double x, y, radius;
16     const char *text;           // what the game writes, or NULL
17     int sfx;                    // a sound, or -1
18     double reach;               // a quelle distance on l'entend
19     enum act act;
20     double rx, ry;              // where the act is aimed
21     int done;
22 };
23
24 // the order of this table is the order of the game. each line is a place
25 // where something happens, and nothing happens anywhere else. the player
26 // walks, the station answers, and he thinks he caused all of it, which
   is
27 // true.
28 static struct trigger table[] = {
29     // the corridor. we leave the airlock and the station keeps getting
30     // wrecked around us, without ever taking the keys away from us.
31     { 12.5, 6.0, 1.8, "DECK THREE  GANGWAY",      -1,      0,
      ACT_NONE, 0, 0, 0 },
```

Ligne par ligne

- 14** Un lieu, une case, un rayon, un texte, un son et une action.
- 28** La table, dans l'ordre du jeu.

PARTIE 4 CHAPITRE 10 LE CODE

SUITE

```
32     { 16.5, 6.0, 1.6, NULL,                      SFX_IMPACT, 1.5, ACT_SHAKE,
    0, 0, 0 },
33     { 19.5, 6.0, 1.6, "HULL BREACH TWO DECKS UP", -1,          0,
    ACT_NONE, 0, 0, 0 },
34     { 22.5, 6.0, 1.6, NULL,                      SFX_SHRIEK, 11.0, ACT_NONE,
    0, 0, 0 },
35     // at the end of the corridor we hear a fight behind the door. that
    is
36     // what makes you go and look, and that door is the one that will
    refuse.
37     { 26.0, 6.0, 1.8, "SOMEONE IS STILL FIGHTING", SFX_SHOT, 9.0,
    ACT_NONE, 0, 0, 0 },
38
39     // the workshop, where the first thing stands in full light, facing
    us,
40     // alone, so that you learn to fight on a clean case
41     { 14.0, 11.0, 1.6, "WORKSHOP",                -1,          0,
    ACT_NONE, 0, 0, 0 },
42     { 15.5, 15.0, 2.6, "CTRL TO SWING",           -1,          0,
    ACT_WAKE, 17.5, 17.5, 0 },
43     { 21.0, 16.0, 2.6, NULL,                      -1,          0,
    ACT_WAKE, 23.5, 15.5, 0 },
44
45     // the machine room, where they come one after another. the gun lies
    in
46     // the middle of the room, so you see it before you need it and you
    run to
47     // it afterwards.
48     { 33.0, 16.0, 2.0, "MACHINE DECK",            -1,          0,
    ACT_WAKE, 38.5, 12.5, 0 },
49     { 40.0, 15.0, 2.6, NULL,                      SFX_GROWL, 6.0, ACT_WAKE,
    45.5, 13.5, 0 },
50     { 43.0, 19.0, 2.6, NULL,                      -1,          0,
    ACT_WAKE, 43.5, 20.5, 0 },
51
52     // the hold, in amber light, where what slept in there gets up in
    three
53     // beats
54     { 41.0, 26.0, 2.0, "CARGO HOLD EMERGENCY POWER", -1,          0,
    ACT_NONE, 0, 0, 0 },
55     { 40.0, 29.5, 3.0, NULL,                      SFX_SHRIEK, 5.0, ACT_WAKE,
    36.5, 30.5, 0 },
56     { 44.0, 32.0, 3.0, NULL,                      -1,          0,
    ACT_WAKE, 43.5, 32.5, 0 },
57     { 46.0, 31.0, 3.0, NULL,                      -1,          0,
    ACT_WAKE, 46.5, 29.5, 0 },
58     { 38.0, 35.0, 3.0, NULL,                      -1,          0,
    ACT_WAKE, 38.5, 36.5, 0 },
59
60     // the bottom. first the thick bulkhead, then the roar, then the
    thing
61     // itself. the shake here is not an effect, it is the thing getting
    up.
62     { 26.5, 33.0, 1.8, "IT IS BEHIND THIS ONE",    -1,          0,
    ACT_NONE, 0, 0, 0 },
63     { 22.0, 33.0, 2.2, NULL,                      SFX_GROWL, 2.0, ACT_SHAKE,
    0, 0, 0 },
64     { 20.0, 33.0, 2.5, "IT ATE THE CREW",          -1,          0,
    ACT_WAKE, 12.5, 33.5, 0 },
65     { 17.0, 33.0, 3.0, NULL,                      -1,          0,
    ACT_WAKE, 9.5, 29.5, 0 },
66     { 15.0, 35.0, 3.0, NULL,                      -1,          0,
    ACT_WAKE, 9.5, 37.5, 0 },
67 };
68 #define TRIGGER_COUNT ((int)(sizeof table / sizeof *table))
69
70 void triggers_reset(void)
71 {
```

PARTIE 4 CHAPITRE 10 LE CODE

SUITE

```
72     for (int i = 0; i < TRIGGER_COUNT; i++)
73         table[i].done = 0;
74 }
75
76 // wakes the one thing nearest to the point asked for, and only that one,
77 // because a room that gets up all at once looks like a switch
78 static void wake_nearest(double x, double y, double now)
79 {
80     struct thing *pick = NULL;
81     double nearest = 4.0;
82     for (int i = 0; i < THINGS_MAX; i++) {
83         struct thing *t = &things[i];
84         if (!t->used || t->state == THING_DEAD || t->roam)
85             continue;
86         double d = hypot(t->x - x, t->y - y);
87         if (d < nearest) {
88             nearest = d;
89             pick = t;
90         }
91     }
92     if (pick && pick->state == THING_IDLE) {
93         pick->state = THING_ALERT;
94         pick->since = now;
95         pick->last_seen_x = x;
96         pick->last_seen_y = y;
97     }
98 }
99
100 const char *triggers_step(const struct player *player, double now)
101 {
102     for (int i = 0; i < TRIGGER_COUNT; i++) {
103         struct trigger *d = &table[i];
104         if (d->done)
105             continue;
106         if (hypot(player->x - d->x, player->y - d->y) > d->radius)
107             continue;
108         d->done = 1;
109         if (d->sfx >= 0)
110             sound_play((enum sfx)d->sfx, d->reach);
111         switch (d->act) {
112             case ACT_WAKE:
113                 wake_nearest(d->rx, d->ry, now);
114                 break;
115             case ACT_SHAKE:
116                 story_hit(now, 1.0, player->x, player->y);
117                 break;
118             case ACT_BLAST:
119                 story_hit(now, 0.5, player->x, player->y);
120                 break;
121             default:
122                 break;
123         }
124         return d->text;
125     }
126     return NULL;
127 }
```

78 Le réveil d'une seule créature, la plus proche du point visé.

106 Le joueur est-il dans le rayon du lieu.

108 Un lieu ne se déclenche qu'une fois.

La borne ne joue jamais deux fois la même partie

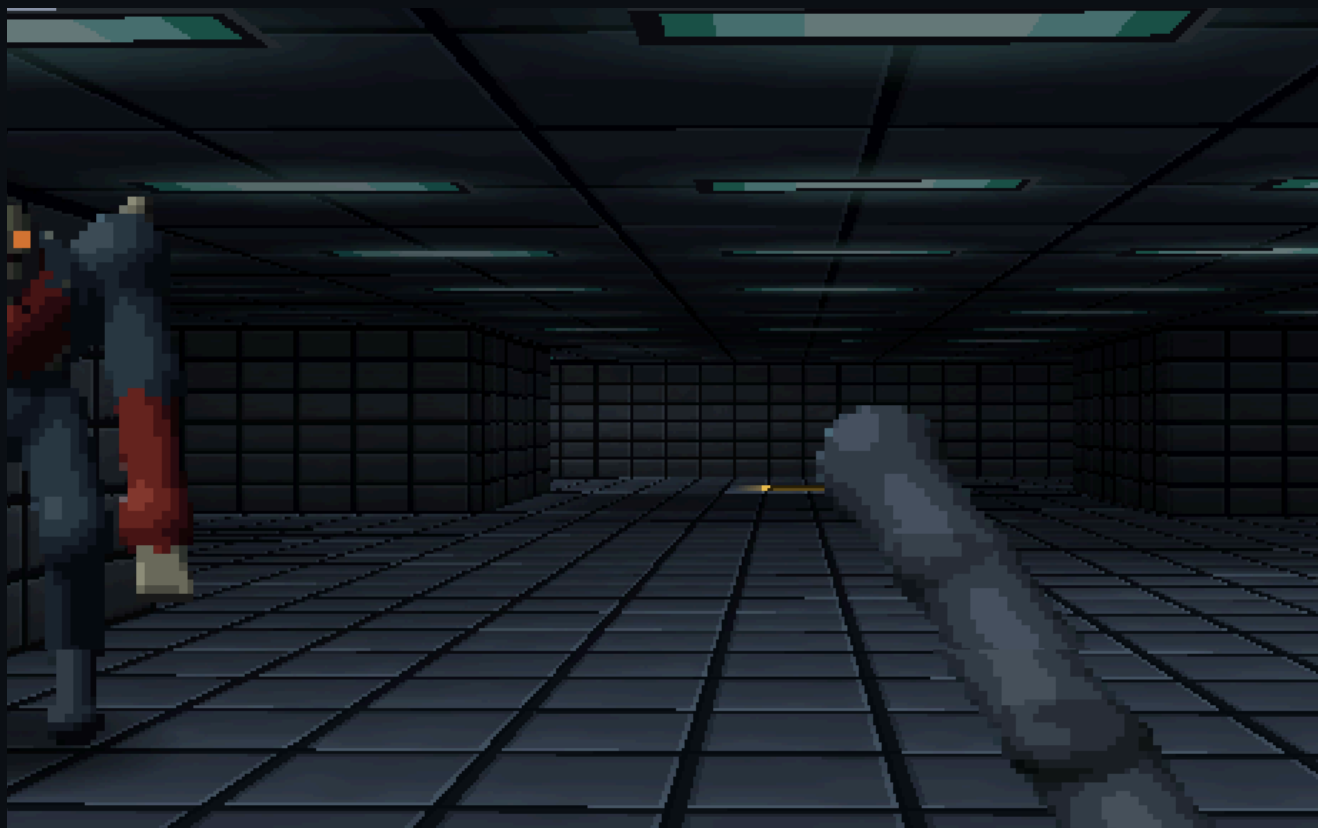


Figure 11.1. La soute, sur le courant de secours. La borne y arrive par un chemin qui varie d'une partie à l'autre.

Dans ce chapitre, vous allez donner un dé à la borne pour qu'elle ne rejoue jamais la même partie.

Une borne qui rejoue exactement la même partie se remarque en deux passages. Nous lui donnons un dé, tiré de l'horloge au démarrage. Le dé décide où elle commence parmi les premiers points de la route, pourvu qu'ils soient en vue, si elle s'arrête à un point, pendant combien de temps, et de quel côté elle tourne la tête avant de repartir.

Le dé est un générateur de nombres pseudo-aléatoires de quelques lignes, trois décalages et trois ou exclusif, que l'on appelle xorshift. Il ne vaut rien pour la cryptographie et tout pour une borne d'arcade. Chaque constante de ce chapitre porte un nom, la probabilité de s'arrêter, la durée minimale et maximale de l'arrêt, la durée du coup d'œil.

VOCABULAIRE

Un **générateur pseudo-aléatoire** est un calcul qui produit une suite de nombres d'apparence imprévisible à partir d'une graine. La même graine donne la même suite.

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

INCLUDE DEMO H LIGNES 27 A 44

```
27 #define DEMO_BODY      0.30
28 #define DEMO_STANDOFF  1.6
29 // the cabinet does not play the same game twice. it rolls a die of its
   own,
30 // seeded on the clock, and the die decides where it starts among the
   first
31 // points of the route, whether it stops at a point, for how long, and
   for how
32 // long it turns its head
33 #define DICE_SHIFT_A    13
34 #define DICE_SHIFT_B    17
35 #define DICE_SHIFT_C    5
36 #define DICE_MASK       0xfffffu
37 #define DEMO_START_SPAN 4
38 #define DEMO_LINGER_ODDS 0.4
39 #define DEMO_LINGER_MIN 0.4
40 #define DEMO_LINGER_MAX 1.4
41 #define DEMO_GLANCE     0.25
42 #define EVEN_ODDS       0.5
43
44 // how many points the file gave
```

SRC DEMO C LIGNES 1 A 5

```
1 #include <math.h>
2 #include <stdio.h>
3 #include <time.h>
4
5 #include "demo.h"
```

SRC DEMO C LIGNES 11 A 28

```
11 // how close it has come to the point so far, and when it last got closer
12 static double best_yet, better_at;
13 // the die, whether the first point has been drawn yet, and the pause at
   a
14 // point, with the moment it walks on, the moment it stops turning its
   head,
15 // and which way it turns
16 static unsigned int dice;
17 static int started, glance;
18 static double linger_until, glance_until;
19
20 static double roll(void)
21 {
22     dice ^= dice << DICE_SHIFT_A;
23     dice ^= dice >> DICE_SHIFT_B;
24     dice ^= dice << DICE_SHIFT_C;
25     return (double)(dice & DICE_MASK) / (DICE_MASK + 1.0);
26 }
27
28 void demo_reset(void)
```

SRC DEMO C LIGNES 31 A 36

```
31     best_yet = -1.0;
32     better_at = 0.0;
33     started = 0;
34     linger_until = glance_until = 0.0;
35 }
36
```

Ligne par ligne

20 Le dé, un xorshift. Trois décalages et trois ou exclusif.

PARTIE 4 CHAPITRE 11 LE CODE

SUITE

SRC DEMO C LIGNES 48 A 52

```
48     }
49     fclose(f);
50     dice = (unsigned int)time(NULL);
51     demo_reset();
52     return points;
```

50 La graine, l'heure au démarrage.

101 Le point de départ, tiré parmi ceux qui sont en vue.

SRC DEMO C LIGNES 90 A 118

```
90 }
91
92 // where it starts is not always the first point. among the first few,
   one
93 // that is in plain sight, drawn at random, so that two runs do not walk
   the
94 // same line from the first second
95 static void pick_start(const struct player *p)
96 {
97     int seen[DEMO_START_SPAN], n = 0;
98     for (int i = 0; i < DEMO_START_SPAN && i < points; i++)
99         if (clear_to(p, route_x[i], route_y[i]))
100             seen[n++] = i;
101     leg = n > 0 ? seen[(int)(roll() * n)] : 0;
102 }
103
104 // at a point it sometimes stops and looks around a little, the way
   someone
105 // who is not in a hurry does before walking on
106 static void next_point(double now)
107 {
108     leg++;
109     best_yet = -1.0;
110     better_at = now;
111     if (roll() < DEMO_LINGER_ODDS) {
112         glance = roll() < EVEN_ODDS ? -1 : 1;
113         glance_until = now + DEMO_GLANCE;
114         linger_until = glance_until + DEMO_LINGER_MIN
115             + roll() * (DEMO_LINGER_MAX - DEMO_LINGER_MIN);
116     }
117 }
118
```

111 Quatre fois sur dix, la borne s'arrête à un point.

SRC DEMO C LIGNES 138 A 147

```
138 static void follow_route(const struct player *p, struct keys *keys,
   double now)
139 {
140     if (now < linger_until) {
141         keys->left = glance < 0 && now < glance_until;
142         keys->right = glance > 0 && now < glance_until;
143         better_at = now;
144         return;
145     }
146     double turn = bearing(p, route_x[leg], route_y[leg]);
147     double d = hypot(route_x[leg] - p->x, route_y[leg] - p->y);
```

SRC DEMO C LIGNES 164 A 171

```
164     if (leg >= points)
165         return;
166     if (!started) {
167         pick_start(p);
168         started = 1;
169     }
170     const struct thing *t = nearest_live(p);
171     if (t)
```

Le niveau, du sas au fond de la soute

Dans ce chapitre, vous allez assembler le niveau entier dans la boucle principale, avec le menu, la scène derrière la porte bloquée, la musique et la fin.

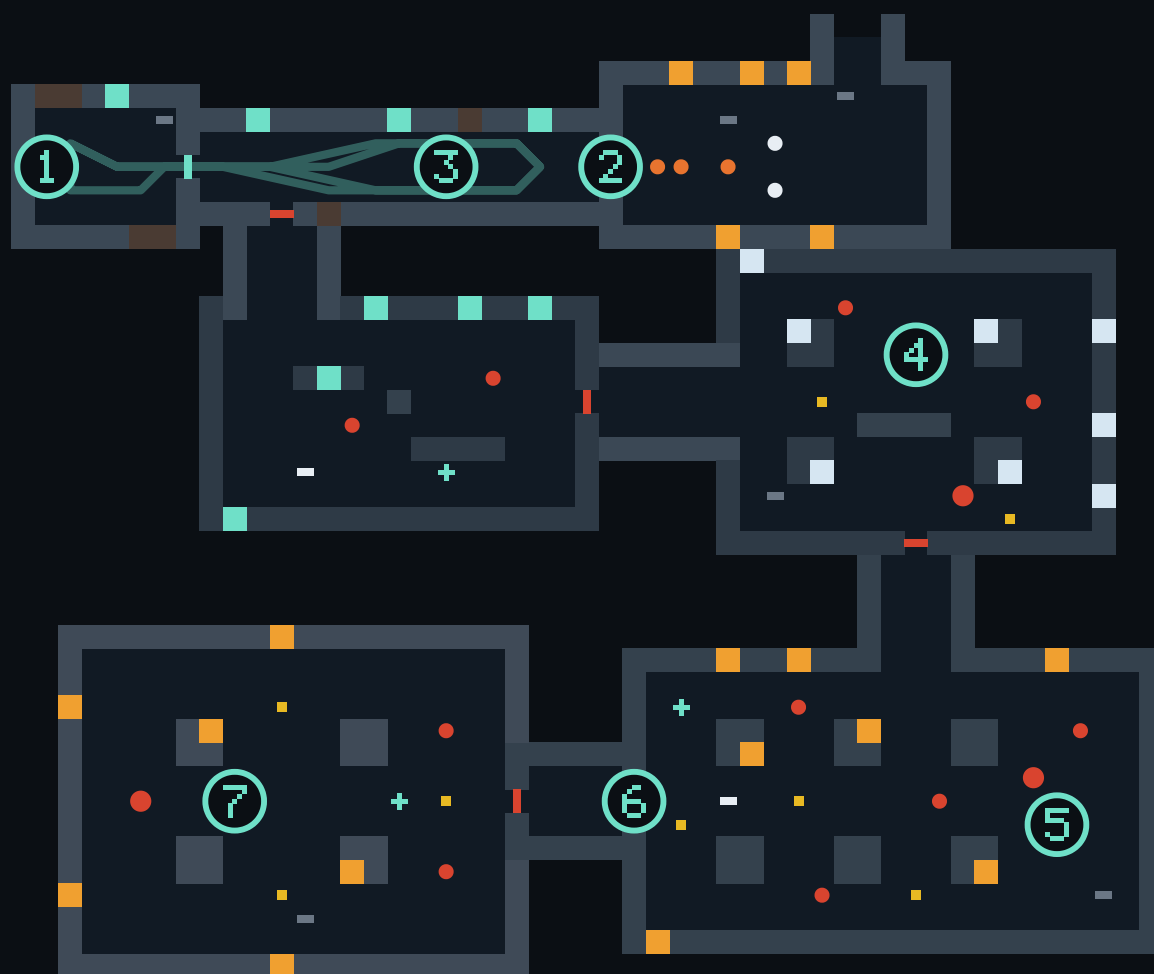
Nous assemblons maintenant tout dans la boucle principale. L'histoire commence au démarrage, les lieux sont remis à zéro, la borne charge sa route. Le menu, ouvert par Échap, fige le jeu et propose de continuer, de passer en plein écran, de recommencer ou de quitter, avec la liste des touches. Il est dessiné dans un vrai panneau, avec ses hauteurs comptées en hauteurs de texte, et le curseur est un bloc, car la police n'a pas de chevron.

La touche Espace est lue une fois et partagée. Pendant le film, elle le saute. Devant une porte, elle la pousse, et la porte répond, elle s'ouvre, elle est bloquée, elle est scellée ou elle attend le badge. La porte bloquée est le pivot du niveau. Elle s'ouvre au tiers et se coince. À travers la fente se joue une scène en huit temps, deux hommes qui tirent, une créature qui tombe, les hommes qui tombent l'un après l'autre, les créatures qui partent, et l'explosion qui coupe le contrôle des portes de tout le pont. Le joueur entend le pont s'ouvrir.

La musique est un événement, pas un fond. Rien ne joue tant que rien ne chasse le joueur, car le silence est l'état normal, et c'est lui qui donne son poids au moment où le riff arrive. La boucle principale compte les créatures éveillées et en chasse, et règle les quatre couches. Le souffle de la station une fois le pont ouvert, la basse quand quelque chose s'est levé, le riff et le métal quand ça vient. Le joueur sait ce qui arrive une seconde avant de le voir.

Le jeu se termine quand la chose du fond tombe, pas sur une case ni sur un compte à rebours, car c'est pour cela qu'on est venu. Le film de fin commence de lui-même, avec le nombre de créatures abattues. Une mort ou le menu reconstruisent le niveau par le même code, sans quoi l'une des deux façons de recommencer oublierait toujours quelque chose. Les pas s'entendent, un tous les 1,6 mètre, sous tout le reste. Le joueur ne glisse plus, il marche.

Le plan du niveau



Le niveau complet est enregistré dans un fichier texte de 42 lignes, avec un caractère par case, exactement comme la petite grille de la première partie. La ligne fine indique le chemin parcouru automatiquement par le jeu lorsque personne ne joue.

- 1 Le départ.** Le joueur se réveille dans le sas. La porte du fond est bloquée.
- 2 La porte bloquée.** Elle ne s'ouvrira jamais, il faut faire le tour.
- 3 Le survivant.** Il est allongé au sol et il lui reste trois phrases.
- 4 L'arme.** Une arme de poing attend dans l'atelier.
- 5 Le badge.** Sans lui, la porte de la soute reste fermée.
- 6 La porte de la soute.** Elle ne s'ouvre qu'avec le badge.
- 7 La chose du pont quatre.** Elle attend dans la soute. Quand elle tombe, le pont est calme.

■ mur ■ néon ■ néon de secours ■ néon froid — porte ■ porte bloquée ■ porte scellée ■ porte au badge ● une créature ■ un corps ■ munitions + soins

Le code à écrire

Voici le code de ce chapitre. Les lignes en clair sont celles que vous ajoutez, avec leur numéro dans le fichier une fois le chapitre terminé. Les lignes grises étaient déjà là et servent de repère.

SRC MAIN C LIGNES 5 A 26

```

5  #include <X11/Xlib.h>
6  #include <X11/Xutil.h>
7  #include <math.h>
8  #include <stdio.h>
9  #include <stdlib.h>
10 #include <time.h>
11 #include <unistd.h>
12
13 #include "render.h"
14 #include "light.h"
15 #include "text.h"
16 #include "screen.h"
17 #include "texture.h"
18 #include "world.h"
19 #include "sound.h"
20 #include "story.h"
21 #include "thing.h"
22 #include "demo.h"
23 #include "fight.h"
24 #include "trigger.h"
25
26 // a game says what can be done at the moment it can be done, and holds
    its

```

SRC MAIN C LIGNES 33 A 164

```

33     notice = line;
34     notice_until = moment + NOTICE_SECONDS;
35 }
36
37 // draws the boss bar at the top, and only during the fight. before the
    fight
38 // it would teach nothing, and after it there is nothing left to say.
39 static void draw_boss_bar(int hud)
40 {
41     double part = boss_health();
42     if (part < 0.0)
43         return;
44     int wide = view_width * 52 / 100, tall = 6 * hud;
45     int x0 = (view_width - wide) / 2, y0 = 4 * hud;
46     // a bar that looks empty has to mean dead, so as long as a single
    point
47     // of health is left, we keep a visible segment.
48     int filled = (int)(wide * part);
49     if (part > 0.0 && filled < 6 * hud)
50         filled = 6 * hud;
51     for (int y = 0; y < tall; y++)
52         for (int x = 0; x < wide; x++) {
53             int px = x0 + x, py = y0 + y;
54             if (px < 0 || py < 0 || px >= view_width || py >=
    view_height)
55                 continue;
56             int edge = y < hud || y >= tall - hud
                    || x < hud || x >= wide - hud;
57             view[py * view_width + px] = edge ? 0x2a2026
                    : (x < filled ? 0xa8342c : 0x14090a);
60         }
61     draw_text_centered(y0 + tall + 2 * hud, "THE THING FROM DECK FOUR",

```

Ligne par ligne

39 La barre du boss, en haut, pendant le combat seulement.

PARTIE 4 CHAPITRE 12 LE CODE

SUITE

```
62         0x8a5a52, hud);
63     }
64
65     // ----- the
        menu
66     // three lines, and the list of keys. a game without a pause forces one
        to
67     // close the window to stop, and it says nowhere how to hit. the menu is
        the
68     // place for that, and it is always one key away.
69     static const char *MENU[] = { "CONTINUE", "FULL SCREEN", "START OVER",
        "QUIT" };
70     #define MENU_COUNT ((int)(sizeof MENU / sizeof *MENU))
71     // the lines by name, so that the choice never counts on the order above
72     enum { MENU_CONTINUE, MENU_FULL_SCREEN, MENU_START_OVER, MENU_QUIT };
73
74     // two columns, the key and then what it does. they have to be aligned,
75     // otherwise it is not a list, it is a heap.
76     static const char *KEYS_LIST[][2] = {
77         { "ARROWS W S", "WALK AND TURN" },
78         { "CLICK MOUSE", "LOOK ESC GIVES IT BACK" },
79         { "A D", "SIDESTEP" },
80         { "CTRL", "ATTACK" },
81         { "1 2", "PIPE SIDEARM" },
82         { "SPACE", "OPEN A DOOR" },
83         { "M", "MAP" },
84         { "N", "SOUND" },
85         { "F11", "FULL SCREEN" },
86         { "ESC", "THIS MENU" },
87     };
88     #define TOUCHES_COUNT ((int)(sizeof KEYS_LIST / sizeof *KEYS_LIST))
89
90     // draws a filled rectangle, for the background of the panel and for its
        edges
91     static void panel(int x0, int y0, int l, int h, unsigned int color,
        double part)
92     {
93         for (int y = y0; y < y0 + h; y++)
94             for (int x = x0; x < x0 + l; x++) {
95                 if (x < 0 || y < 0 || x >= view_width || y >= view_height)
96                     continue;
97                 unsigned int c = view[y * view_width + x];
98                 int r = (int)((c >> 16) & 0xff) * (1 - part)
99                     + ((color >> 16) & 0xff) * part;
100                 int g = (int)((c >> 8) & 0xff) * (1 - part)
101                     + ((color >> 8) & 0xff) * part;
102                 int b = (int)((c & 0xff) * (1 - part) + (color & 0xff) *
        part);
103                 view[y * view_width + x] =
104                     (unsigned int)((r << 16) | (g << 8) | b);
105             }
106     }
107
108     // we draw a real panel, not lines thrown in the middle of the screen.
        heights
109     // are counted in text height rather than in interface units, so a line
        takes
110     // the height of its font times its scale, plus some air, and nothing
        else.
111     static void draw_menu(int choice, int u)
112     {
113         for (int i = 0; i < view_width * view_height; i++) {
114             unsigned int c = view[i];
115             view[i] = (((c >> 16) & 0xff) * 2 / 7) << 16)
116                 | (((c >> 8) & 0xff) * 2 / 7) << 8) | ((c & 0xff) * 2 / 7);
```

69 Les quatre lignes du menu.

111 Le panneau du menu, avec ses hauteurs comptées en hauteurs de texte.

PARTIE 4 CHAPITRE 12 LE CODE

SUITE

```
117     }
118     int st = 4 * u, se = 3 * u, sk = 2 * u;
119     int ht = GLYPH_H * st, he = GLYPH_H * se, hk = GLYPH_H * sk;
120     int air = 5 * u, margin = 9 * u;
121     int total = ht + air * 3 + u + air * 2 + MENU_COUNT * (he + air)
122         + air * 2 + u + air * 2 + hk + air
123         + TOUCHES_COUNT * (hk + air / 2 + u);
124     int ph = total + margin * 2;
125     int pl = view_width * 88 / 100;
126     int px = (view_width - pl) / 2, py = (view_height - ph) / 2;
127     if (py < 0)
128         py = 0;
129     panel(px, py, pl, ph, 0x0a0e14, 0.90);
130     panel(px, py, pl, u, 0x3f4c5a, 1.0);
131     panel(px, py + ph - u, pl, u, 0x3f4c5a, 1.0);
132     panel(px, py, u, ph, 0x3f4c5a, 1.0);
133     panel(px + pl - u, py, u, ph, 0x3f4c5a, 1.0);
134
135     int y = py + margin;
136     draw_text_centered(y, "THE KEEP", 0x9fe8d8, st);
137     y += ht + air * 2;
138     panel(px + margin, y, pl - margin * 2, u, 0x27313c, 1.0);
139     y += air * 2;
140
141     int left_x = px + margin + 4 * u;
142     for (int i = 0; i < MENU_COUNT; i++) {
143         // the cursor is a block rather than a character, because the
font
144         // only has letters and digits and there is no chevron in it.
145         if (i == choice)
146             panel(left_x, y + he / 4, se * 2, he / 2, 0xf0c060, 1.0);
147         draw_text(left_x + (GLYPH_W + 1) * se * 2, y, MENU[i],
148             i == choice ? 0xf0e4c8 : 0x5a6674, se);
149         y += he + air;
150     }
151     y += air;
152     panel(px + margin, y, pl - margin * 2, u, 0x27313c, 1.0);
153     y += air * 2;
154     draw_text(left_x, y, "CONTROLS", 0x4e5c6a, sk);
155     y += hk + air;
156     // the right column starts right after the longest key name
157     int column_x = left_x + (GLYPH_W + 1) * sk * 13;
158     for (int i = 0; i < TOUCHES_COUNT; i++) {
159         draw_text(left_x, y, KEYS_LIST[i][0], 0x8fa2b4, sk);
160         draw_text(column_x, y, KEYS_LIST[i][1], 0x5f6d7c, sk);
161         y += hk + air / 2 + u;
162     }
163 }
164
```

SRC MAIN C LIGNES 204 A 238

```
204     double last = now_in_seconds();
205     sound_open();
206     // the station is falling apart while he wakes up, and none of it
stops
207     // him from walking. it all happens around him.
208     story_begin(last);
209     things_clear();
210     fight_reset();
211     things_from_level();
212     triggers_reset();
213     demo_load(ROUTE_FILE);
214     int doors_opened = 0;
215     double died_at = 0.0, over_at = 0.0;
216     // when the keys were last touched, whether anyone has ever touched
them,
217     // and whether the cabinet is playing right now
218     double last_key = last;
```


PARTIE 4 CHAPITRE 12 LE CODE

SUITE

```
219     int hand_seen = 0, cabinet = 0;
220     // the scene behind the jammed door, where it is in its beats and
    since
221     // when
222     double breach_at = 0.0;
223     int breach_step = 0;
224     int menu_on = 0, menu_choice = 0, up_before = 0, down_before = 0;
225     int restart = 0;
226     // these are the first three things the player learns, and he learns
    them
227     // in the quiet. the swing is taught in the airlock, where there is
228     // nothing to hit, because the tool always comes before the danger.
229     say("ARROWS TO MOVE    CLICK TO USE THE MOUSE", last);
230     double lesson_at = last;
231     int lesson = 0;
232     int said_window = 0;
233
234     // a trace that we only use to tune the test runs, never on air
235     const char *trace = getenv("TEC_TRACE");
236     double said_at = 0.0;
237
238     while (!keys.quit) {
```

SRC MAIN C LIGNES 245 A 321

```
245         keys.any = 0;
246         last_key = moment;
247         hand_seen = 1;
248         // the hand is back: the cabinet lets go of what it held
249         if (cabinet)
250             keys.forward = keys.back = keys.left = keys.right = 0;
251         cabinet = 0;
252     }
253     // if nobody has touched the keys since the start, the level
    plays
254     // itself. once someone has played, it never takes the hand
    again,
255     // because a cabinet walks off on its own from the start screen,
    not
256     // in the middle of somebody's game.
257     if (!hand_seen && moment - last_key > DEMO_AFTER &&
    story_control()
258         && fight.health > 0.0 && !menu_on) {
259         demo_drive(&player, &keys, moment);
260         cabinet = 1;
261     }
262     if (keys.menu) {
263         keys.menu = 0;
264         menu_on = !menu_on;
265         menu_choice = 0;
266         // the menu gives the mouse back, otherwise you cannot even
    click
267         // somewhere else to leave the game
268         if (menu_on)
269             screen_release_mouse(&screen);
270     }
271     // the menu freezes everything. the room is still drawn behind
    it,
272     // darkened, and the game waits, so nothing moves and nothing
    hits.
273     if (menu_on) {
274         int go_up = keys.forward && !up_before;
275         int go_down = keys.back && !down_before;
276         up_before = keys.forward;
277         down_before = keys.back;
278         if (go_up)
```

257 La borne ne prend la main qu'à l'écran de départ, jamais au milieu d'une partie.

273 Le menu fige tout. La salle est dessinée derrière, assombrie.

PARTIE 4 CHAPITRE 12 LE CODE

SUITE

```
279         menu_choice = (menu_choice + MENU_COUNT - 1) %
MENU_COUNT;
280     if (go_down)
281         menu_choice = (menu_choice + 1) % MENU_COUNT;
282     if (keys.push || keys.validate) {
283         keys.push = keys.validate = 0;
284         if (menu_choice == MENU_CONTINUE) {
285             menu_on = 0;
286         } else if (menu_choice == MENU_FULL_SCREEN) {
287             screen_toggle_fullscreen(&screen);
288             menu_on = 0;
289         } else if (menu_choice == MENU_START_OVER) {
290             menu_on = 0;
291             fight.health = 0.0;
292             died_at = moment - 2.0;
293             restart = 1;
294         } else {
295             keys.quit = 1;
296         }
297     }
298     keys.hit = 0;
299     fit_view_to_window(&player);
300     render_floor_and_ceiling(&player);
301     render_walls(&player);
302     things_draw(&player, moment);
303     int hudm = view_width / HUD_DIVISOR;
304     draw_menu(menu_choice, hudm < 1 ? 1 : hudm);
305     screen_present(&screen);
306     usleep(16000);
307     continue;
308 }
309 up_before = keys.forward;
310 down_before = keys.back;
311
312 // a door that opens by itself teaches nothing; a door that
313 // asks for a key and says so does
314 if (door_ahead(&player) && story_control())
315     say("PRESS SPACE TO OPEN", moment);
316 // the space key is read once and then shared, because it skips
the
317 // film, pushes a door and restarts after a death. if each of
them
318 // cleared it on its own, the last one would never see it. we
also
319 // handle death before the door, because a dead man pushes
nothing.
320 int push_key = keys.push;
321 keys.push = 0;
```

SRC MAIN C LIGNES 324 A 361

```
324     push_key = 0;
325     if (push_key) {
326         // during the film, space only skips the film and does
nothing
327         // else
328         enum push which = story_control()
? push_door(&player) : PUSH_NOTHING;
329         if (!story_control())
330             story_skip(moment);
331         if (trace)
332             fprintf(stderr, "PUSH %d seals %d\n",
333                 (int)which, world_seals_released());
334         switch (which) {
335             case PUSH_OPENS:
336                 doors_opened++;
337                 sound_play(SFX_D00R, 0.0);
338                 break;
```

PARTIE 4 CHAPITRE 12 LE CODE

SUITE

```
340          // the jammed door is the pivot of the whole level. it opens
341  a          // third of the way, it sticks, and through the gap you see
342  what       // you will not reach. that same jam is what releases the
343  seals,     // so its failure is what opens the game.
344  case PUSH_JAMMED:
345      sound_play(SFX_DOOR, 0.0);
346      say("JAMMED IT IS NOT GOING TO OPEN", moment);
347      breach_at = moment;
348      breach_step = 0;
349      break;
350  case PUSH_SEALED:
351      say("NO POWER IN THIS ONE", moment);
352      break;
353  case PUSH_LOCKED:
354      sound_play(SFX_IMPACT, 4.0);
355      say("VAULT THE BADGE OPENS THIS", moment);
356      break;
357  default:
358      break;
359  }
360  }
361  // picked up by walking over it, no stop and no panel
```

344 La porte bloquée
déclenche la scène
derrière la fente.

SRC MAIN C LIGNES 365 A 487

```
365      fight.ammo += AMMO_PACK;
366      say("SIDEARM IT STILL FEEDS", moment);
367      sound_play(SFX_PICKUP, 0.0);
368      break;
369  case LOOT_AMMO:
370      fight.ammo += AMMO_PACK;
371      say("ROUNDS", moment);
372      sound_play(SFX_PICKUP, 0.0);
373      break;
374  case LOOT_MED:
375      fight_heal(MED_PACK);
376      say("PATCHED UP", moment);
377      sound_play(SFX_PICKUP, 0.0);
378      break;
379  case LOOT_BADGE:
380      say("BADGE THE VAULT WILL OPEN NOW", moment);
381      sound_play(SFX_PICKUP, 0.0);
382      break;
383  default:
384      break;
385  }
386  if (lesson < 2 && story_control() && moment - lesson_at > 7.0) {
387      say(lesson ? "ESC FOR THE MENU AND THE KEYS"
388          : "CTRL SWINGS WHAT YOU ARE HOLDING", moment);
389      lesson_at = moment;
390      lesson++;
391  }
392
393  // the pattern is taught once and only once. the boss charges,
394  misses // and rests, and it is while it rests that it really gets hurt.
395  a
396  // player who does not find that out loses without knowing why,
397  so we // say it in one line at the first rest and never again.
398  if (!said_window && boss_resting()) {
399      said_window = 1;
400      say("IT IS WIDE OPEN HIT IT NOW", moment);
401  }
```

PARTIE 4 CHAPITRE 12 LE CODE

SUITE

```
402         // the scene behind the jammed door plays in eight beats. the
door is
403         // forced, it sticks, and through the gap you see a fight at the
far
404         // end. there are shots, then a cry, then it stops, and what
blows up
405         // down there cuts the door controls of the whole deck.
406         if (breach_at > 0.0) {
407             double t = moment - breach_at;
408             // the battle, beat by beat. two men hold the far end of the
deck
409             // and fire, one beast falls, then the men fall one after the
410             // other, and the beasts leave by the far end. none of it
comes
411             // towards us, because the gap is only a window.
412             static const struct { double when; int which; } scene[] = {
413                 { 0.4, 0 }, { 0.8, 1 }, { 1.2, 0 }, { 1.6, 1 },
414                 { 2.0, 2 }, { 2.4, 3 }, { 2.8, 0 }, { 3.2, 1 },
415                 { 3.6, 0 }, { 4.2, 4 }, { 4.7, 1 }, { 5.2, 1 },
416                 { 5.9, 5 }, { 6.6, 6 }, { 8.2, 7 }, { 11.0, 8 },
417             };
418             int amount = (int)(sizeof scene / sizeof *scene);
419             while (breach_step < amount && t > scene[breach_step].when) {
420                 switch (scene[breach_step].which) {
421                     case 0:
422                         crew_fire(0, moment);
423                         things_roam_graze(moment);
424                         break;
425                     case 1:
426                         crew_fire(1, moment);
427                         things_roam_graze(moment);
428                         break;
429                     case 2:
430                         // this shot lands, and a beast falls
431                         things_roam_kill();
432                         sound_play(SFX_DIE, 9.0);
433                         break;
434                     case 3:
435                         // the beasts shriek and close in on the men
436                         sound_play(SFX_SHRIEK, 9.0);
437                         things_roam_advance();
438                         break;
439                     case 4:
440                         crew_fall(0);
441                         sound_play(SFX_HURT, 9.0);
442                         break;
443                     case 5:
444                         crew_fall(1);
445                         sound_play(SFX_HURT, 9.0);
446                         break;
447                     case 6:
448                         // and the beasts leave by the far end
449                         things_roam_away();
450                         sound_play(SFX_GROWL, 9.0);
451                         break;
452                     case 7:
453                         sound_play(SFX_BLAST, 1.5);
454                         story_hit(moment, 2.4,
455                             player.x + player.dir_x * 6.0,
456                             player.y + player.dir_y * 6.0);
457                         story_blast(moment);
458                         say("DOOR CONTROL IS GONE", moment);
459                         world_release_seals();
460                         // we hear the deck open, three doors at different
461                         // distances, and the deck becomes another place
```

412 Les huit temps de la scène, en secondes depuis la poussée.

459 L'explosion coupe le contrôle des portes, et toutes les portes scellées s'ouvrent.

PARTIE 4 CHAPITRE 12 LE CODE

SUITE

```
462             sound_play(SFX_D00R, 6.0);
463             sound_play(SFX_D00R, 12.0);
464             break;
465         default:
466             say("EVERY SEAL ON THE DECK JUST OPENED", moment);
467             breach_at = 0.0;
468             break;
469     }
470     breach_step++;
471 }
472 }
473 const char *scene = triggers_step(&player, moment);
474 if (scene)
475     say(scene, moment);
476 // this is what the survivors have to say. they lie on the floor,
you
477 // lean over them, they let out a line, then another one, and the
478 // third never comes out whole. they are the only crew we will
ever
479 // see, and that is what makes it real.
480 const char *word = crew_speak(&player, moment);
481 if (word)
482     say(word, moment);
483 story_shake_here(player.x, player.y);
484 story_tick(moment);
485 story_update(moment, doors_opened);
486 // 1 and 2 pick what is held. asking for the gun with no
487 // rounds does nothing, and the hud already says why.
```

SRC MAIN C LIGNES 496 A 625

```
496     keys.weapon = 0;
497 }
498 if (keys.mute) {
499     sound_mute(!sound_muted());
500     say(sound_muted() ? "SOUND OFF" : "SOUND ON", moment);
501     keys.mute = 0;
502 }
503 if (keys.hit) {
504     if (story_control())
505         fight_strike(&player, moment);
506     keys.hit = 0;
507 }
508 // the magazine has just run dry. we say it once, the bar comes
back
509 // on its own, and the player knows why.
510 if (fight.emptyed) {
511     fight.emptyed = 0;
512     say("EMPTY BACK TO THE PIPE", moment);
513 }
514 // on the test bench we take every hit without falling, so that
the
515 // whole level can be walked and photographed
516 double damage_taken = things_update(&player, elapsed, moment);
517 fight_take(getenv("TEC_ESSAI") ? 0.0 : damage_taken);
518 fight_step(elapsed);
519 // the music is an event, not a background. nothing plays while
520 // nothing hunts us, because silence is the normal state, and
that
521 // silence is what gives weight to the moment the riff comes in.
522 int hunted = 0, awake = 0;
523 for (int i = 0; i < THINGS_MAX; i++) {
524     if (!things[i].used || things[i].roam)
525         continue;
526     if (things[i].state == THING_HUNT
527         || things[i].state == THING_STRIKE)
```

PARTIE 4 CHAPITRE 12 LE CODE

SUITE

```
528         hunted = 1;
529         else if (things[i].state == THING_ALERT)
530             awake = 1;
531     }
532     // three states, and you hear them before you see them. the
breath of
533     // the station once the deck gives way, a bass when something has
got
534     // up, and the riff when it comes at us. the player knows what is
535     // coming a second before he sees it.
536     sound_layer(LAYER_BREATH, world_seals_released() ? 0.30 : 0.0);
537     sound_layer(LAYER_PULSE, hunted || awake ? 0.75 : 0.0);
538     sound_layer(LAYER_DRIVE, hunted ? 0.85 : 0.0);
539     sound_layer(LAYER_METAL, hunted ? 0.70 : 0.0);
540     move_doors(elapsed);
541     lamp_flicker(moment);
542     remember(&player);
543
544     // the game ends when the thing at the bottom falls, not on a
cell or
545     // on a countdown, because that is what we came here for. the
closing
546     // film then starts on its own.
547     if (boss_down() && over_at == 0.0) {
548         over_at = moment;
549         story_score(fight.put_down);
550         say("IT IS DOWN", moment);
551         sound_layer(LAYER_DRIVE, 0.0);
552         sound_layer(LAYER_METAL, 0.0);
553     }
554     if (over_at > 0.0 && moment - over_at > 2.2)
555         story_finish(moment);
556     if (story_finished())
557         keys.quit = 1;
558     // dying must not mean starting the program again, because a
player
559     // sent back to a terminal does not retry. death also has to be
seen,
560     // so the world freezes, the colour drains, the bars come back,
and
561     // what to do next is written large in the middle.
562     if (fight.health <= 0.0) {
563         if (!died_at) {
564             died_at = moment;
565             story_death(moment);
566             sound_layer(LAYER_DRIVE, 0.0);
567             sound_layer(LAYER_METAL, 0.0);
568             sound_layer(LAYER_PULSE, 0.0);
569         }
570         if (moment - died_at > 1.3 && death_push)
571             restart = 1;
572     }
573     // the level is rebuilt in place, after a death or from the menu,
and
574     // both go through the same code. with two ways to start over and
two
575     // resets, one of them would always forget something.
576     if (restart) {
577         restart = 0;
578         died_at = 0.0;
579         load_level(level_file);
580         level_marks(&player.x, &player.y, &exit_x, &exit_y,
581                 &player.dir_x, &player.dir_y);
582         // the eyes turn back to the start, so the camera plane has
to
```

538 Le riff n'entre que quand une créature chasse le joueur.

547 La fin du jeu, la chute du boss.

PARTIE 4 CHAPITRE 12 LE CODE

SUITE

```
583         // follow them. left where it was, it is no longer square to
584         // the eyes, and the whole picture shears as soon as we turn
585         player.plane_x = -player.dir_y * FIELD_OF_VIEW;
586         player.plane_y = player.dir_x * FIELD_OF_VIEW;
587         things_clear();
588         things_from_level();
589         triggers_reset();
590         fight_reset();
591         story_restart(moment);
592         demo_reset();
593         doors_opened = 0;
594         over_at = breach_at = 0.0;
595         breach_step = 0;
596         say("AGAIN", moment);
597     }
598
599     // every move is scaled by the time the last frame took, so the
game
600     // runs at the same speed whatever the machine is doing
601     int alive = story_control() && fight.health > 0.0;
602     double forward = alive
603         ? (keys.forward - keys.back) * WALK_SPEED * elapsed : 0.0;
604     double sideways = alive
605         ? (keys.strafe_right - keys.strafe_left) * WALK_SPEED *
elapsed : 0.0;
606     double turn = (alive ? (keys.right - keys.left) * TURN_SPEED :
0.0)
607         * elapsed + story_drift() * elapsed;
608     if (alive)
609         turn += keys.look;
610
611     double step_x = player.dir_x * forward + player.plane_x *
sideways;
612     double step_y = player.dir_y * forward + player.plane_y *
sideways;
613
614     // you hear the steps. without them the character slides, and a
615     // character that slides has no weight. there is one step every
metre
616     // sixty, and the sound stays under everything else.
617     static double walked;
618     walked += hypot(step_x, step_y);
619     if (walked > 1.6) {
620         walked = 0.0;
621         if (alive)
622             sound_play(SFX_STEP, 0.0);
623     }
624     move_player(&player, step_x, step_y);
625     turn_player(&player, turn);
```

SRC MAIN C LIGNES 636 A 686

```
636     // filled for them to know what hides them
637     things_draw(&player, moment);
638     // during the film there is only the film. the weapon and the
gauges
639     // inside a cinema frame are the one thing that could break the
effect
640     // once the bars are in place.
641     int film = storyBars() > 0.005;
642     if (!film)
643         fight_draw(&player, moment, keys.forward || keys.back
|| keys.strafe_left || keys.strafe_right);
644     if (keys.map && story_control() && seen_count() > 12)
645         render_map(&player, hud * MAP_CELL, hud * MAP_LEFT,
hud * MAP_TOP);
646
647
```

585 Au redémarrage, le plan de la caméra est remis d'équerre avec la direction, sans quoi l'image se cisaille dès que le joueur tourne.

619 Un pas tous les 1,6 mètre.

PARTIE 4 CHAPITRE 12 LE CODE

SUITE

```
648         // the blast is drawn over the world and over the weapon, because
        it
649         // is light, not an interface layer
650         story_draw_blast();
651         // the death screen is drawn over the world and under the bars
652         story_draw_death();
653         story_notice(moment < notice_until || story_dead());
654         story_draw_dust(&player, moment);
655         if (!film) {
656             fight_hud(moment);
657             draw_boss_bar(hud);
658         }
659         story_draw_log();
660         if (moment < notice_until)
661             draw_text_centered(view_height - hud * NOTICE_UP, notice,
662                               TEXT_COLOR, hud * TEXT_SCALE);
663         // the bars are drawn over everything else, because they are a
        frame,
664         // not a layer of the game
665         story_draw_film();
666         screen_present(&screen);
667         if (trace && moment - said_at > 0.05) {
668             said_at = moment;
669             // we also print where the nearest awake thing stands,
        otherwise
670         // the test bench shoots into nothing and no screenshot ever
        shows
671         // a fight
672         double cx = 0.0, cy = 0.0, nearest = VERY_FAR;
673         for (int i = 0; i < THINGS_MAX; i++) {
674             struct thing *t = &things[i];
675             if (!t->used || t->roam || t->state == THING_DEAD
676                 || t->state == THING_IDLE)
677                 continue;
678             double d = hypot(t->x - player.x, t->y - player.y);
679             if (d < nearest) { nearest = d; cx = t->x; cy = t->y; }
680         }
681         fprintf(stderr, "%.2f %.2f  cap %.2f  health %.0f  rounds %d"
682                "  target %.2f %.2f  boss %.2f\n",
683                player.x, player.y, atan2(player.dir_y, player.dir_x),
684                fight.health, fight.ammo, cx, cy, boss_health());
685     }
686
```



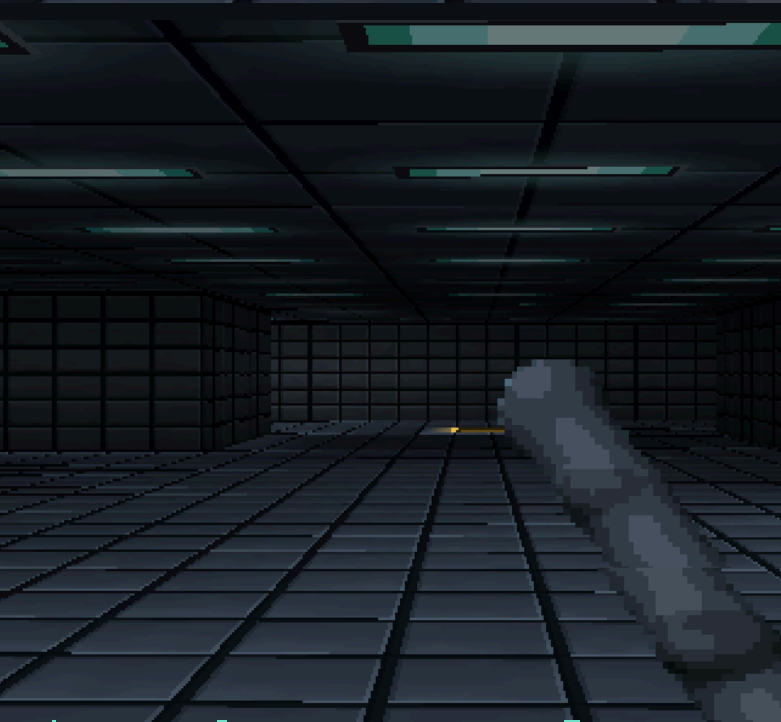
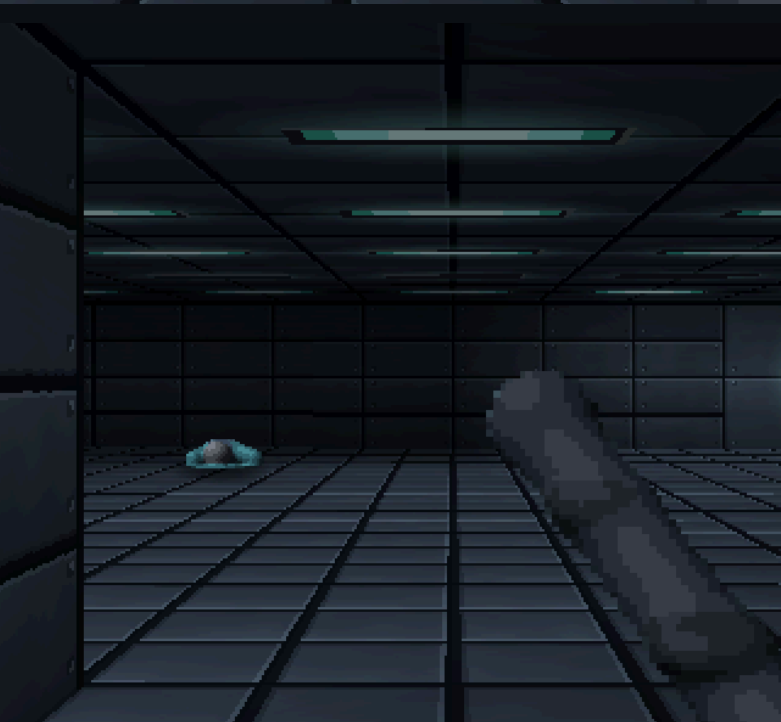
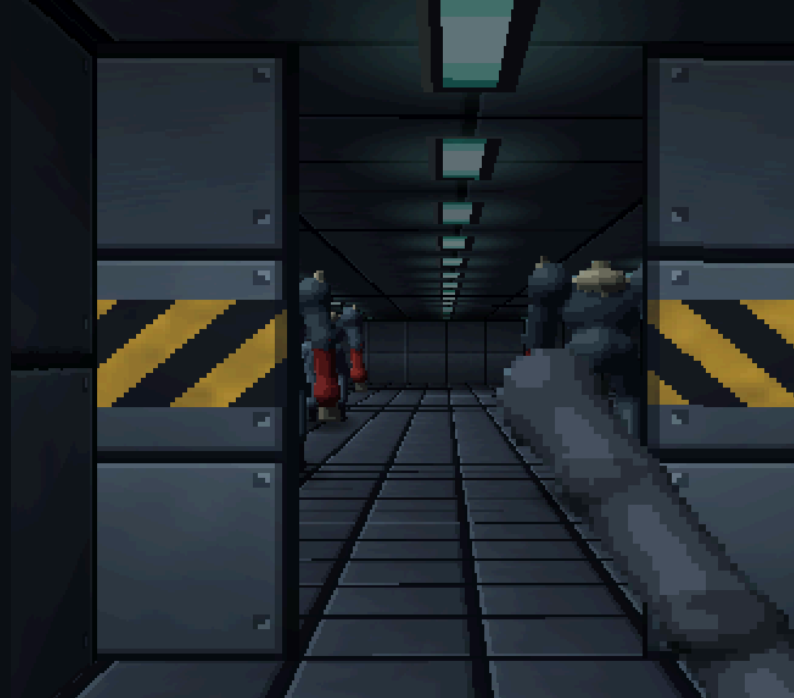
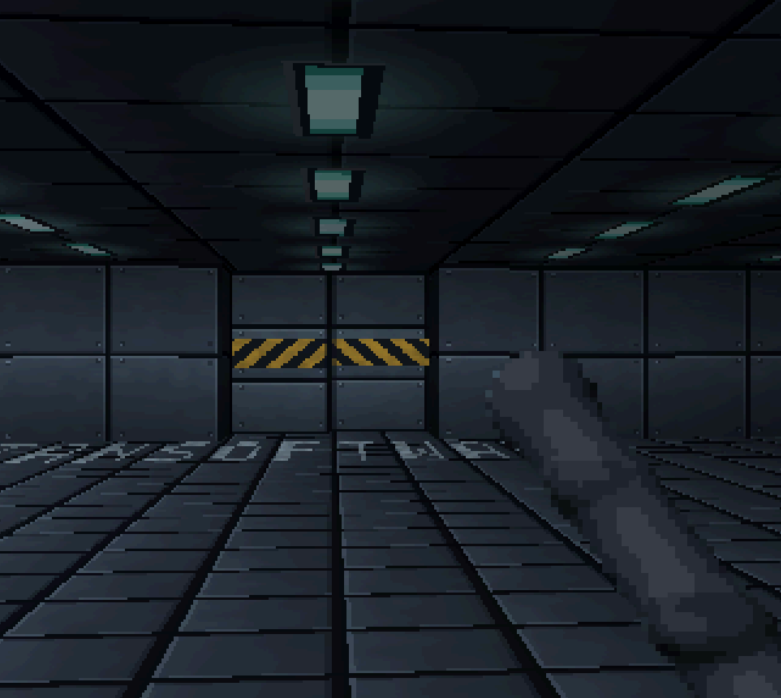

Figure 12.1. Le film de fin. Le pont trois est calme, et la seconde ligne compte ce que le joueur a abattu.

VOCABULAIRE

Le pivot d'un niveau est l'endroit où ce que le joueur ne peut pas faire décide de ce qu'il va faire. Ici, la porte bloquée.

EXERCICE

Jouez la partie entière, puis relancez sans toucher une touche. La borne joue à votre place, par un autre chemin.



1

2

3

4

5

6

326

LE SAS

LA PORTE BLOQUEE

L ATELIER

UNE CREATURE

LA SOUTE

LA CHOSE

Ce qui reste à faire

Le jeu est terminé, et le cours avec lui. Tout ce qui est à l'écran a été écrit dans ces soixante-trois chapitres, du premier appel de fonction à la chute de la chose du pont quatre. Il n'y a ni moteur, ni bibliothèque graphique, ni fichier d'image ou de son, seulement du C, Xlib et ALSA.

Le dépôt GitHub contient le code de chaque partie, avec un commit par chapitre, ce qui permet de suivre le cours commit après commit. Le jeu se joue dans le navigateur, compilé en WebAssembly, sur le site de la chaîne. Les quatre séances où il a été écrit sont sur YouTube, dans la liste de lecture The Keep.

Ce que ce cours a montré se prolonge. Le même moteur peut recevoir des murs de hauteurs différentes, des escaliers, des textures chargées depuis des fichiers, un réseau pour jouer à deux. Chaque ajout suit la même méthode, une idée, un calcul, quelques dizaines de lignes, et un lancement pour voir.

Le prochain jeu de la chaîne sera un vrai moteur en trois dimensions, en fil de fer, sur un rail, avec un son calculé qui se joue au rythme du joueur. Il aura son propre livre, dans la même collection.

LES ADRESSES

Le code github.com/tanguychenier/live-coding

Le jeu dans le navigateur tanguychenier.github.io/live-coding/keep/

Les séances youtube.com/playlist?list=PLBPRrs12VQXc

TEO LE COURS

Un moteur de jeu en 2.5D, écrit en C à partir d'un fichier vide, expliqué de bout en bout. Le lancer de rayons, les textures dessinées par le code, les portes, la lumière, les créatures, le son calculé et l'histoire, en soixante-trois chapitres, avec le code de chaque chapitre et les calculs refaits à la main.

Le jeu obtenu, The Keep, se joue dans le navigateur, et son code est public, un commit par chapitre.

Le code github.com/tanguychenier/live-coding

Le jeu tanguychenier.github.io/live-coding/keep/

La chaîne youtube.com/@tanguy_tec

